

INTRODUCTION TO ARTIFICIAL INTELLIGENCE



INTRODUCTION

AI is one of the newest disciplines, formally initiated in 1956 when the name was coined. However, the study of intelligence is one of the oldest disciplines being approximately 2000 years old. The advent of computers made it possible for the first time for people to test models they proposed for learning, reasoning, perceiving, etc.

Artificial Intelligence is composed of two words **Artificial** and **Intelligence**, where **Artificial** defines "man-made," and **intelligence** defines "thinking power", hence AI means "a man-made thinking power."

WHAT IS ARTIFICIAL INTELLIGENCE?

Artificial intelligence is **the simulation of human intelligence processes by machines, especially computer systems**. Specific applications of AI include expert systems, natural language processing, speech recognition and machine vision.

Thinking Humanly

“The exciting new effort to make computers think . . . *machines with minds*, in the full and literal sense.” (Haugeland, 1985)

“[The automation of] activities that we associate with human thinking, activities such as decision-making, problem solving, learning . . .” (Bellman, 1978)

Thinking Rationally

“The study of mental faculties through the use of computational models.”
(Charniak and McDermott, 1985)

“The study of the computations that make it possible to perceive, reason, and act.”
(Winston, 1992)

Acting Humanly

“The art of creating machines that perform functions that require intelligence when performed by people.” (Kurzweil, 1990)

“The study of how to make computers do things at which, at the moment, people are better.” (Rich and Knight, 1991)

Acting Rationally

“Computational Intelligence is the study of the design of intelligent agents.” (Poole *et al.*, 1998)

“AI . . . is concerned with intelligent behavior in artifacts.” (Nilsson, 1998)

Some definitions of artificial intelligence, organized into four categories.

ACTING HUMANLY

- The first proposal for success in building a program and acts humanly was the Turing Test. To be considered intelligent a program must be able to act sufficiently like a human to fool an interrogator.
- A human interrogates the program and another human via a terminal simultaneously. If after a reasonable period, the interrogator cannot tell which is which, the program passes.

To pass this test requires:

- ✓ Natural language processing
 - ✓ Knowledge representation
 - ✓ Automated reasoning
 - ✓ Machine learning
- This test avoids physical contact and concentrates on "higher level" mental faculties. A total Turing test would require the program to also do:
 - ✓ Computer vision
 - ✓ Robotics

THINKING HUMANLY

- This requires "getting inside" of the human mind to see how it works and then comparing our computer programs to this. This is what cognitive science attempts to do.
- Another way to do this is to observe a human problem solving and argue that one's programs go about problem solving in a similar way.

EXAMPLE:

- GPS (General Problem Solver) was an early computer program that attempted to model human thinking.
- The developers were not so much interested in whether or not GPS solved problems correctly.
- They were more interested in showing that it solved problems like people, going through the same steps and taking around the same amount of time to perform those steps.

THINKING RATIONALLY

Aristotle was one of the first to attempt to codify "thinking". His **syllogisms** provided patterns of argument structure that always gave correct conclusions, given correct premises.

EXAMPLE: All computers use energy. Using energy always generates heat. Therefore, all computers generate heat.

This initiated the field of **logic**. Formal logic was developed in the late nineteenth century. This was the first step toward enabling computer programs to reason logically.

By **1965**, programs existed that could, given enough time and memory, take a description of the problem in logical notation and find the solution, if one existed. The **logician** tradition in AI hopes to build on such programs to create intelligence.

There are two main obstacles to this approach: First, it is difficult to make informal knowledge precise enough to use the logicist approach particularly when there is uncertainty in the knowledge.

Second, there is a big difference between being able to solve a problem in principle and doing so in practice.

ACTING RATIONALLY / THE RATIONAL AGENT APPROACH

Acting rationally means acting so as to achieve one's goals, given one's beliefs. An agent is just something that perceives and acts.

In the logical approach to AI, the emphasis is on correct inferences. This is often part of being a rational agent because one way to act rationally is to reason logically and then act on one's conclusions. But this is not all of rationality because agents often find themselves in situations where there is no provably correct thing to do, yet they must do something. There are also ways to act rationally that do not seem to involve inference, e.g., reflex actions.

The study of AI as rational agent design has two advantages:

1. It is more general than the logical approach because correct inference is only a useful mechanism for achieving rationality, not a necessary one.
2. It is more amenable to scientific development than approaches based on human behaviour or human thought because a standard of rationality can be defined independent of humans.

Achieving perfect rationality in complex environments is not possible because the computational demands are too high.

However, we will study perfect rationality as a starting place.

FOUNDATIONS OF AI

Like any history, this one is forced to concentrate on a small number of people, events, and ideas and to ignore others that also were important. We organize the history around a series of questions. We certainly would not wish to give the impression that these questions are the only ones the disciplines address or that the disciplines have all been working toward AI as their ultimate fruition.

1. PHILOSOPHY

- ✓ Can formal rules be used to draw valid conclusions?
- ✓ How does the mind arise from a physical brain?
- ✓ Where does knowledge come from?
- ✓ How does knowledge lead to action?

Aristotle (384–322 B.C.), was the first to formulate a precise set of laws governing the rational part of the mind. He developed an informal system of syllogisms for proper reasoning, which in principle allowed one to generate conclusions mechanically, given initial premises.

Much later, Ramon Lull (d. 1315) had the idea that useful reasoning could actually be carried out by a mechanical artifact.

Thomas Hobbes (1588–1679) proposed that reasoning was like numerical computation that “we add and subtract in our silent thoughts.”

2 MATHEMATICS

- ✓ What are the formal rules to draw valid conclusions?
- ✓ What can be computed?
- ✓ How do we reason with uncertain information?

Philosophers staked out most of the important ideas of AI, but to move to a formal science requires a level of mathematical formalism in three main areas: computation, logic and probability.

Mathematicians have proved that there exists an algorithm to prove any true statement in first-order logic.

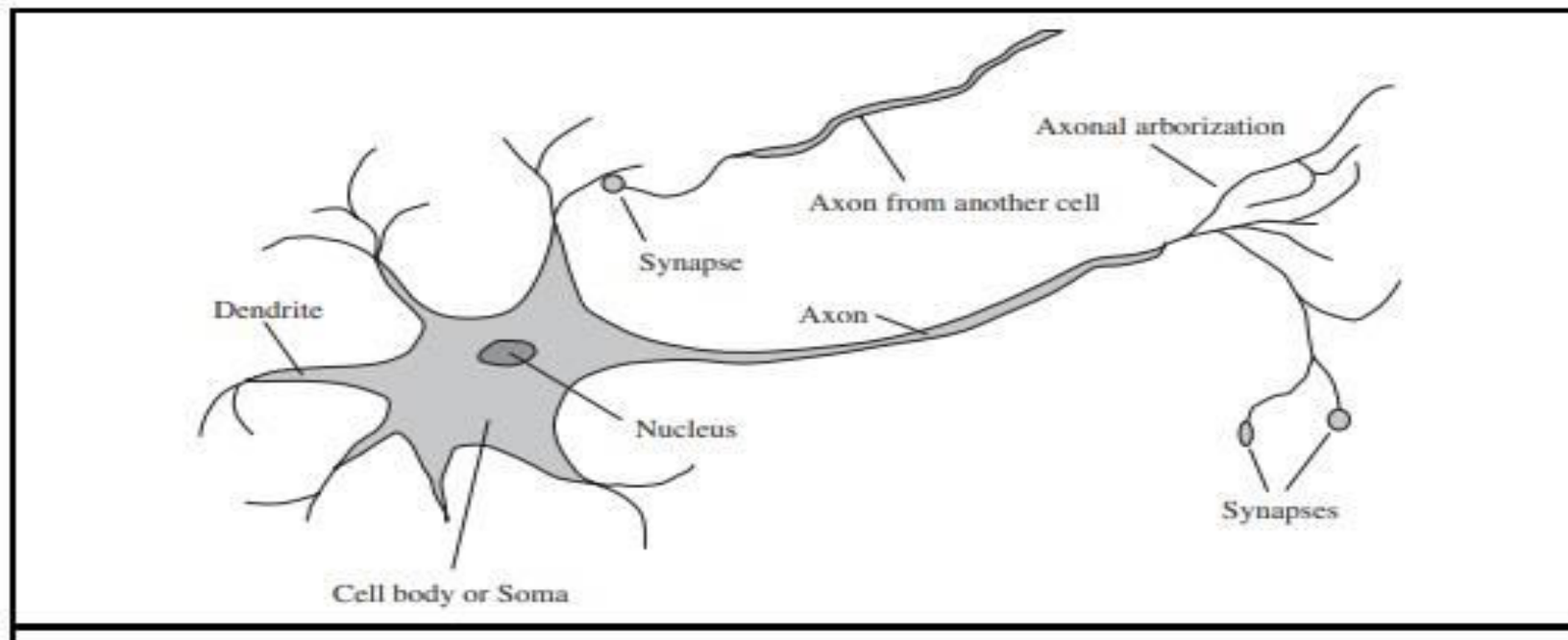
However, if one adds the principle of induction required to capture the semantics of the natural numbers, then this is no longer the case. Specifically, the **incompleteness theorem** showed that in any language expressive enough to describe the properties of the natural numbers, there are true statements that are undecidable: their truth cannot be established by any algorithm.

3 ECONOMICS

- ✓ How should we make decisions so as to maximize payoff?
- ✓ How should we do this when others may not go along?
- ✓ How should we do this when the payoff may be far in the future?

4 NEUROSCIENCE

- ✓ How do brains process information?
- Neuroscience is the study of the nervous system, particularly the brain. Although the exact way in which the brain enables thought is one of the great mysteries of science, the fact that it does enable thought has been appreciated for thousands of years because of the evidence that strong blows to the head can lead to mental incapacitation.
- It has also long been known that human brains are somehow different; in about 335 B.C. Aristotle wrote, “Of all the animals, man has the largest brain in proportion to his size.”⁵ Still, it was not until the middle of the 18th century that the brain was widely recognized as the seat of consciousness. Before then, candidate locations included the heart and the spleen.



The parts of a nerve cell or neuron. Each neuron consists of a cell body, or soma, that contains a cell nucleus. Branching out from the cell body are a number of fibers called dendrites and a single long fiber called the axon. The axon stretches out for a long distance, much longer than the scale in this diagram indicates.

	Supercomputer	Personal Computer	Human Brain
Computational units	10^4 CPUs, 10^{12} transistors	4 CPUs, 10^9 transistors	10^{11} neurons
Storage units	10^{14} bits RAM 10^{15} bits disk	10^{11} bits RAM 10^{13} bits disk	10^{11} neurons 10^{14} synapses
Cycle time	10^{-9} sec	10^{-9} sec	10^{-3} sec
Operations/sec	10^{15}	10^{10}	10^{17}
Memory updates/sec	10^{14}	10^{10}	10^{14}

5 PSYCHOLOGY

- ✓ How do humans and animals think and act?

The principle characteristic of **cognitive psychology** is that the brain processes and processes information. The claim is that beliefs, goals, and reasoning steps can be useful components of a theory of human behaviour. The knowledge-based agent has three key steps:

1. Stimulus is translated into an internal representation
2. The representation is manipulated by cognitive processes to derive new internal representations

These are translated into actions

6 COMPUTER ENGINEERING

- ✓ How can we build an efficient computer?

For artificial intelligence to succeed, we need two things: intelligence and an artifact. The computer has been the artifact of choice. The modern digital electronic computer was invented independently and almost simultaneously by scientists in three countries embattled in World War II.

7 CONTROL THEORY AND CYBERNETICS

- ✓ How can artifacts operate under their own control?

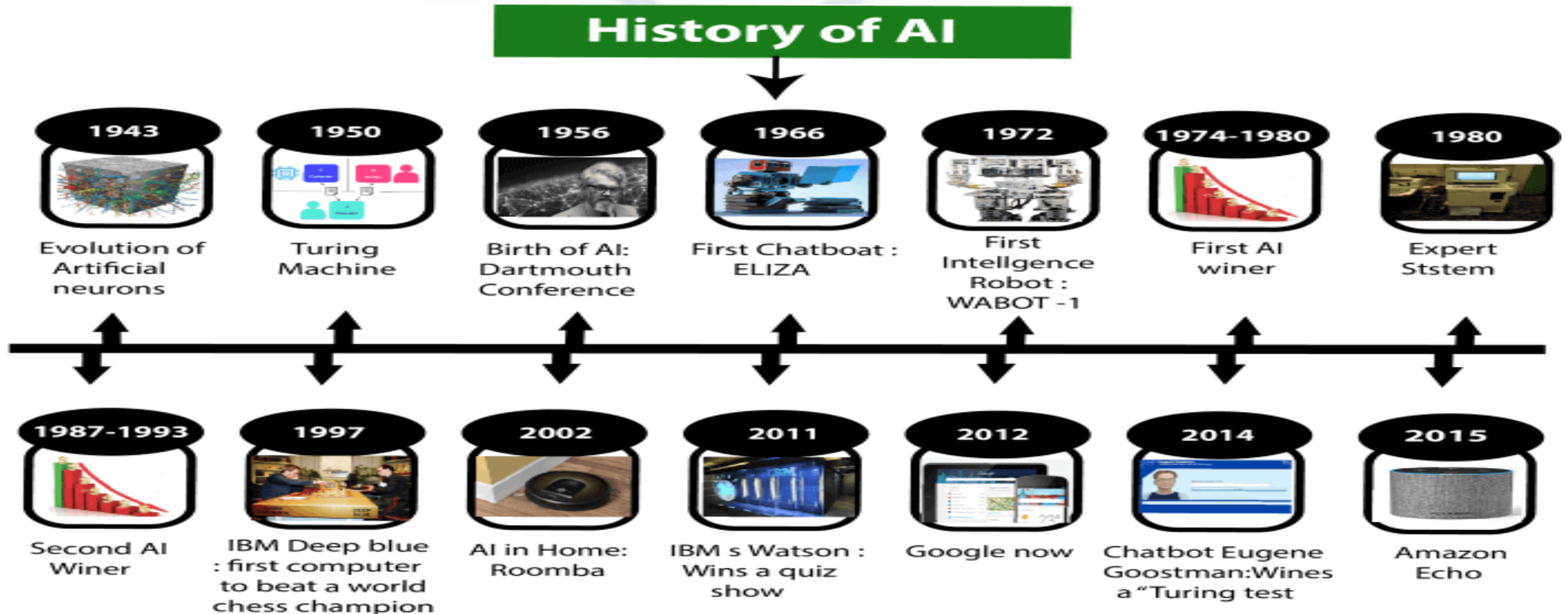
Modern control theory, especially the branch known as stochastic optimal control, has as its goal the design of systems that maximize an objective function over time. This roughly OBJECTIVE FUNCTION matches our view of AI: designing systems that behave optimally. Why, then, are AI and control theory two different fields, despite the close connections among their founders? The answer lies in the close coupling between the mathematical techniques that were familiar to the participants and the corresponding sets of problems that were encompassed in each world view.

8. LINGUISTICS

- ✓ Having a theory of how humans successfully process natural language is an AI-complete problem - if we could solve this problem then we would have created a model of intelligence.
- ✓ Much of the early work in knowledge representation was done in support of programs that attempted natural language understanding.

HISTORY OF ARTIFICIAL INTELLIGENCE

Artificial Intelligence is not a new word and not a new technology for researchers. This technology is much older than you would imagine. Even there are the myths of Mechanical men in Ancient Greek and Egyptian Myths. Following are some milestones in the history of AI which defines the journey from the AI generation to till date development.



Maturation of Artificial Intelligence (1943-1952)

- **Year 1943:** The first work which is now recognized as AI was done by Warren McCulloch and Walter Pitts in 1943. They proposed a model of artificial neurons.
- **Year 1949:** Donald Hebb demonstrated an updating rule for modifying the connection strength between neurons. His rule is now called Hebbian learning.
- **Year 1950:** The Alan Turing who was an English mathematician and pioneered Machine learning in 1950. Alan Turing publishes "Computing Machinery and Intelligence" in which he proposed a test. The test can check the machine's ability to exhibit intelligent behavior equivalent to human intelligence, called a Turing test.

The birth of Artificial Intelligence (1952-1956)

- **Year 1955:** Allen Newell and Herbert A. Simon created the "first artificial intelligence program" which was named as "Logic Theorist". This program had proved 38 of 52 Mathematics theorems, and find new and more elegant proofs for some theorems.
- **Year 1956:** The word "Artificial Intelligence" first adopted by American Computer scientist John McCarthy at the Dartmouth Conference. For the first time, AI coined as an academic field.
- At that time high-level computer languages such as FORTRAN, LISP, or COBOL were invented. And the enthusiasm for AI was very high at that time.

The golden years-Early enthusiasm (1956-1974)

- **Year 1966:** The researchers emphasized developing algorithms which can solve mathematical problems. Joseph Weizenbaum created the first chatbot in 1966, which was named as ELIZA.
- **Year 1972:** The first intelligent humanoid robot was built in Japan which was named as WABOT-1.

The first AI winter (1974-1980)

- The duration between years 1974 to 1980 was the first AI winter duration. AI winter refers to the time period where computer scientist dealt with a severe shortage of funding from government for AI researches.
- During AI winters, an interest of publicity on artificial intelligence was decreased.

A boom of AI (1980-1987)

- **Year 1980:** After AI winter duration, AI came back with "Expert System". Expert systems were programmed that emulate the decision-making ability of a human expert.
- **In the Year 1980,** the first national conference of the American Association of Artificial Intelligence was held at Stanford University.

The second AI winter (1987-1993)

- The duration between the years 1987 to 1993 was the second AI Winter duration.
- Again Investors and government stopped in funding for AI research as due to high cost but not efficient result. The expert system such as XCON was very cost effective.

The emergence of intelligent agents (1993-2011)

- Year 1997: In the year 1997, IBM Deep Blue beats world chess champion, Gary Kasparov, and became the first computer to beat a world chess champion.
- Year 2002: for the first time, AI entered the home in the form of Roomba, a vacuum cleaner.
- Year 2006: AI came in the Business world till the year 2006. Companies like Facebook, Twitter, and Netflix also started using AI.

TEAM Y not Learning...

Deep learning, big data and artificial general intelligence (2011-present)

- **Year 2011:** In the year 2011, IBM's Watson won jeopardy, a quiz show, where it had to solve the complex questions as well as riddles. Watson had proved that it could understand natural language and can solve tricky questions quickly.
- **Year 2012:** Google has launched an Android app feature "Google now", which was able to provide information to the user as a prediction.
- **Year 2014:** In the year 2014, Chatbot "Eugene Goost man" won a competition in the infamous "Turing test."
- **Year 2018:** The "Project Debater" from IBM debated on complex topics with two master debaters and also performed extremely well.
- Google has demonstrated an AI program "Duplex" which was a virtual assistant and which had taken hairdresser appointment on call, and lady on other side didn't notice that she was talking with the machine.

Need for Artificial Intelligence



1. COMPETITIVE EDGE

The organizations which mean to have a serious edge over their adversaries are banking upon AI advancements to acquire this.

Take the case of the **Autopilot highlight** offered by **Tesla** in its vehicles. Tesla is utilizing Deep Learning Algorithms to accomplish Autonomous driving. This was before, when there was only one element out of many, yet now it is characterizing the brand.

2. ACCESSIBILITY

The establishment **speed, availability, and sheer scale** have enabled bolder computations to deal with progressively exciting issues. Not solely is the gear faster, expanded by specific assortments of processors (e.g., GPUs), it is moreover available looking like cloud organizations.

What used to run in explicit labs with access to super PCs would now pass on to the cloud at a lower cost. This has democratized access to the significant hardware stages to run AI, enabling duplication of new organizations.

3. FEAR OF MISSING OUT (FOMO)

No typo, you read that right! Not simply us, organizations additionally feel the **dread of passing up** a major opportunity. To stay competitive and not get tossed out of the market, they need to adjust appropriately. This is done by putting resources into advances that would upset their enterprises. Take the case of the financial part, where practically all the banks have put vigorously in chatbots with the goal that they won't pass up the following rush of interruption.

4. COST-EFFECTIVENESS

As with all other technologies, with time, AI is becoming more and more affordable. This has made it **feasible** for a lot of organizations that couldn't bear the cost of them in the past to use these advances.

Organizations do not have that barrier of cost to implement AI.

5. FUTURE PROOF

One thing that we all need to comprehend is that future in AI is very safe .

Organizations can and ought to **guarantee** themselves to be future confirmation by actualizing AI advancements. On the off chance that this is where the world is going, why not to head in that equivalent course and be versatile to that change.

TEAM Y not Learnings...

WHY ARTIFICIAL INTELLIGENCE?

Before Learning about Artificial Intelligence, we should know that what is the importance of AI and why should we learn it.

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk.

AI opens a path for other new technologies, new devices, and new Opportunities.

TEAM Y not Learning

GOALS OF ARTIFICIAL INTELLIGENCE

- Replicate human intelligence
- Solve Knowledge-intensive tasks
- An intelligent connection of perception and action
- Building a machine which can perform tasks that requires human intelligence such as:
 - ✓ Proving a theorem
 - ✓ Playing chess
 - ✓ Plan some surgical operation
 - ✓ Driving a car in traffic
- Creating some system which can exhibit intelligent behaviour, learn new things by itself, demonstrate, explain, and can advise to its user.

ADVANTAGES OF ARTIFICIAL INTELLIGENCE

- **High Accuracy with less errors:** AI machines or systems are prone to less errors and high accuracy as it takes decisions as per pre-experience or information.
- **High-Speed:** AI systems can be of very high-speed and fast-decision making, because of that AI systems can beat a chess champion in the Chess game.
- **High reliability:** AI machines are highly reliable and can perform the same action multiple times with high accuracy.
- **Useful for risky areas:** AI machines can be helpful in situations such as defusing a bomb, exploring the ocean floor, where to employ a human can be risky.
- **Digital Assistant:** AI can be very useful to provide digital assistant to the users such as AI technology is currently used by various E-commerce websites to show the products as per customer requirement.
- **Useful as a public utility:** AI can be very useful for public utilities such as a self-driving car which can make our journey safer and hassle-free, facial recognition for security purpose, Natural language processing to communicate with the human in human-language, etc.

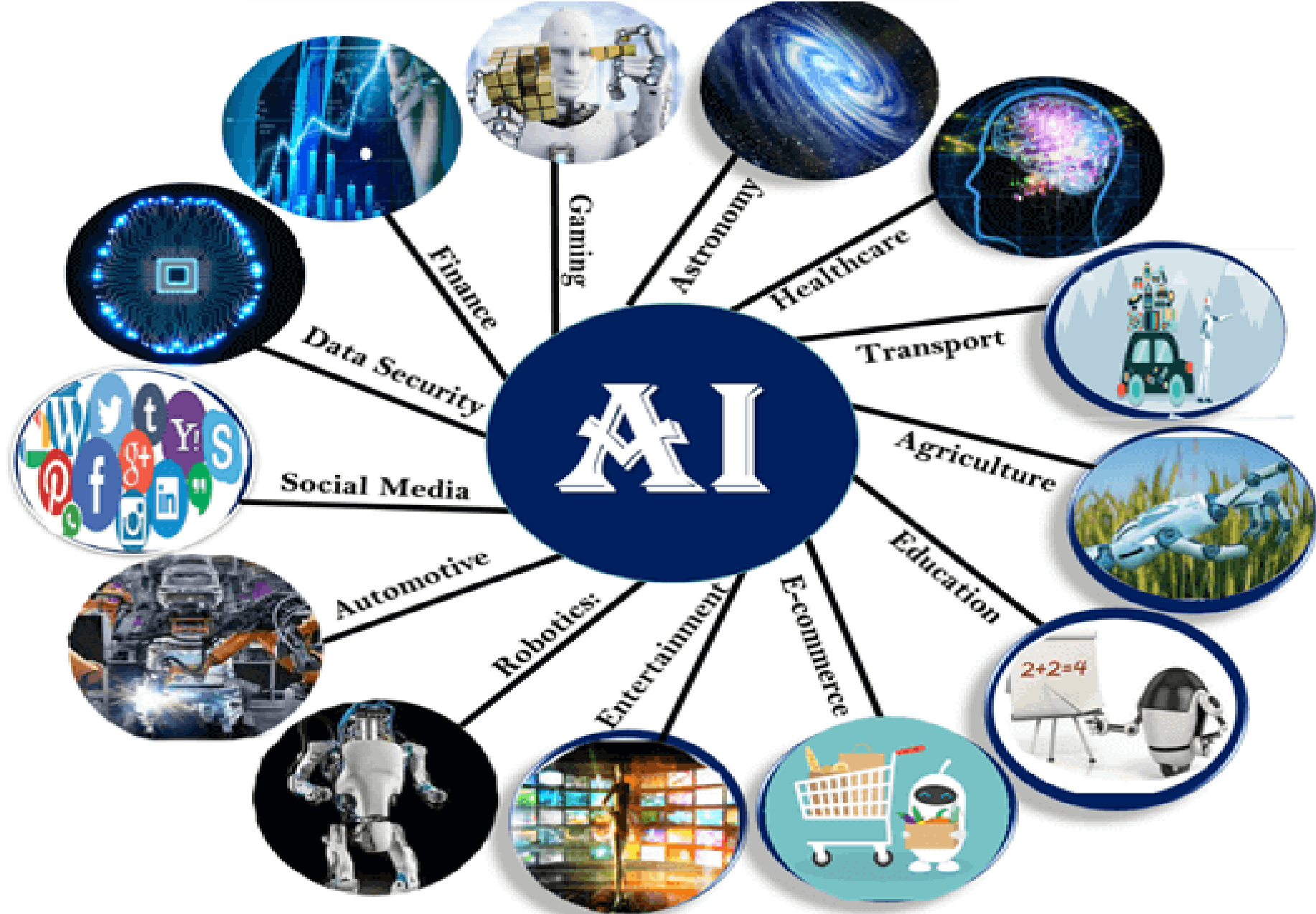
DISADVANTAGES OF ARTIFICIAL INTELLIGENCE

Every technology has some disadvantages, and the same goes for Artificial intelligence. Being so advantageous technology still, it has some disadvantages which we need to keep in our mind while creating an AI system.

Following are the disadvantages of AI:

- **High Cost:** The hardware and software requirement of AI is very costly as it requires lots of maintenance to meet current world requirements.
- **Can't think out of the box:** Even we are making smarter machines with AI, but still they cannot work out of the box, as the robot will only do that work for which they are trained, or programmed.
- **No feelings and emotions:** AI machines can be an outstanding performer, but still it does not have the feeling so it cannot make any kind of emotional attachment with human, and may sometime be harmful for users if the proper care is not taken.
- **Increase dependency on machines:** With the increment of technology, people are getting more dependent on devices and hence they are losing their mental capabilities.
- **No Original Creativity:** As humans are so creative and can imagine some new ideas but still AI machines cannot beat this power of human intelligence and cannot be creative and imaginative.

APPLICATION OF AI



1. AI in Astronomy

Artificial Intelligence can be very useful to solve complex universe problems. AI technology can be helpful for understanding the universe such as how it works, origin, etc.

2. AI in Healthcare

In the last, five to ten years, AI becoming more advantageous for the healthcare industry and going to have a significant impact on this industry.

Healthcare Industries are applying AI to make a better and faster diagnosis than humans. AI can help doctors with diagnoses and can inform when patients are worsening so that medical help can reach to the patient before hospitalization.

3. AI in Gaming

AI can be used for gaming purpose. The AI machines can play strategic games like chess, where the machine needs to think of a large number of possible places.

4. AI in Finance

AI and finance industries are the best matches for each other. The finance industry is implementing automation, chatbot, adaptive intelligence, algorithm trading, and machine learning into financial processes.

5. AI in Data Security

The security of data is crucial for every company and cyber-attacks are growing very rapidly in the digital world. AI can be used to make your data more safe and secure. Some examples such as AEG bot, AI2 Platform, are used to determine software bug and cyber-attacks in a better way.

6. AI in Social Media

Social Media sites such as Facebook, Twitter, and Snapchat contain billions of user profiles, which need to be stored and managed in a very efficient way. AI can organize and manage massive amounts of data. AI can analyze lots of data to identify the latest trends, hashtag, and requirement of different users.

7. AI in Travel & Transport

AI is becoming highly demanding for travel industries. AI is capable of doing various travel related works such as from making travel arrangement to suggesting the hotels, flights, and best routes to the customers. Travel industries are using AI-powered chatbots which can make human-like interaction with customers for better and fast response.

8. AI in Automotive Industry

Some Automotive industries are using AI to provide virtual assistant to their user for better performance. Such as Tesla has introduced TeslaBot, an intelligent virtual assistant.

Various Industries are currently working for developing self-driven cars which can make your journey more safe and secure.

9. AI in Robotics:

Artificial Intelligence has a remarkable role in Robotics. Usually, general robots are programmed such that they can perform some repetitive task, but with the help of AI, we can create intelligent robots which can perform tasks with their own experiences without pre-programmed.

Humanoid Robots are best examples for AI in robotics, recently the intelligent Humanoid robot named as Erica and Sophia has been developed which can talk and behave like humans.

10. AI in Entertainment

We are currently using some AI based applications in our daily life with some entertainment services such as Netflix or Amazon. With the help of ML/AI algorithms, these services show the recommendations for programs or shows.

TEAM Y not Learning

11. AI in Agriculture

Agriculture is an area which requires various resources, labor, money, and time for best result. Now a day's agriculture is becoming digital, and AI is emerging in this field. Agriculture is applying AI as agriculture robotics, solid and crop monitoring, predictive analysis. AI in agriculture can be very helpful for farmers

12. AI in E-commerce

AI is providing a competitive edge to the e-commerce industry, and it is becoming more demanding in the e-commerce business. AI is helping shoppers to discover associated products with recommended size, color, or even brand.

13. AI in education:

AI can automate grading so that the tutor can have more time to teach. AI chatbot can communicate with students as a teaching assistant.



Thank You

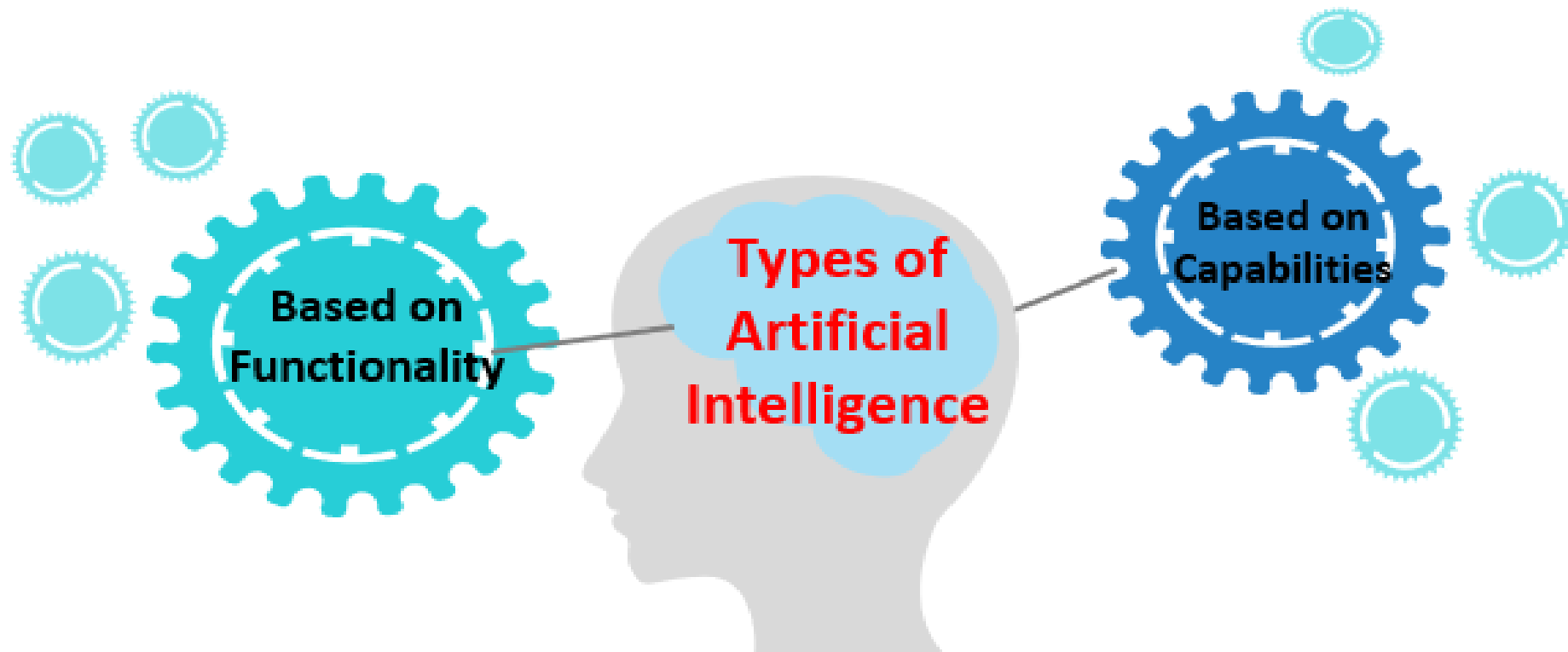
Types of Artificial Intelligence



TYPES OF ARTIFICIAL INTELLIGENCE

The main aim of Artificial Intelligence is to enable machines to perform a human-like function.

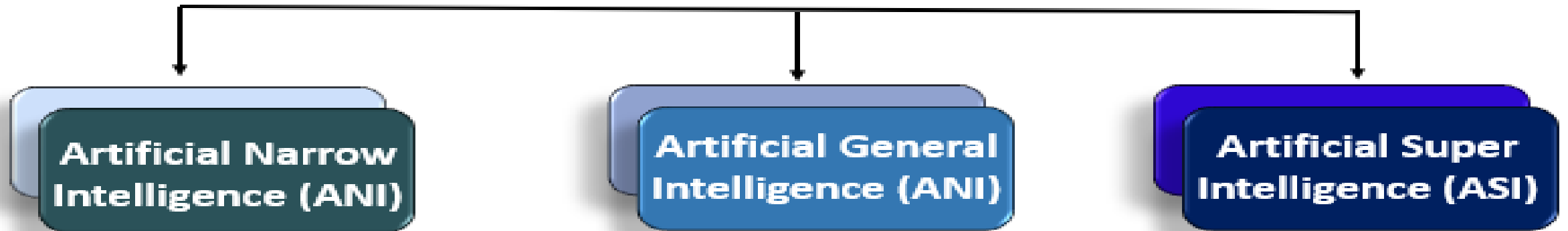
Artificial Intelligence can be divided into various types, there are mainly two types of main categorization which are **based on capabilities** and **based on functionality** of AI.



Type 1 - Based on Functionality



Type 2 - Based on Capabilities



BASED ON FUNCTIONALITY

1. REACTIVE MACHINES

- Purely reactive machines are the most **basic types** of Artificial Intelligence.
- Such AI systems **do not store memories** or **past experiences** for **future actions**.
- These machines **only focus** on **current scenarios** and **react on it** as per possible **best action**.
- **IBM's Deep Blue system** is an example of reactive machines.
- **Google's AlphaGo** is also an example of reactive machines.

2. LIMITED MEMORY

- Limited memory machines can store **past experiences** or **some data** for **a short period** of time.
- These machines can use stored data for a **limited time period only**.
- **Self-driving cars are one of the best examples** of Limited Memory systems. These cars can store recent speed of nearby cars, the distance of other cars, speed limit, and other information to navigate the road.

3. THEORY OF MIND

- Theory of Mind AI should understand the **human emotions**, people, beliefs, and be able to **interact socially like humans**.
- This type of AI machines are still not developed, but researchers are making lots of efforts and improvement for developing such AI machines.

4. SELF-AWARENESS

- Self-awareness AI is the **future** of **Artificial Intelligence**. These machines will be **super intelligent**, and will have their own **consciousness, sentiments**, and **self-awareness**.
- These machines will be **smarter than human mind**.
- Self-Awareness **AI does not exist in reality still** and it is a **hypothetical concept**.

BASED ON CAPABILITIES

1. WEAK AI OR NARROW AI:

- Narrow AI is a type of AI which is able to perform a **dedicated task** with intelligence. **The most common and currently available AI is Narrow AI in the world of Artificial Intelligence.**
- Narrow AI cannot **perform beyond its field or limitations**, as it is only **trained for one specific task**. Hence it is also termed as weak AI. Narrow AI can fail in unpredictable ways if it goes beyond its limits.
- **Apple Siri** is a good example of Narrow AI, but it operates with a limited pre-defined range of functions.
- **IBM's Watson supercomputer** also comes under Narrow AI, as it uses an Expert system approach combined with Machine learning and natural language processing.
- **Some Examples of Narrow AI are playing chess, purchasing suggestions on e-commerce site, self-driving cars, speech recognition, and image recognition.**

2. GENERAL AI:

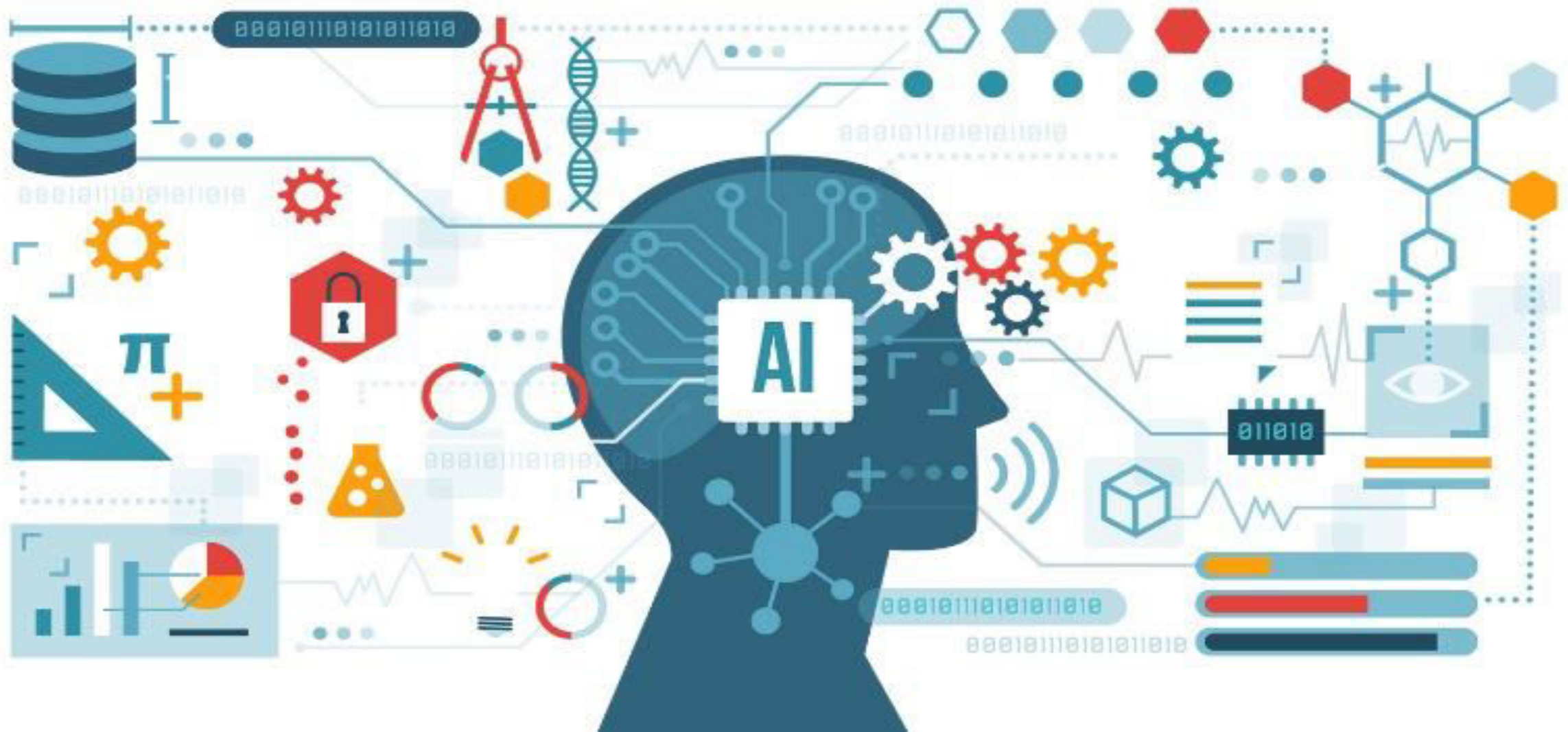
- General AI is a type of intelligence which could perform any intellectual task with efficiency like a human.
- The idea behind the general AI to make such a system which could be **smarter** and **think like a human by its own**.
- Currently, there is **no such system exist which could come under general AI** and can perform any task **as perfect as a human**.
- The worldwide researchers are now focused on developing machines with General AI.
- As systems with general AI are still under research, and it will take lots of efforts and time to develop such systems.

3. SUPER AI:

- Super AI is a level of Intelligence of Systems at which machines could surpass human intelligence, and can perform any task better than human with cognitive properties. It is an outcome of general AI.
- Some key characteristics of strong AI include capability include the ability to think, to reason, solve the puzzle, make judgments, plan, learn, and communicate by its own.
- Super AI is still a hypothetical concept of Artificial Intelligence. Development of such systems in real is still world changing task.



BENEFITS OF ARTIFICIAL INTELLIGENCE



BENEFITS OF ARTIFICIAL INTELLIGENCE

- 1. INCREASED EFFICIENCY**
- 2. REDUCING HUMAN RISK**
- 3. AID IN DECISION MAKING**
- 4. AVAILABILITY**

1. INCREASED EFFICIENCY

Machines are highly efficient in contrast to the human workforce. This is because they can complete monotonous tasks effectively without taking a break and irrespective of the working conditions.

Take the example of Power BI. Earlier, time was spent in **sifting, isolating, and utilizing the information** in **MS Excel**, **Power BI** utilizes AI to do the entirety of this (and significantly more) autonomously.

This expands the effectiveness of the framework and spares significant time for the investigator to utilize it somewhere else.

2. REDUCING HUMAN RISK

A lot of places wherein it is **hazardous** for a person to be truly present.

Man-made brainpower can be useful in such cases since it would empower the machines to have the option to settle on choices continuously. A case of this is a remote ocean boring at the sea bed where there is a high danger of human life.

3. AID IN DECISION MAKING

This is the domain where AI has successfully paid dividends. AI has the capacity to analyze a great deal of information in a brief time frame. This is helping people improve decision making. Take the case of an investor who needs to conclude whether to give out a credit or not.

This can be highly effective in verticals like **portfolio management**. This would help in a more astute and quicker dynamic for the investor.

4. AVAILABILITY

Machines, in contrast to people, don't have to take rests. They can work nonstop.

Now, we can depend on machines for keeping up required manufacturing units running with their judgment which would prompt **24×7** creation units and complete mechanization.



Improve efficiency



Save costs



Avoid problems



Anticipate future scenarios

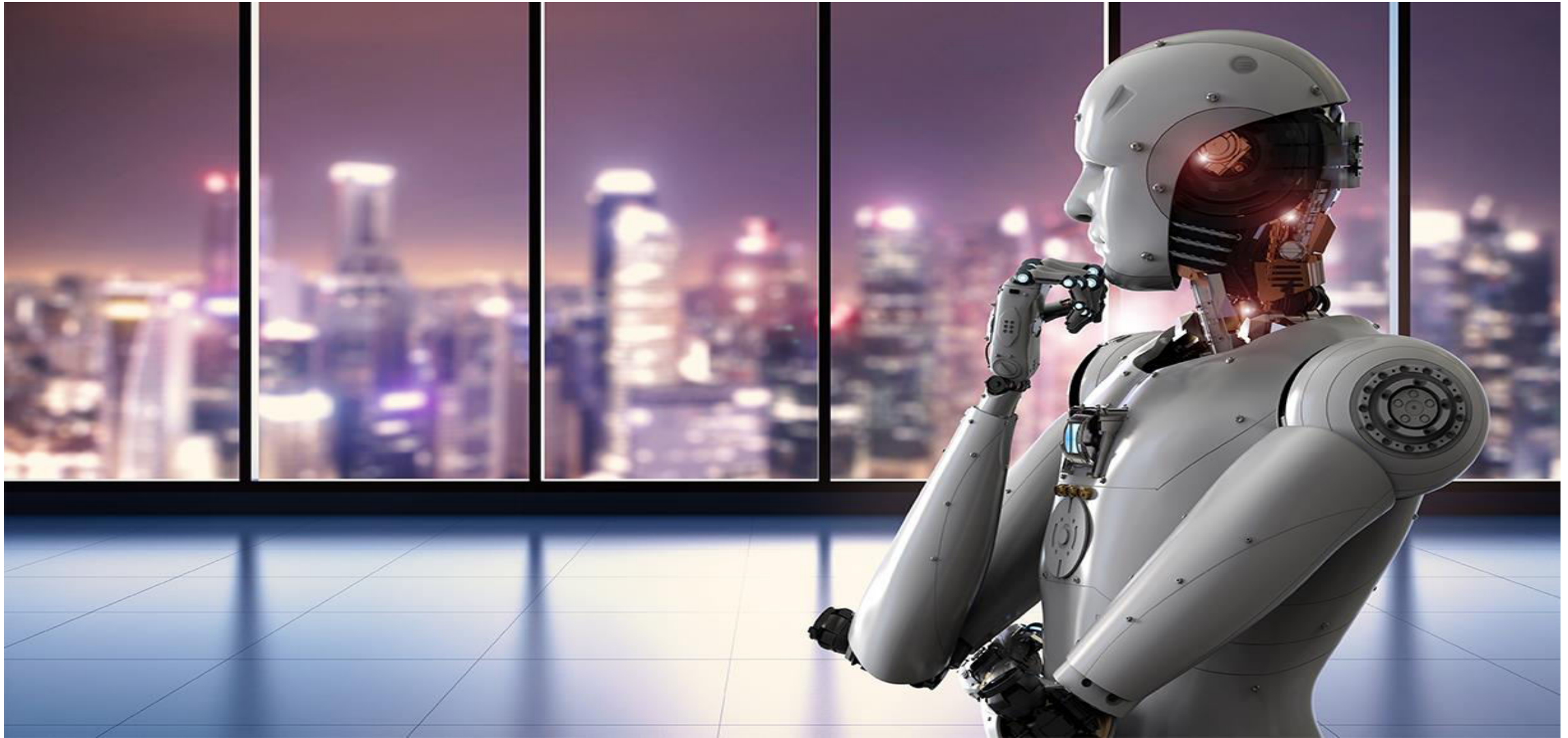


Reduce human error



Automate processes

THE STATE OF ART **OR** WHAT CAN AI DO TODAY?



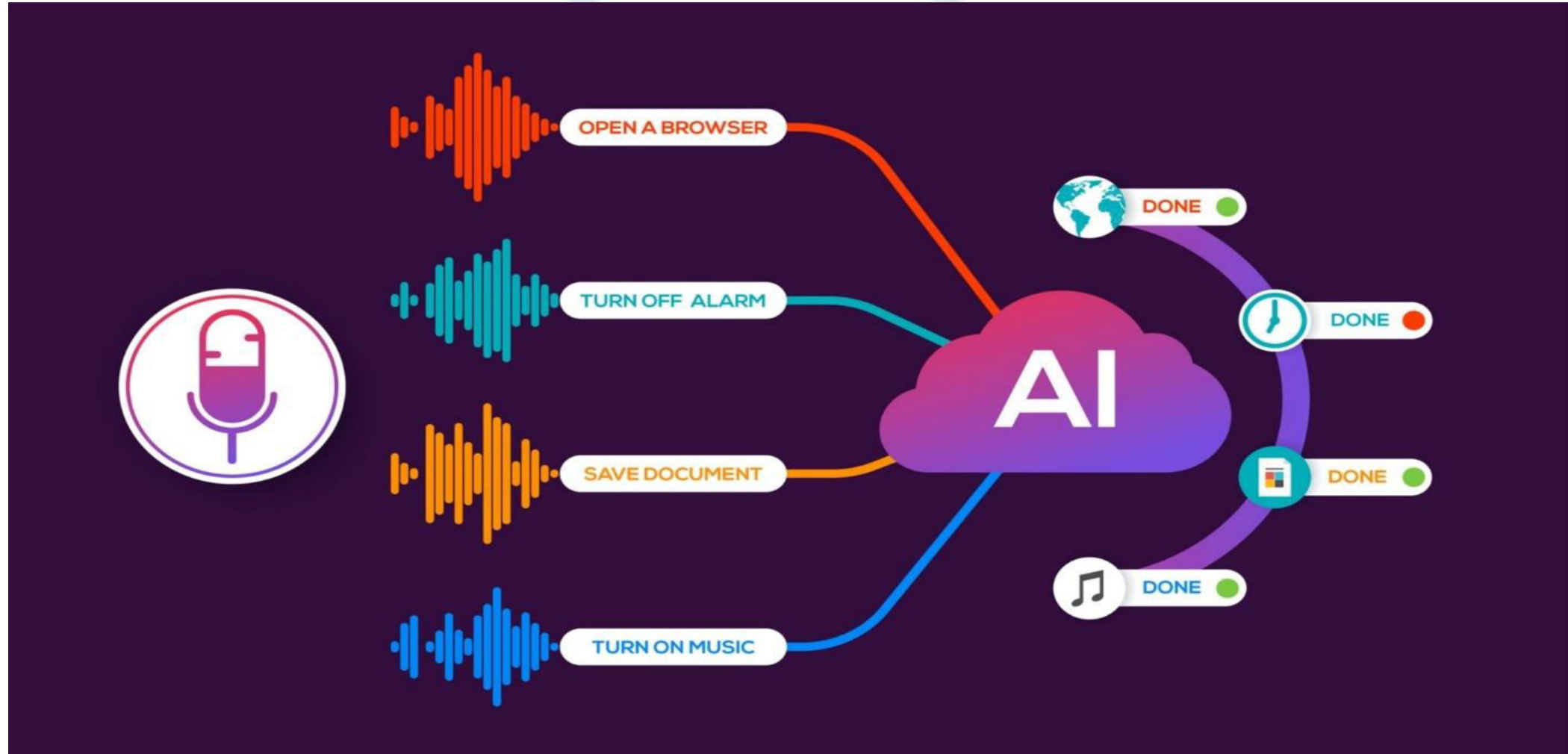
➤ **ROBOTIC VEHICLES:** A driverless robotic car named STANLEY sped through the rough terrain of the Mojave Desert at 22 mph, finishing the 132-mile course first to win the 2005 DARPA Grand Challenge.

STANLEY is a Volkswagen Touareg outfitted with cameras, radar, and laser rangefinders to sense the environment and onboard software to command the steering, braking, and acceleration (Thrun, 2006).

The following year CMU's BOSS won the Urban Challenge, safely driving in traffic through the streets of a closed Air Force base, obeying traffic rules and avoiding pedestrians and other vehicles.

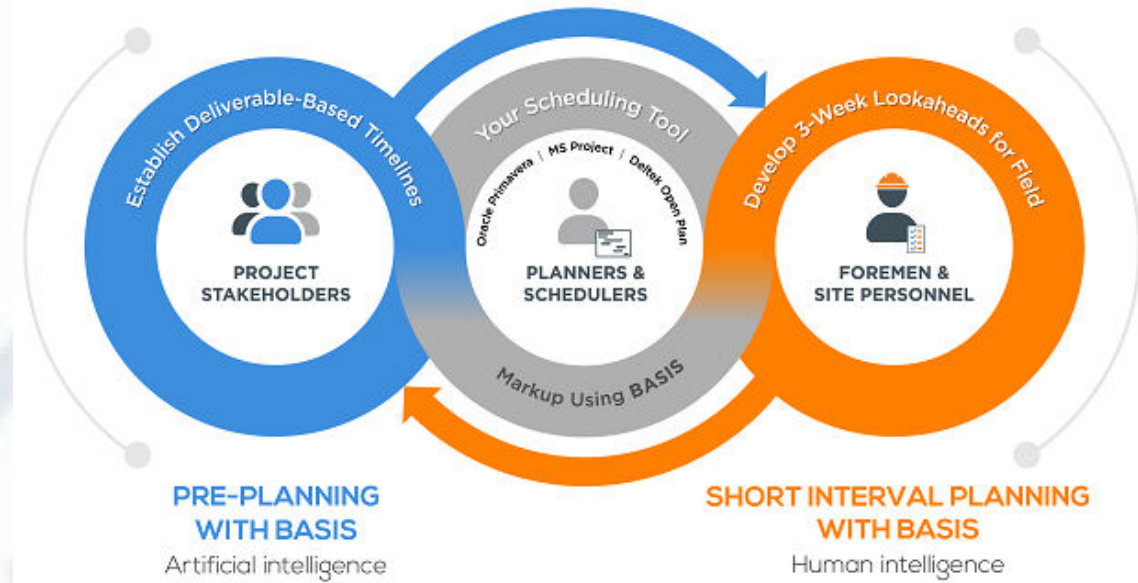


- **SPEECH RECOGNITION:** A traveller calling United Airlines to book a flight can have the entire conversation guided by an automated speech recognition and dialog management system.



➤ AUTONOMOUS PLANNING AND

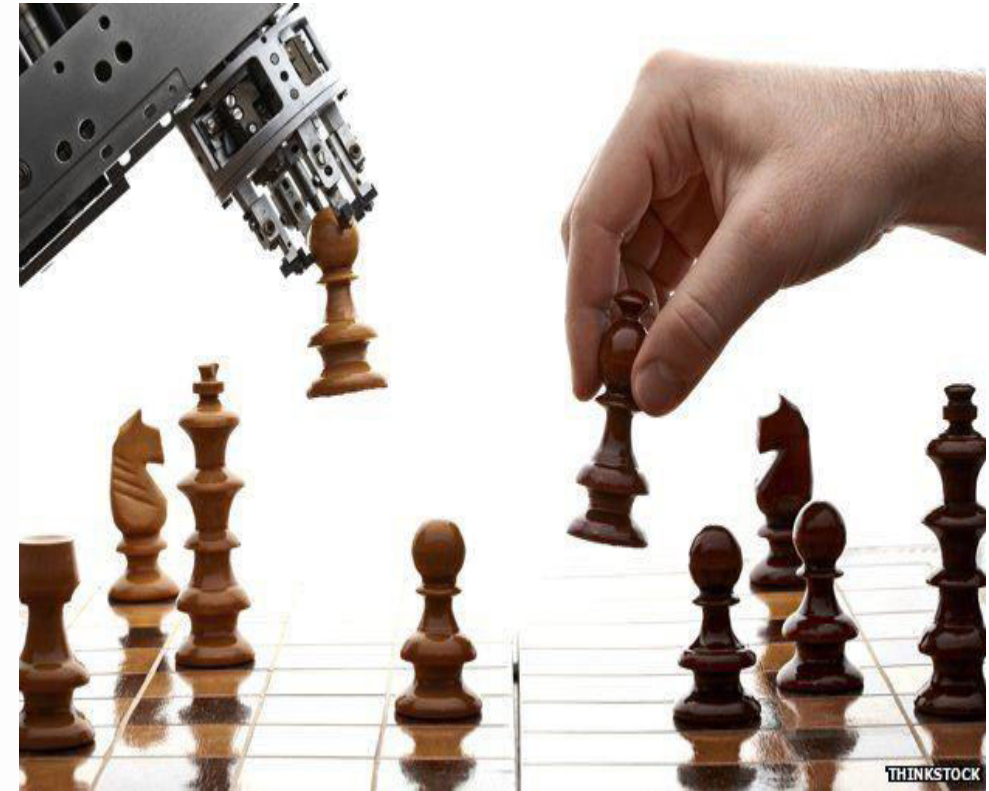
SCHEDULING: A hundred million miles from Earth, NASA's Remote Agent program became the first on-board autonomous planning program to control the scheduling of operations for a spacecraft (Jonsson *et al.*, 2000).



REMOTE AGENT generated plans from high-level goals specified from the ground and monitored the execution of those plans—detecting, diagnosing, and recovering from problems as they occurred.

Successor program MAPGEN (Al-Chang *et al.*, 2004) plans the daily operations for NASA's Mars Exploration Rovers, and MEXAR2 (Cesta *et al.*, 2007) did mission planning—both logistics and science planning—for the European Space Agency's Mars Express mission in 2008.

➤ **GAME PLAYING:** IBM's DEEP BLUE became the first computer program to defeat the world champion in a chess match when it bested Garry Kasparov by a score of 3.5 to 2.5 in an exhibition match (Goodman and Keene, 1997). Kasparov said that he felt a “new kind of intelligence” across the board from him. Newsweek magazine described the match as “The brain’s last stand.” The value of IBM’s stock increased by \$18 billion. Human champions studied Kasparov’s loss and were able to draw a few matches in subsequent years, but the most recent human-computer matches have been won convincingly by the computer.



- **SPAM FIGHTING:** Each day, learning algorithms classify over a billion messages as spam, saving the recipient from having to waste time deleting what, for many users, could comprise 80% or 90% of all messages, if not classified away by algorithms. Because the spammers are continually updating their tactics, it is difficult for a static programmed approach to keep up, and learning algorithms work best (Sahami *et al.*, 1998; Goodman and Heckerman, 2004).
- **LOGISTICS PLANNING:** During the Persian Gulf crisis of 1991, U.S. forces deployed a Dynamic Analysis and Replanning Tool, DART (Cross and Walker, 1994), to do automated logistics planning and scheduling for transportation. This involved up to 50,000 vehicles, cargo, and people at a time, and had to account for starting points, destinations, routes, and conflict resolution among all parameters. The AI planning techniques generated in hours a plan that would have taken weeks with older methods. The Defence Advanced Research Project Agency (DARPA) stated that this single application more than paid back DARPA's 30-year investment in AI.

- **ROBOTICS:** The iRobot Corporation has sold over two million Roomba robotic vacuum cleaners for home use. The company also deploys the more rugged PackBot to Iraq and Afghanistan, where it is used to handle hazardous materials, clear explosives, and identify the location of snipers.
- **MACHINE TRANSLATION:** A computer program automatically translates from Arabic to English, allowing an English speaker to see the headline “Ardogan Confirms That Turkey Would Not Accept Any Pressure, Urging Them to Recognize Cyprus.” The program uses a statistical model built from examples of Arabic-to-English translations and from examples of English text totalling two trillion words (Brants *et al.*, 2007). None of the computer scientists on the team speak Arabic, but they do understand statistics and machine learning algorithms.

THANK YOU



AGENTS AND ENVIRONMENTS

TEAM Y not Learnings...

AGENTS AND ENVIRONMENTS

- **Agents** in Artificial Intelligence are the **associated concepts** that the **AI technologies** work upon.
- The **AI software or AI-enabled devices** with **sensors** generally captures the information **from** the **environment setup** and **process** the **data for further actions**.
- There are mainly two ways the agents interact with the environment, such as **perception** and **action**.
- The **person** is only **passive** for capturing the **information** without **changing** the **actual environment**, whereas **action** is the **active** form of **interaction** by **changing** the **actual environment**.
- AI technologies such as **virtual assistance catboats**, AI-enabled devices to work based on the **previous persecution data processing and learning for the actions**.

WHAT IS AN AGENT?

An Agent is anything that takes actions according to the information that it gains from the environment.

- **HUMAN-AGENT:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
- **ROBOTIC AGENT:** A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.
- **SOFTWARE AGENT:** Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

EXAMPLE FOR AGENT AND ENVIRONMENTS?

Agents

Environments

Robot → Room

Chatbot → Chatting

Vehicle → Road

Program → Data & Rules

Machine → Working Field

HOW DOES THE AGENT INTERACT WITH THE ENVIRONMENT?

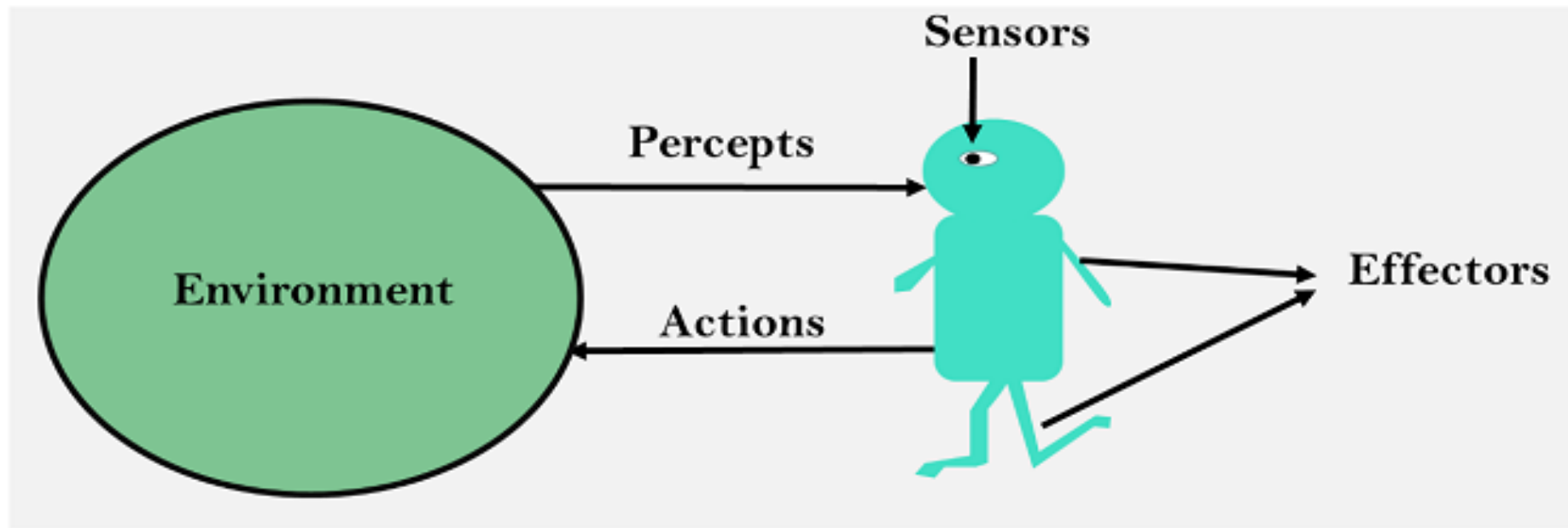
The agents interact with the environment in two ways:

1. PERCEPTION

Perception is a **passive interaction**, where the **agent gains information** about the **environment without changing the environment**. The **sensors** of the **robot help** it to **gain information** about the **surroundings** without **affecting** the **surrounding**. Hence, gaining information through sensors is called perception.

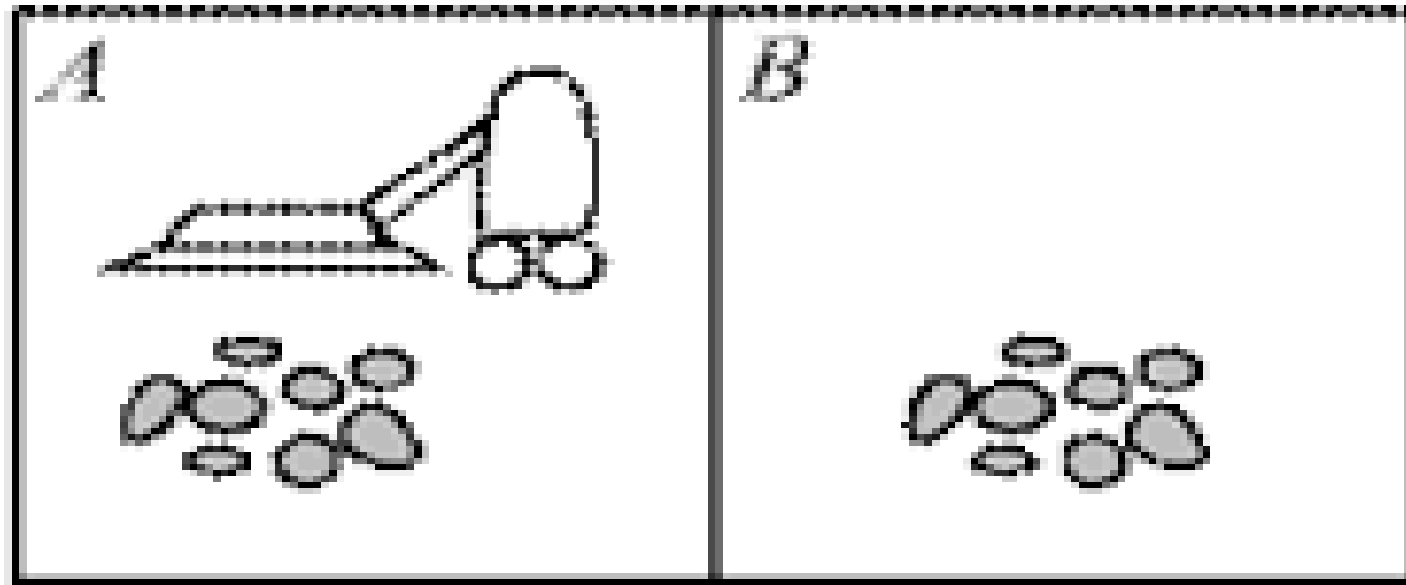
2. ACTION

Action is an **active interaction** where the **environment is changed**. When the **robot moves** an **obstacle** using its **arm**, it is called an **action** as the **environment is changed**. The arm of the robot is called an **“Effector”** as it performs the action.



- **SENSOR:** Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.
- **ACTUATORS:** Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.
- **EFFECTORS:** Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.

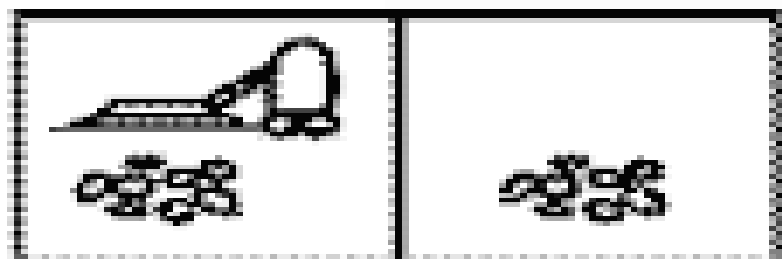
CONSIDER A VACUUM CLEANER WORLD



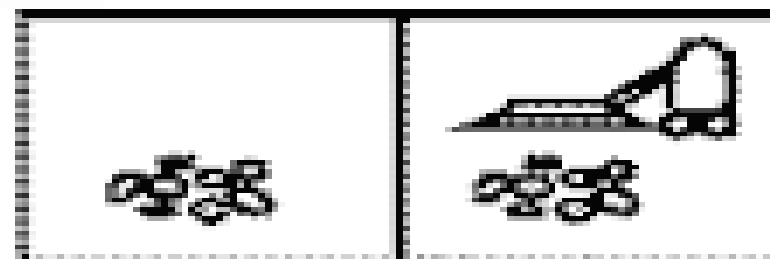
Imagine that our **intelligent agent** is a robot vacuum cleaner.

Let's suppose that the world has just two rooms. The robot can be in either room and there can be dirt in zero, one, or two rooms.

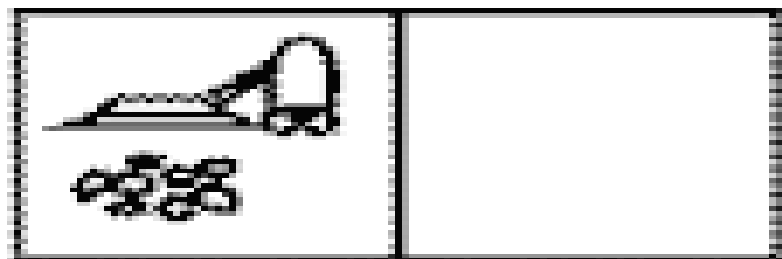
1



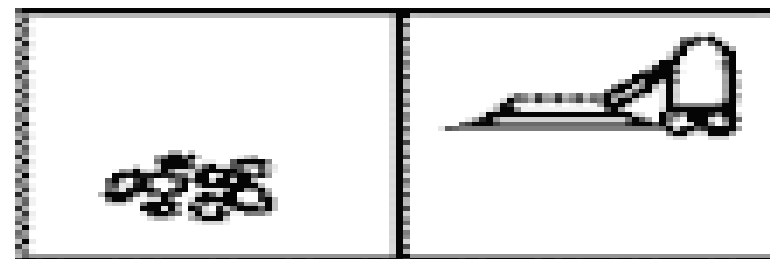
2



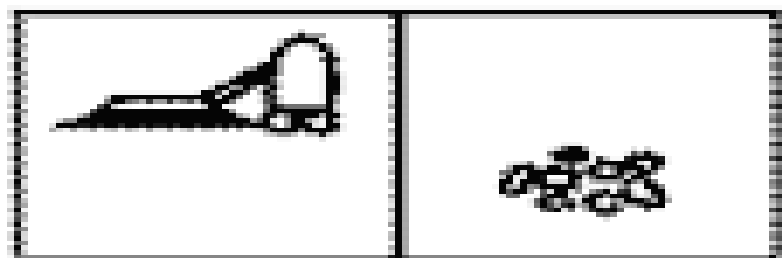
3



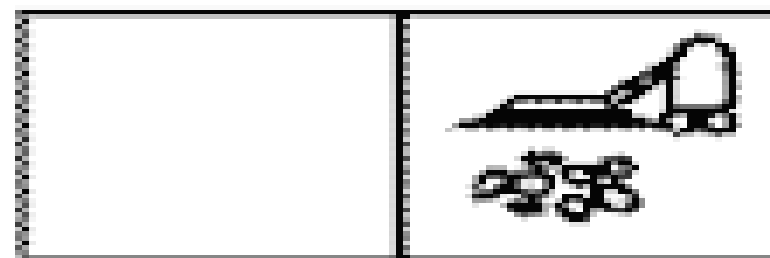
4



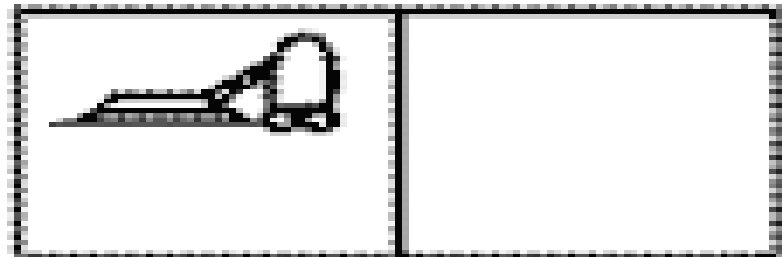
5



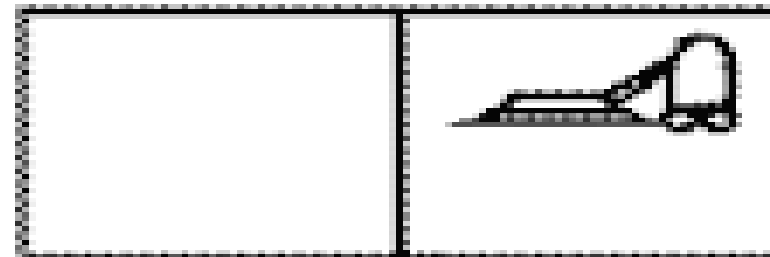
6



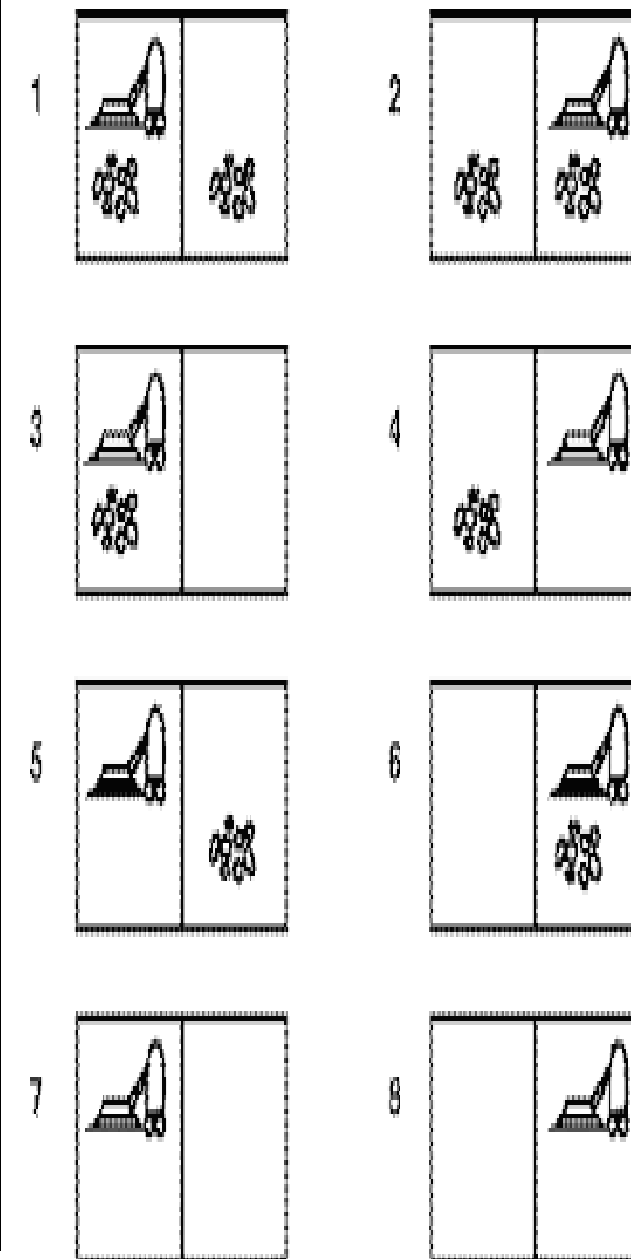
7



8



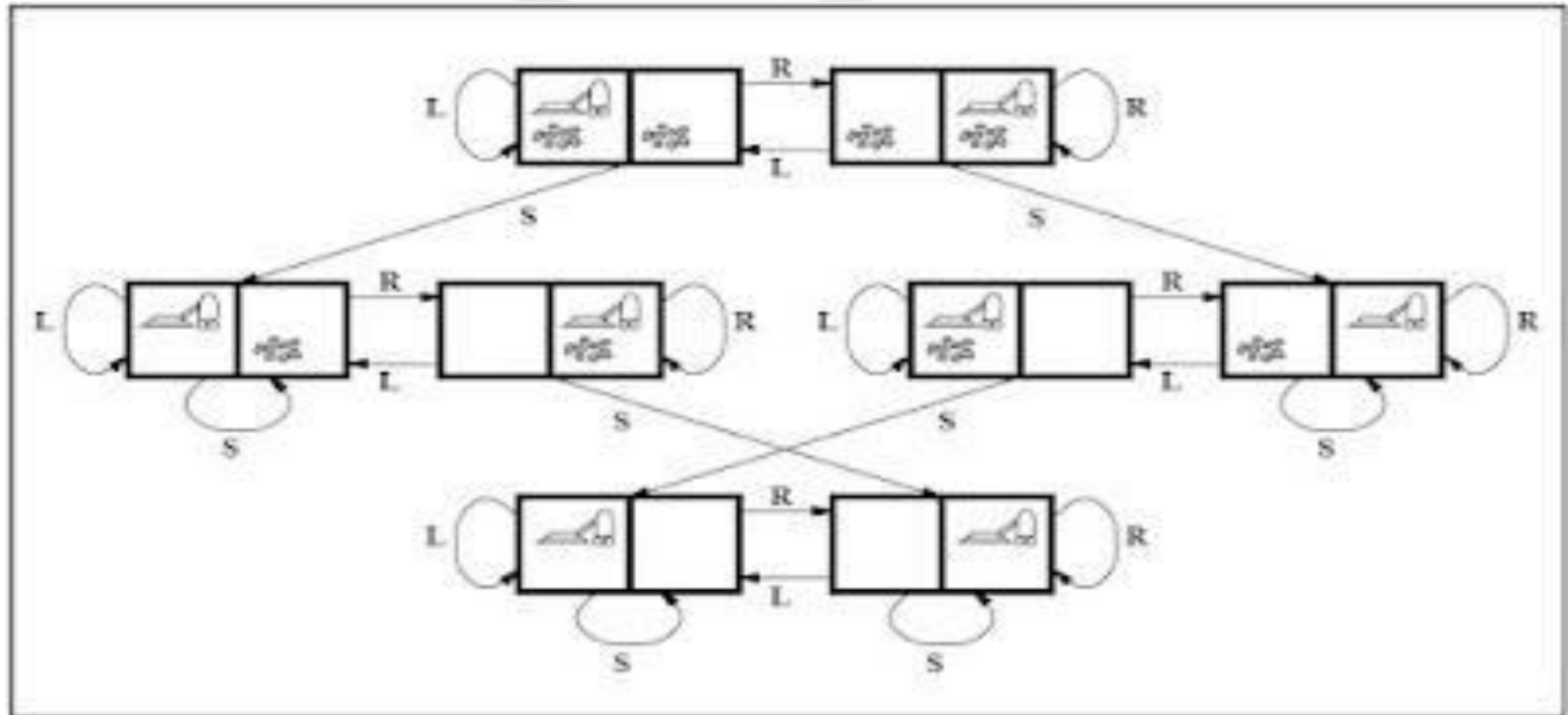
Percept sequence	Action
$[A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Dirty}]$	<i>Suck</i>
$[B, \textit{Clean}]$	<i>Left</i>
$[B, \textit{Dirty}]$	<i>Suck</i>
$[A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Clean}]$	<i>Right</i>
$[A, \textit{Clean}], [A, \textit{Clean}], [A, \textit{Dirty}]$	<i>Suck</i>
$\vdots$	$\vdots$
Partial tabulation of a simple agent function for the vacuum-cleaner world	



Goal formulation: intuitively, we want all the dirt cleaned up. Formally, the goal is {State 7, state 8}.

Problem formulation (Actions):Left, Right, Suck, NoOp

STATE SPACE GRAPH:




```
function REFLEX-VACUUM-AGENT( [location,status]) returns  
action
```

```
if status = Dirty then return Suck  
else if location = A then return Right  
else if location = B then return Left
```

MEASURING PERFORMANCE

With any intelligent agent, we want it to find a (good) solution and not spend forever doing it.

The interesting quantities are, therefore,

THE SEARCH COST--how long the agent takes to come up with the solution to the problem,
and

THE PATH COST--how expensive the actions in the solution are.

The total cost of the solution is the sum of the above two quantities.

HOW SHOULD THE AGENTS ACT IN ARTIFICIAL INTELLIGENCE?

- A rational agent does the right thing. The right action is the one that causes the agent to be the most successful.
- An omniscient agent knows what impact the action will have and can act accordingly, but it is not possible in reality.
- The degree of success which is defined by the performance measure
- The percept sequence which is the entire sequence of perceptions by the agent until the present moment

Agent Type	Percepts	Action	Goals	Environment
Car Driver	Speedometer, GPS, Microphone, Cameras	Steering control, accelerate, brake, talk to passenger	Safe, legal, comfortable journey	Road, Traffic, Pedestrian etc.

GOOD BEHAVIOUR: THE CONCEPT OF RATIONALITY

INTELLIGENT AGENTS:

An **intelligent agent** is an autonomous entity which act upon an environment using sensors and actuators for achieving goals. An intelligent agent may learn from the environment to achieve their goals. A thermostat is an example of an intelligent agent.

Following are the main four rules for an AI agent:

- **Rule 1:** An AI agent must have the ability to perceive the environment.
- **Rule 2:** The observation must be used to make decisions.
- **Rule 3:** Decision should result in an action.
- **Rule 4:** The action taken by an AI agent must be a rational action.

RATIONAL AGENT:

- A rational agent is an agent which has **clear preference**, models uncertainty, and acts in a way to maximize its **performance measure with all possible actions**.
- A rational agent is said to perform the **right things**. AI is about creating **rational agents** to use for **game theory** and **decision theory** for **various real- world scenarios**.
- For an AI agent, the **rational action is most important** because in AI reinforcement learning algorithm, **for each best possible action**, agent **gets the positive reward** and for each wrong action, an **agent gets a negative reward**.

NOTE: Rational agents in AI are very similar to intelligent agents.

TEAM Y not Learning

RATIONALITY:

The rationality of an agent is measured by its performance measure. Rationality can be judged on the basis of following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.
- The actions that the agent can perform.
- The agent's percept sequence to date.

NOTE: Rationality differs from Omniscience because an Omniscient agent knows the actual outcome of its action and act accordingly, which is not possible in reality.

TEAM not Learning

MAPPING OF PERCEPT SEQUENCES TO ACTIONS

When it is known that the action of agent depends completely on the perceptual history – the percept sequence, then the agent can be described by using a mapping. Mapping is a list that maps the percept sequence to the action. When we specify which action an agent should take corresponding to the given percept sequence, we specify the design for an ideal agent.

AUTONOMY

The behaviour of an agent depends on its own experience as well as the built-in knowledge of the agent instilled by the agent designer. A system is autonomous if it takes actions according to its experience. So for the initial phase, as it does not have any experience, it is good to provide built-in knowledge. The agent learns then through evolution. A truly autonomous intelligent agent should be able to operate successfully in a wide variety of environments if given sufficient time to adapt.

TASK ENVIRONMENTS

To design a rational agent we need to specify a task environment

- A problem specification for which the agent is a solution

PEAS Representation

PEAS is a type of model on which an AI agent works upon. When we define an AI agent or rational agent, then we can group its properties under PEAS representation model. It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

Here performance measure is the objective for the success of an agent's behaviour.



PEAS: SPECIFYING AN AUTOMATED TAXI DRIVER

Performance measure: ?

Environment: ?

Actuators: ?

Sensors: ?

Performance measure:

- safe, fast, legal, comfortable, maximize profits

Environment:

- roads, other traffic, pedestrians, customers

Actuators:

- steering, accelerator, brake, signal, horn

Sensors:

- cameras, sonar, speedometer, GPS



PEAS: MEDICAL DIAGNOSIS SYSTEM

- **PERFORMANCE MEASURE:** Healthy patient, minimize costs, lawsuits
- **ENVIRONMENT:** Patient, hospital, staff
- **ACTUATORS:** Screen display (form including: questions, tests, diagnoses, treatments, referrals)
- **SENSORS:** Keyboard (entry of symptoms, findings, patient's answers)

TEAM Y not Learning...

Example of Agents with their PEAS representation

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Color pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry

Examples of agent types and their PEAS descriptions.

THE NATURE OF ENVIRONMENTS

TEAM Y not Learnings...

THE NATURE OF ENVIRONMENTS

- An environment in artificial intelligence is **the surrounding of the agent**. **The agent takes input from the environment through sensors** and **delivers the output to the environment through actuators**.
- An environment is **everything** in the world **which surrounds the agent**, but it is not a part of an **agent itself**. An environment can be described as a situation in which an agent is present.
- The environment is where agent lives, operate and provide the agent with something to sense and act upon it. **An environment is mostly said to be non-feministic.**

TEAM Y not Learning

FEATURES OF ENVIRONMENT

As per Russell and Norvig, an environment can have various features from the point of view of an agent:

1. Fully observable vs Partially Observable
2. Static vs Dynamic
3. Discrete vs Continuous
4. Deterministic vs Stochastic
5. Single-agent vs Multi-agent
6. Episodic vs sequential
7. Known vs Unknown
8. Accessible vs Inaccessible

1. FULLY OBSERVABLE VS PARTIALLY OBSERVABLE:

- ✓ If an agent sensor can sense or access the complete state of an environment at each point of time then it is a **fully observable** environment, else it is **partially observable**.
- ✓ A fully observable environment is easy as there is no need to maintain the internal state to keep track history of the world.
- ✓ An agent with no sensors in all environments then such an environment is called as **unobservable**.

2. DETERMINISTIC VS STOCHASTIC:

- ✓ If an agent's current state and selected action can completely determine the next state of the environment, then such environment is called a deterministic environment.
- ✓ A stochastic environment is random in nature and cannot be determined completely by an agent.
- ✓ In a deterministic, fully observable environment, agent does not need to worry about uncertainty.

3. EPISODIC VS SEQUENTIAL:

- ✓ In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
- ✓ However, in Sequential environment, an agent requires memory of past actions to determine the next best actions.

4. SINGLE-AGENT VS MULTI-AGENT

- ✓ If only one agent is involved in an environment, and operating by itself then such an environment is called single agent environment.
- ✓ However, if multiple agents are operating in an environment, then such an environment is called a multi-agent environment.
- ✓ The agent design problems in the multi-agent environment are different from single agent environment.

5. STATIC VS DYNAMIC:

- ✓ If the environment can change itself while an agent is deliberating then such environment is called a dynamic environment else it is called a static environment.
- ✓ Static environments are easy to deal because an agent does not need to continue looking at the world while deciding for an action.
- ✓ However for dynamic environment, agents need to keep looking at the world at each action.
- ✓ Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

6. DISCRETE VS CONTINUOUS:

- ✓ If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.
- ✓ A chess game comes under discrete environment as there is a finite number of moves that can be performed.
- ✓ A self-driving car is an example of a continuous environment.

7. ACCESSIBLE VS INACCESSIBLE

- ✓ If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
- ✓ An empty room whose state can be defined by its temperature is an example of an accessible environment.

8. KNOWN VS UNKNOWN

- ✓ Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
- ✓ In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works in order to perform an action.
- ✓ It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.

TEAM Y not Learning

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

Examples of task environments and their characteristics.

THE STRUCTURE OF AGENTS

TEAM Y not Learnings...

THE STRUCTURE OF AGENTS

The task of AI is to design an agent program which implements the agent function. The structure of an intelligent agent is a combination of architecture and agent program. It can be viewed as:

$$\text{Agent} = \text{Architecture} + \text{Agent program}$$

Architecture: Architecture is machinery that an AI agent executes on.

Agent Function: Agent function is used to map a percept to an action.

$$f:P^* \rightarrow A$$

Agent program: Agent program is an implementation of agent function. An agent program executes on the physical architecture to produce function f .

TYPES OF AI AGENTS

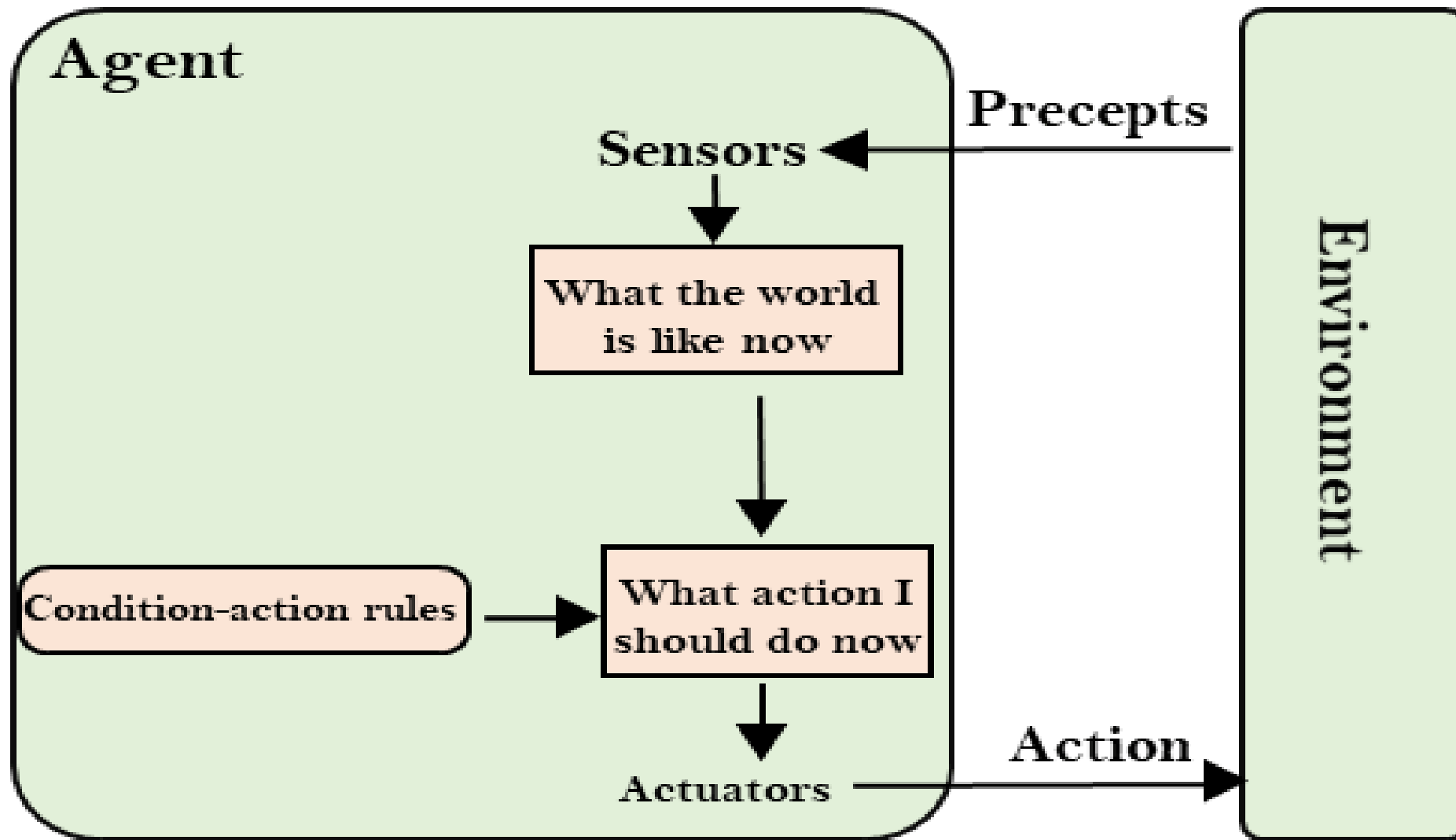
Agents can be grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better action over the time.

These are given below:

- Simple Reflex Agent
- Model-based reflex agent
- Goal-based agents
- Utility-based agent
- Learning agent

1. SIMPLE REFLEX AGENT:

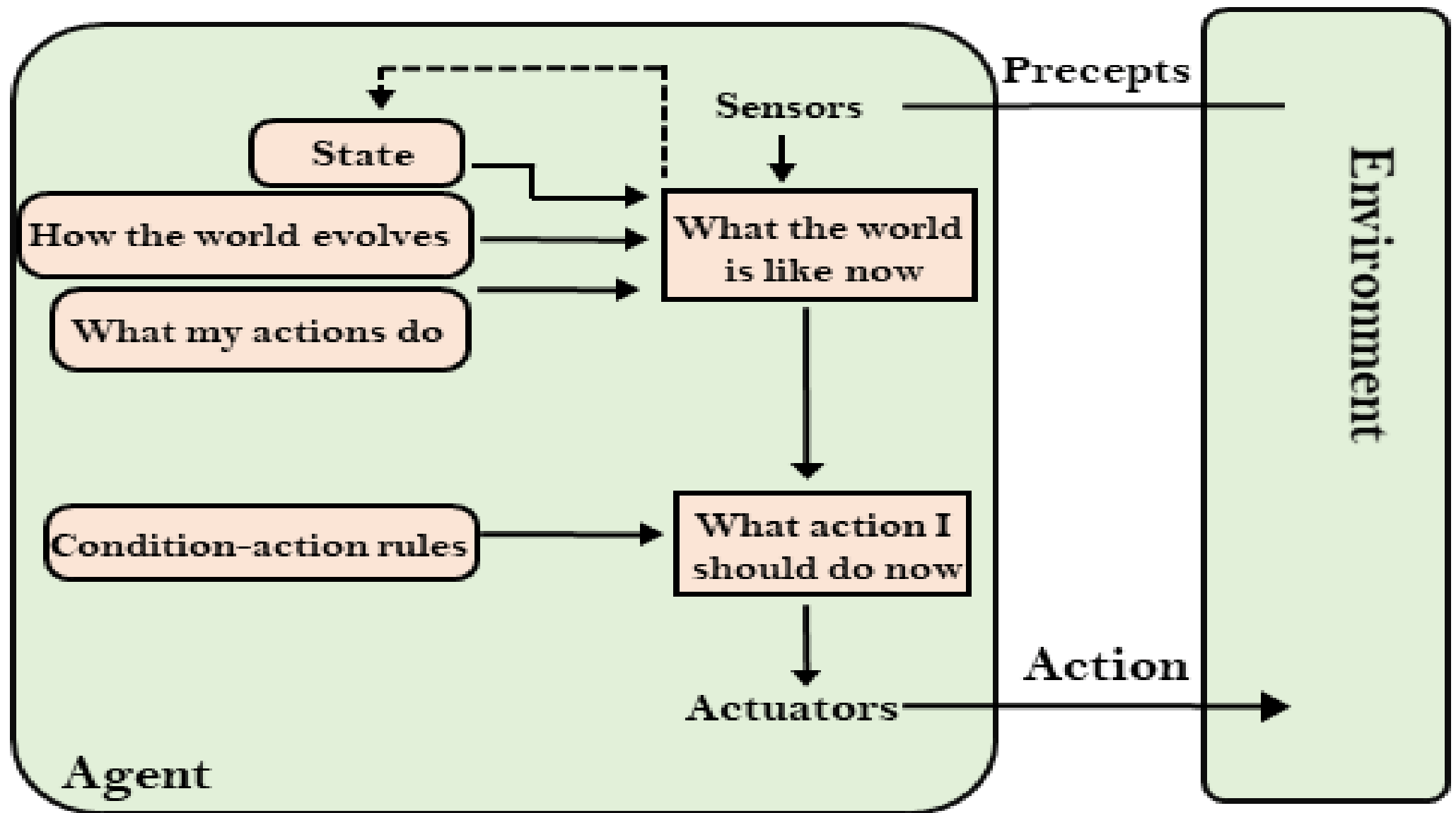
- The Simple reflex agents are the simplest agents. These agents take decisions on the basis of the current percepts and ignore the rest of the percept history.
- These agents only succeed in the fully observable environment.
- The Simple reflex agent does not consider any part of percepts history during their decision and action process.
- The Simple reflex agent works on Condition-action rule, which means it maps the current state to action. Such as a Room Cleaner agent, it works only if there is dirt in the room.
- Problems for the simple reflex agent design approach:
 - ✓ They have very limited intelligence
 - ✓ They do not have knowledge of non-perceptual parts of the current state
 - ✓ Mostly too big to generate and to store.
 - ✓ Not adaptive to changes in the environment.



Schematic diagram of a simple reflex agent.

2. MODEL-BASED REFLEX AGENT

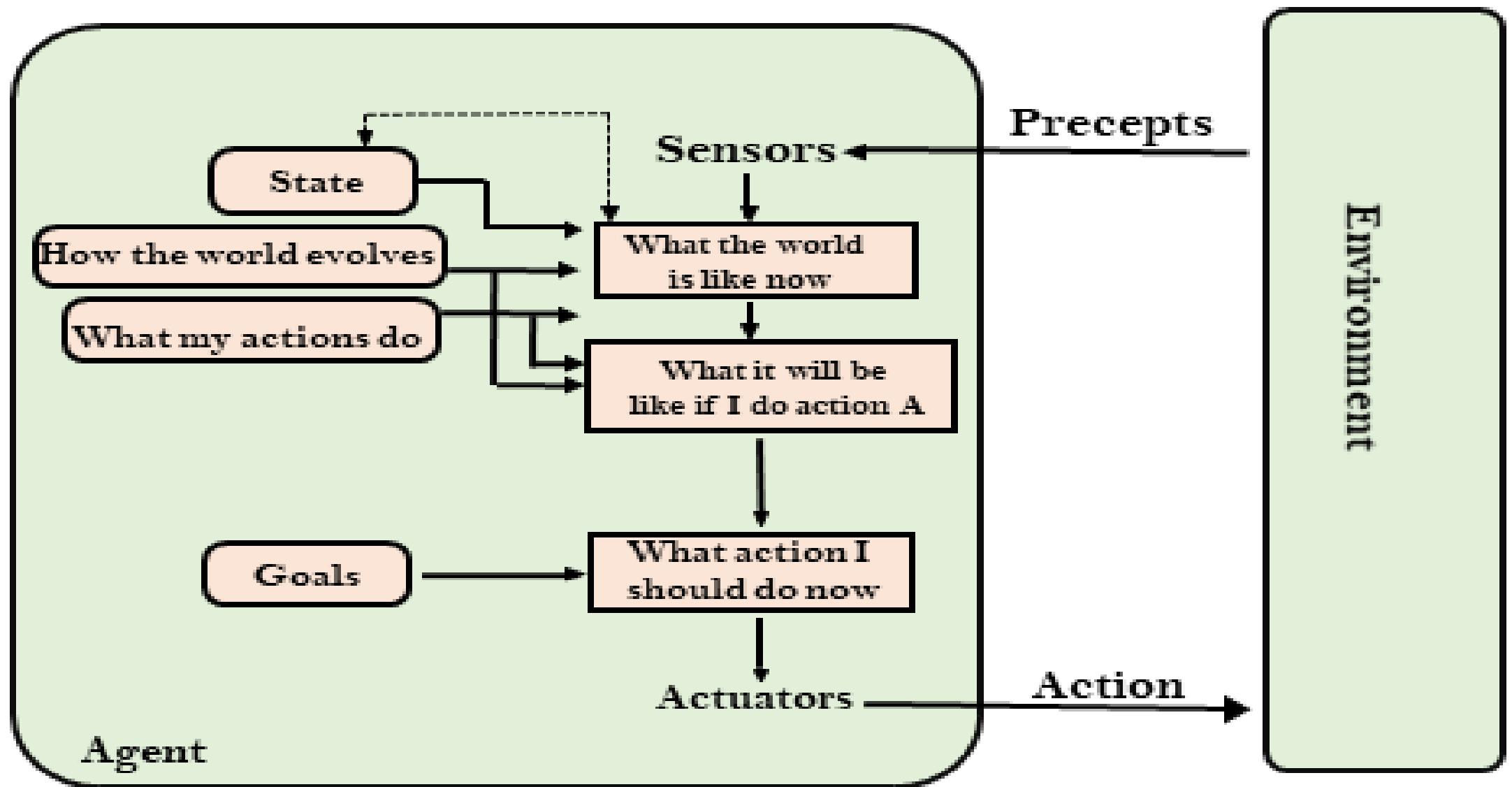
- The Model-based agent can work in a partially observable environment, and track the situation.
- A model-based agent has two important factors:
 - ✓ **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
 - ✓ **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
 - ✓ How the world evolves
 - ✓ How the agent's action affects the world.



A model-based reflex agent.

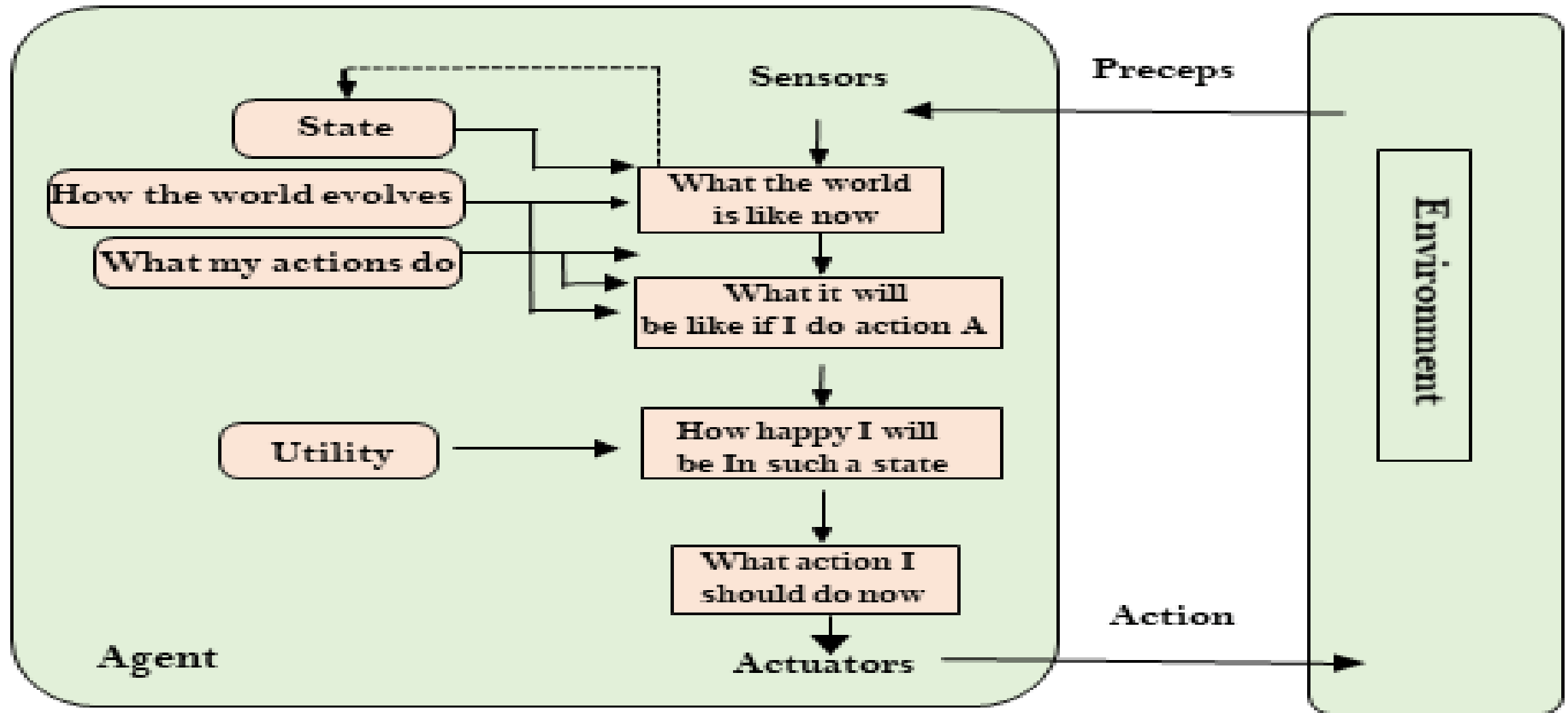
3. GOAL-BASED AGENTS

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its goal which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenario are called searching and planning, which makes an agent proactive.



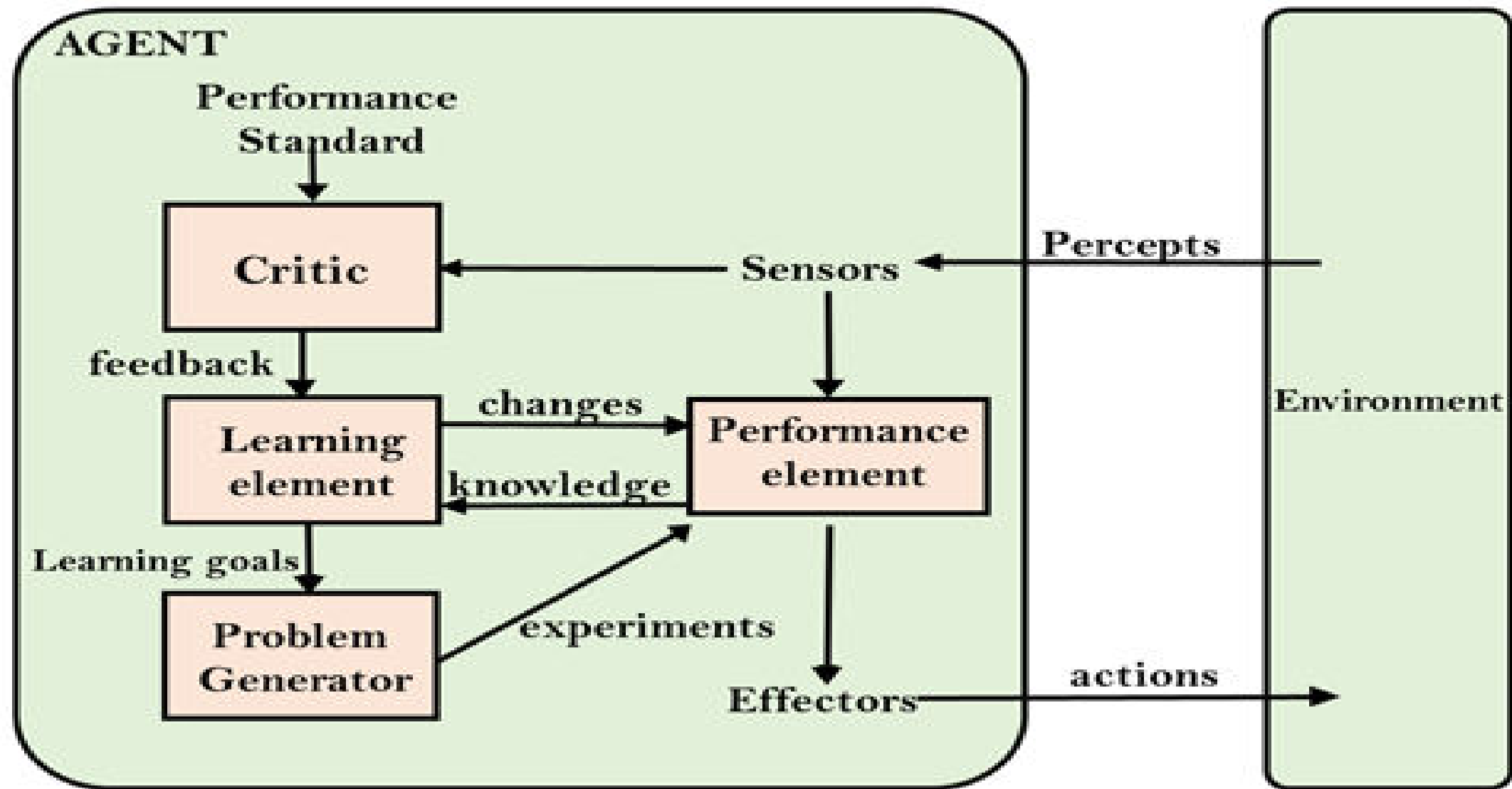
4. UTILITY-BASED AGENTS

- These agents are similar to the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agent act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent has to choose in order to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.



5. LEARNING AGENTS

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
 - ✓ **Learning element:** It is responsible for making improvements by learning from environment
 - ✓ **Critic:** Learning element takes feedback from critic which describes that how well the agent is doing with respect to a fixed performance standard.
 - ✓ **Performance element:** It is responsible for selecting external action
 - ✓ **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents are able to learn, analyze performance, and look for new ways to improve the performance.



A general learning agent.



THANK YOU



PROBLEM-SOLVING AGENT

PROBLEM-SOLVING AGENT

The problem-solving agent performs **precisely** by **defining problems** and its **several solutions**.

- According to **psychology**, “a problem-solving refers to a state where we wish to reach to a **definite goal from a present state or condition.**”
- According to computer science, a problem-solving is a part of artificial intelligence which encompasses a number of techniques such as algorithms, heuristics to solve a problem.

STEPS PERFORMED BY PROBLEM-SOLVING AGENT

Goal Formulation: It is the first and simplest step in problem-solving.

- It organizes the steps/sequence required to formulate one goal out of multiple goals as well as actions to achieve that goal.
- Goal formulation is based on the current situation and the agent's performance measure (discussed below).

Problem Formulation: It is the most **important** step **of problem-solving** which decides what actions should be taken to achieve the formulated goal.

There are following five components involved in problem formulation:

- **Initial State:** It is the starting state or initial step of the agent towards its goal.
- **Actions:** It is the description of the possible actions available to the agent.
- **Transition Model:** It describes what each action does.
- **Goal Test:** It determines if the given state is a goal state.
- **Path cost:** It assigns a numeric cost to each path that follows the goal. The problem-solving agent selects a cost function, which reflects its performance measure. Remember, **an optimal solution has the lowest path cost among all the solutions.**

- **Search:** It identifies all the best possible sequence of actions to reach the goal state from the current state. It takes a problem as an input and returns solution as its output.
- **Solution:** It finds the best algorithm out of various algorithms, which may be proven as the best optimal solution.
- **Execution:** It executes the best optimal solution from the searching algorithms to reach the goal state from the current state.

NOTE: Initial state, actions, and transition model together define the **state-space** of the problem implicitly. State-space of a problem is a set of all states which can be reached from the initial state followed by any sequence of actions. The state-space forms a directed map or graph where nodes are the states, links between the nodes are actions, and the path is a sequence of states connected by the sequence of actions.

function SIMPLE-PROBLEM-SOLVING-AGENT(*percept*) **returns** an action

persistent: *seq*, an action sequence, initially empty

state, some description of the current world state

goal, a goal, initially null

problem, a problem formulation

state $\leftarrow$ UPDATE-STATE(*state*, *percept*)

if *seq* is empty **then**

goal $\leftarrow$ FORMULATE-GOAL(*state*)

problem $\leftarrow$ FORMULATE-PROBLEM(*state*, *goal*)

seq $\leftarrow$ SEARCH(*problem*)

if *seq* = *failure* **then return** a null action

action $\leftarrow$ FIRST(*seq*)

seq $\leftarrow$ REST(*seq*)

return *action*

A simple problem-solving agent. It first formulates a goal and a problem, searches for a sequence of actions that would solve the problem, and then executes the actions one at a time. When this is complete, it formulates another goal and starts over.

EXAMPLE PROBLEMS

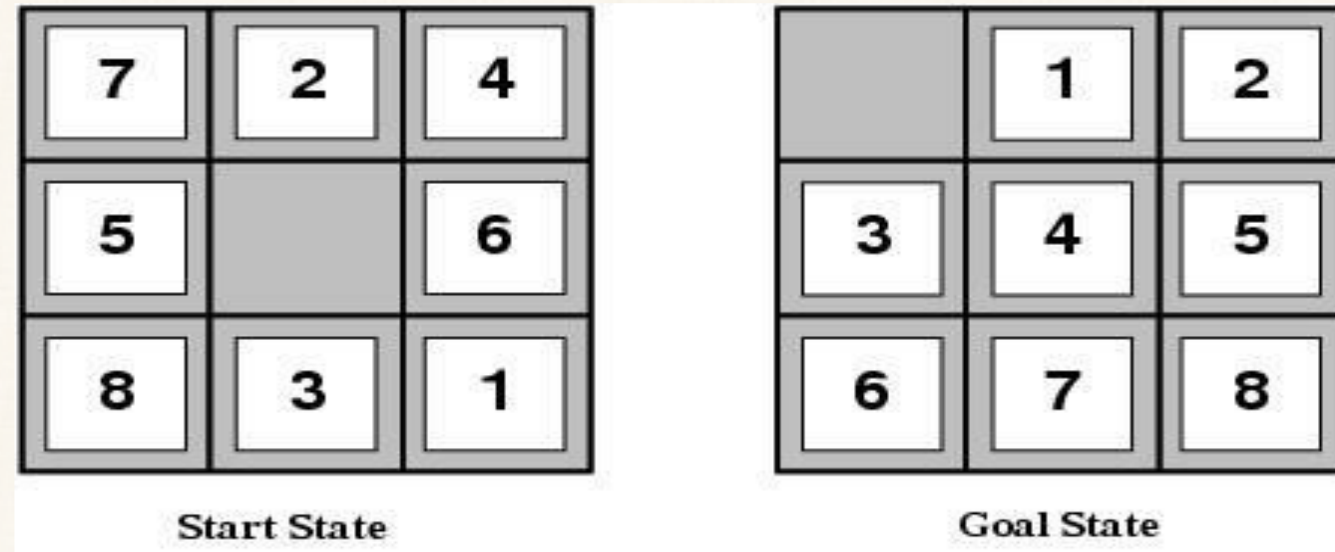
Basically, there are two types of problem approaches:

- **Toy Problem:** It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.
- **Real-world Problem:** It is real-world based problems which require solutions. Unlike a toy problem, it does not depend on descriptions, but we can have a general formulation of the problem.

SOME TOY PROBLEMS

8 Puzzle Problem: Here, we have a 3×3 matrix with movable tiles numbered from 1 to 8 with a blank space. The tile adjacent to the blank space can slide into that space. The objective is to reach a specified goal state similar to the goal state, as shown in the below figure.

SOME TOY PROBLEMS



THE PROBLEM FORMULATION IS AS FOLLOWS:

- **States:** It describes the location of each numbered tiles and the blank tile.
- **Initial State:** We can start from any state as the initial state.
- **Actions:** Here, actions of the blank space is defined, i.e., either **left, right, up or down**
- **Transition Model:** It returns the resulting state as per the given state and actions.
- **Goal test:** It identifies whether we have reached the correct goal-state.
- **Path cost:** The path cost is the number of steps in the path where the cost of each step is 1.

EIGHT PUZZLE PROBLEM

- We also know the **eight-puzzle problem** by the name of **N puzzle problem** or **sliding puzzle problem**.
- **N-puzzle** that consists of N tiles (N+1 tiles with an empty tile) where N can be 8, 15, 24 and so on.
- In our example **N = 8**. (that is **square root of (8+1) = 3 rows and 3 columns**).
- In the same way, if we have N = 15, 24 in this way, then they have Row and columns as follow (**square root of (N+1) rows and square root of (N+1) columns**).
- That is if **N=15** than number of rows and columns= 4, and if **N= 24** number of rows and columns= 5.
- So, basically in these types of problems we have given a **initial state or initial configuration (Start state) and a Goal state or Goal Configuration**.

Here We are solving a problem of 8 puzzle that is a 3x3 matrix.

Initial state

1	2	3
	4	6
7	5	8

Goal state

1	2	3
4	5	6
7	8	

The puzzle can be solved by moving the tiles one by one in the single empty space and thus achieving the Goal state.

Rules of solving puzzle

Instead of moving the tiles in the empty space we can visualize moving the empty space in place of the tile.

The empty space can only **move in four directions** (Movement of empty space)

Up Down Right or Left

The empty space **cannot move diagonally** and can take **only one step at a time**.

Start Node

1	2	3
	4	6
7	5	8

Right Move

1	2	3
4		6
7	5	8

Up move

	2	3
1	4	6
7	5	8

Down move

1	2	3
7	4	6
	5	8

Explore all the move for each node (i.e. U,D, R,L)

Down move

1	2	3
4	5	6
7		8

Left move

1	2	3
	4	6
7	5	8

Right Move

1	2	3
4	6	
7	5	8

Up move

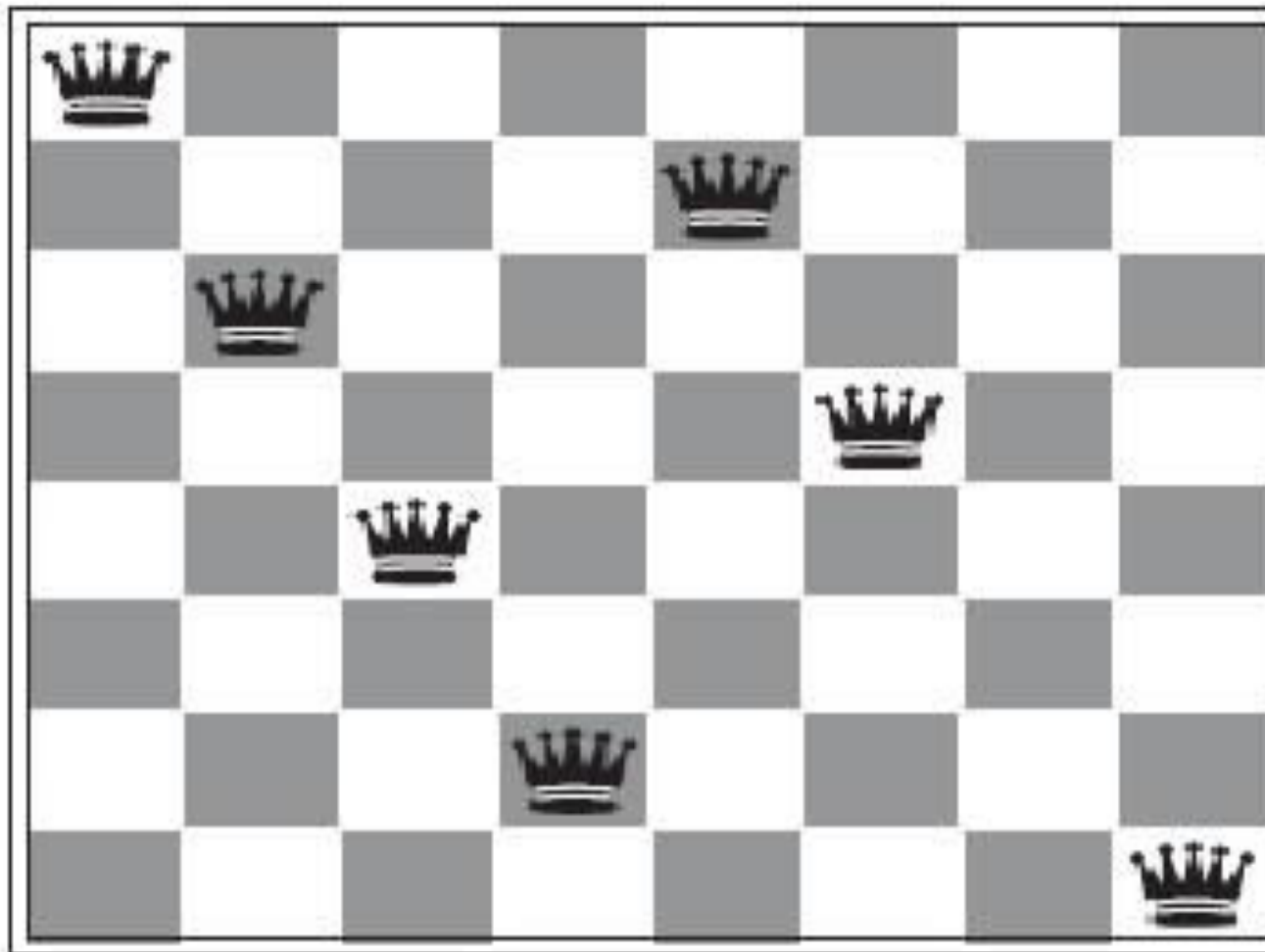
1		3
4	2	6
7	5	8

Explore all possible move for each node

1	2	3
4	5	6
7		8

Goal Node

1	2	3
4	5	6
	7	8



Almost a solution to the 8-queens problem. (Solution is left as an exercise.)

For this problem, there are two main kinds of formulation

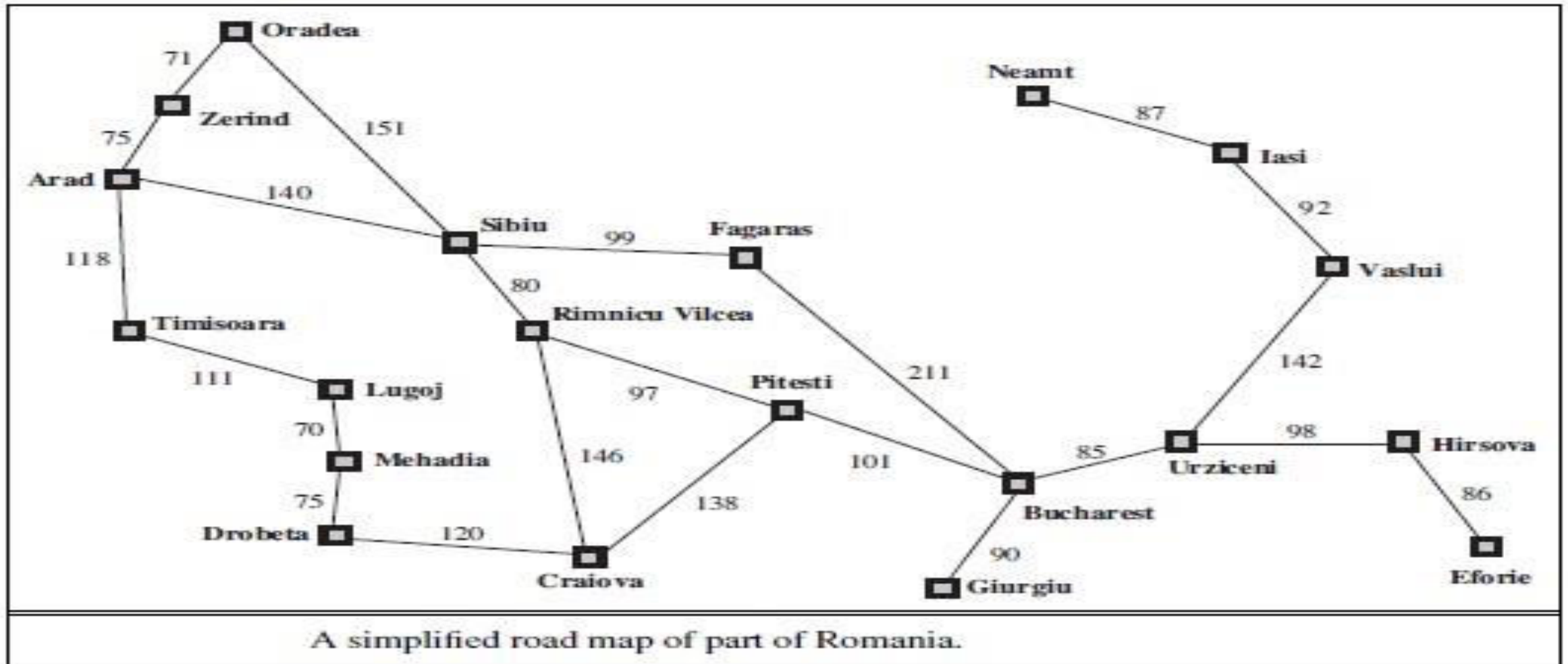
- **Incremental formulation:** It starts from an empty state where the operator augments a queen at each step.

Following steps are involved in this formulation:

- **States:** Arrangement of any 0 to 8 queens on the chessboard.
- **Initial State:** An empty chessboard
- **Actions:** Add a queen to any empty box.
- **Transition model:** Returns the chessboard with the queen added in a box.
- **Goal test:** Checks whether 8-queens are placed on the chessboard without any attack.
- **Path cost:** There is no need for path cost because only final states are counted.
- In this formulation, there is approximately 1.8×10^{14} possible sequence to investigate.
- **Complete-state formulation:** It starts with all the 8-queens on the chessboard and moves them around, saving from the attacks.

SOME REAL-WORLD PROBLEMS

- **Traveling salesperson problem (TSP):** It is a **touring problem** where the salesman can visit each city only once. The objective is to find the shortest tour and sell-out the stuff in each city.



SOME REAL-WORLD PROBLEMS

- **VLSI Layout problem:** In this problem, millions of components and connections are positioned on a chip in order to minimize the area, circuit-delays, stray-capacitances, and maximizing the manufacturing yield.

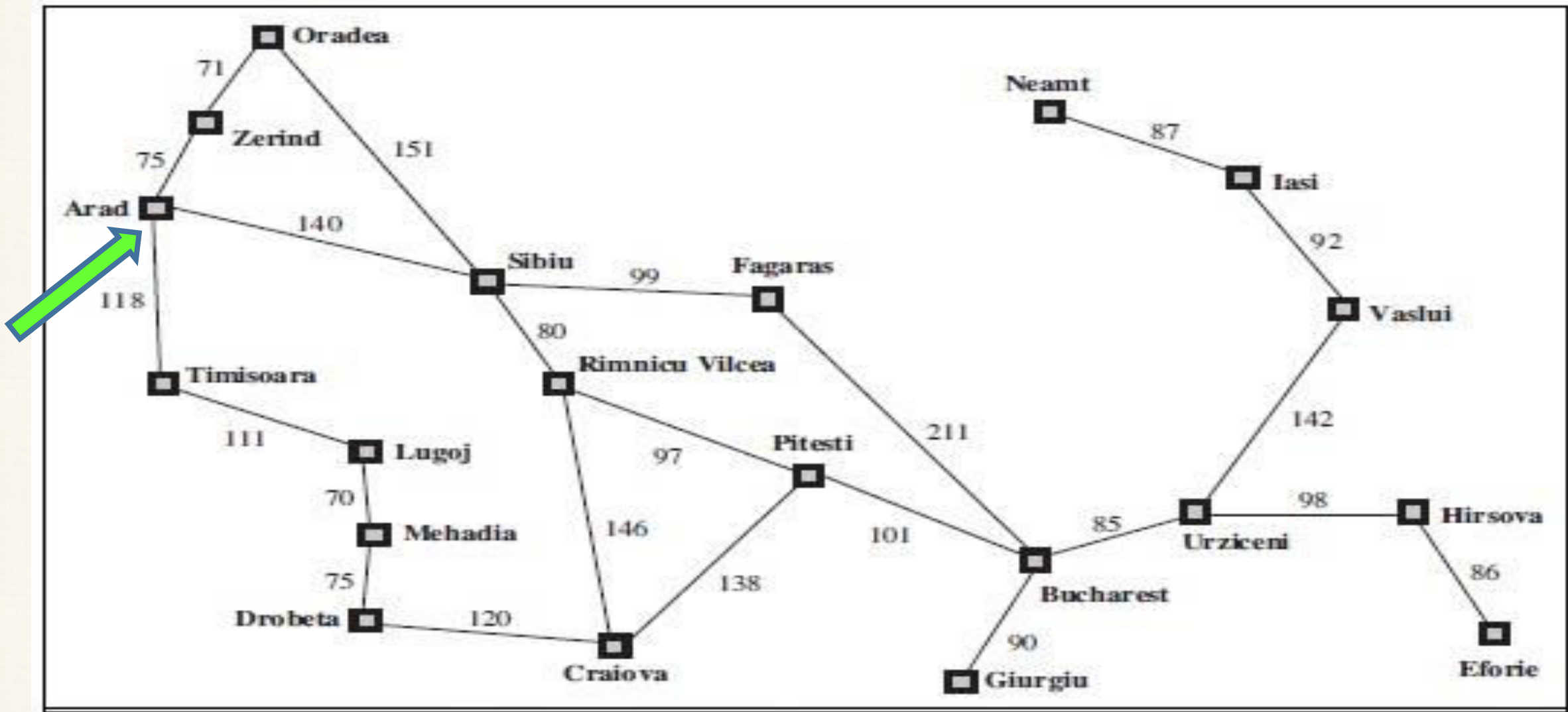
THE LAYOUT PROBLEM IS SPLIT INTO TWO PARTS:

- **Cell layout:** Here, the primitive components of the circuit are grouped into cells, each performing its specific function. Each cell has a fixed shape and size. The task is to place the cells on the chip without overlapping each other.
- **Channel routing:** It finds a specific route for each wire through the gaps between the cells.
- **Protein Design:** The objective is to find a sequence of amino acids which will fold into 3D protein having a property to cure some disease.

ROBOT NAVIGATION is a generalization of the route-finding problem described earlier. Rather than following a discrete set of routes, a robot can move in a continuous space with (in principle) an infinite set of possible actions and states. For a circular robot moving on a flat surface, the space is essentially two-dimensional. When the robot has arms and legs or wheels that must also be controlled, the search space becomes many-dimensional. Advanced techniques are required just to make the search space finite. We examine some of these methods in Chapter 25. In addition to the complexity of the problem, real robots must also deal with errors in their sensor readings and motor controls.

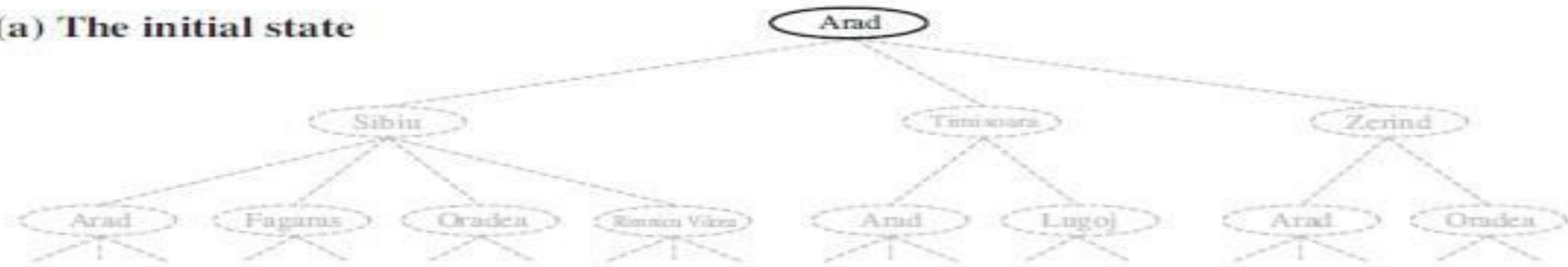
AUTOMATIC ASSEMBLY SEQUENCING of complex objects by a robot was first demonstrated by FREDDY (Michie, 1972). Progress since then has been slow but sure, to the point where the assembly of intricate objects such as electric motors is economically feasible. In assembly problems, the aim is to find an order in which to assemble the parts of some object. If the wrong order is chosen, there will be no way to add some part later in the sequence without undoing some of the work already done. Checking a step in the sequence for feasibility is a difficult geometrical search problem closely related to robot navigation. Thus, the generation of legal actions is the expensive part of assembly sequencing. Any practical algorithm must avoid exploring all but a tiny fraction of the state space. Another important assembly problem is **protein design**.

SEARCHING FOR SOLUTIONS

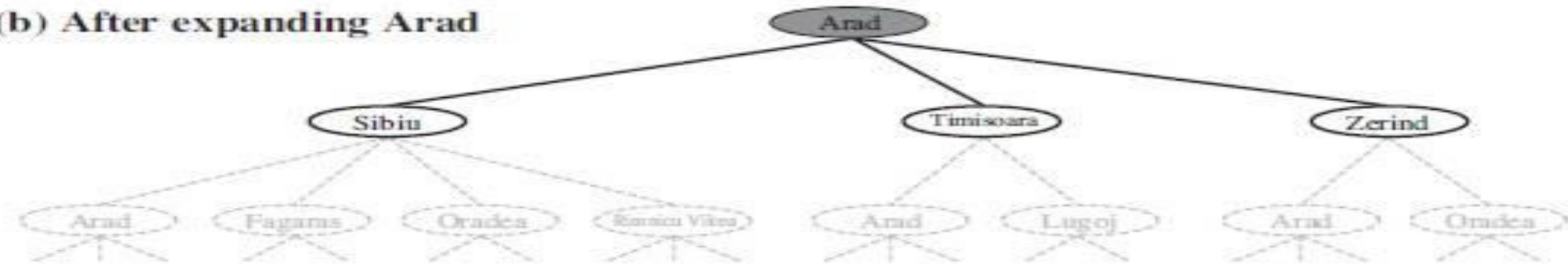


A simplified road map of part of Romania.

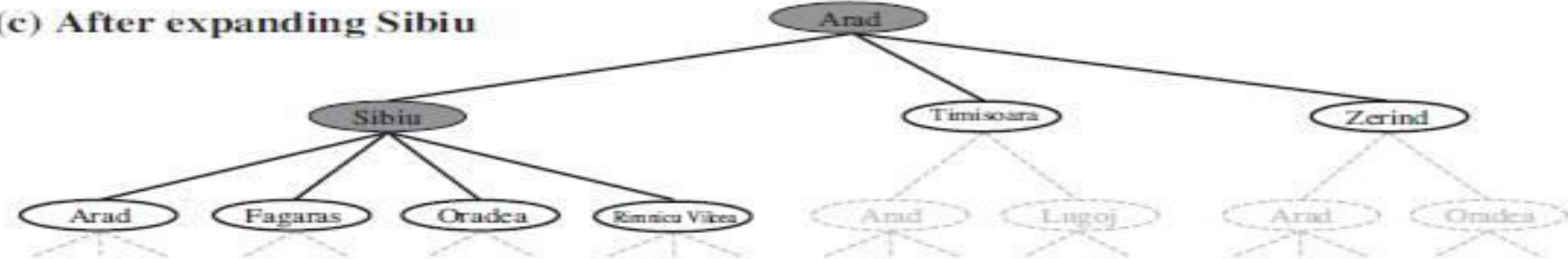
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



Partial search trees for finding a route from Arad to Bucharest. Nodes that have been expanded are shaded; nodes that have been generated but not yet expanded are outlined in bold; nodes that have not yet been generated are shown in faint dashed lines.

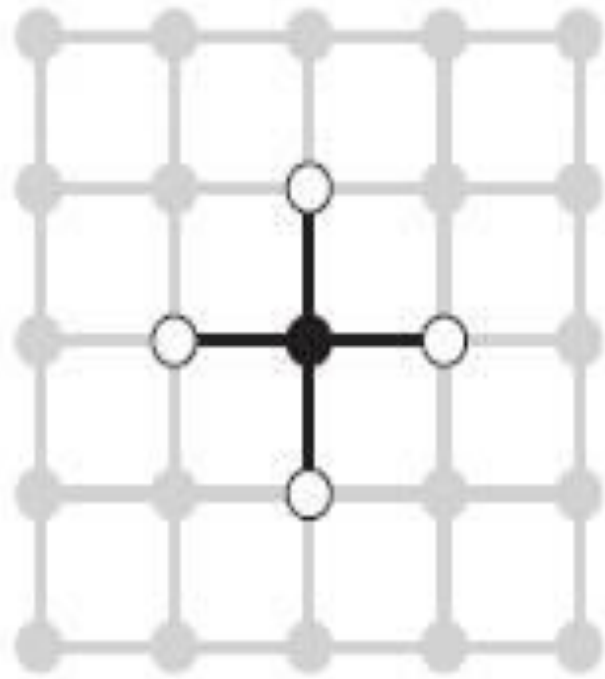
function TREE-SEARCH(*problem*) **returns** a solution, or failure
 initialize the frontier using the initial state of *problem*
 loop do
 if the frontier is empty **then return** failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state **then return** the corresponding solution
 expand the chosen node, adding the resulting nodes to the frontier

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
 initialize the frontier using the initial state of *problem*
 initialize the explored set to be empty
 loop do
 if the frontier is empty **then return** failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state **then return** the corresponding solution
 add the node to the explored set
 expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set

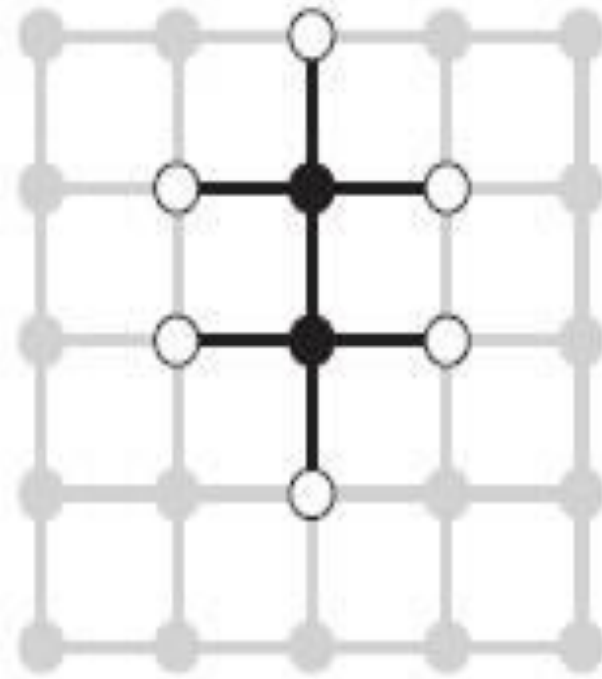
An informal description of the general tree-search and graph-search algorithms. The parts of GRAPH-SEARCH marked in bold italic are the additions needed to handle repeated states.



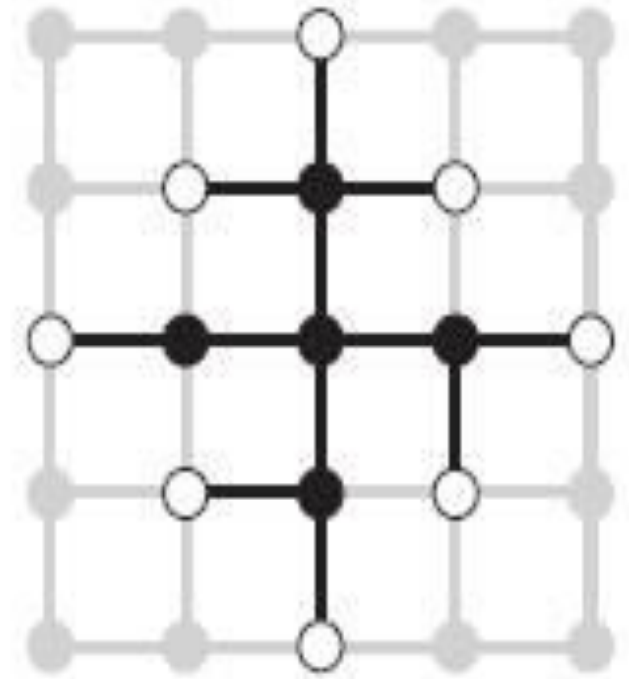
A sequence of search trees generated by a graph search on the Romania problem of Figure . At each stage, we have extended each path by one step. Notice that at the third stage, the northernmost city (Oradea) has become a dead end: both of its successors are already explored via other paths.



(a)



(b)



(c)

The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

INFRASTRUCTURE FOR SEARCH ALGORITHMS

Search algorithms require a data structure to keep track of the search tree that is being constructed.

For each node n of the tree, we have a structure that contains four components:

- **n . STATE:** the state in the state space to which the node corresponds;
- **n . PARENT:** the node in the search tree that generated this node;
- **n . ACTION:** the action that was applied to the parent to generate the node;
- **n . PATH-COST:** the cost, traditionally denoted by $g(n)$, of the path from the initial state to the node, as indicated by the parent pointers.

MEASURING PROBLEM-SOLVING PERFORMANCE

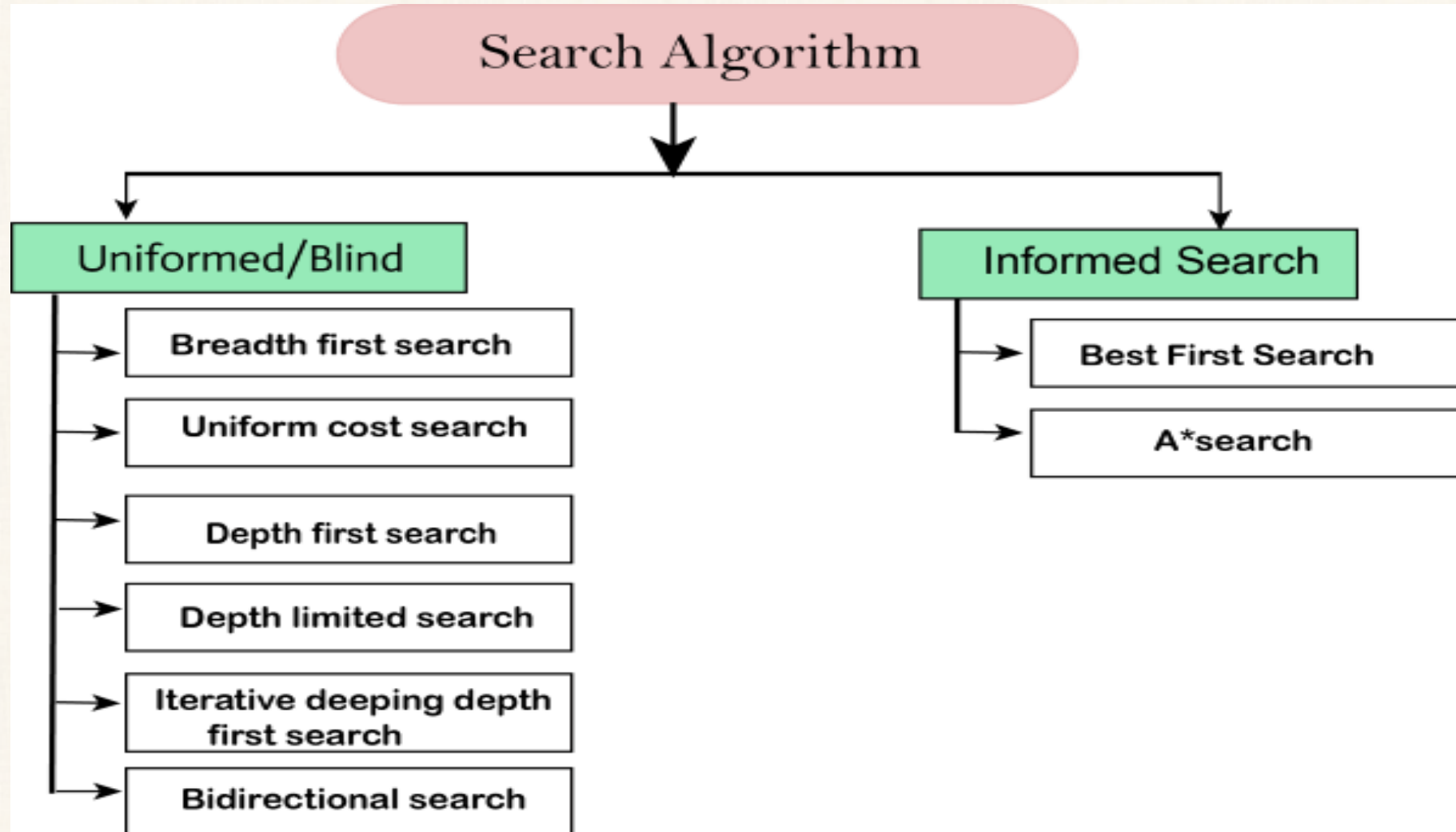
Before discussing different search strategies, the **performance measure** of an algorithm should be measured. Consequently, there are four ways to measure the performance of an algorithm:

- **Completeness:** It measures if the algorithm guarantees to find a solution (if any solution exists).
- **Optimality:** It measures if the strategy searches for an optimal solution.
- **Time Complexity:** The time taken by the algorithm to find a solution.
- **Space Complexity:** Amount of memory required to perform a search.

The complexity of an algorithm depends on **branching factor** or **maximum number of successors**, **depth of the shallowest goal node** (i.e., number of steps from root to the path) and **the maximum length of any path in a state space**.

TYPES OF SEARCH ALGORITHMS

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.

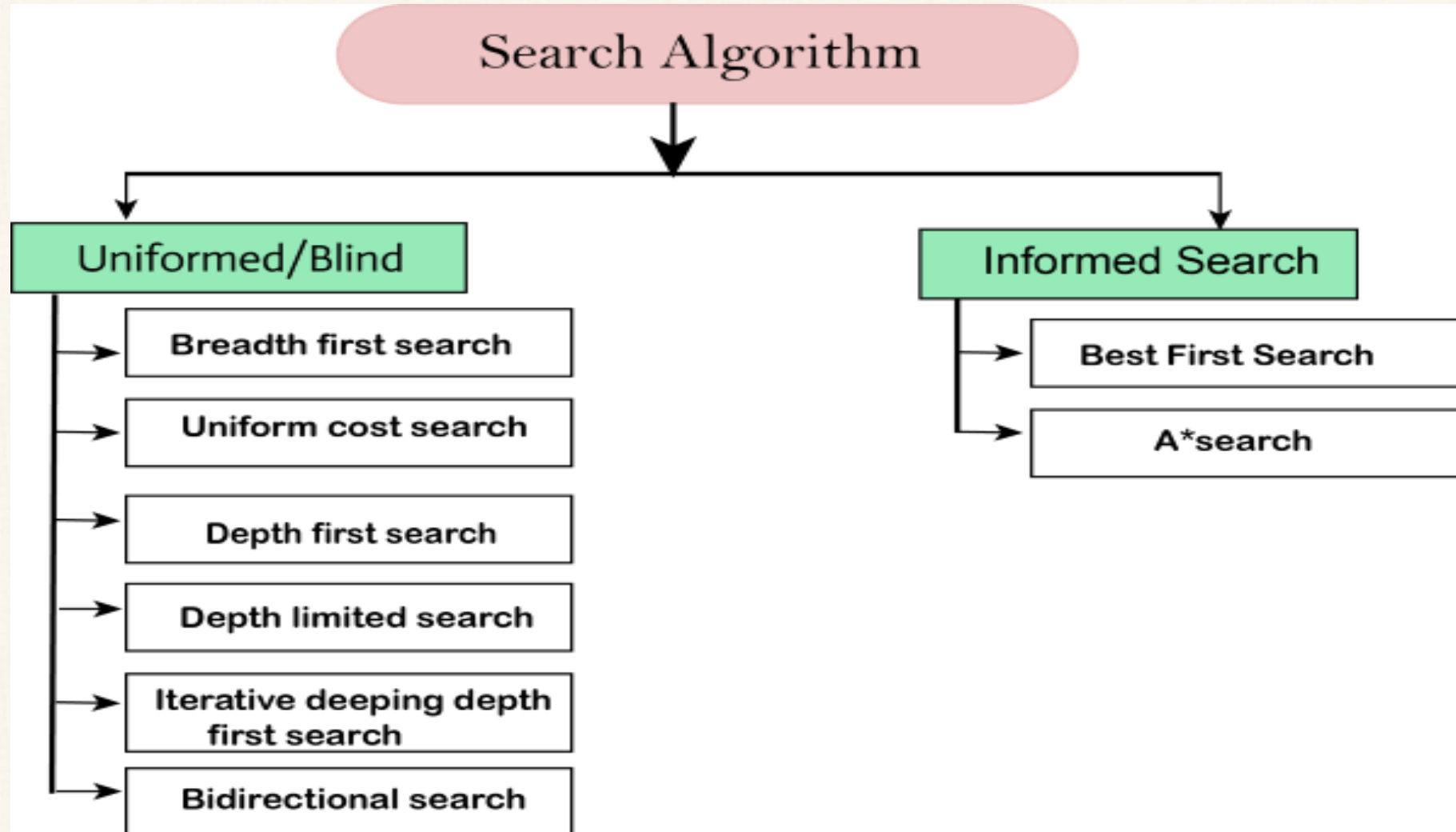


THANK YOU



TYPES OF SEARCH ALGORITHMS

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



UNINFORMED/BLIND SEARCH:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes.

Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Iterative deepening depth-first search
- Bidirectional Search

1. BREADTH-FIRST SEARCH:-

- Breadth-first search is the most common search strategy for traversing a tree or graph.
- This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

ADVANTAGES:

- BFS will provide a solution if any solution exists.
- If there are more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

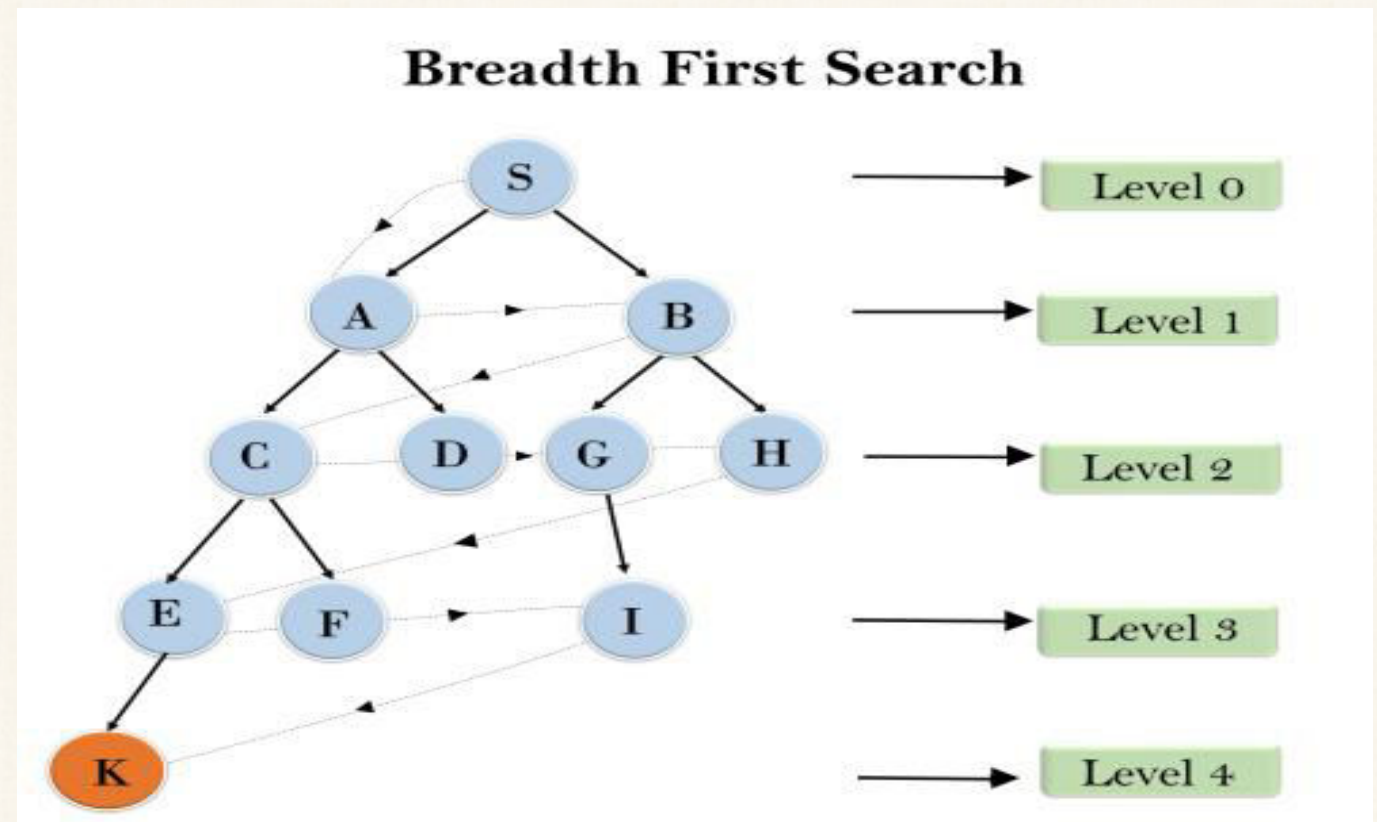
DISADVANTAGES:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

EXAMPLE:

- The traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

S---> A--->B---->C--->D---->G--->H--->E---->F---->I---->K



➤ **Time Complexity:** Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d = depth of shallowest solution and b is a node at every state.

$$\text{➤ } T(b) = 1 + b^1 + b^2 + \dots + b^d = O(b^d)$$

➤ **Space Complexity:** Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

➤ **Completeness:** BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

➤ **Optimality:** BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. DEPTH-FIRST SEARCH

- Depth-first search is a recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is similar to the BFS algorithm.

ADVANTAGE:

- DFS requires very less memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach to the goal node than BFS algorithm (if it traverses in the right path).

DISADVANTAGE:

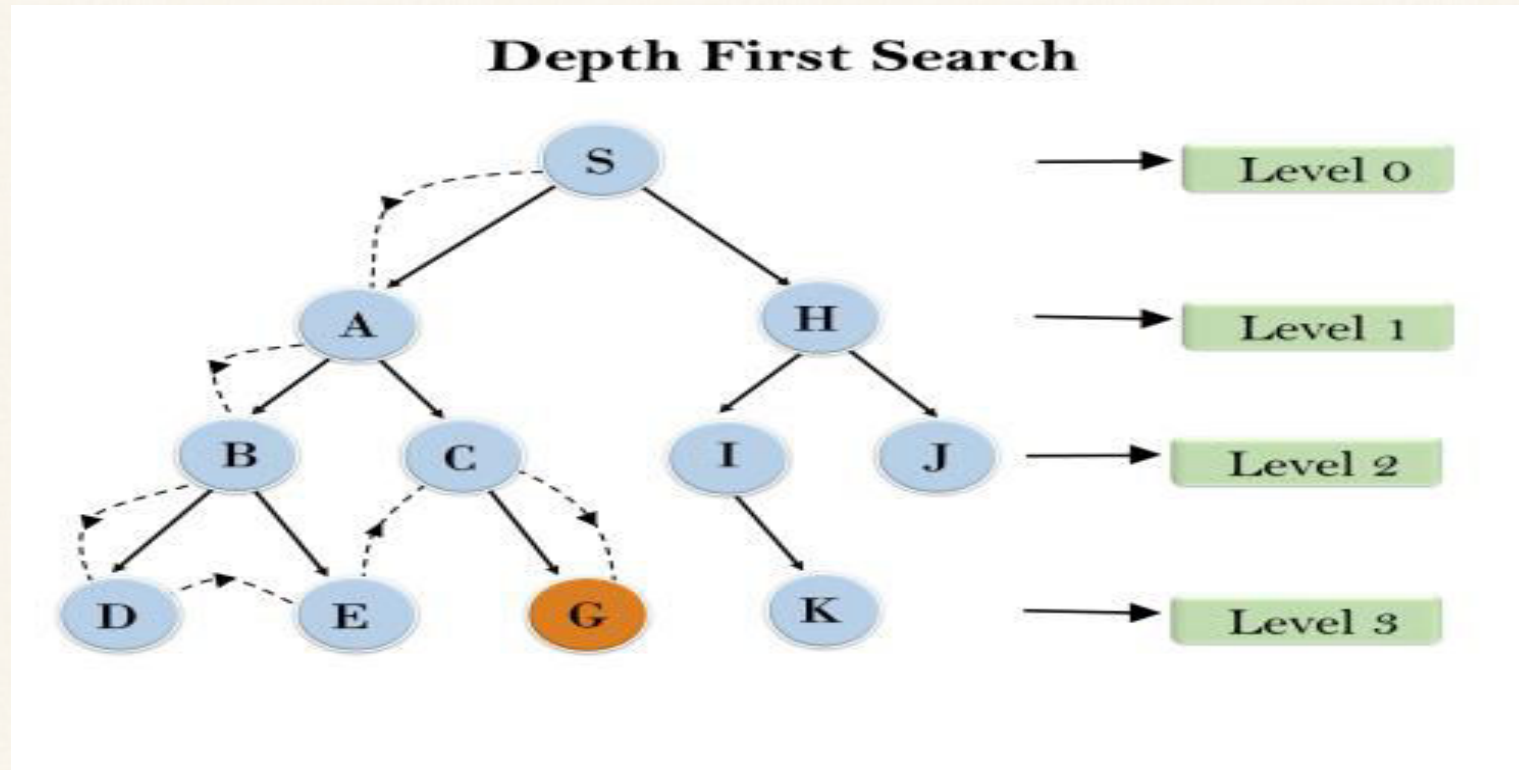
- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep down searching and sometime it may go to the infinite loop.

EXAMPLE:

Depth-first search, and it will follow the order as:

Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E, after traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$$T(n) = 1 + n^2 + n^3 + \dots + n^m = O(n^m)$$

Where, m = maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is $O(bm)$.

Optimal: DFS search algorithm is non-optimal, as it may generate a large number of steps or high cost to reach to the goal node.

3. DEPTH-LIMITED SEARCH ALGORITHM:

A depth-limited search algorithm is similar to depth-first search with a predetermined limit. Depth-limited search can solve the drawback of the infinite path in the Depth-first search. In this algorithm, the node at the depth limit will treat as it has no successor nodes further.

Depth-limited search can be terminated with two Conditions of failure:

- **Standard failure value:** It indicates that problem does not have any solution.
- **Cut off failure value:** It defines no solution for the problem within a given depth limit.

Advantages:

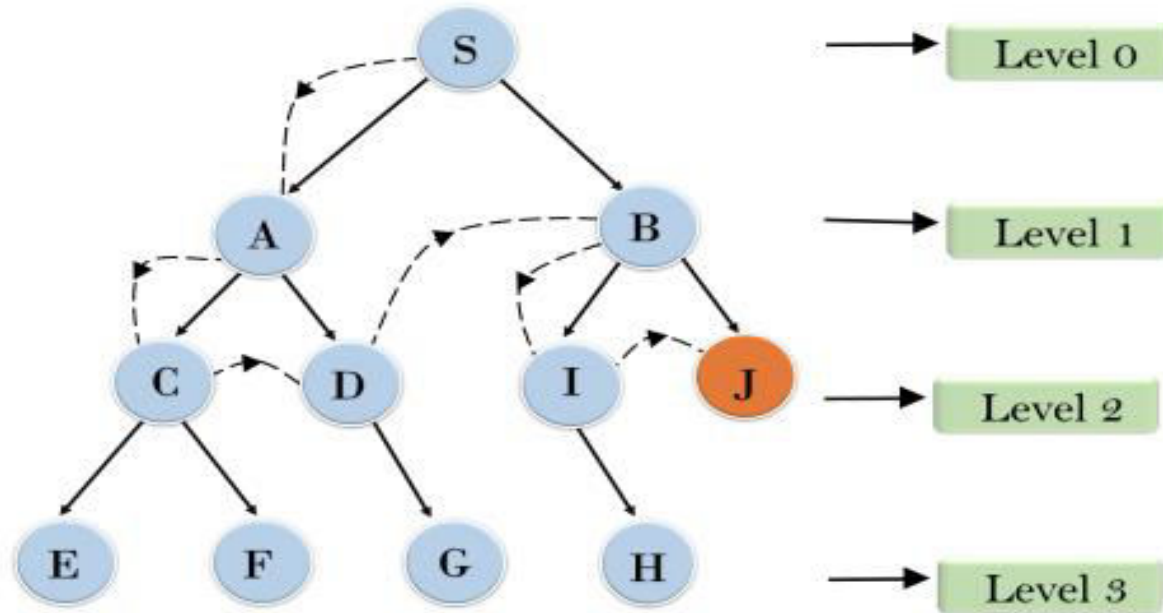
Depth-limited search is Memory efficient.

Disadvantages:

- Depth-limited search also has a disadvantage of incompleteness.
- It may not be optimal if the problem has more than one solution.

EXAMPLE

Depth Limited Search



Completeness: DLS search algorithm is complete if the solution is above the depth-limit.

Time Complexity: Time complexity of DLS algorithm is $O(b^\ell)$.

Space Complexity: Space complexity of DLS algorithm is $O(b \times \ell)$.

Optimal: Depth-limited search can be viewed as a special case of DFS, and it is also not optimal even if $\ell > d$.

4. UNIFORM-COST SEARCH ALGORITHM

- Uniform-cost search is a searching algorithm used for traversing a weighted tree or graph.
- This algorithm comes into play when a different cost is available for each edge.
- The primary goal of the uniform-cost search is to find a path to the goal node which has the lowest cumulative cost.
- Uniform-cost search expands nodes according to their path costs from the root node.
- It can be used to solve any graph/tree where the optimal cost is in demand.
- A uniform-cost search algorithm is implemented by the priority queue.
- It gives maximum priority to the lowest cumulative cost.
- Uniform cost search is equivalent to BFS algorithm if the path cost of all edges is the same.

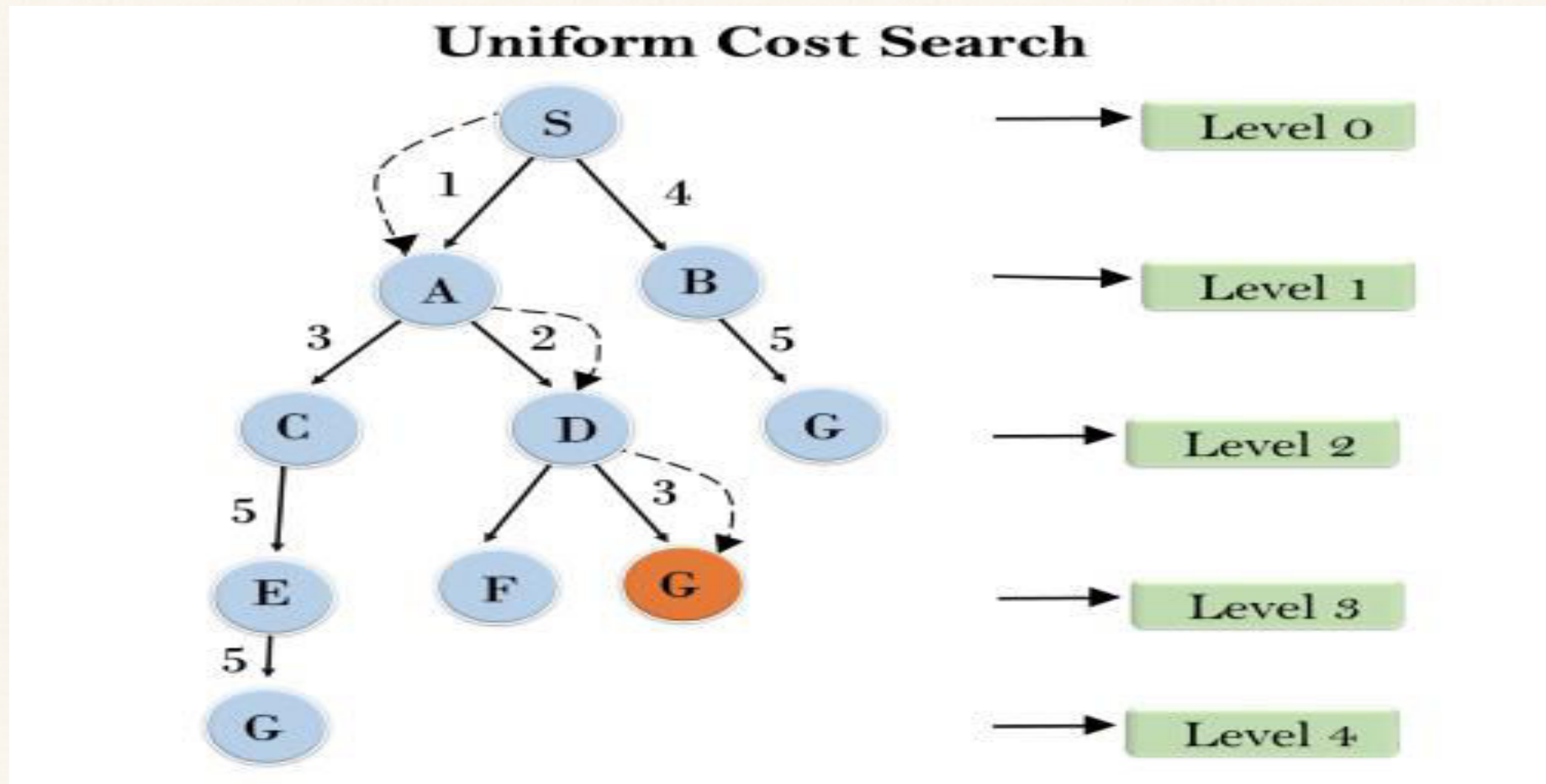
Advantages:

- Uniform cost search is optimal because at every state the path with the least cost is chosen.

Disadvantages:

- It does not care about the number of steps involved in searching and is only concerned about path cost. Due to which this algorithm may be stuck in an infinite loop.

EXAMPLE



Completeness: Uniform-cost search is complete, such as if there is a solution, UCS will find it.

Time Complexity: Let C^* is **Cost of the optimal solution**, and ϵ is each step to get closer to the goal node. Then the number of steps is $= C^*/\epsilon + 1$. Here we have taken $+1$, as we start from state 0 and end to C^*/ϵ .

Hence, the worst-case time complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Space Complexity: The same logic is for space complexity so, the worst-case space complexity of Uniform-cost search is $O(b^{1 + \lceil C^*/\epsilon \rceil})$.

Optimal: Uniform-cost search is always optimal as it only selects a path with the lowest path cost.

5. ITERATIVE DEEPENING DEPTH-FIRST SEARCH

- The iterative deepening algorithm is a combination of DFS and BFS algorithms.
- This search algorithm finds out the best depth limit and does it by gradually increasing the limit until a goal is found.
- This algorithm performs depth-first search up to a certain "depth limit", and it keeps increasing the depth limit after each iteration until the goal node is found.
- This Search algorithm combines the benefits of Breadth-first search's fast search and depth-first search's memory efficiency.
- The iterative search algorithm is useful uninformed search when search space is large, and depth of goal node is unknown.

Advantages:

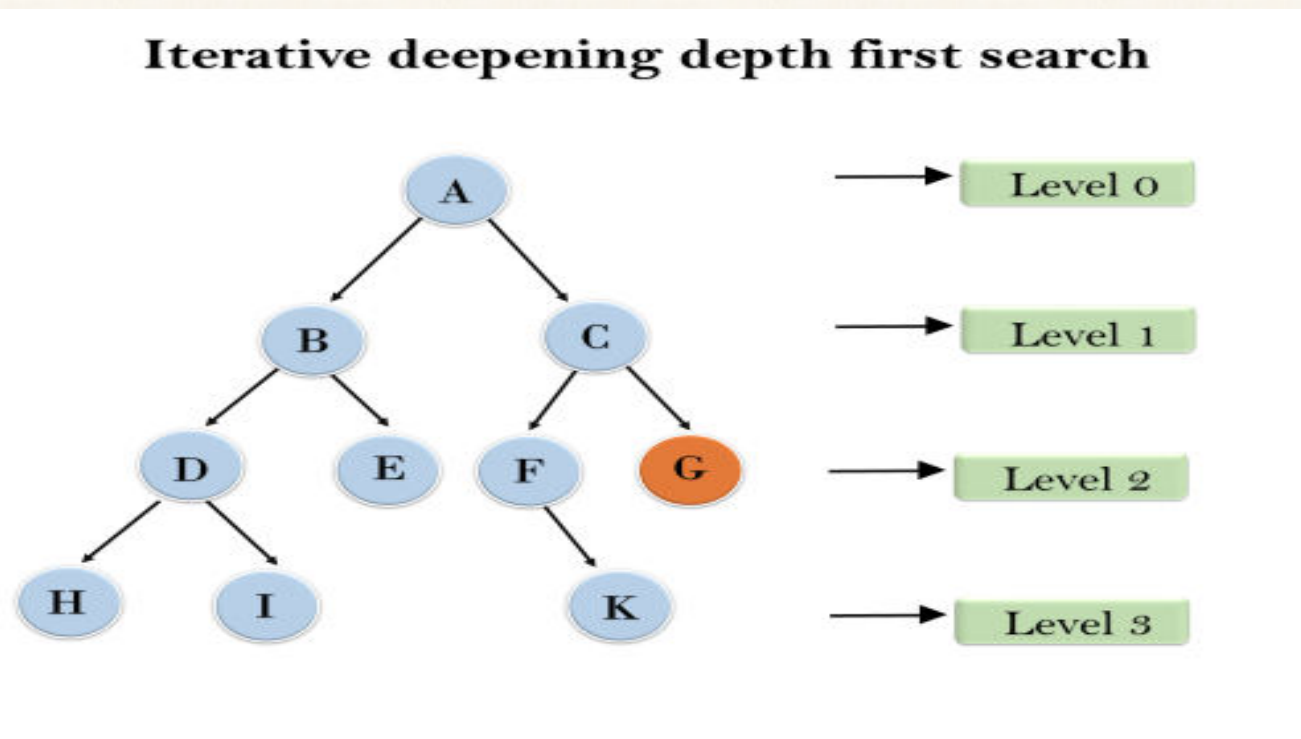
- It combines the benefits of BFS and DFS search algorithm in terms of fast search and memory efficiency.

Disadvantages:

- The main drawback of IDDFS is that it repeats all the work of the previous phase.

EXAMPLE

- Following tree structure is showing the iterative deepening depth-first search.
- IDDFS algorithm performs various iterations until it does not find the goal node. The iteration performed by the algorithm is given as:



1'st Iteration-----> A

2'nd Iteration-----> A, B, C

3'rd Iteration-----> A, B, D, E, C, F, G

4'th Iteration-----> A, B, D, H, I, E, C, F, K, G

In the fourth iteration, the algorithm will find the goal node

Completeness: This algorithm is complete if the branching factor is finite.

Time Complexity: Let's suppose b is the branching factor and depth is d then the worst-case time complexity is $O(b^d)$.

Space Complexity: The space complexity of IDDFS will be $O(bd)$.

Optimal: IDDFS algorithm is optimal if path cost is a non-decreasing function of the depth of the node.

6. BIDIRECTIONAL SEARCH ALGORITHM

- Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search, to find the goal node.
- Bidirectional search replaces one single search graph with two small subgraphs in which one starts the search from an initial vertex and other starts from goal vertex.
- The search stops when these two graphs intersect each other.
- Bidirectional search can use search techniques such as BFS, DFS, DLS, etc.

Advantages:

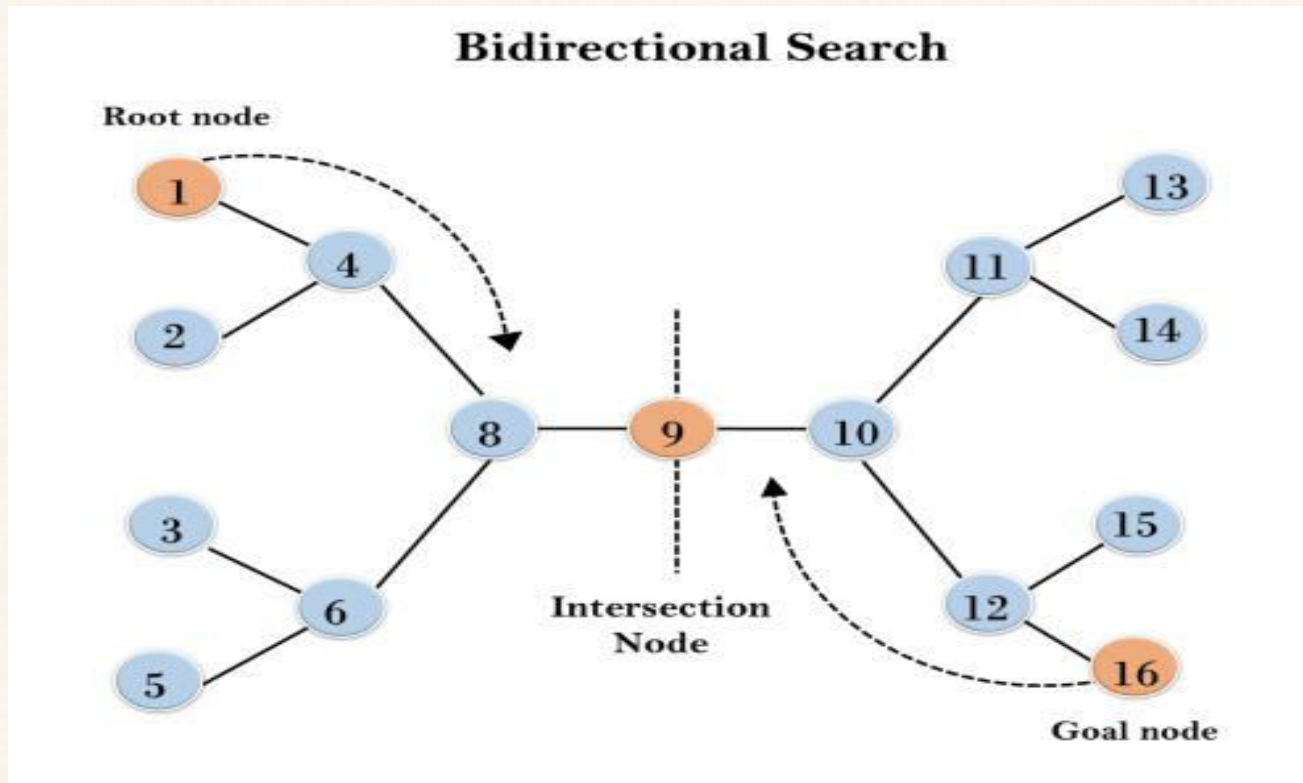
- Bidirectional search is fast.
- Bidirectional search requires less memory

Disadvantages:

- Implementation of the bidirectional search tree is difficult.
- **In bidirectional search, one should know the goal state in advance.**

EXAMPLE

- In the below search tree, bidirectional search algorithm is applied.
- This algorithm divides one graph/tree into two sub-graphs.
- It starts traversing from node 1 in the forward direction and starts from goal node 16 in the backward direction.
- The algorithm terminates at node 9 where two searches meet.



Completeness: Bidirectional Search is complete if we use BFS in both searches.

Time Complexity: Time complexity of bidirectional search using BFS is $O(b^d)$.

Space Complexity: Space complexity of bidirectional search is $O(b^d)$.

Optimal: Bidirectional search is Optimal.

INFORMED SEARCH ALGORITHMS

INFORMED SEARCH ALGORITHMS

The informed search algorithm is more useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called Heuristic search.

Heuristics function

Heuristic is a function which is used in Informed Search, and it finds the most promising path. It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal. The heuristic method, however, might not always give the best solution, but it guaranteed to find a good solution in reasonable time. Heuristic function estimates how close a state is to the goal. It is represented by $h(n)$, and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

$$h(n) \leq h^*(n)$$

Here $h(n)$ is heuristic cost, and $h^*(n)$ is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

PURE HEURISTIC SEARCH:

- Pure heuristic search is the simplest form of heuristic search algorithms.
- It expands nodes based on their heuristic value $h(n)$.
- It maintains two lists, OPEN and CLOSED list.
- In the CLOSED list, it places those nodes which have already expanded and in the OPEN list, it places nodes which have yet not been expanded.
- On each iteration, each node n with the lowest heuristic value is expanded and generates all its successors and n is placed to the closed list. The algorithm continues until a goal state is found.
- In the informed search we will discuss two main algorithms which are given below:
 - **Best First Search Algorithm (Greedy search)**
 - **A* Search Algorithm**

1.) BEST-FIRST SEARCH ALGORITHM (GREEDY SEARCH)

- Greedy best-first search algorithm always selects the path which appears best at that moment.
- It is the combination of depth-first search and breadth-first search algorithms.
- It uses the heuristic function and search. Best-first search allows us to take the advantages of both algorithms.
- With the help of best-first search, at each step, we can choose the most promising node.
- In the best first search algorithm, we expand the node which is closest to the goal node and the closest cost is estimated by heuristic function, i.e.

$$f(n) = g(n).$$

Where, $h(n)$ = estimated cost from node n to the goal.

The greedy best first algorithm is implemented by the priority queue.

BEST FIRST SEARCH ALGORITHM:

- **Step 1:** Place the starting node into the OPEN list.
- **Step 2:** If the OPEN list is empty, Stop and return failure.
- **Step 3:** Remove the node n , from the OPEN list which has the lowest value of $h(n)$, and places it in the CLOSED list.
- **Step 4:** Expand the node n , and generate the successors of node n .
- **Step 5:** Check each successor of node n , and find whether any node is a goal node or not. If any successor node is goal node, then return success and terminate the search, else proceed to Step 6.
- **Step 6:** For each successor node, algorithm checks for evaluation function $f(n)$, and then check if the node has been in either OPEN or CLOSED list. If the node has not been in both list, then add it to the OPEN list.
- **Step 7:** Return to Step 2.

ADVANTAGES:

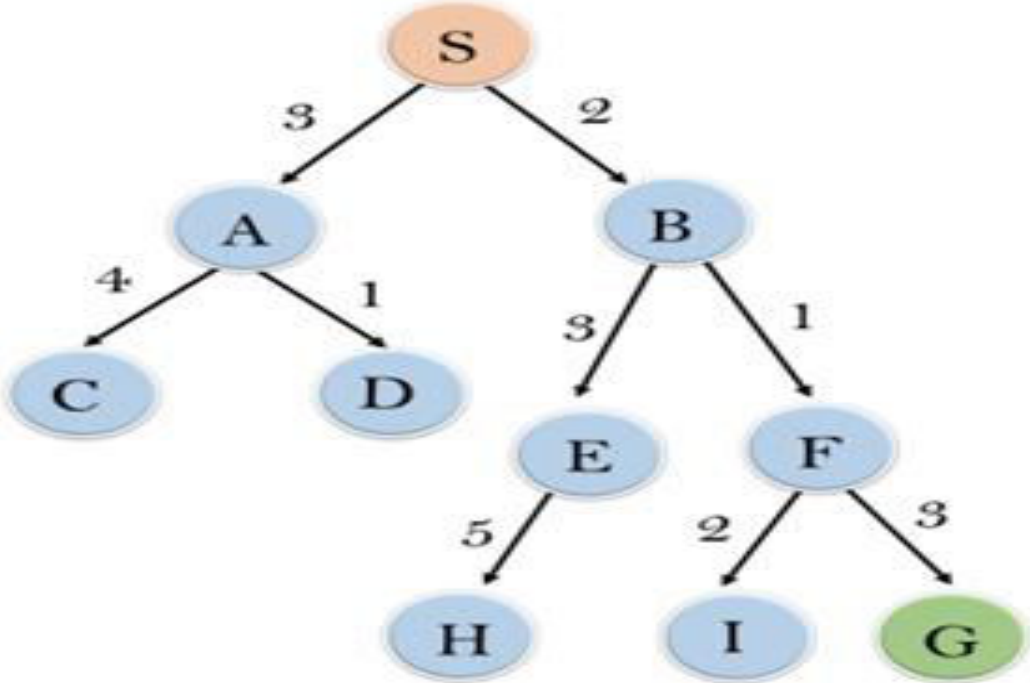
- Best first search can switch between BFS and DFS by gaining the advantages of both the algorithms.
- This algorithm is more efficient than BFS and DFS algorithms.

DISADVANTAGES:

- It can behave as an unguided depth-first search in the worst case scenario.
- It can get stuck in a loop as DFS.
- This algorithm is not optimal.

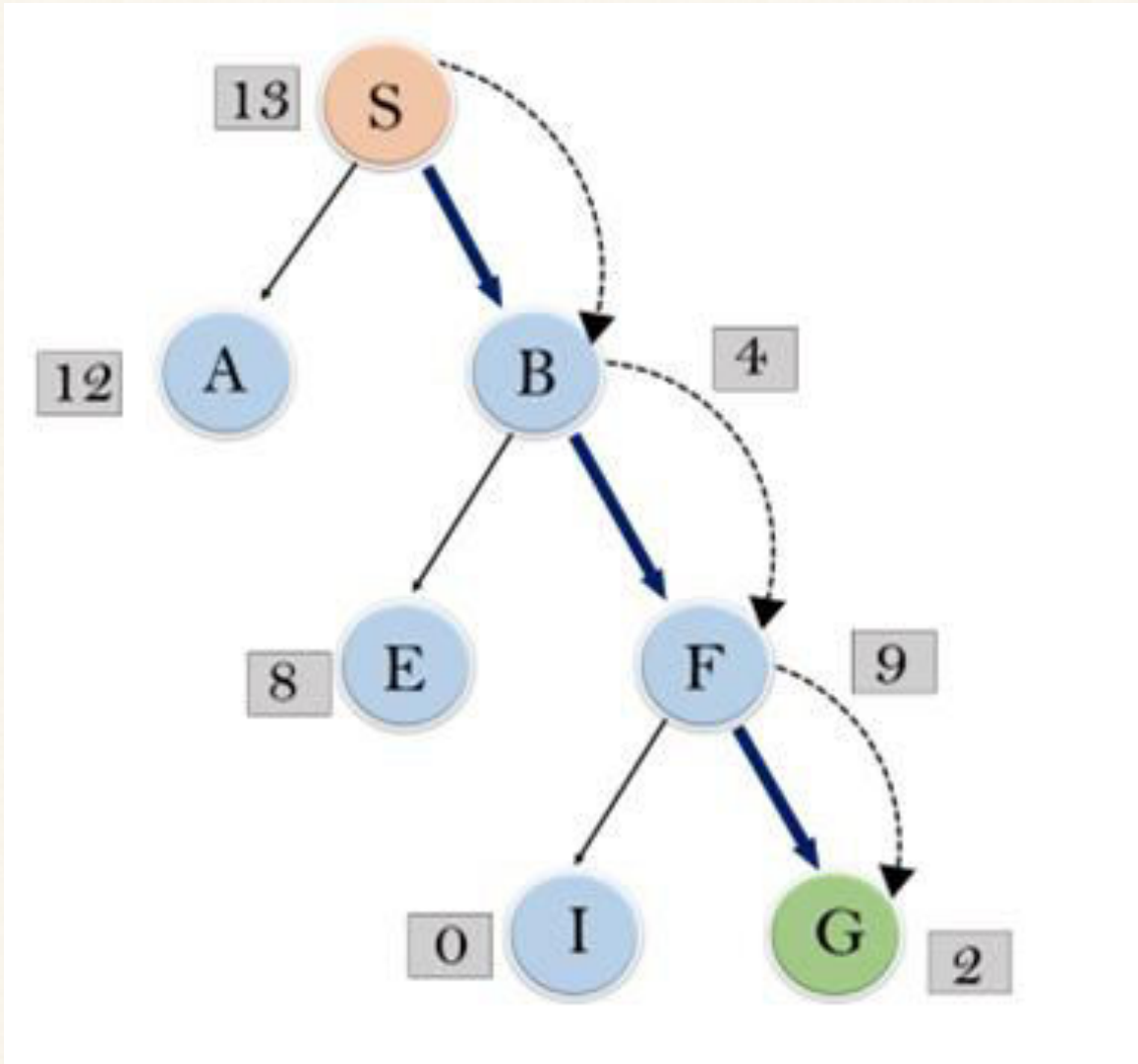
EXAMPLE

Consider the below search problem, and we will traverse it using greedy best-first search. At each iteration, each node is expanded using evaluation function $f(n)=h(n)$, which is given in the below table.



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

In this search example, we are using two lists which are **OPEN** and **CLOSED** Lists. Following are the iteration for traversing the above example.



Expand the nodes of S and put in the CLOSED list

Initialization: Open [A, B], Closed [S]

Iteration 1: Open [A], Closed [S, B]

Iteration 2: Open [E, F, A], Closed [S, B]

: Open [E, A], Closed [S, B, F]

Iteration 3: Open [I, G, E, A], Closed [S, B, F]

: Open [I, E, A], Closed [S, B, F, G]

Hence the final solution path will be:

S-----> B----->F-----> G

Time Complexity: The worst-case time complexity of Greedy best first search is $O(b^m)$.

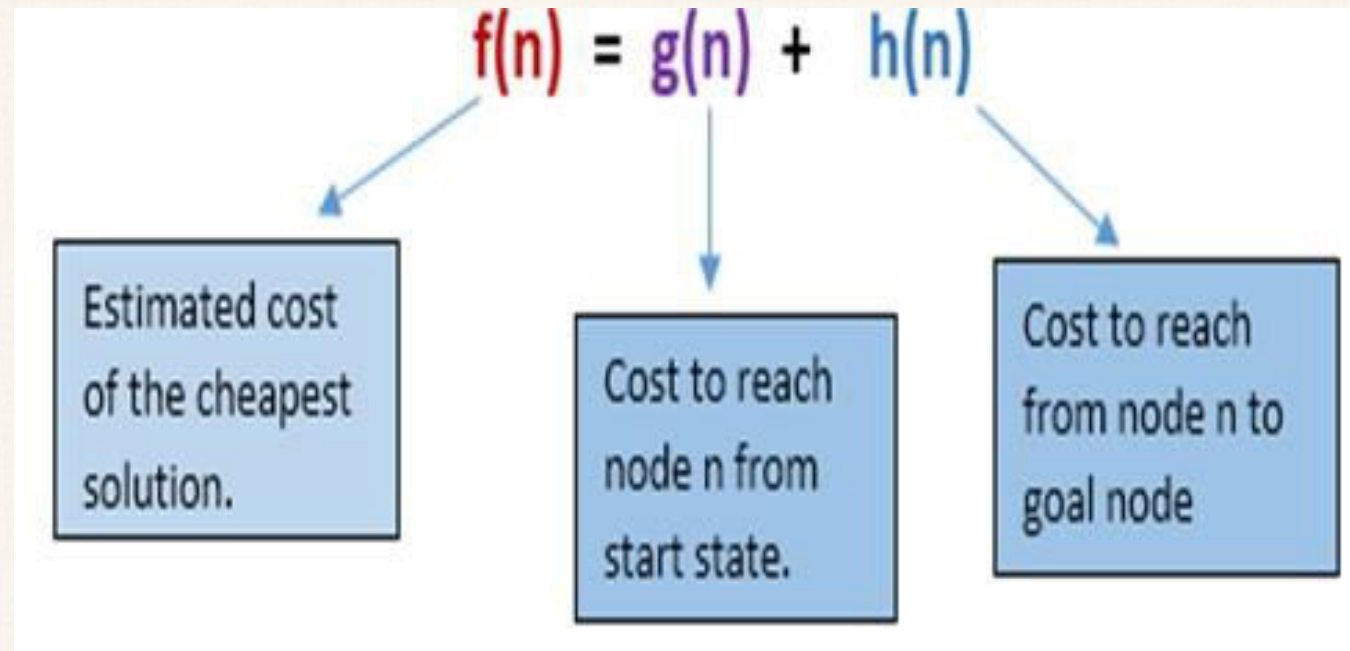
Space Complexity: The worst-case space complexity of Greedy best first search is $O(b^m)$. Where, m is the maximum depth of the search space.

Complete: Greedy best-first search is also incomplete, even if the given state space is finite.

Optimal: Greedy best first search algorithm is not optimal.

2.) A* SEARCH ALGORITHM:

- A* search is the most commonly known form of best-first search.
- It uses heuristic function $h(n)$, and cost to reach the node n from the start state $g(n)$ to find the **best shortest path**.
- It has combined features of UCS and greedy best-first search, by which it solve the problem efficiently.
- A* search algorithm finds the shortest path through the search space using the heuristic function.
- This search algorithm expands less search tree and provides **optimal result faster**.
- A* algorithm is similar to UCS except that it uses $g(n)+h(n)$ instead of $g(n)$.
- In A* search algorithm, we use search heuristic as well as the cost to reach the node.
- Hence, we can combine both costs as following, and this sum is called as a **fitness number**.



NOTE: At each point in the search space, only those nodes is expanded which have the lowest value of $f(n)$, and the algorithm terminates when the goal node is found.

ALGORITHM OF A* SEARCH:

- **Step1:** Place the starting node in the OPEN list.
- **Step 2:** Check if the OPEN list is empty or not, if the list is empty then return failure and stops.
- **Step 3:** Select the node from the OPEN list which has the smallest value of evaluation function ($g+h$), if node n is goal node then return success and stop, otherwise
- **Step 4:** Expand node n and generate all of its successors, and put n into the closed list. For each successor n' , check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for n' and place into Open list.
- **Step 5:** Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest $g(n')$ value.
- **Step 6:** Return to **Step 2**.

ADVANTAGES:

- A* search algorithm is the best algorithm than other search algorithms.
- A* search algorithm is optimal and complete.
- This algorithm can solve very complex problems.

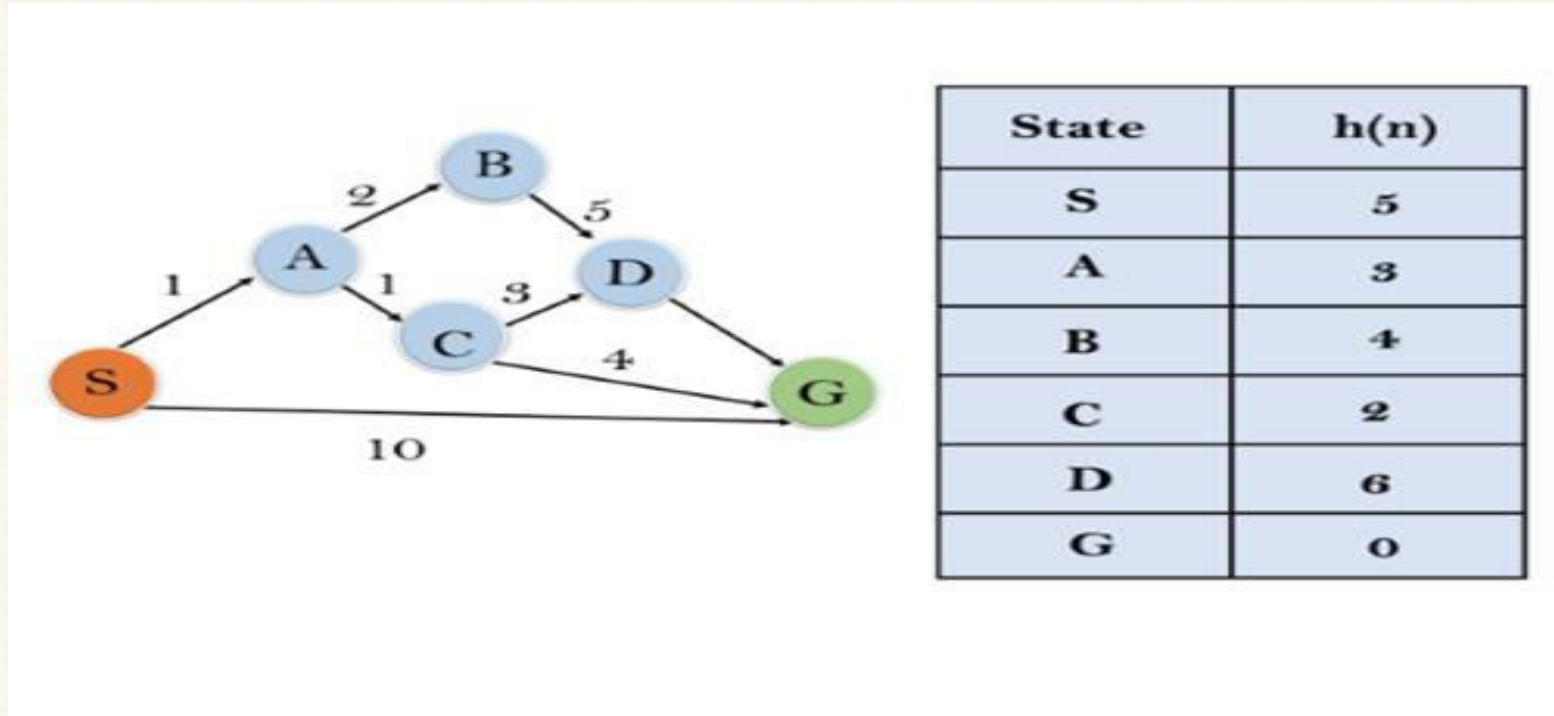
DISADVANTAGES:

- It does not always produce the shortest path as it mostly based on heuristics and approximation.
- A* search algorithm has some complexity issues.
- The main drawback of A* is memory requirement as it keeps all generated nodes in the memory, so it is not practical for various large-scale problems.

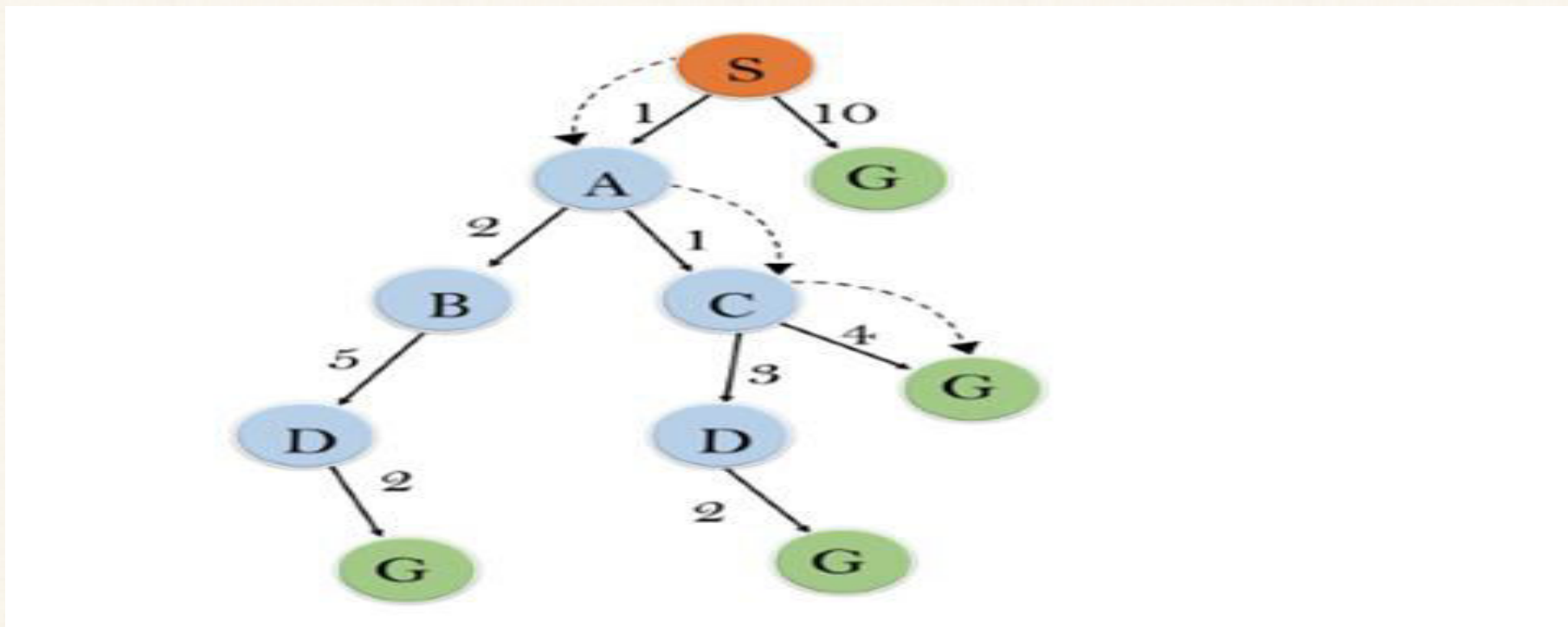
EXAMPLE

- In this example, we will traverse the given graph using the A* algorithm.
- The heuristic value of all states is given in the below table so we will calculate the $f(n)$ of each state using the formula $f(n) = g(n) + h(n)$, where $g(n)$ is the cost to reach any node from start state.

Here we will use OPEN and CLOSED list.



SOLUTION:



Initialization: $\{(S, 5)\}$

Iteration1: $\{(S \rightarrow A, 4), (S \rightarrow G, 10)\}$

Iteration2: $\{(S \rightarrow A \rightarrow C, 4), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration3: $\{(S \rightarrow A \rightarrow C \rightarrow G, 6), (S \rightarrow A \rightarrow C \rightarrow D, 11), (S \rightarrow A \rightarrow B, 7), (S \rightarrow G, 10)\}$

Iteration 4 will give the final result, as $S \rightarrow A \rightarrow C \rightarrow G$ it provides the optimal path with cost 6.

POINTS TO REMEMBER:

- A* algorithm returns the path which occurred first, and it does not search for all remaining paths.
- The efficiency of A* algorithm depends on the quality of heuristic.
- A* algorithm expands all nodes which satisfy the condition $f(n) \leq c(n)$

COMPLETE:

A* algorithm is complete as long as:

- Branching factor is finite.
- Cost at every action is fixed.

OPTIMAL: A* search algorithm is optimal if it follows below two conditions:

- **Admissible:** the first condition requires for optimality is that $h(n)$ should be an admissible heuristic for A* tree search. An admissible heuristic is optimistic in nature.
- **Consistency:** Second required condition is consistency for only A* graph-search.

Thank you.



HEURISTIC FUNCTIONS

HEURISTIC FUNCTIONS

- As we have already seen that an informed search makes use of heuristic functions in order to reach the goal node in a more prominent way. Therefore, there are several pathways in a search tree to reach the goal node from the current node. The selection of a good heuristic function matters certainly. A good heuristic function is determined by its efficiency. More is the information about the problem, more is the processing time.
- Some toy problems, such as 8-puzzle, 8-queen, tic-tac-toe, etc., can be solved more efficiently with the help of a heuristic function. Let's see how:
- Consider the following 8-puzzle problem where we have a start state and a goal state. Our task is to slide the tiles of the current/start state and place it in an order followed in the goal state. There can be four moves either **left, right, up, or down**. There can be several ways to convert the current/start state to the goal state, but, we can use a heuristic function $h(n)$ to solve the problem more efficiently.

8 PUZZLE PROBLEM USING HEURISTIC FUNCTION

1	2	3
8	6	
7	5	4

Start State

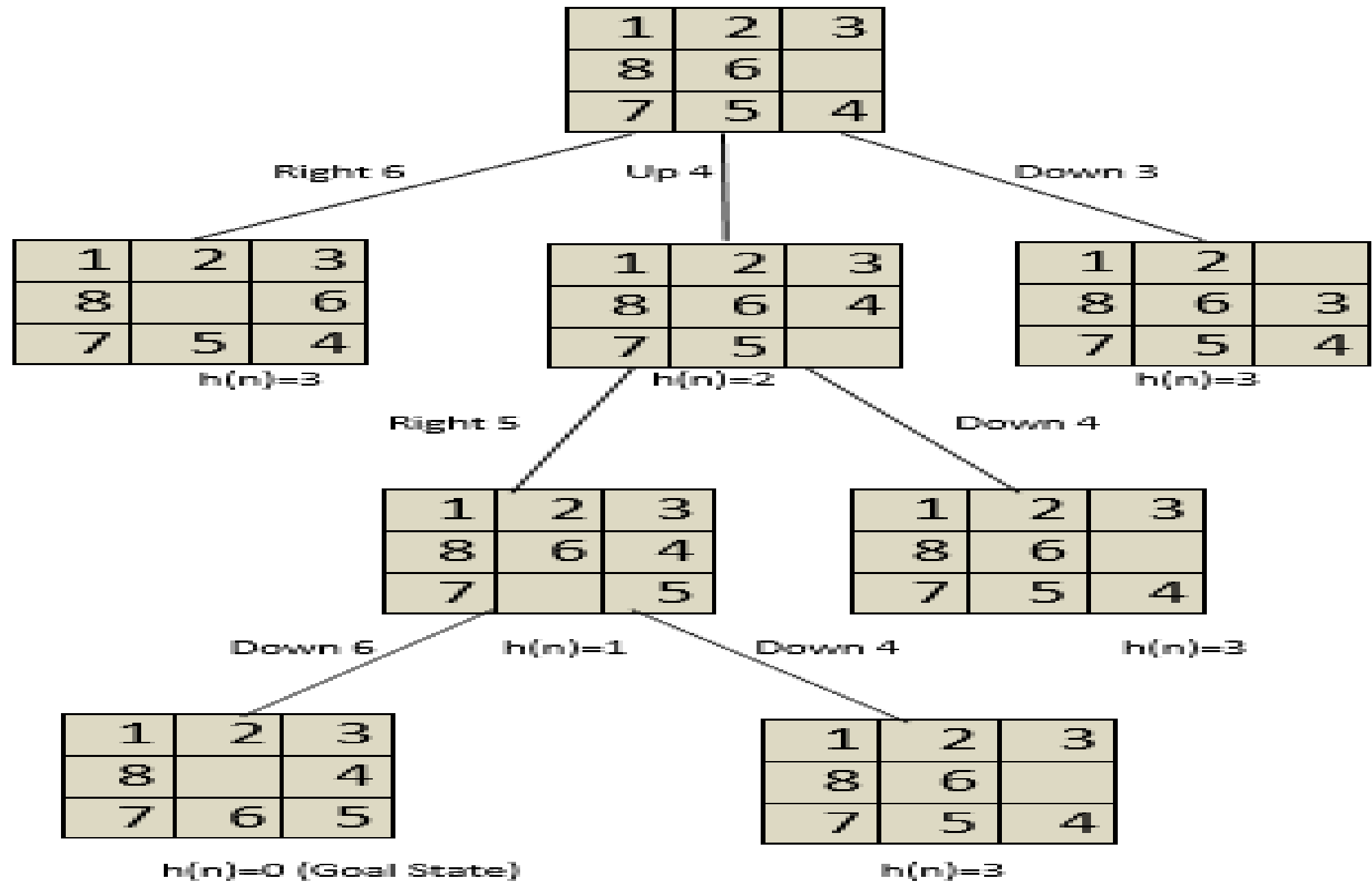
1	2	3
8		4
7	6	5

Goal State

A heuristic function for the 8-puzzle problem is defined below:

$h(n)$ =Number of tiles out of position.

So, there is total of three tiles out of position i.e., 6,5 and 4. Do not count the empty tile present in the goal state). i.e. $h(n)=3$. Now, we require to minimize the value of **$h(n) = 0$** .



- It is seen from the above state space tree that the goal state is minimized from $h(n)=3$ to $h(n)=0$
- However, we can create and use several heuristic functions as per the requirement. It is also clear from the above example that a heuristic function $h(n)$ can be defined as the information required to solve a given problem more efficiently. The information can be related to the **nature of the state, cost of transforming from one state to another, goal node characteristics**, etc., which is expressed as a heuristic function.

PROPERTIES OF A HEURISTIC SEARCH ALGORITHM

Use of heuristic function in a heuristic search algorithm leads to following properties of a heuristic search algorithm:

- **Admissible Condition:** An algorithm is said to be admissible, if it returns an optimal solution.
- **Completeness:** An algorithm is said to be complete, if it terminates with a solution (if the solution exists).
- **Dominance Property:** If there are two admissible heuristic algorithms **A1** and **A2** having **h1** and **h2** heuristic functions, then **A1** is said to dominate **A2** if **h1** is better than **h2** for all the values of node **n**.
- **Optimality Property:** If an algorithm is **complete**, **admissible**, and **dominating** other algorithms, it will be the best one and will definitely give an optimal solution

LOCAL SEARCH ALGORITHMS AND OPTIMIZATION PROBLEMS

The informed and uninformed search expands the nodes systematically in two ways:

- keeping different paths in the memory and
- selecting the best suitable path,

Which leads to a solution state required to reach the goal node. But beyond these “**classical search algorithms**,” we have some “**local search algorithms**” where the **path cost does not matters, and only focus on solution-state needed to reach the goal node.**

A local search algorithm completes its task by traversing on a single current node rather than multiple paths and following the neighbours of that node generally.

Although local search algorithms are not systematic, still they have the following two advantages:

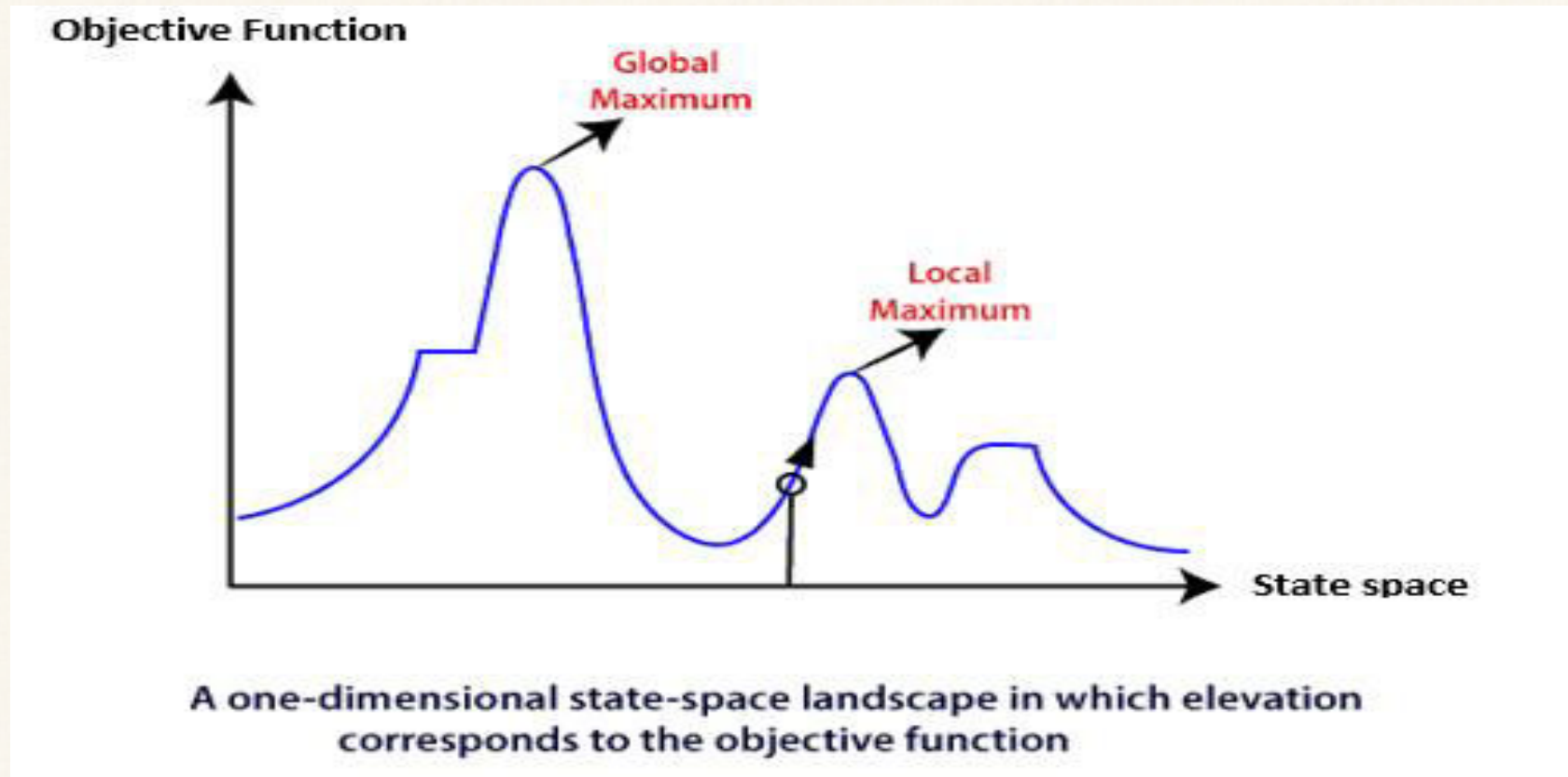
- Local search algorithms use a very little or constant amount of memory as they operate only on a single path.
- Most often, they find a reasonable solution in large or infinite state spaces where the classical or systematic algorithms do not work.

Does the local search algorithm work for a pure optimized problem?

Yes, the local search algorithm works for pure optimized problems. A pure optimization problem is one where all the nodes can give a solution. But the target is to find the best state out of all according to the **objective function**. Unfortunately, the pure optimization problem fails to find high-quality solutions to reach the goal state from the current state.

WORKING OF A LOCAL SEARCH ALGORITHM

- **Location:** It is defined by the state.
- **Elevation:** It is defined by the value of the objective function or heuristic cost function.



The local search algorithm explores the above landscape by finding the following two points:

- **Global Minimum:** If the elevation corresponds to the cost, then the task is to find the lowest valley, which is known as **Global Minimum**.
- **Global Maxima:** If the elevation corresponds to an objective function, then it finds the highest peak which is called as **Global Maxima**. It is the highest point in the valley.
- **We will understand the working of these points better in Hill-climbing search.**

Below are some different types of local searches:

- Hill-climbing Search
- Simulated Annealing
- Local Beam Search

HILL CLIMBING ALGORITHM

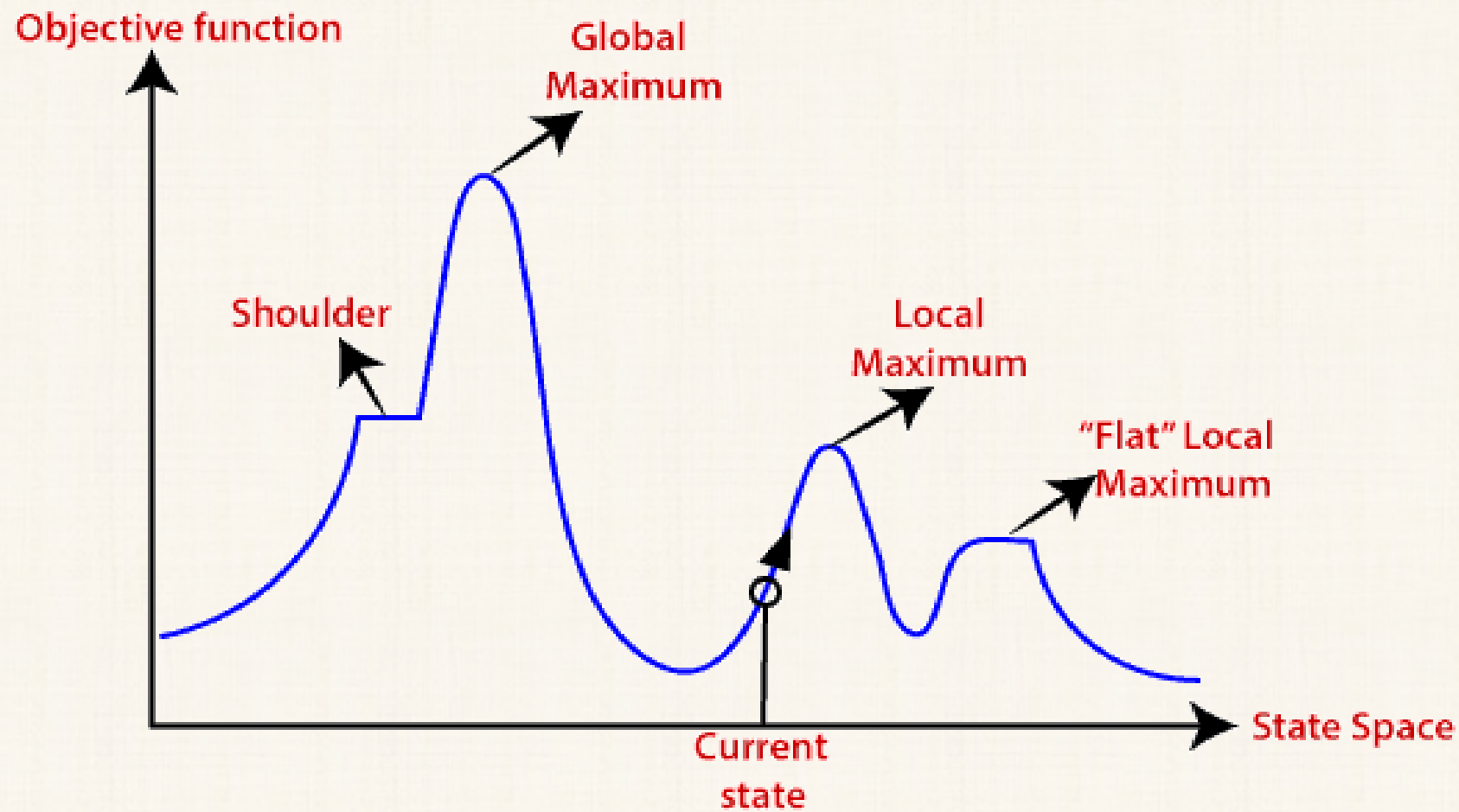
HILL CLIMBING ALGORITHM

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak value where no neighbour has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing the mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance travelled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbour state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

STATE-SPACE LANDSCAPE OF HILL CLIMBING ALGORITHM

To understand the concept of hill climbing algorithm, consider the below landscape representing the **goal state/peak** and the **current state** of the climber. The topographical regions shown in the figure can be defined as:

- **Global Maximum:** It is the highest point on the hill, which is the goal state.
- **Local Maximum:** It is the peak higher than all other peaks but lower than the global maximum.
- **Flat local maximum:** It is the flat area over the hill where it has no uphill or downhill. It is a saturated point of the hill.
- **Shoulder:** It is also a flat area where the summit is possible.
- **Current state:** It is the current position of the person.

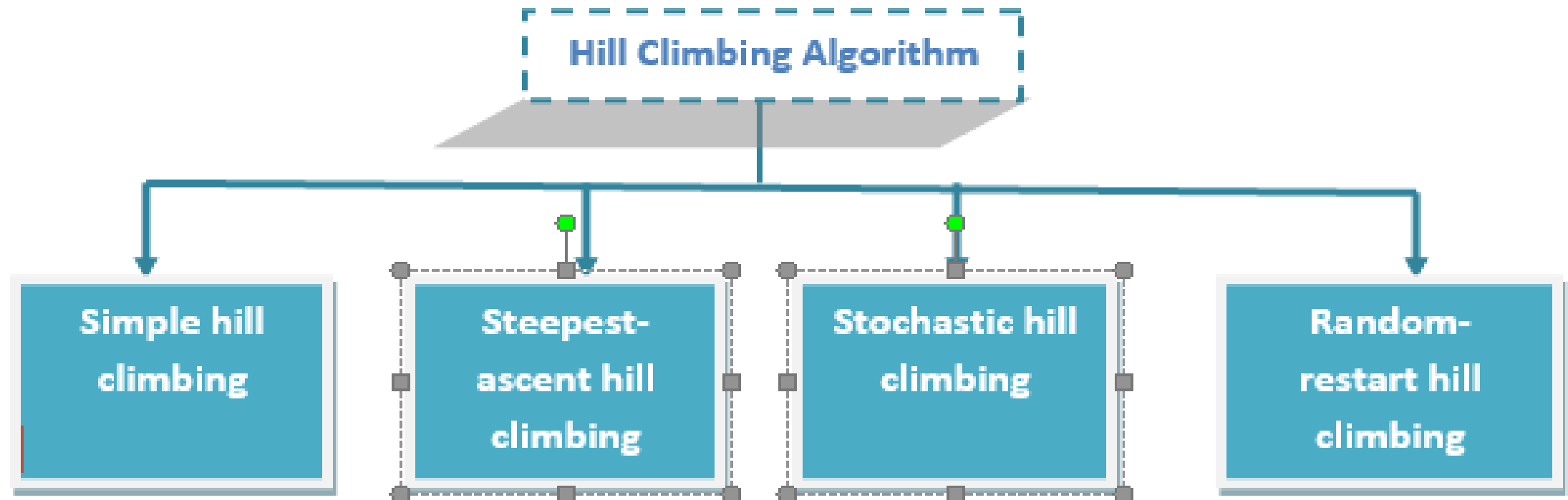


A one-dimensional state-space landscape in which elevation corresponds to the objective function

TYPES
OF
HILL CLIMBING
SEARCH ALGORITHM

TYPES OF HILL CLIMBING SEARCH ALGORITHM

- Simple hill climbing
- Steepest-ascent hill climbing
- Stochastic hill climbing
- Random-restart hill climbing



SIMPLE HILL CLIMBING SEARCH

Simple hill climbing is the simplest technique to climb a hill. The task is to reach the highest peak of the mountain. Here, the movement of the climber depends on his move/steps. If he finds his next step better than the previous one, he continues to move else remain in the same state. This search focus only on his previous and next step.

SIMPLE HILL CLIMBING ALGORITHM

1. Create a **CURRENT** node, **NEIGHBOUR** node, and a **GOAL** node.
2. If the **CURRENT node=GOAL node**, return **GOAL** and terminate the search.
3. Else **CURRENT node<= NEIGHBOUR node**, move ahead.
4. Loop until the goal is not reached or a point is not found.

STEEPEST-ASCENT HILL CLIMBING

Steepest-ascent hill climbing is different from simple hill climbing search. Unlike simple hill climbing search, it considers all the successive nodes, compares them, and choose the node which is closest to the solution. Steepest hill climbing search is similar to **best-first search** because it focuses on each node instead of one.

NOTE: Both simple, as well as steepest-ascent hill climbing search, fails when there is no closer node.

STEEPEST-ASCENT HILL CLIMBING ALGORITHM

1. Create a **CURRENT** node and a **GOAL** node.
2. If the **CURRENT node=GOAL** node, return **GOAL** and terminate the search.
3. Loop until a better node is not found to reach the solution.
4. If there is any better successor node present, expand it.
5. When the **GOAL** is attained, return **GOAL** and terminate.

STOCHASTIC HILL CLIMBING

Stochastic hill climbing does not focus on all the nodes. It selects one node at random and decides whether it should be expanded or search for a better one.

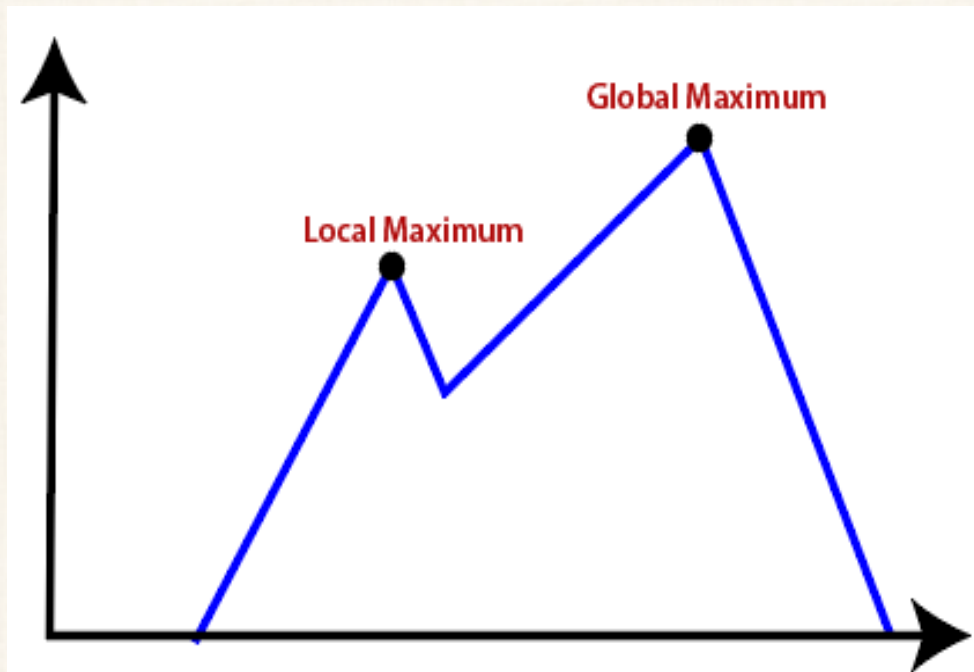
RANDOM-RESTART HILL CLIMBING

Random-restart algorithm is based on **try and try strategy**. It iteratively searches the node and selects the best one at each step until the goal is not found. The success depends most commonly on the shape of the hill. If there are few plateaus, local maxima, and ridges, it becomes easy to reach the destination.

LIMITATIONS OF HILL CLIMBING ALGORITHM

Hill climbing algorithm is a fast and furious approach. It finds the solution state rapidly because it is quite easy to improve a bad state. But, there are following limitations of this search:

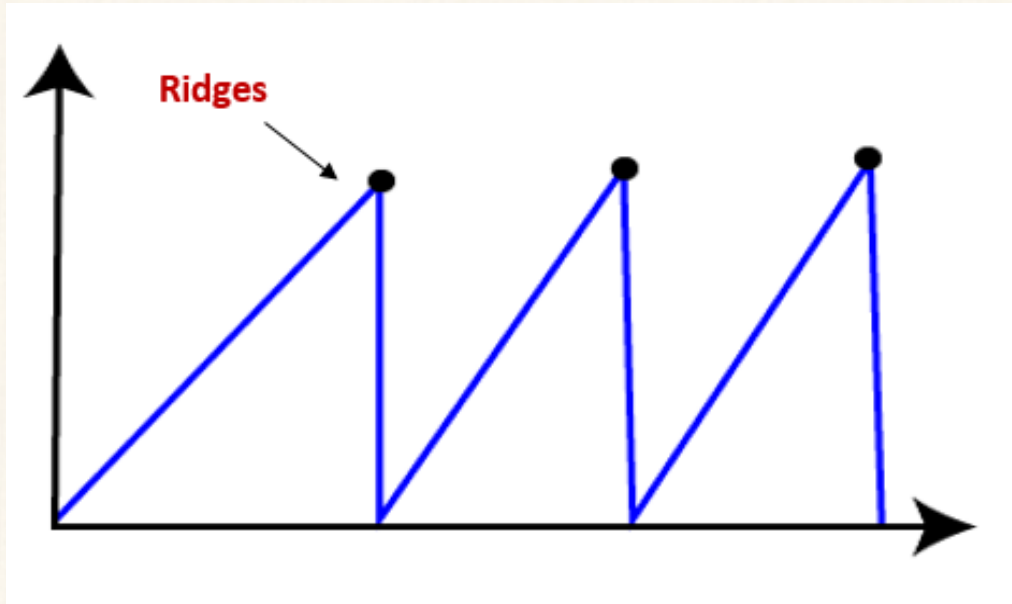
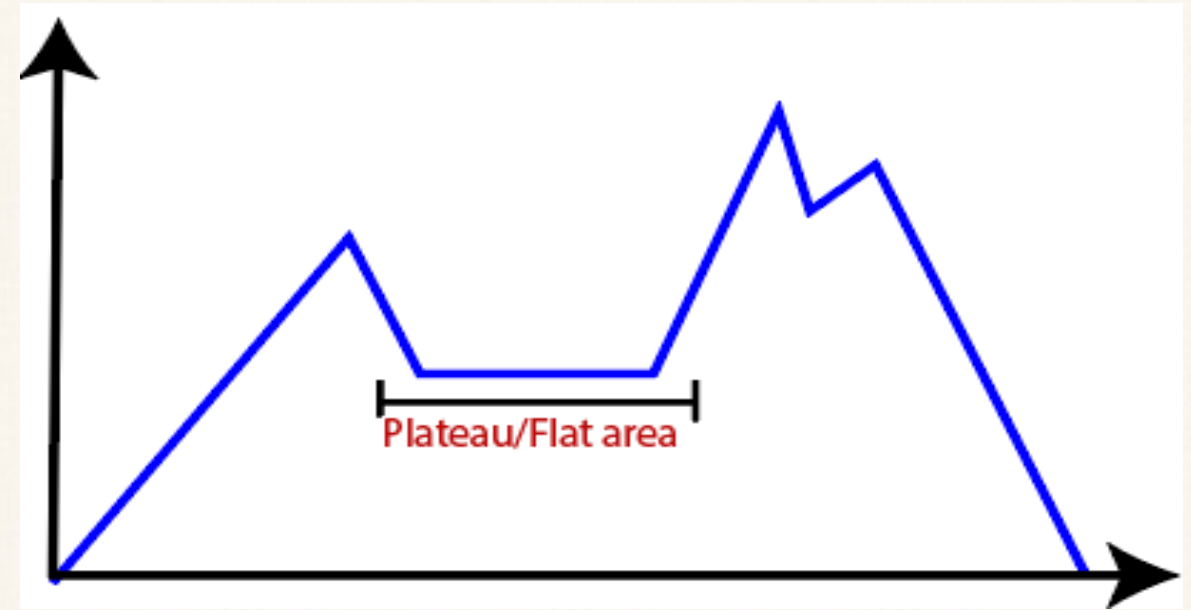
- **Local Maxima:** It is that peak of the mountain which is highest than all its neighbouring states but lower than the global maxima. It is not the goal peak because there is another peak higher than it.



SOLUTION :- Back tracking

- **Plateau:** It is a flat surface area where no uphill exists. It becomes difficult for the climber to decide that in which direction he should move to reach the goal point. Sometimes, the person gets lost in the flat area.

SOLUTION :- Taking a large steps or small steps



- **Ridges:** It is a challenging problem where the person finds two or more local maxima of the same height commonly. It becomes difficult for the person to navigate the right point and stuck to that point itself.

SOLUTION :- By using Bi directional

FEATURES OF HILL CLIMBING:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states.

SIMULATED ANNEALING

SIMULATED ANNEALING

Simulated annealing is similar to the hill climbing algorithm. It works on the current situation. It picks **a random move** instead of picking **the best move**. If the move leads to the improvement of the current situation, it is always accepted as a step towards the solution state, else it accepts the move having **a probability less than 1**. This search technique was first used in **1980** to solve **VLSI layout** problems. It is also applied for factory scheduling and other large optimization tasks.

LOCAL BEAM SEARCH

Local beam search is quite different from random-restart search. It keeps track of **k** states instead of just one. It selects **k** randomly generated states, and expand them at each step. If any state is a goal state, the search stops with success. Else it selects the best **k** successors from the complete list and repeats the same process. In random-restart search where each search process runs independently, but in local beam search, the necessary information is shared between the parallel search processes.

DISADVANTAGES OF LOCAL BEAM SEARCH

- This search can suffer from a lack of diversity among the k states.
- It is an expensive version of hill climbing search.

LOCAL SEARCH IN CONTINUOUS SPACES

Gradient Descent is one of the most used machine learning algorithms in the industry. And yet it confounds a lot of newcomers.

What is a Cost Function?

It is a **function** that measures the performance of a model for any given data. **Cost Function** quantifies the error between predicted values and expected values and presents it in the form of a single real number.

After making a hypothesis with initial parameters, we calculate the Cost function. And with a goal to reduce the cost function, we modify the parameters by using the Gradient descent algorithm over the given data. Here's the mathematical representation for it:

Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$

Parameters: θ_0, θ_1

Cost Function: $J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$

Goal: $\underset{\theta_0, \theta_1}{\text{minimize}} J(\theta_0, \theta_1)$

What is Gradient Descent?

Let's say you are playing a game where the players are at the top of a mountain, and they are asked to reach the lowest point of the mountain. Additionally, they are blindfolded. So, what approach do you think would make you reach the lake?

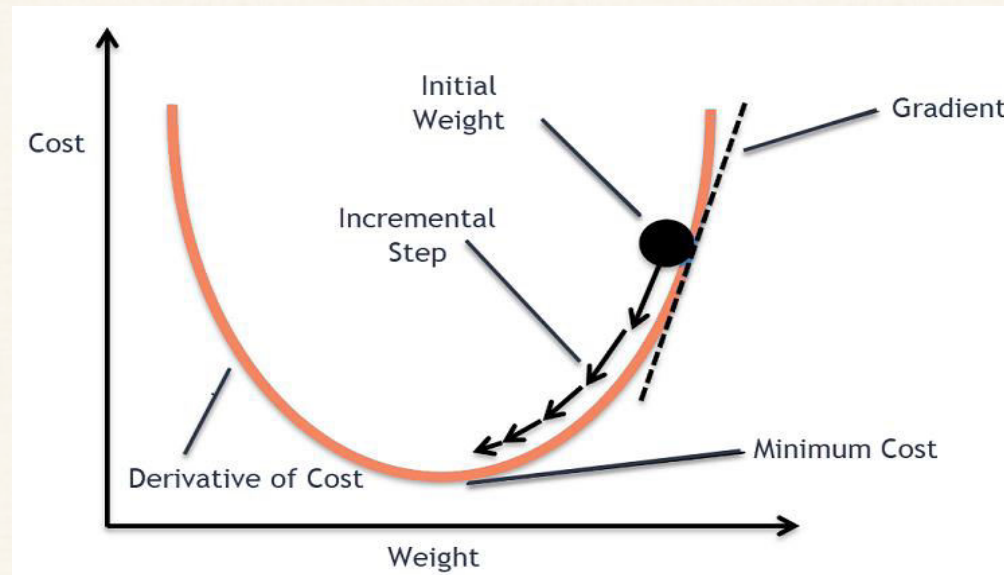
The best way is to observe the ground and find where the land descends. From that position, take a step in the descending direction and iterate this process until we reach the lowest point.



Gradient descent is an iterative optimization algorithm for finding the local minimum of a function.

To find the local minimum of a function using gradient descent, we must take steps proportional to the negative of the gradient (move away from the gradient) of the function at the current point. If we take steps proportional to the positive of the gradient (moving towards the gradient), we will approach a local maximum of the function, and the procedure is called **Gradient Ascent**.

Gradient descent was originally proposed by **CAUCHY** in 1847. It is also known as steepest descent.



The goal of the gradient descent algorithm is to minimize the given function (say cost function). To achieve this goal, it performs two steps iteratively:

1. **Compute the gradient** (slope), the first order derivative of the function at that point
2. **Make a step (move) in the direction opposite to the gradient**, opposite direction of slope increases from the current point by alpha times the gradient at that point

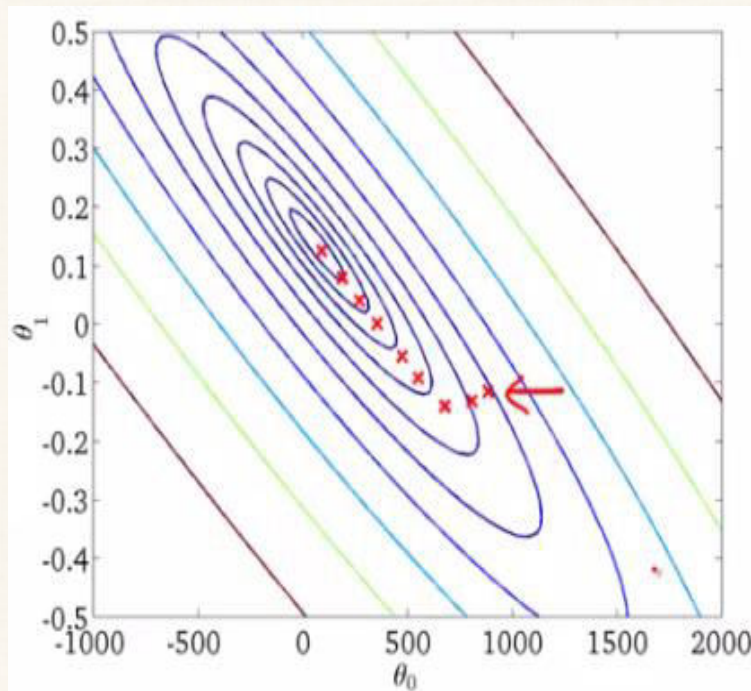
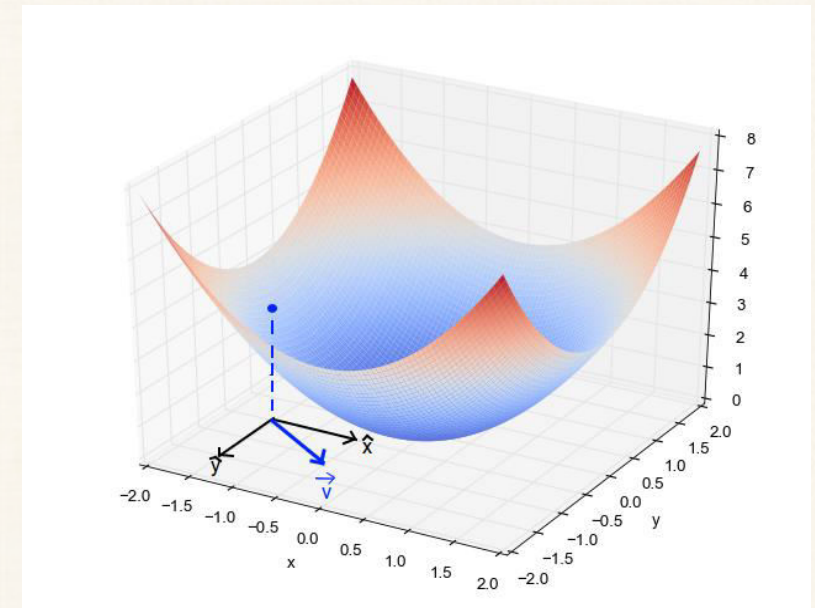
Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}

Alpha is called **Learning rate** – a tuning parameter in the optimization process. It decides the length of the steps.

Plotting the Gradient Descent Algorithm

When we have a single parameter (theta), we can plot the dependent variable cost on the y-axis and theta on the x-axis. If there are two parameters, we can go with a 3-D plot, with cost on one axis and the two parameters (thetas) along the other two axes.



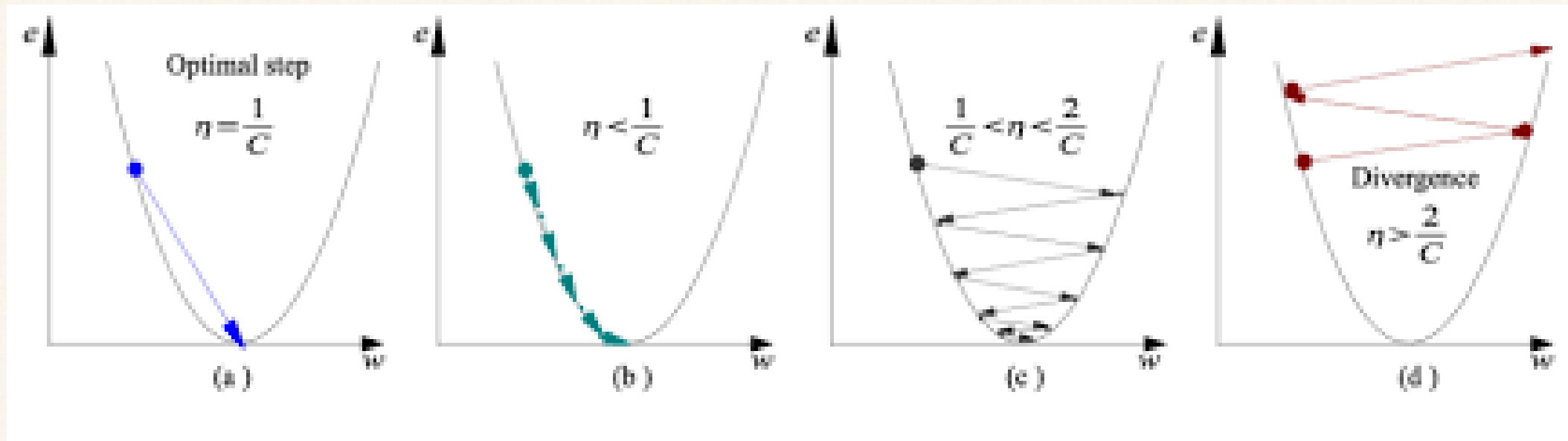
It can also be visualized by using **Contours**. This shows a 3-D plot in two dimensions with parameters along both axes and the response as a contour. The value of the response increases away from the center and has the same value along with the rings. The response is directly proportional to the distance of a point from the center (along a direction).

Alpha – The Learning Rate

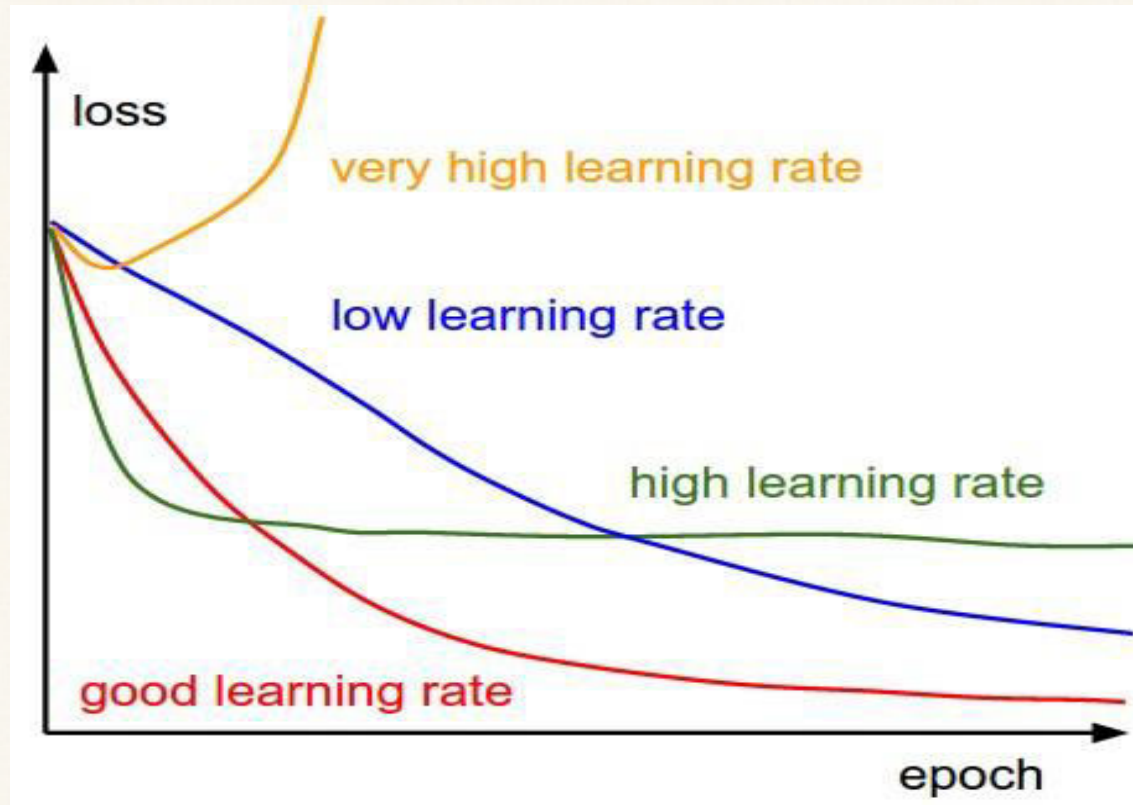
We have the direction we want to move in, now we must decide the size of the step we must take.

***It must be chosen carefully to end up with local minima.**

- If the learning rate is too high, we might **OVERSHOOT** the minima and keep bouncing, without reaching the minima
- If the learning rate is too small, the training might turn out to be too long



1. a) Learning rate is optimal, model converges to the minimum
2. b) Learning rate is too small, it takes more time but converges to the minimum
3. c) Learning rate is higher than the optimal value, it overshoots but converges ($1/C < \eta < 2/C$)
4. d) Learning rate is very large, it overshoots and diverges, moves away from the minima, performance decreases on learning



SEARCHING
WITH
NONDETERMINISTIC
ACTIONS

SEARCHING WITH NONDETERMINISTIC ACTIONS

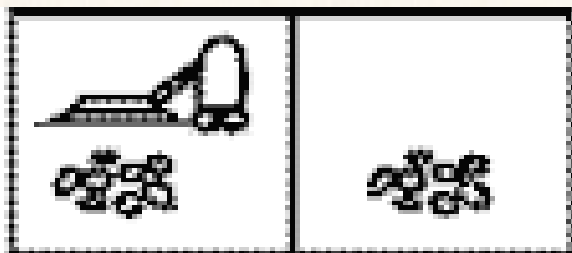
- The environment is **fully observable** and **deterministic** and that the agent knows what the effects of each action are.
- Therefore, the agent can calculate exactly which state results from any sequence of actions and always knows which state it is in.
- Its precepts provide no new information after each action, although of course they tell the agent the initial state.
- When the **environment** is **either partially observable** or **nondeterministic** (or both), precepts become useful.
- In a partially observable environment, every percept helps narrow down the set of possible states the agent might be in, thus making it easier for the agent to achieve its goals.
- When the environment is nondeterministic, precepts tell the agent which of the possible outcomes of its actions has actually occurred.

- In both cases, the future precepts cannot be determined in advance and the agent's future actions will depend on those future precepts.
- So, the solution to a problem is not a sequence but a contingency plan (also known as a strategy) that specifies what to do depending on what precepts are received.

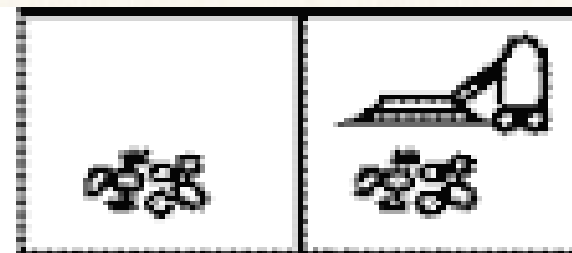
THE ERRATIC VACUUM WORLD

- As an example, we use the vacuum world, recall that the state space has eight states, as shown in Figure. There are three **actions** — **Left**, **Right**, and **Suck** — and the goal is to clean up all the dirt (states 7 and 8).
- If the environment is **observable**, **deterministic**, and completely known, then the problem is trivially solvable by any of the algorithm and the solution is an action sequence.

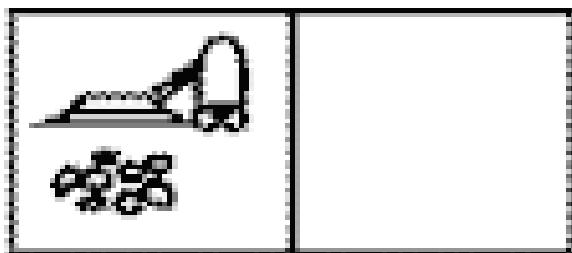
1



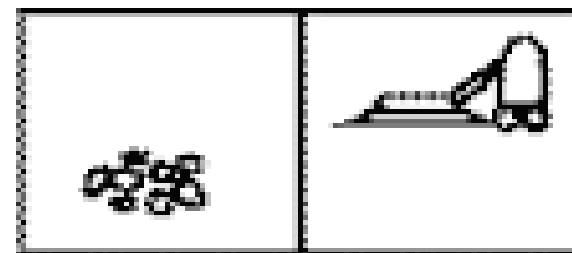
2



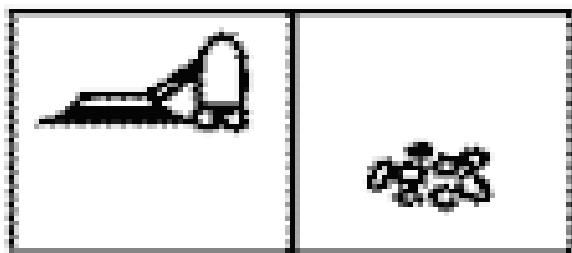
3



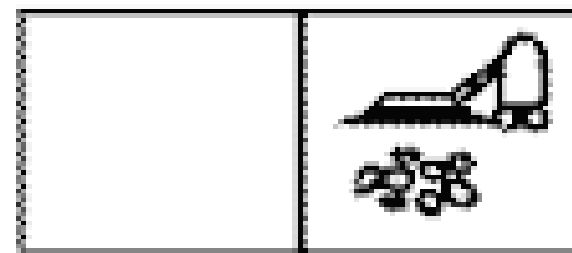
4



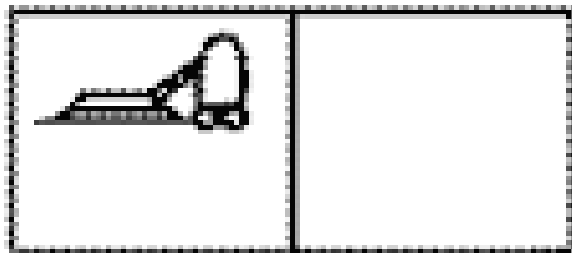
5



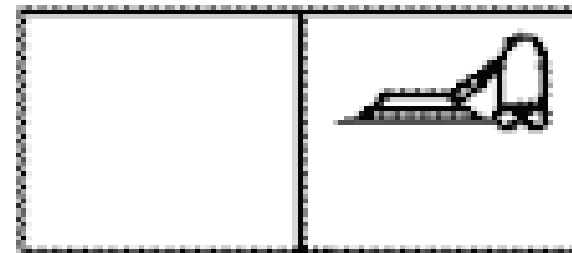
6



7



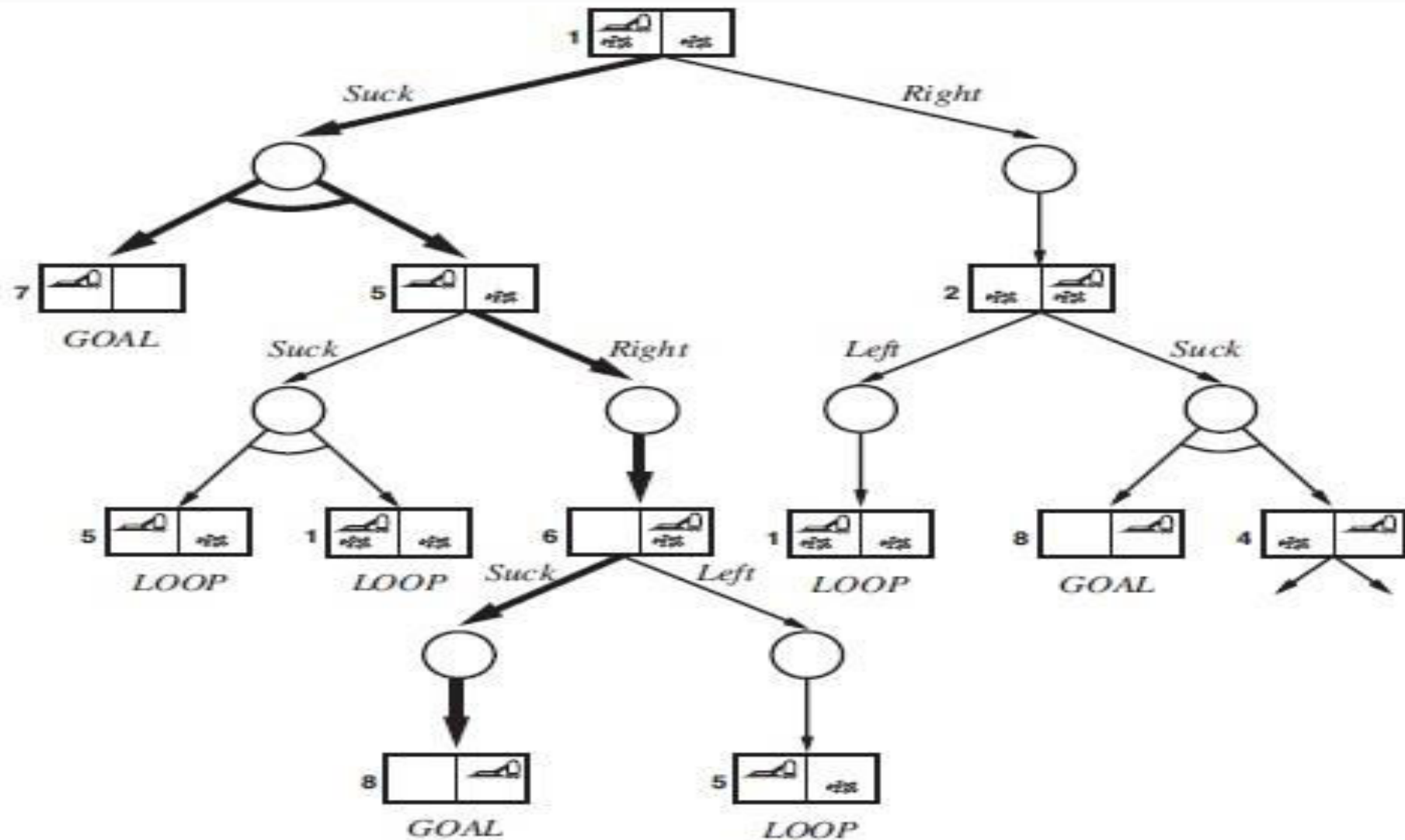
8



The next question is how to find contingent solutions to **nondeterministic** problems.

- we begin by **constructing** search **trees**, but here the trees have a different character.
- In a deterministic environment, the only branching is introduced by the agent's own choices in each state. We call these nodes **OR nodes**.
- In the vacuum world, for example, at an **OR node** the agent chooses **Left or Right or Suck**.
- In a nondeterministic environment, **branching** is also introduced by the **environment's choice** of outcome for each action. We call these **nodes AND nodes**.

A solution for an **AND-OR** search problem is a subtree that (1) has a goal node at every leaf, (2) specifies one action at each of its OR nodes, and (3) includes every outcome branch at each of its AND nodes.



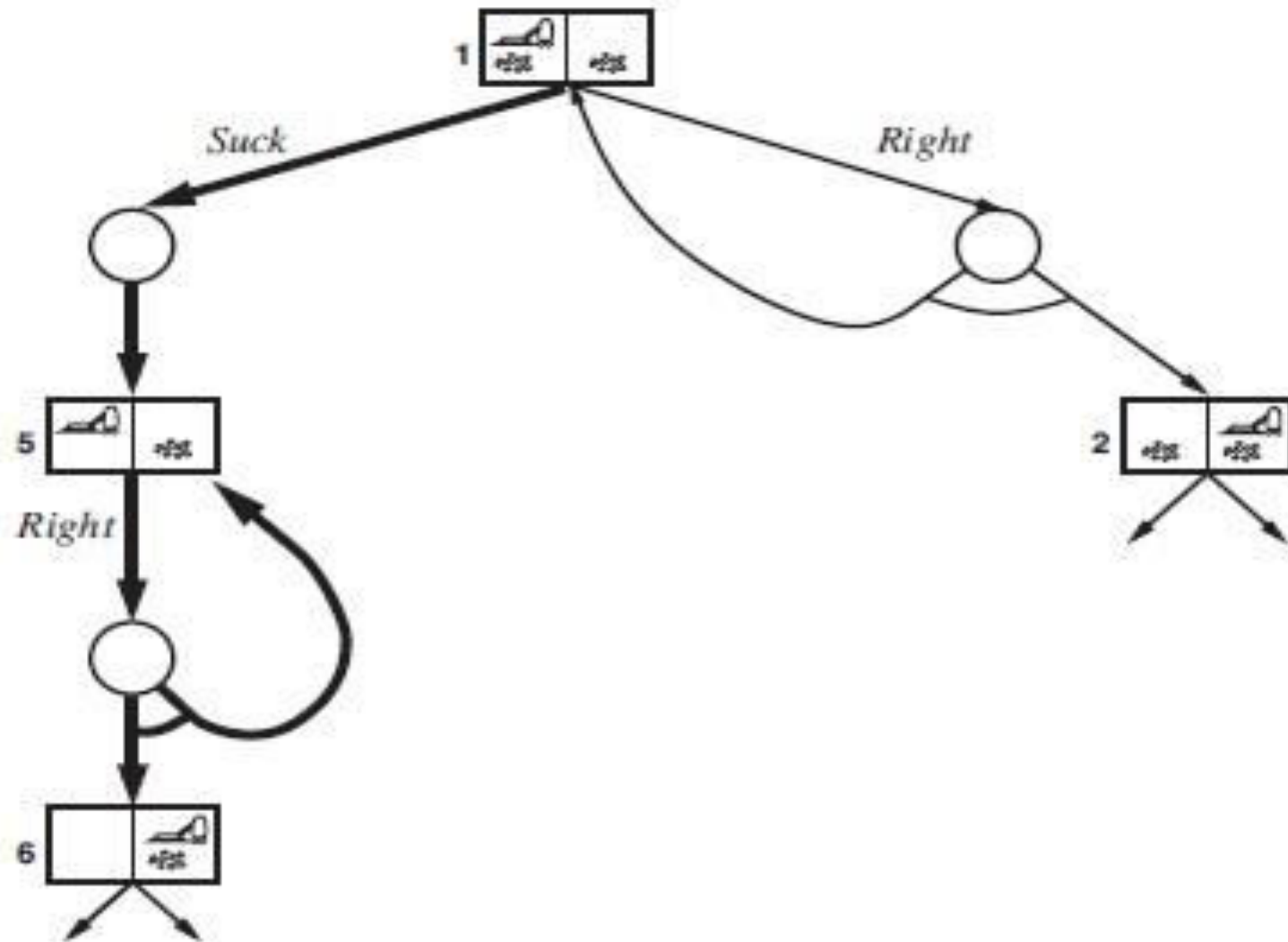
The first two levels of the search tree for the erratic vacuum world. State nodes are OR nodes where some action must be chosen. At the AND nodes, shown as circles, every outcome must be handled, as indicated by the arc linking the outgoing branches. The solution found is shown in bold lines.

function AND-OR-GRAPH-SEARCH(*problem*) *returns a conditional plan, or failure*
OR-SEARCH(*problem*.INITIAL-STATE, *problem*, [])

function OR-SEARCH(*state*, *problem*, *path*) *returns a conditional plan, or failure*
if *problem*.GOAL-TEST(*state*) **then return** the empty plan
if *state* is on *path* **then return failure**
for each *action* **in** *problem*.ACTIONS(*state*) **do**
 plan $\leftarrow$ AND-SEARCH(RESULTS(*state*, *action*), *problem*, [*state* | *path*])
 if *plan* $\neq$ failure **then return** [*action* | *plan*]
return failure

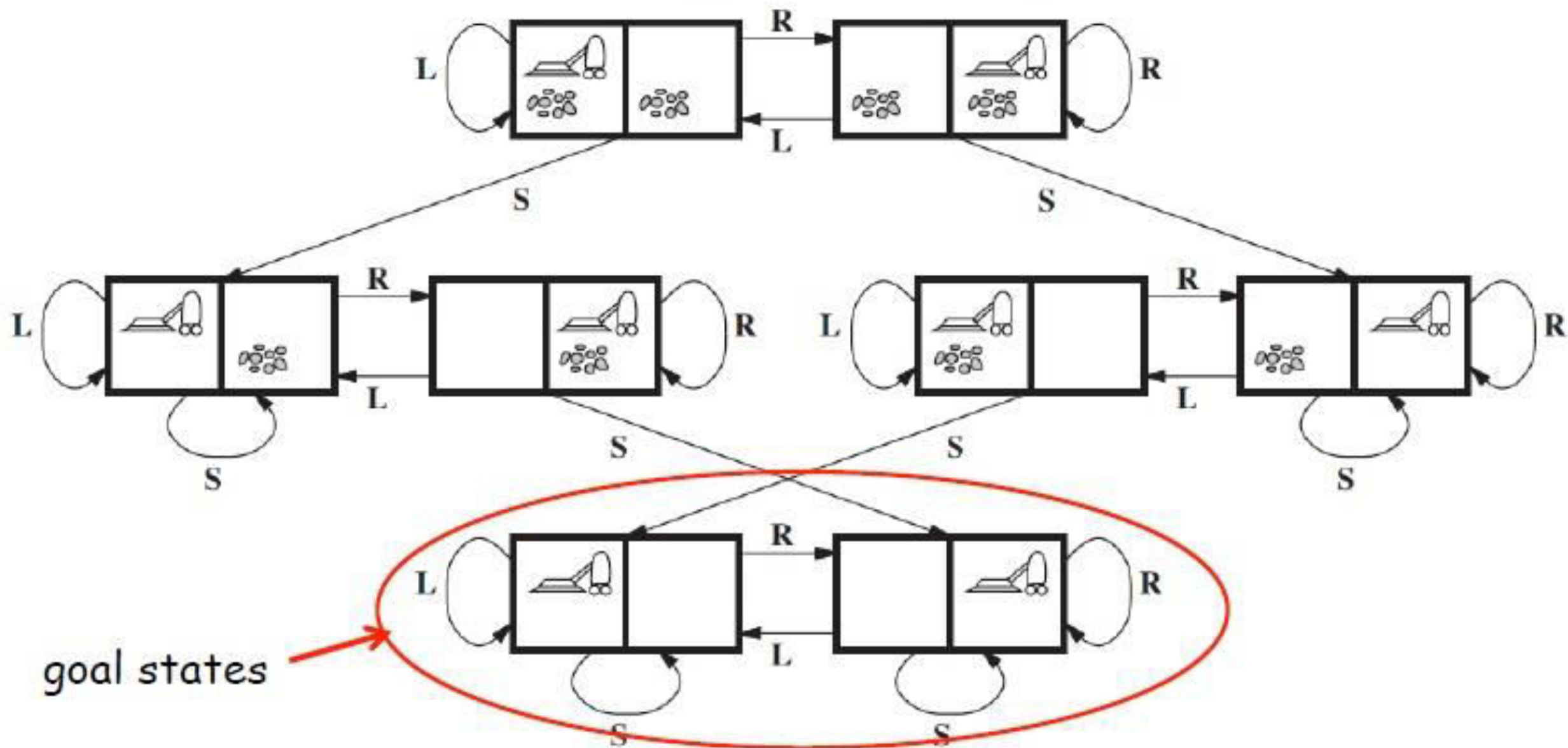
function AND-SEARCH(*states*, *problem*, *path*) *returns a conditional plan, or failure*
for each s_i **in** *states* **do**
 *plan*_{*i*} $\leftarrow$ OR-SEARCH(s_i , *problem*, *path*)
 if *plan*_{*i*} = failure **then return failure**
return [if s_1 **then** *plan*₁ **else if** s_2 **then** *plan*₂ **else** ... if s_{n-1} **then** *plan*_{*n-1*} **else** *plan*_{*n*}]

An algorithm for searching AND-OR graphs generated by nondeterministic environments. It returns a conditional plan that reaches a goal state in all circumstances. (The notation [x | l] refers to the list formed by adding object x to the front of list l .)

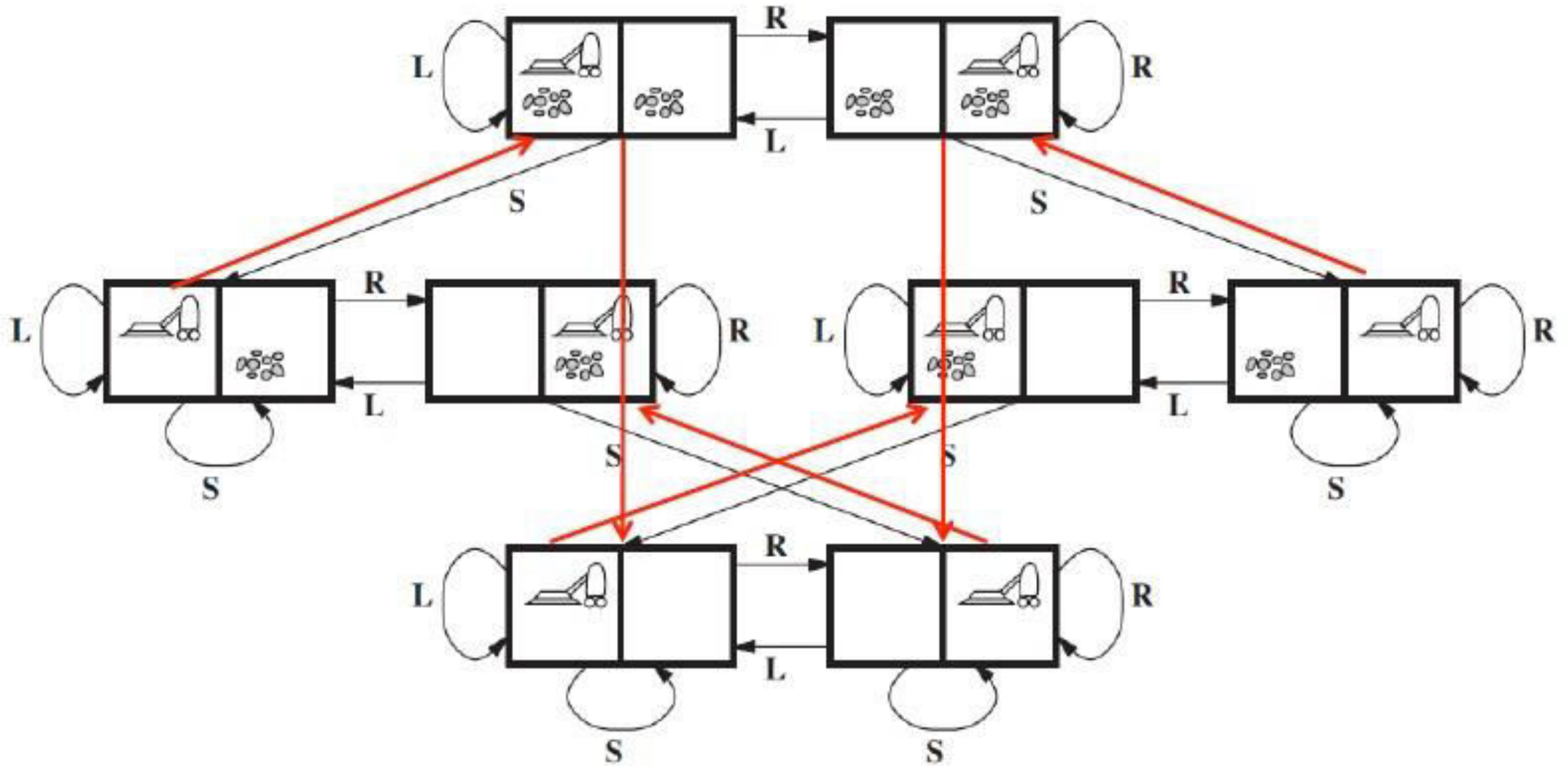


Part of the search graph for the slippery vacuum world, where we have shown (some) cycles explicitly. All solutions for this problem are cyclic plans because there is no way to move reliably.

Deterministic actions, observable, known



Suck action: applied to a dirty square, cleans the square but sometimes cleans adjacent square; applied to a clean square, sometimes deposits dirt.

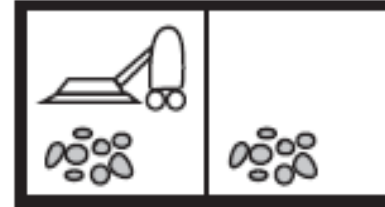


SENSOR-LESS VACUUM WORLD

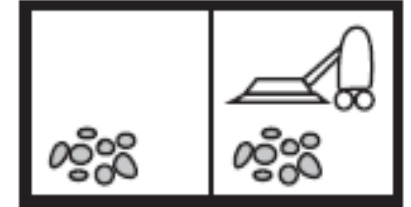
Assume belief states are the same but no location or dust sensors

- Initial state = {1, 2, 3, 4, 5, 6, 7, 8}
- Action: Right
 - Result = {2, 4, 6, 8}
- Right, Suck
 - Result = {4, 8}
- Right, Suck, Left, Suck
 - Result = {7} guaranteed!

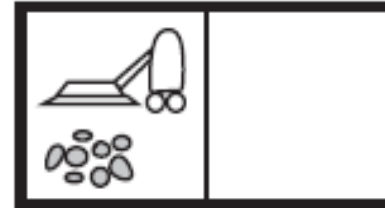
1



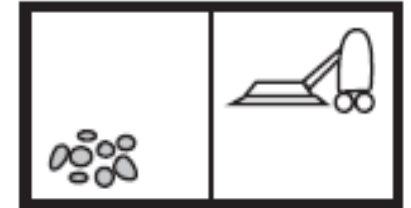
2



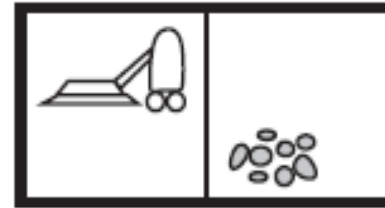
3



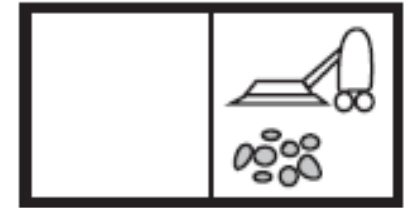
4



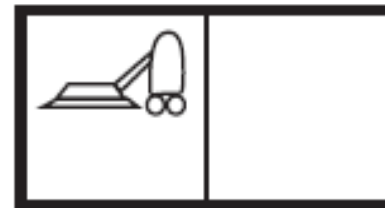
5



6



7



8

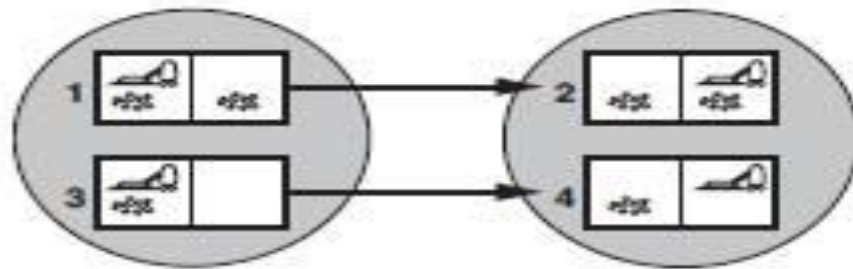


WHAT IS THE CORRESPONDING SENSOR-LESS PROBLEM

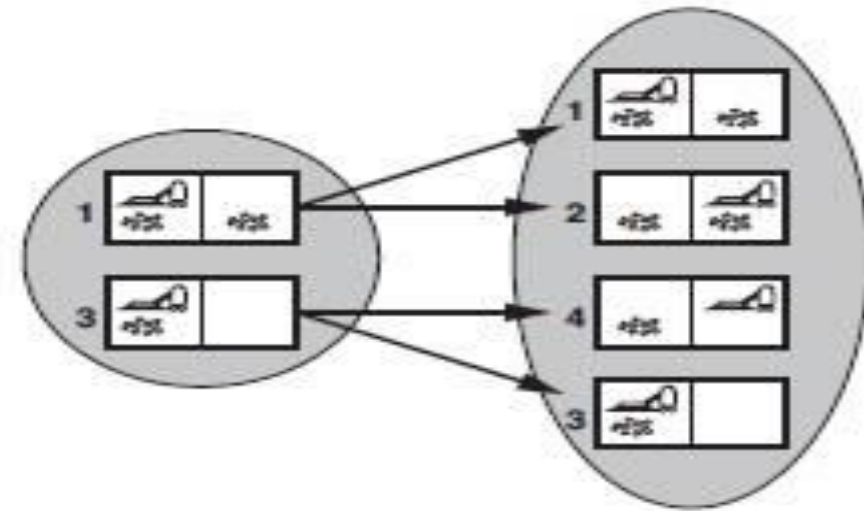
- States Belief States: every possible set of physical states
- If N physical states, number of belief states can be $2N$
- Initial State: Typically, the set of all states in P
- Actions: Consider $\{s_1, s_2\}$
- If $Actionsp(s_1) \neq Actionsp(s_2)$ should we take the Union of both sets of actions or the Intersection?
- Union if all actions are legal, intersection if not

TRANSITION MODEL

- Union of all states that $Resultp(s)$ returns for all states, s , in your current belief state
- $b' = Result\ b, a = \{s' : s' = Resultp(s, a) \text{ and } s \in b\}$
- This is the prediction step, $Predict\ p(b, a)$
- Goal-Test: If all physical states in belief state satisfy $Goal-Testp$
- Path cost □ Tricky in general. Consider what happens if actions in different physical states have different costs. For now assume cost of an action is the same in all states



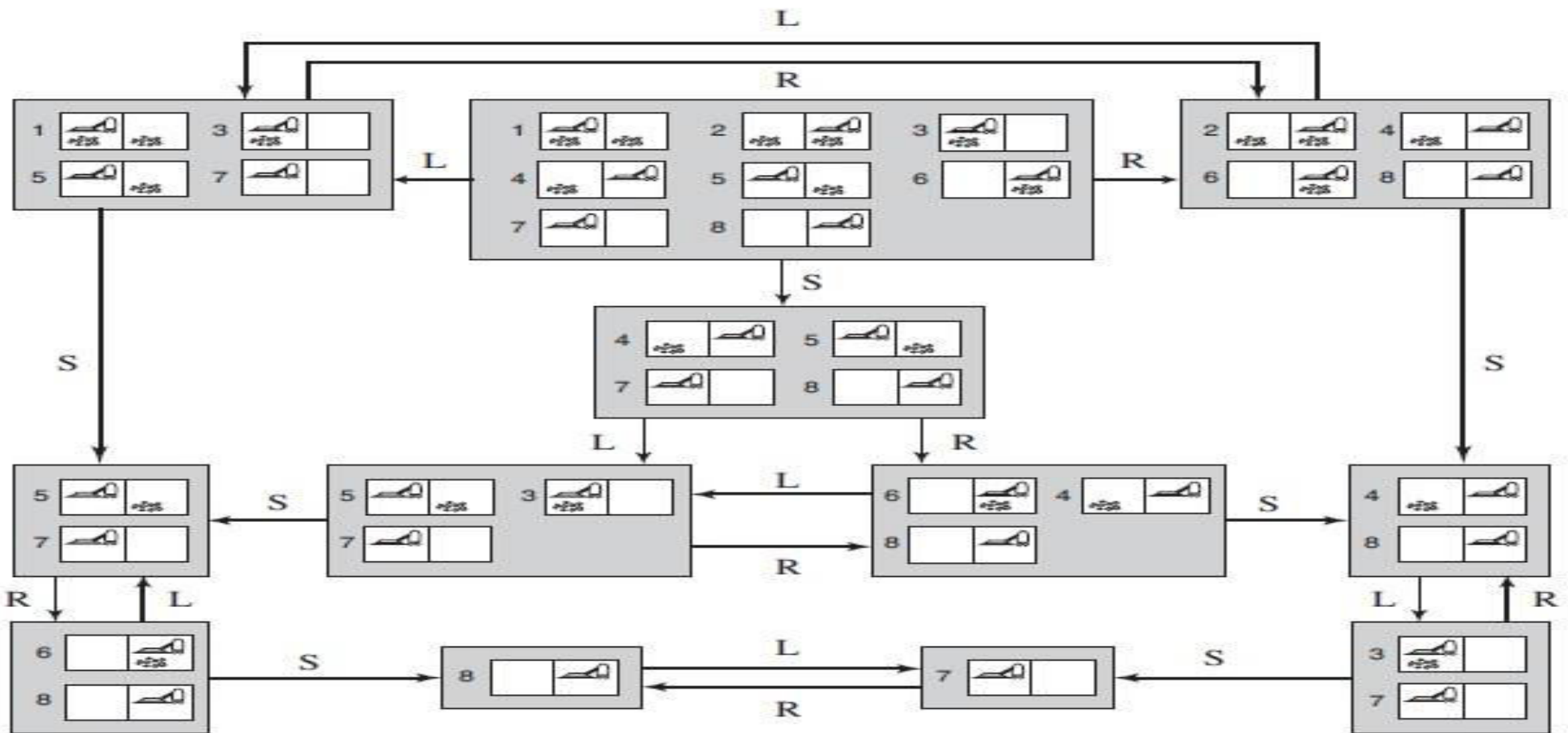
(a)



(b)

(a) Predicting the next belief state for the sensorless vacuum world with a deterministic action, *Right*. (b) Prediction for the same belief state and action in the slippery version of the sensorless vacuum world.

One solution is to represent the belief state by some more compact description. In English, we could say the agent knows “Nothing” in the initial state; after moving *Left*, we could say, “Not in the rightmost column,” and so on. Chapter 7 explains how to do this in a formal representation scheme. Another approach is to avoid the standard search algorithms, which treat belief states as black boxes just like any other problem state. Instead, we can look



The reachable portion of the belief-state space for the deterministic, sensor-less vacuum world. Each shaded box corresponds to a single belief state. At any given point, the agent is in a particular belief state but does not know which physical state it is in. The initial belief state (complete ignorance) is the top center box. Actions are represented by labeled links. Self-loops are omitted for clarity.

SEARCHING
WITH
OBSERVATIONS

SEARCHING WITH OBSERVATIONS

- When observations are partial, it will usually be the case that several states could have produced any given percept. For example, the percept [A, Dirty] is produced by state 3 as well as by state 1.
- Hence, given this as the initial percept, the initial belief state for the local-sensing vacuum world will be {1, 3}.
- The **ACTIONS**, **STEP-COST**, and **GOAL-TEST** are constructed from the underlying physical problem just as for sensor less problems, but the transition model is a bit more complicated.
- We can think of transitions from one belief state to the next for a particular action as occurring in three stages.
- The **prediction** stage is the same as for sensor less problems: given the action a in belief state b , the predicted belief state is $\hat{b} = \text{PREDICT}(b, a)$

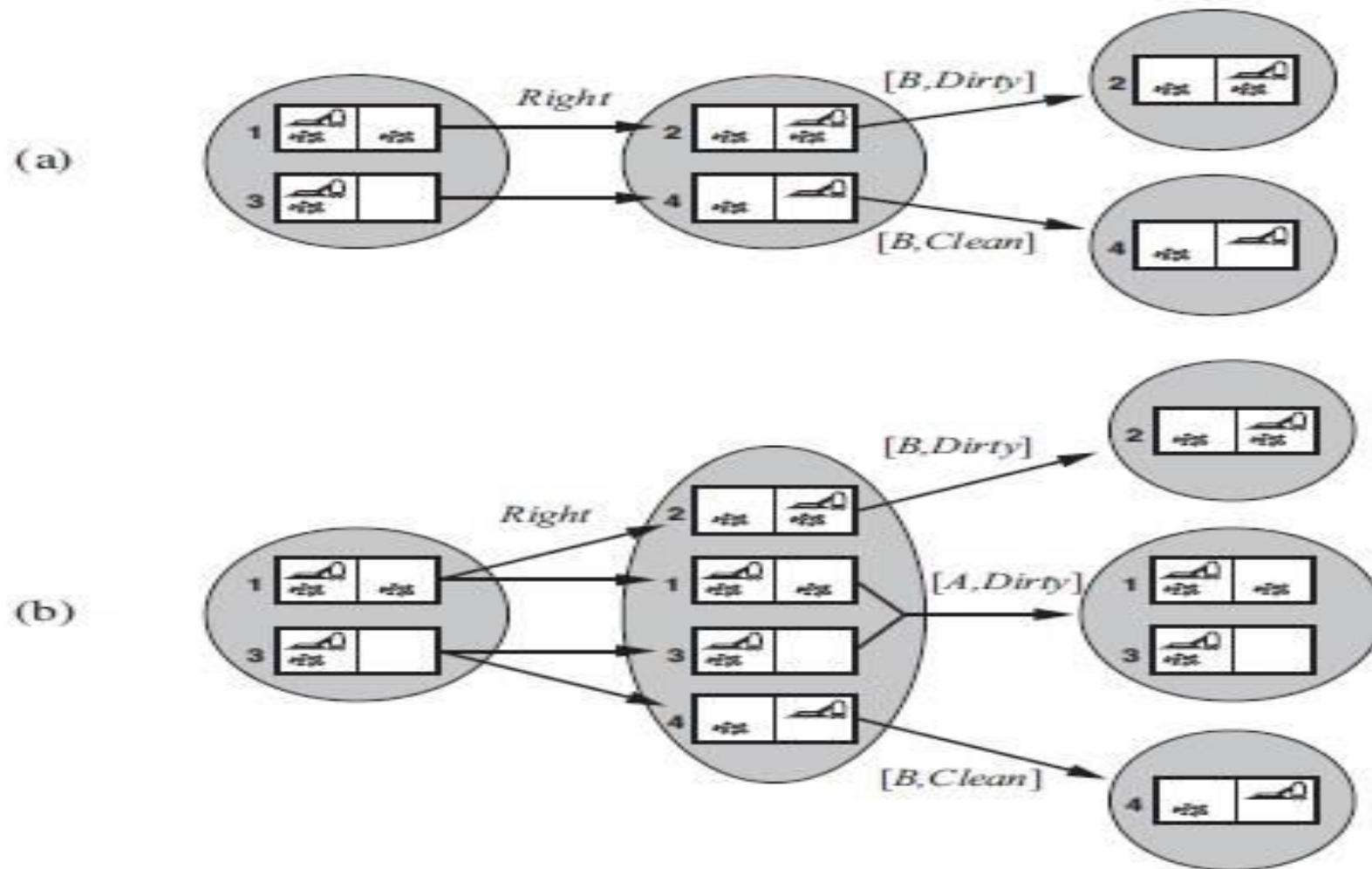
- The **observation prediction** stage determines the set of precepts **o** that could be observed in the predicted belief state:

$$\text{POSSIBLE-PERCEPTS}(\hat{b}) = \{o : o = \text{PERCEPT}(s) \text{ and } s \in \hat{b}\}$$

- The **update** stage determines, for each possible percept, the belief state that would result from the percept. The new belief state **bo** is just the set of states in $\hat{b}$ that could have produced the percept:

$$b_o = \text{UPDATE}(\hat{b}, o) = \{s : o = \text{PERCEPT}(s) \text{ and } s \in \hat{b}\}$$

- **Notice** that each updated belief state b_o can be no larger than the predicted belief state $\hat{b}$; observations can only help reduce uncertainty compared to the sensor less case.
- Moreover, for deterministic sensing, the belief states for the different possible precepts will be disjoint, forming a partition of the original predicted belief state.



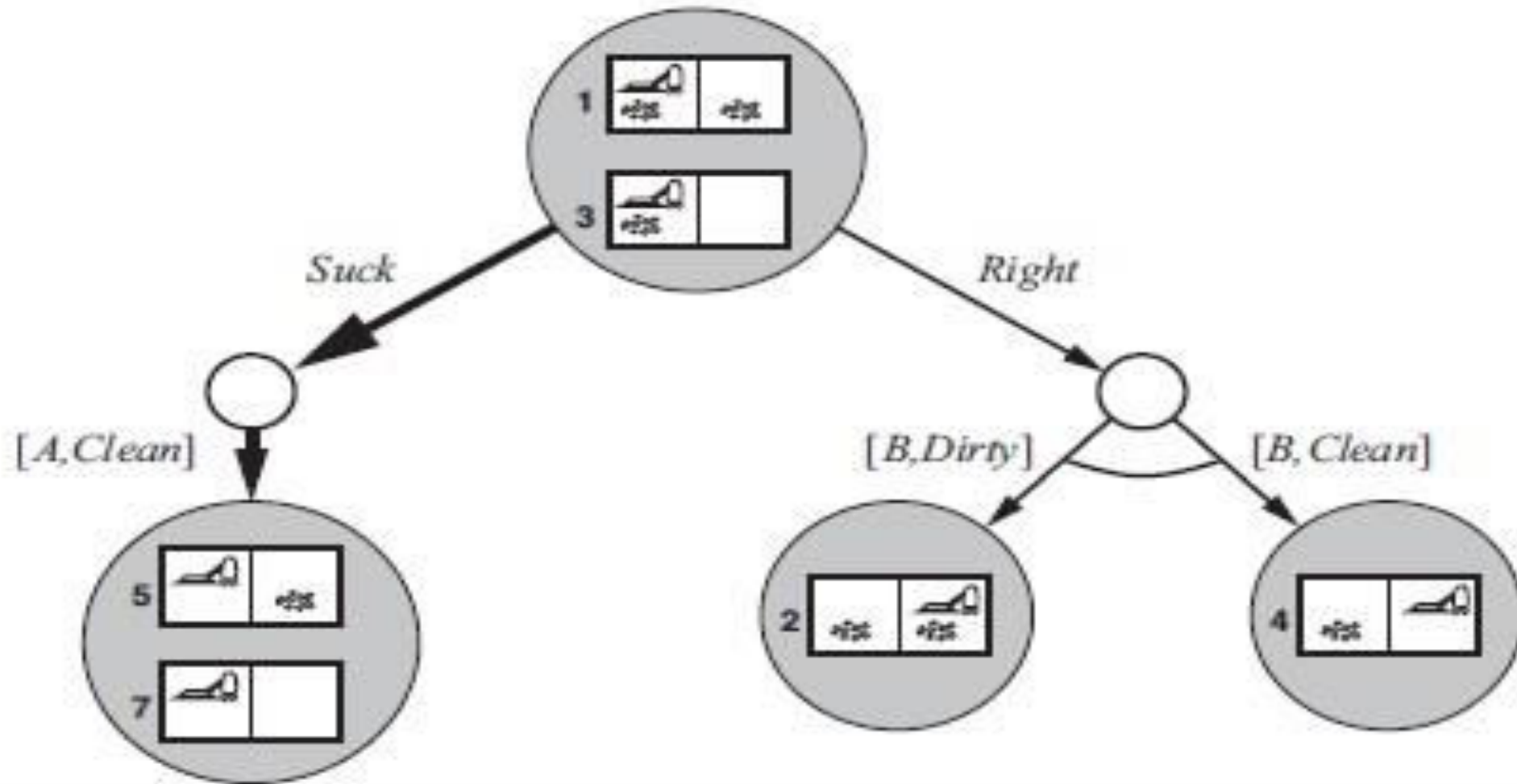
Two example of transitions in local-sensing vacuum worlds. (a) In the deterministic world, *Right* is applied in the initial belief state, resulting in a new belief state with two possible physical states; for those states, the possible percepts are $[B, Dirty]$ and $[B, Clean]$, leading to two belief states, each of which is a singleton. (b) In the slippery world, *Right* is applied in the initial belief state, giving a new belief state with four physical states; for those states, the possible percepts are $[A, Dirty]$, $[B, Dirty]$, and $[B, Clean]$, leading to three belief states as shown.

RESULTS (b, a) = {bo : bo = UPDATE(PREDICT(b, a), o) and
o ∈ POSSIBLE-PERCEPTS (PREDICT (b, a))}. (4.5)

SOLVING PARTIALLY OBSERVABLE PROBLEMS

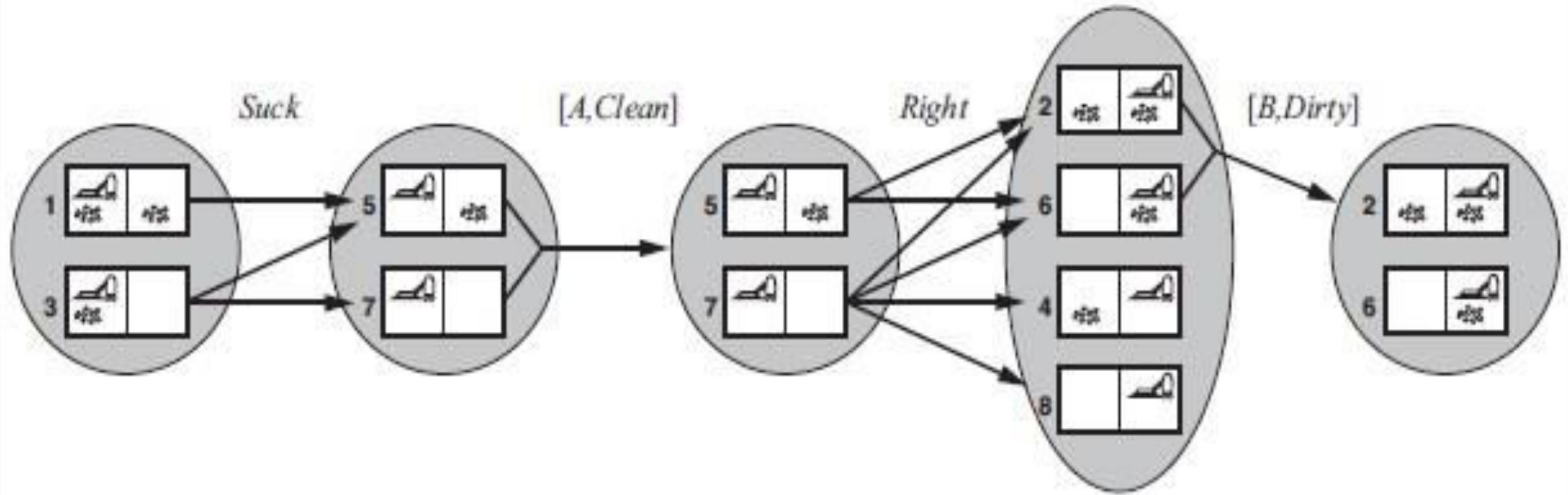
- The preceding section showed how to derive the **RESULTS** function for a nondeterministic belief-state problem from an underlying physical problem and the **PERCEPT** function.
- Given such a formulation, the **AND-OR** search algorithm of Figure can be applied directly to derive a solution.
- Figure shows part of the search tree for the local-sensing vacuum world, assuming an initial percept [A, Dirty]. The solution is the conditional plan.

$[Suck, Right, \text{if } Bstate = \{6\} \text{ then } Suck \text{ else } []]$.



The first level of the AND-OR search tree for a problem in the local-sensing vacuum world; *Suck* is the first step of the solution.

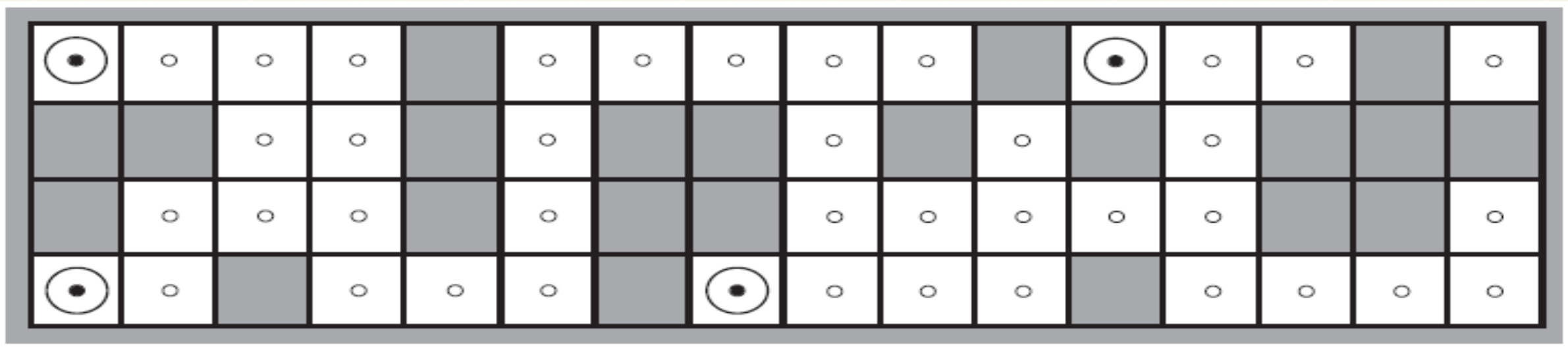
$b_ = \text{UPDATE}(\text{PREDICT}(b, a), o).$



Two prediction–update cycles of belief-state maintenance in the kindergarten vacuum world with local sensing.

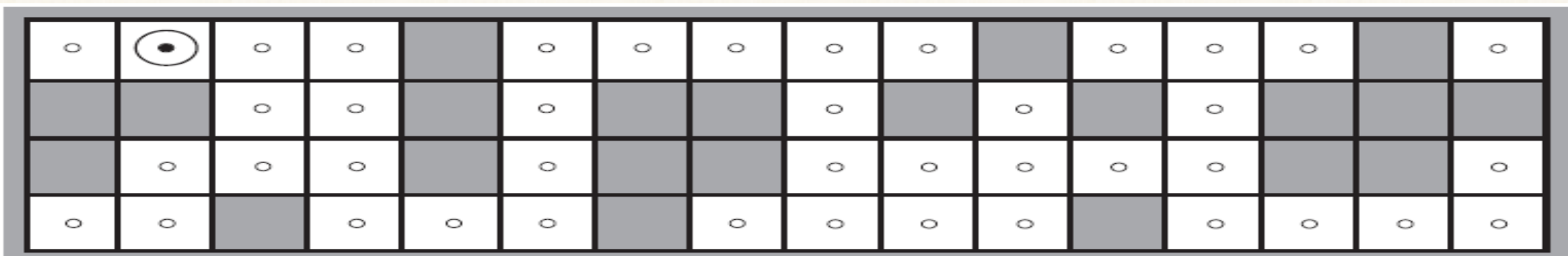
ROBOT LOCALIZATION

It must be in one of the following squares after [NSW]



Now it gets [NS], where can it be?

Only one location possible



(b) Possible locations of robot After $E_1 = \text{NSW}$, $E_2 = \text{NS}$

ONLINE SEARCH

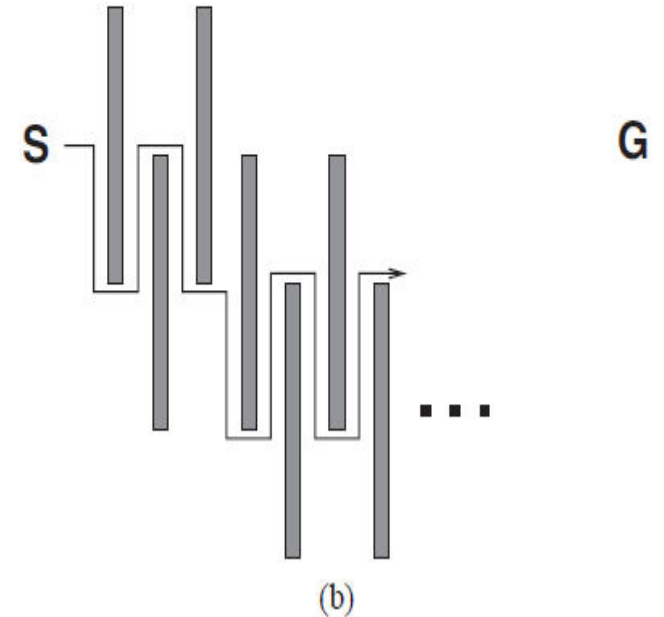
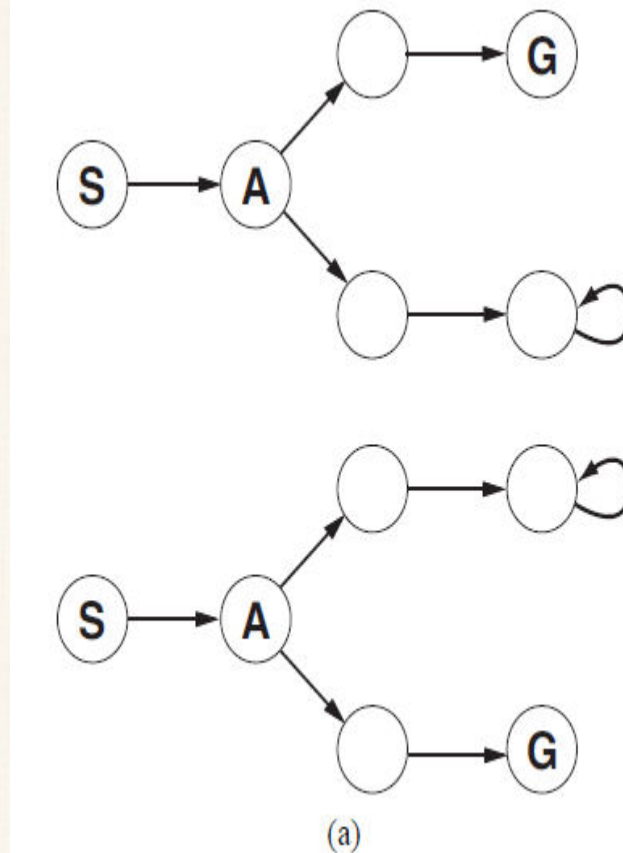
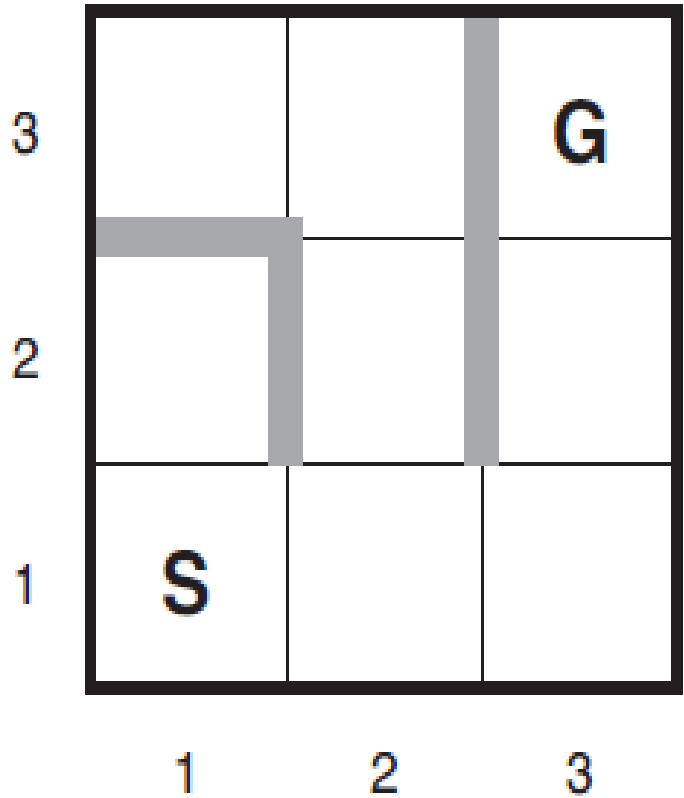
- Not find plan then execute then stop
- Compute, execute, observe, compute, execute, ...
- Interleave computation and action
- Great for
 - ✓ dynamic domains
 - ✓ Non-deterministic domains

NECESSARY IN UNKNOWN ENVIRONMENTS

- ✓ Robot localization in an unknown environment (no map)
- ✓ Does not know about obstacles, where the goal is, that UP from (1,1) goes to (1, 2)
- ✓ Once in (1, 2) does not know that down will go to (1, 1)

SOME KNOWLEDGE MIGHT BE AVAILABLE

- ✓ If location of goal is known, might use Manhattan distance heuristic
- ✓ Competitive Ratio = Cost of shortest path without exploration / Cost of actual agent path
- ✓ Irreversible actions can lead to dead ends and CR can become infinite



ONLINE SEARCH ALGORITHMS

function ONLINE-DFS-AGENT(s') **returns** an action

inputs: s' , a percept that identifies the current state

persistent: *result*, a table indexed by state and action, initially empty

untried, a table that lists, for each state, the actions not yet tried

unbacktracked, a table that lists, for each state, the backtracks not yet tried

s , a , the previous state and action, initially null

if GOAL-TEST(s') **then return** *stop*

if s' is a new state (not in *untried*) **then** $untried[s'] \leftarrow \text{ACTIONS}(s')$

if s is not null **then**

$result[s, a] \leftarrow s'$

add s to the front of $unbacktracked[s']$

if $untried[s']$ is empty **then**

if $unbacktracked[s']$ is empty **then return** *stop*

else $a \leftarrow$ an action b such that $result[s', b] = \text{POP}(unbacktracked[s'])$

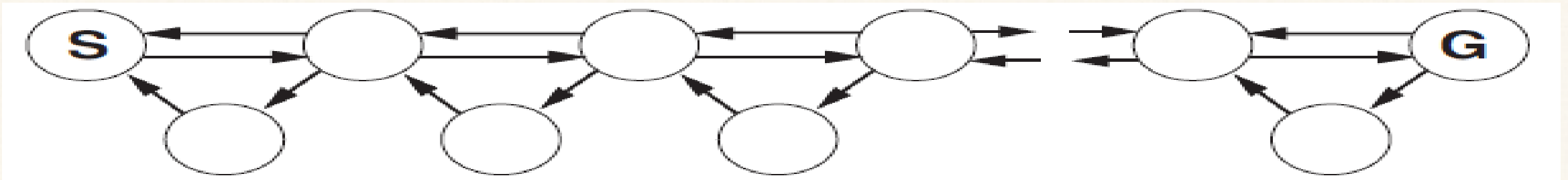
else $a \leftarrow \text{POP}(untried[s'])$

$s \leftarrow s'$

return a

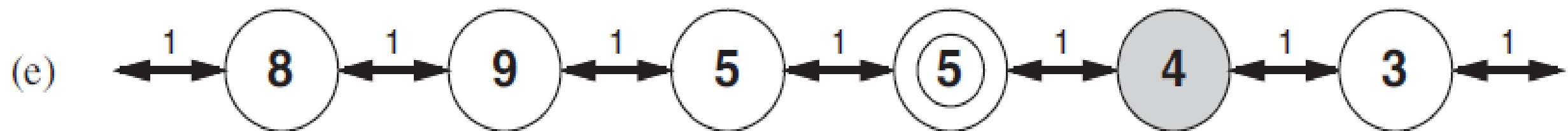
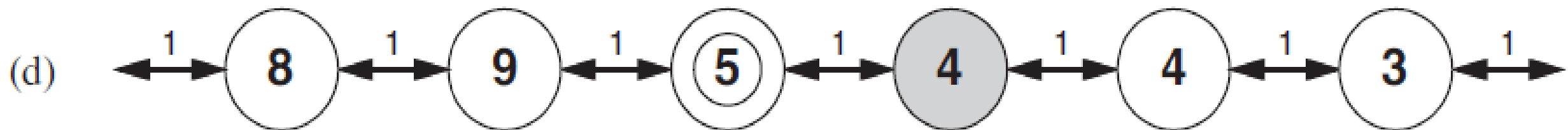
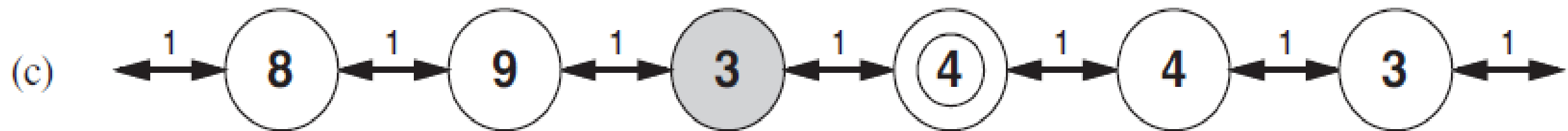
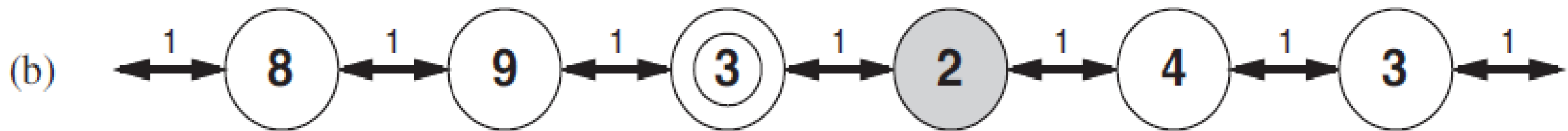
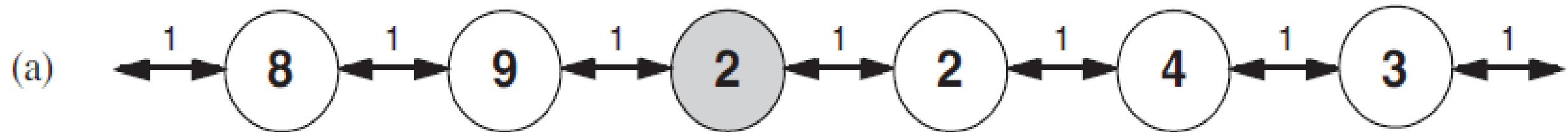
Hill-climbing is already an online search algorithm but stops at local optimum. How about randomization?

- Cannot do random restart (you can't teleport a robot)
- How about just a random walk instead of hill-climbing?



Can be very bad (two ways back for every way forward above)

- Let's augment HC with memory
- Learning real-time A* (LRTA*)
- Updates cost estimates, $g(s)$, for the state it leaves
- Likes unexplored states
- $f(s) = h(s)$ not $g(s) + h(s)$ for unexplored states



function LRTA*-AGENT(s') **returns** an action

inputs: s' , a percept that identifies the current state

persistent: *result*, a table, indexed by state and action, initially empty

H , a table of cost estimates indexed by state, initially empty

s , a , the previous state and action, initially null

if GOAL-TEST(s') **then return** *stop*

if s' is a new state (not in H) **then** $H[s'] \leftarrow h(s')$

if s is not null

$result[s, a] \leftarrow s'$

$H[s] \leftarrow \min_{b \in \text{ACTIONS}(s)} \text{LRTA}^*\text{-COST}(s, b, result[s, b], H)$

$a \leftarrow$ an action b in $\text{ACTIONS}(s')$ that minimizes $\text{LRTA}^*\text{-COST}(s', b, result[s', b], H)$

$s \leftarrow s'$

return a

function LRTA*-COST(s, a, s', H) **returns** a cost estimate

if s' is undefined **then return** $h(s)$

else return $c(s, a, s') + H[s']$

LRTA*-AGENT selects an action according to the values of neighboring states, which are updated as the agent moves about the state space.

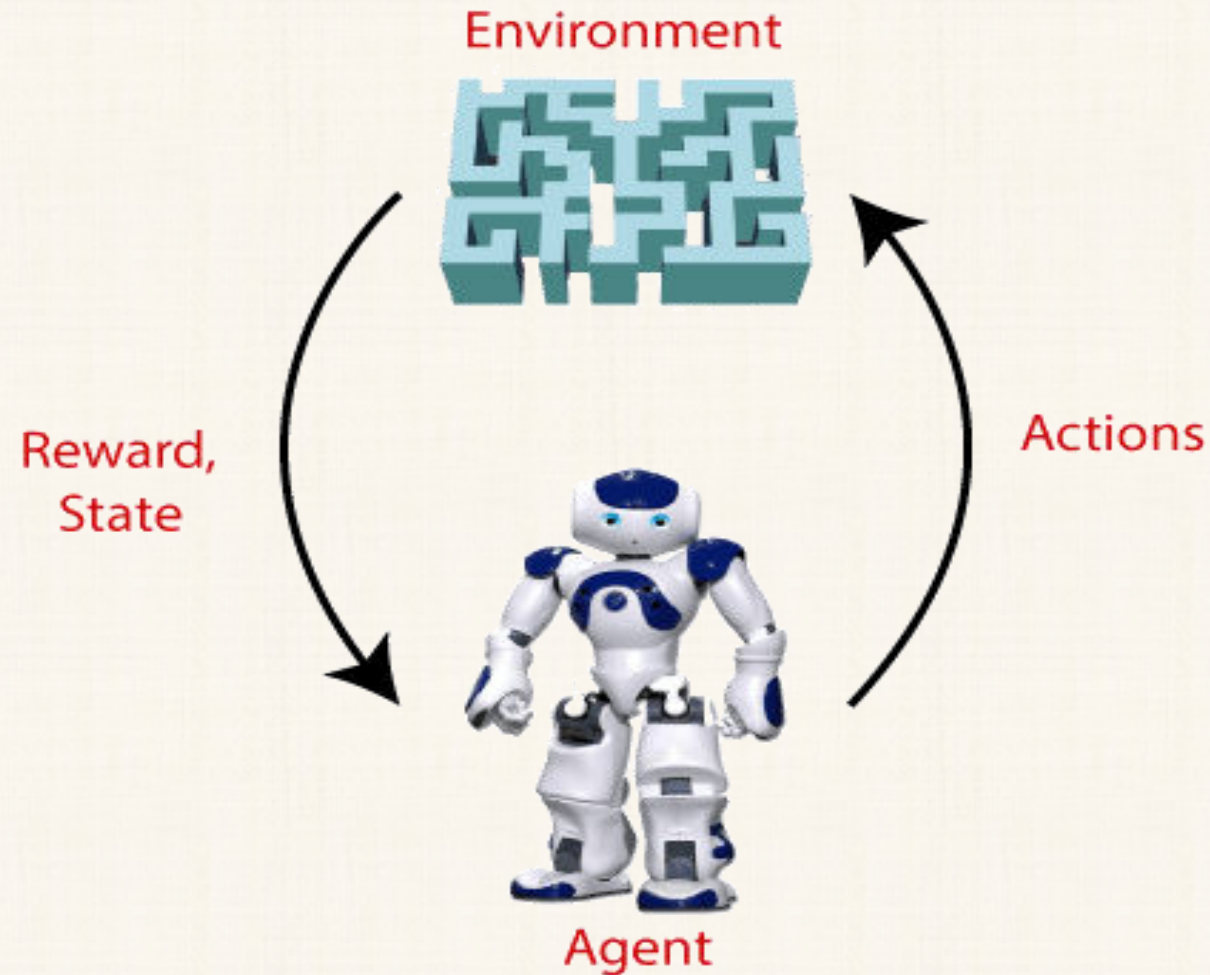
REINFORCEMENT

LEARNING

REINFORCEMENT LEARNING

- Reinforcement Learning is a **feedback-based Machine learning** technique in which an agent learns to behave in an environment by performing the actions and seeing the results of actions.
- For each **good action**, the agent **gets positive feedback**, and for each **bad action**, the agent gets **negative feedback** or penalty.
- In Reinforcement Learning, the agent **learns automatically** using **feedbacks** without any labelled data, unlike supervised learning.
- RL solves a specific type of problem where **decision making is sequential**, and the goal is long-term, such as game-playing, robotics, etc.
- The agent learns with the process of hit and trial, and based on the experience, it learns to perform the task in a better way. Hence, we can say that "**Reinforcement learning is a type of machine learning method where an intelligent agent (computer program) interacts with the environment and learns to act within that.**" How a Robotic dog learns the movement of his arms is an example of Reinforcement learning.

- **EXAMPLE:** Suppose there is an AI agent present within a maze environment, and his goal is to find the diamond. The agent interacts with the environment by performing some actions, and based on those actions, the state of the agent gets changed, and it also receives a reward or penalty as feedback.



TERMS USED IN REINFORCEMENT LEARNING

- **Agent ()**: An entity that can perceive/explore the environment and act upon it.
- **Environment ()**: A situation in which an agent is present or surrounded by. In RL, we assume the stochastic environment, which means it is random in nature.
- **Action ()**: Actions are the moves taken by an agent within the environment.
- **State ()**: State is a situation returned by the environment after each action taken by the agent.
- **Reward ()**: A feedback returned to the agent from the environment to evaluate the action of the agent.
- **Policy ()**: Policy is a strategy applied by the agent for the next action based on the current state.
- **Value ()**: It is expected long-term return with the discount factor and opposite to the short-term reward.
- **Q-value ()**: It is mostly similar to the value, but it takes one additional parameter as a current action (a).

KEY FEATURES OF REINFORCEMENT LEARNING

- In RL, the agent is not instructed about the environment and what actions need to be taken.
- It is based on the hit and trial process.
- The agent takes the next action and changes states according to the feedback of the previous action.
- The agent may get a delayed reward.
- The environment is stochastic, and the agent needs to explore it to reach to get the maximum positive rewards.

APPROACHES TO IMPLEMENT REINFORCEMENT LEARNING

There are mainly three ways to implement reinforcement-learning in ML, which are:

➤ **Value-based**

➤ **Policy-based**

➤ **Model-based**

1. VALUE-BASED:

The value-based approach is about to find the optimal value function, which is the maximum value at a state under any policy. Therefore, the agent expects the long-term return at any state(s) under policy π .

2. POLICY-BASED:

Policy-based approach is to find the optimal policy for the maximum future rewards without using the value function. In this approach, the agent tries to apply such a policy that the action performed in each step helps to maximize the future reward.

The policy-based approach has mainly two types of policy:

- ✓ **Deterministic:** The same action is produced by the policy (π) at any state.
- ✓ **Stochastic:** In this policy, probability determines the produced action.

3.MODEL-BASED:

In the model-based approach, a virtual model is created for the environment, and the agent explores that environment to learn it. There is no particular solution or algorithm for this approach because the model representation is different for each environment.

ELEMENTS OF REINFORCEMENT LEARNING

There are four main elements of Reinforcement Learning, which are given below:

1. Policy
2. Reward Signal
3. Value Function
4. Model of the environment

1) POLICY:

A policy can be defined as a way how an agent behaves at a given time. It maps the perceived states of the environment to the actions taken on those states. A policy is the core element of the RL as it alone can define the behaviour of the agent. In some cases, it may be a simple function or a lookup table, whereas, for other cases, it may involve general computation as a search process. It could be deterministic or a stochastic policy:

For deterministic policy: $a = \pi(s)$

For stochastic policy: $\pi(a | s) = P[A_t = a | S_t = s]$

2) REWARD SIGNAL:

The goal of reinforcement learning is defined by the reward signal. At each state, the environment sends an immediate signal to the learning agent, and this signal is known as a **reward signal**. These rewards are given according to the good and bad actions taken by the agent. The agent's main objective is to maximize the total number of rewards for good actions. The reward signal can change the policy, such as if an action selected by the agent leads to low reward, then the policy may change to select other actions in the future.

3) VALUE FUNCTION:

The value function gives information about how good the situation and action are and how much reward an agent can expect. A reward indicates the **immediate signal for each good and bad action**, whereas a value function specifies **the good state and action for the future**. The value function depends on the reward as, without reward, there could be no value. The goal of estimating values is to achieve more rewards.

4) MODEL:

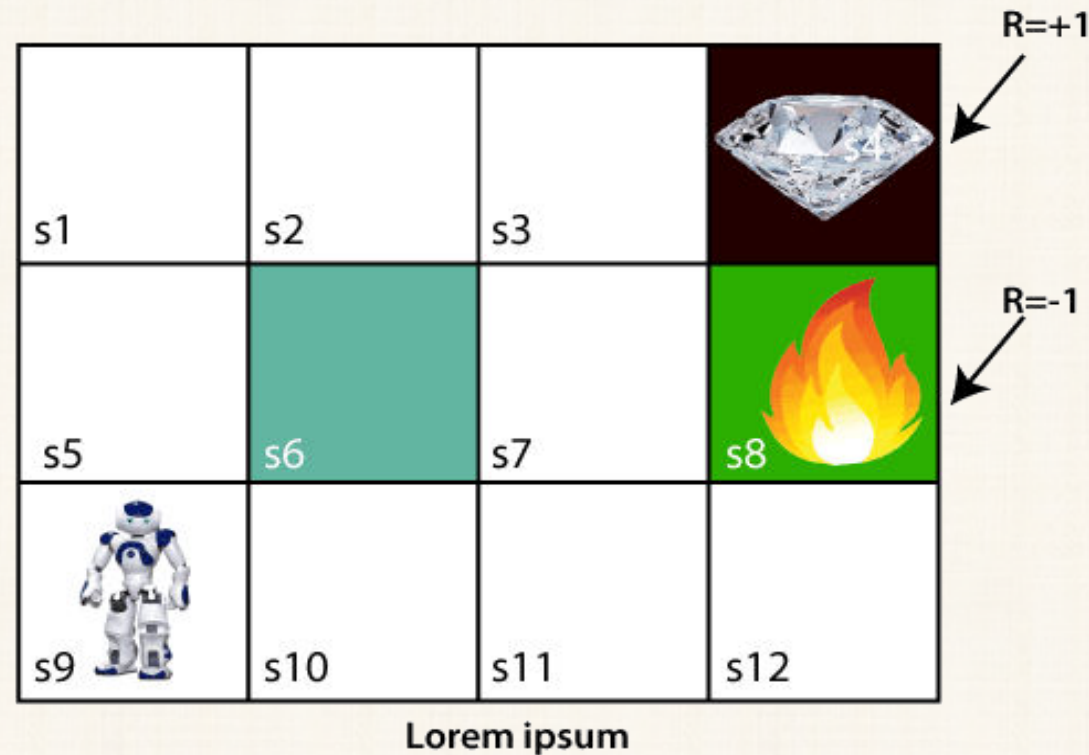
The last element of reinforcement learning is the model, which mimics the behaviour of the environment. With the help of the model, one can make inferences about how the environment will behave. Such as, if a state and an action are given, then a model can predict the next state and reward. The model is used for planning, which means it provides a way to take a course of action by considering all future situations before actually experiencing those situations. The approaches for solving the RL problems **with the help of the model** are termed as the **model-based approach**. Comparatively, an approach **without using a model** is called a **model-free approach**.

HOW DOES REINFORCEMENT LEARNING WORK?

To understand the working process of the RL, we need to consider two main things:

- ✓ **Environment:** It can be anything such as a room, maze, football ground, etc.
- ✓ **Agent:** An intelligent agent such as AI robot.

Let's take an example of a maze environment that the agent needs to explore. Consider the below image:



In the above image, the agent is at the very first block of the maze. The maze is consisting of an S_6 block, which is a **wall**, S_8 a **fire pit**, and S_4 a **diamond block**.

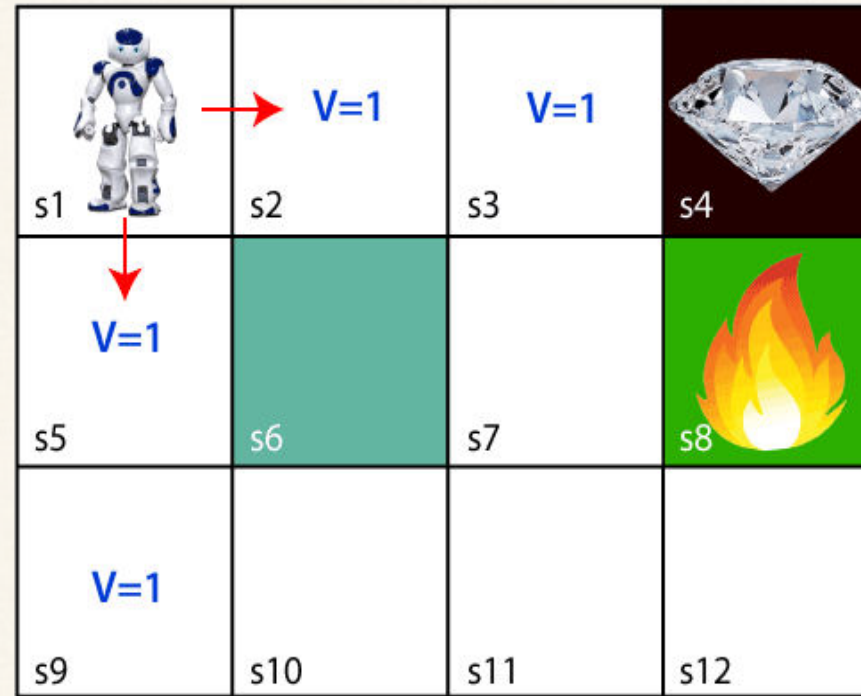
The agent cannot cross the S_6 block, as it is a solid wall. If the agent reaches the S_4 block, then get the **+1 reward**; if it reaches the fire pit, then gets **-1 reward point**. It can take four actions: **move up, move down, move left, and move right**.

The agent can take any path to reach to the final point, but he needs to make it in possible fewer steps. Suppose the agent considers the path **S9-S5-S1-S2-S3**, so he will get the +1 reward point.

The agent will try to remember the preceding steps that it has taken to reach the final step. To memorize the steps, it assigns 1 value to each previous step.

Now, the agent has successfully stored the previous steps assigning the 1 value to each previous block. But what will the agent do if he starts moving from the block, which has 1 value block on both sides?

$V=1$ s1	$V=1$ s2	$V=1$ s3	 s4
$V=1$ s5	 s6	s7	 s8
 $V=1$ s9	s10	s11	s12



It will be a difficult condition for the agent whether he should go up or down as each block has the same value. So, the above approach is not suitable for the agent to reach the destination. Hence to solve the problem, we will use the **Bellman equation**, which is the main concept behind reinforcement learning.

THE BELLMAN EQUATION

- The Bellman equation was introduced by the Mathematician **Richard Ernest Bellman** in the year **1953**, and hence it is called as a Bellman equation.
- It is associated with dynamic programming and used to calculate the values of a decision problem at a certain point by including the values of previous states.

It is a way of calculating the value functions in dynamic programming or environment that leads to modern reinforcement learning.

The key-elements used in Bellman equations are:

- **Action** performed by the agent is referred to as "**a**"
- **State occurred** by performing the action is "**s**."
- The reward/feedback obtained for each good and bad action is "**R**."
- A discount factor is Gamma " **γ** ."

The **Bellman equation** can be written as:

$$V(s) = \max [R(s,a) + \gamma V(s')]$$

Where,

$V(s)$ = value calculated at a particular point.

$R(s,a)$ = Reward at a particular state s by performing an action.

γ = Discount factor

$V(s')$ = The value at the previous state.

- In the above equation, we are taking the max of the complete values because the agent tries to find the optimal solution always.
- So now, using the Bellman equation, we will find value at each state of the given environment. We will start from the block, which is next to the target block.

For 1st block:

$V(s_3) = \max [R(s,a) + \gamma V(s')]$, here $V(s')= 0$ because there is no further state to move.

$$V(s_3)= \max[R(s,a)]\Rightarrow V(s_3)= \max[1]\Rightarrow \mathbf{V(s_3)= 1.}$$

For 2nd block:

$V(s_2) = \max [R(s,a) + \gamma V(s')]$, here $\gamma= 0.9$ (lets), $V(s')= 1$, and $R(s, a)= 0$, because there is no reward at this state.

$$V(s_2)= \max[0.9(1)]\Rightarrow V(s_2)= \max[0.9]\Rightarrow \mathbf{V(s_2) =0.9}$$

For 3rd block:

$V(s_1) = \max [R(s,a) + \gamma V(s')]$, here $\gamma= 0.9$ (lets), $V(s')= 0.9$, and $R(s, a)= 0$, because there is no reward at this state also.

$$V(s_1)= \max[0.9(0.9)]\Rightarrow V(s_1)= \max[0.81]\Rightarrow \mathbf{V(s_1) =0.81}$$

For 4th block:

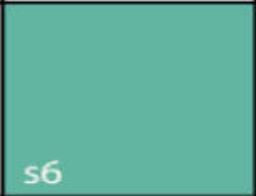
$V(s_5) = \max [R(s,a) + \gamma V(s')]$, here $\gamma= 0.9$ (lets), $V(s')= 0.81$, and $R(s, a)= 0$, because there is no reward at this state also.

$$V(s_5)= \max[0.9(0.81)]\Rightarrow V(s_5)= \max[0.73]\Rightarrow \mathbf{V(s_5) =0.73}$$

For 5th block:

$V(s_9) = \max [R(s,a) + \gamma V(s')]$, here $\gamma= 0.9$ (lets), $V(s')= 0.73$, and $R(s, a)= 0$, because there is no reward at this state also.

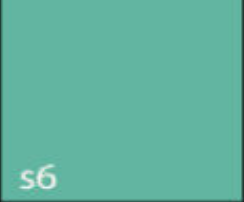
$$V(s_9)= \max[0.9(0.73)]\Rightarrow V(s_4)= \max[0.66]\Rightarrow \mathbf{V(s_4) =0.66}$$

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	 s6	s7	 s8
 V=0.66 s9	s10	s11	s12

Now, we will move further to the 6th block, and here agent may change the route because it always tries to find the optimal path. So now, let's consider from the block next to the fire pit.

Now, the agent has three options to move; if he moves to the blue box, then he will feel a bump if he moves to the fire pit, then he will get the -1 reward. But here we are taking only positive rewards, so for this, he will move to upwards only. The complete block values will be calculated using this formula.

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	 s6	 s7	 s8
V=0.66 s9	s10	  s11	s12

V=0.81 s1	V=0.9 s2	V=1 s3	 s4
V=0.73 s5	 s6	V=0.9 s7	 s8
V=0.66 s9	V=0.73 s10	V=0.81 s11	V=0.73 s12

TYPES OF REINFORCEMENT LEARNING

There are mainly two types of reinforcement learning, which are:

- **Positive Reinforcement**
- **Negative Reinforcement**

POSITIVE REINFORCEMENT:

- The positive reinforcement learning means adding something to increase the tendency that expected behaviour would occur again. It impacts positively on the behaviour of the agent and increases the strength of the behaviour.
- This type of reinforcement can sustain the changes for a long time, but too much positive reinforcement may lead to an overload of states that can reduce the consequences.

NEGATIVE REINFORCEMENT:

- The negative reinforcement learning is opposite to the positive reinforcement as it increases the tendency that the specific behaviour will occur again by avoiding the negative condition.
- It can be more effective than the positive reinforcement depending on situation and behaviour, but it provides reinforcement only to meet minimum behaviour.

HOW TO REPRESENT THE AGENT STATE?

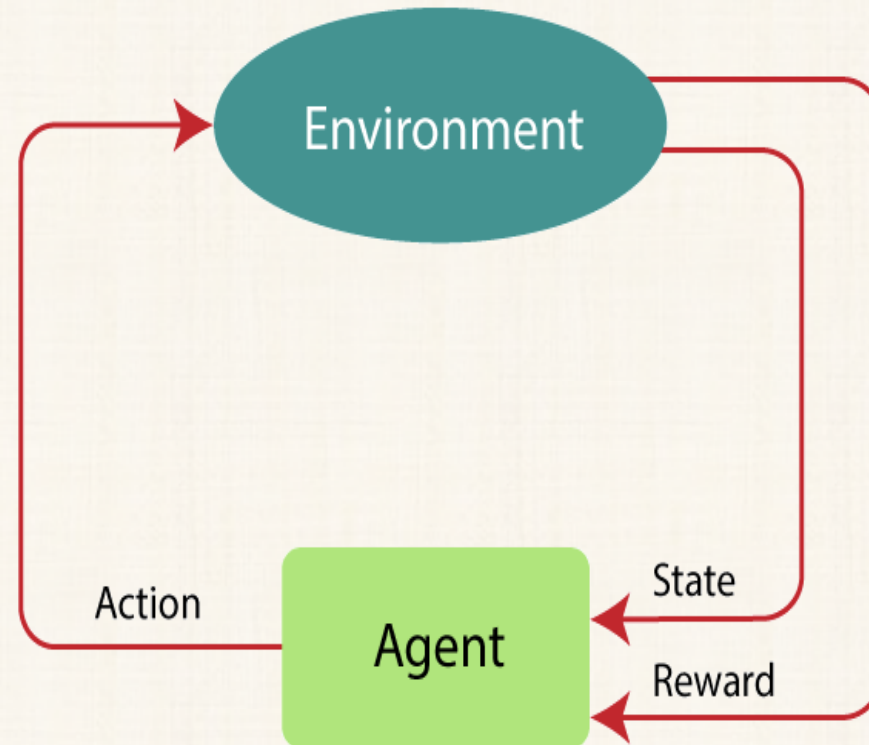
We can represent the agent state using the **Markov State** that contains all the required information from the history. The State S_t is Markov state if it follows the given condition:

$$P[S_{t+1} | S_t] = P[S_{t+1} | S_1, \dots, S_t]$$

The Markov state follows the **Markov property**, which says that the future is independent of the past and can only be defined with the present. The RL works on fully observable environments, where the agent can observe the environment and act for the new state. The complete process is known as Markov Decision process.

MARKOV DECISION PROCESS

Markov Decision Process or **MDP**, is used to **formalize the reinforcement learning problems**. If the environment is completely observable, then its dynamic can be modelled as a **Markov Process**. In MDP, the agent constantly interacts with the environment and performs actions; at each action, the environment responds and generates a new state.



MDP is used to describe the environment for the RL, and almost all the RL problem can be formalized using **MDP**.

MDP contains a tuple of four elements (S, A, P_a, R_a):

- ✓ A set of finite States **S**
- ✓ A set of finite Actions **A**
- ✓ Rewards received after transitioning from state **S** to state **S'**, due to action **a**.
- ✓ Probability **P_a**.

MDP uses **Markov property**, and to better understand the MDP, we need to learn about it.

MARKOV PROPERTY

- It says that "If the agent is present in the current state **S1**, performs an action **a1** and move to the state **s2**, then the state transition from **s1** to **s2** only depends on the current state and future action and states do not depend on past actions, rewards, or states."

MARKOV PROPERTY

- Or, in other words, as per Markov Property, the current state transition does not depend on any past action or state. Hence, MDP is an RL problem that satisfies the Markov property. Such as in a **Chess game, the players only focus on the current state and do not need to remember past actions or states.**

FINITE MDP

A finite MDP is when there are finite states, finite rewards, and finite actions. In RL, we consider only the finite MDP.

MARKOV PROCESS

Markov Process is a memoryless process with a sequence of random states $S_1, S_2, \dots, S_t$ that uses the Markov Property. Markov process is also known as Markov chain, which is a tuple (S, P) on state S and transition function P . These two components (S and P) can define the dynamics of the system.

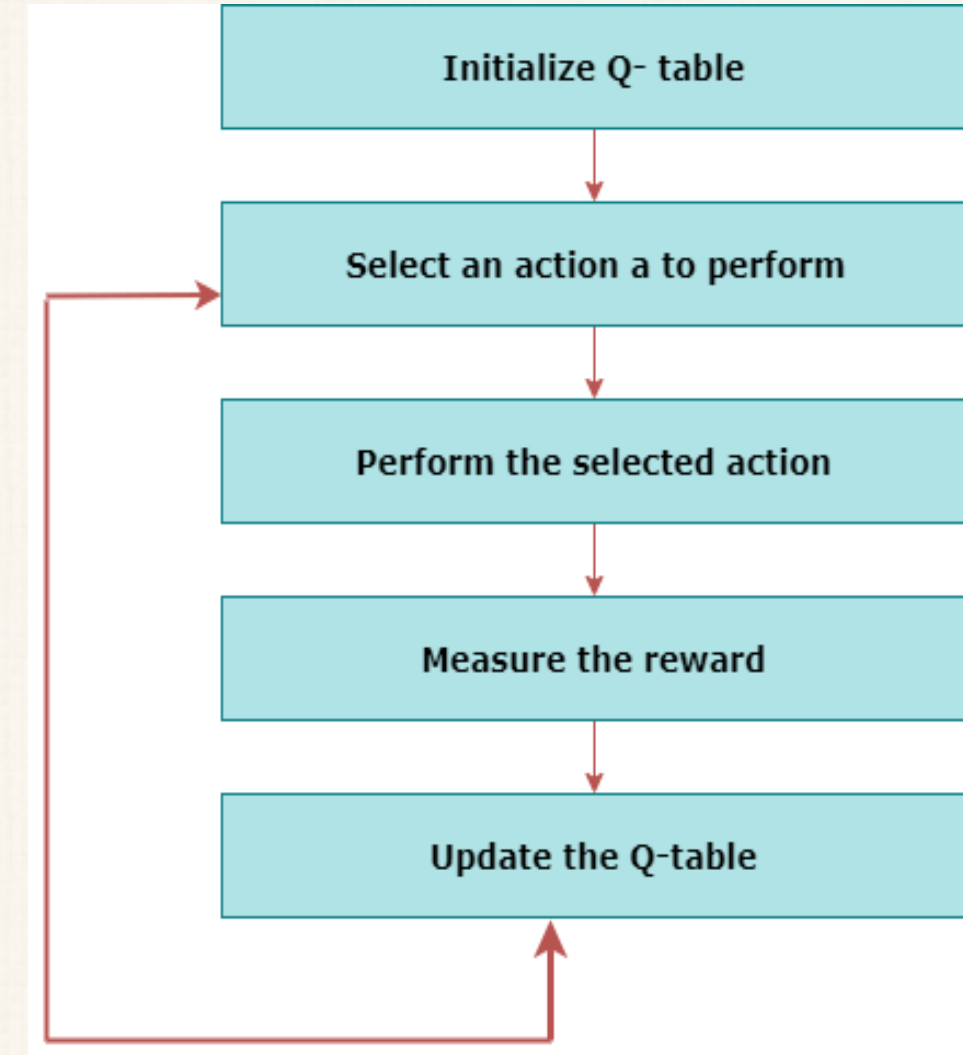
REINFORCEMENT LEARNING ALGORITHMS

Reinforcement learning algorithms are mainly used in AI applications and gaming applications.

The main used algorithms are:

Q-Learning:

- Q-learning is an **Off policy RL algorithm**, which is used for the temporal difference Learning. The temporal difference learning methods are the way of comparing temporally successive predictions.
- It learns the value function $Q(s, a)$, which means how good to take action "**a**" at a particular state "**s**."
- The below flowchart explains the working of Q- learning:



State Action Reward State action (SARSA):

- SARSA stands for **State Action Reward State action**, which is an **on-policy** temporal difference learning method. The on-policy control method selects the action for each state while learning using a specific policy.
- The goal of SARSA is to calculate the **$Q \pi (s, a)$** for the **selected current policy π and all pairs of (s-a)**.
- The main difference between Q-learning and SARSA algorithms is that **unlike Q-learning, the maximum reward for the next state is not required for updating the Q-value in the table.**
- In SARSA, new action and reward are selected using the same policy, which has determined the original action.

- ✓ The SARSA is named because it uses the quintuple $Q(s, a, r, s', a')$. Where,

S: original state

a: Original action

r: reward observed while following the states

s' and a': New state, action pair.

Deep Q Neural Network (DQN)

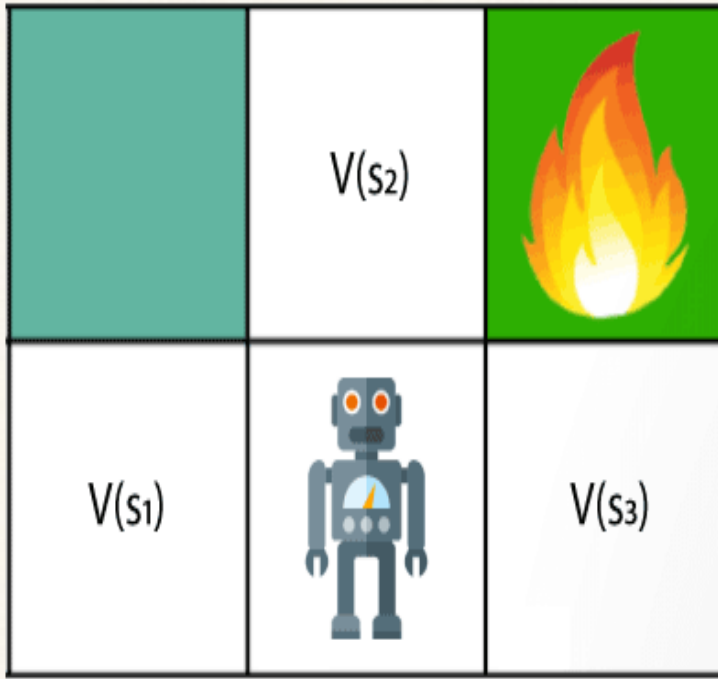
- As the name suggests, DQN is a **Q-learning using Neural networks**.
- For a big state space environment, it will be a challenging and complex task to define and update a Q-table.
- To solve such an issue, we can use a DQN algorithm. Where, instead of defining a Q-table, neural network approximates the Q-values for each action and state.
- Now, we will expand the Q-learning.

Q-LEARNING EXPLANATION

- Q-learning is a popular model-free reinforcement learning algorithm based on the Bellman equation.
- **The main objective of Q-learning is to learn the policy which can inform the agent that what actions should be taken for maximizing the reward under what circumstances.**
- It is an **off-policy RL** that attempts to find the best action to take at a current state.
- The goal of the agent in Q-learning is to maximize the value of Q.
- The value of Q-learning can be derived from the Bellman equation. Consider the Bellman equation given below:

$$V(s) = \max [R(s,a) + \gamma \sum_{s'} P(s, a, s') V(s')]$$

In the equation, we have various components, including reward, discount factor (γ), probability, and end states s' . But there is no any Q-value is given so first consider the below image:



In the image, we can see there is an agent who has three values options, $V(s_1)$, $V(s_2)$, $V(s_3)$. As this is MDP, so agent only cares for the current state and the future state. The agent can go to any direction (Up, Left, or Right), so he needs to decide where to go for the optimal path. Here agent will take a move as per probability bases and changes the state. But if we want some exact moves, so for this, we need to make some changes in terms of Q-value.

$$Q(S, a) = R(s, a) + \gamma \sum_{s'} P(s, a, s') V(s')$$

Hence, we can say that, $V(s) = \max [Q(s, a)]$

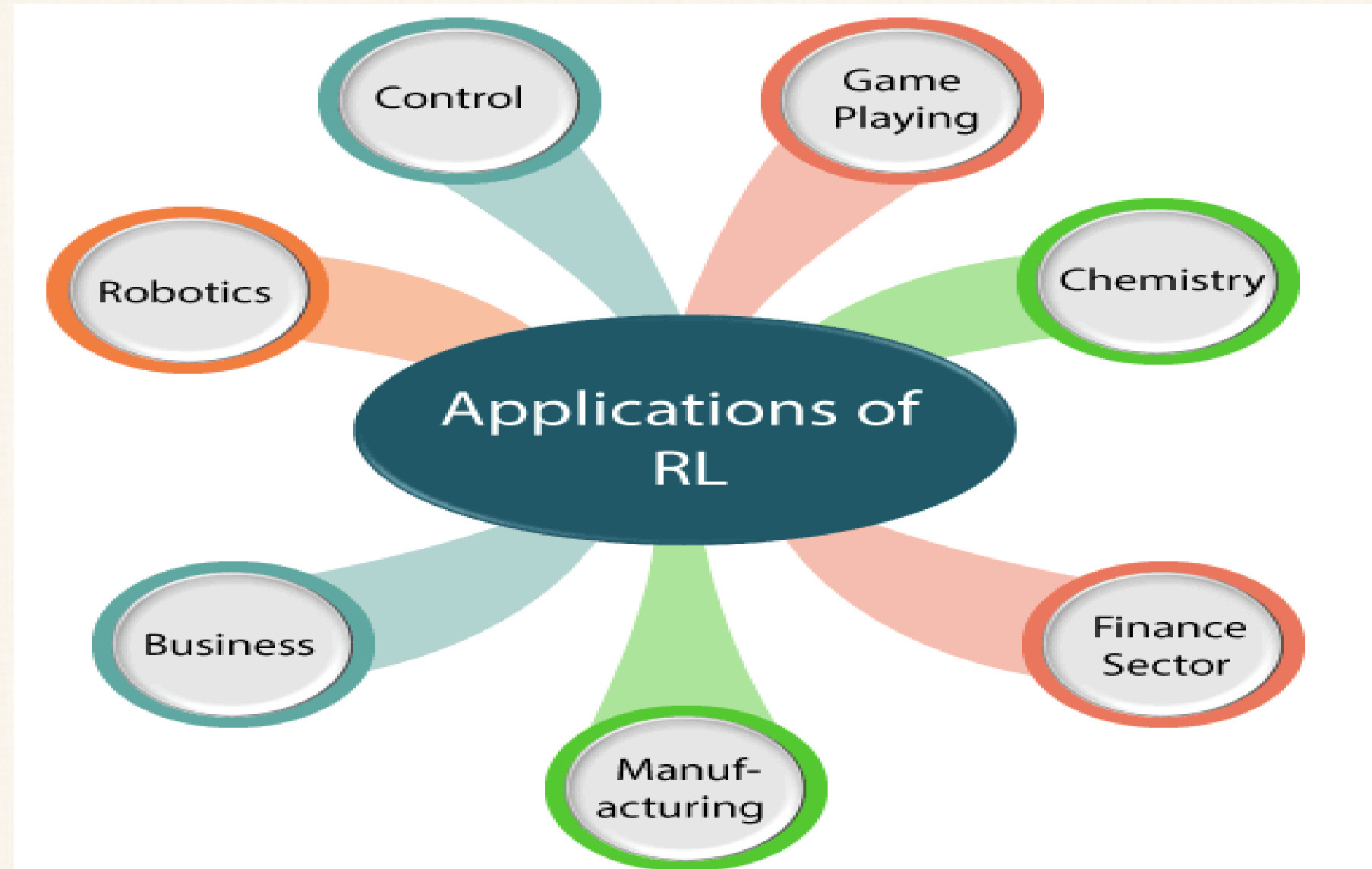
$$Q(S, a) = R(s, a) + \gamma \sum_{s'} (P(s, a, s') \max Q(s', a'))$$

DIFFERENCE BETWEEN REINFORCEMENT LEARNING AND SUPERVISED LEARNING

The Reinforcement Learning and Supervised Learning both are the part of machine learning, but both types of learnings are far opposite to each other. The RL agents interact with the environment, explore it, take action, and get rewarded. Whereas supervised learning algorithms learn from the labelled dataset and, on the basis of the training, predict the output.

REINFORCEMENT LEARNING	SUPERVISED LEARNING
✓ RL works by interacting with the environment.	✓ Supervised learning works on the existing dataset.
✓ The RL algorithm works like the human brain works when making some decisions.	✓ Supervised Learning works as when a human learns things in the supervision of a guide.
✓ There is no labelled dataset is present	✓ The labelled dataset is present.
✓ No previous training is provided to the learning agent.	✓ Training is provided to the algorithm so that it can predict the output.
✓ RL helps to take decisions sequentially.	✓ In Supervised learning, decisions are made when input is given

REINFORCEMENT LEARNING APPLICATIONS



1. Robotics:

RL is used in **Robot navigation, Robo-soccer, walking, juggling**, etc.

2. Control:

RL can be used for **adaptive control** such as Factory processes, admission control in telecommunication, and Helicopter pilot is an example of reinforcement learning.

3. Game Playing:

RL can be used in **Game playing** such as tic-tac-toe, chess, etc.

4. Chemistry:

RL can be used for optimizing the chemical reactions.

5. Business:

RL is now used for business strategy planning.

6. Manufacturing:

In various automobile manufacturing companies, the robots use deep reinforcement learning to pick goods and put them in some containers.

7. Finance Sector:

The RL is currently used in the finance sector for evaluating trading strategies.

PASSIVE AND ACTIVE REINFORCEMENT LEARNING

1.DIRECT UTILITY ESTIMATION

In this method, the agent executes a **sequence of trials or runs** (sequences of states-actions transition that continue until the agent reaches the terminal state). Each trial gives a sample value and the agent estimates the utility based on the sample's values. Can be calculated as **running averages of sample values**. The main drawback is that this method makes a wrong assumption that **state utilities are independent** while in reality they are Markovian. Also, Suppose we have a 4x3 grid as the environment in which the agent can move either Left, Right, Up or Down (set of available actions). An example of a run

$$(1,1)_{-0.04} \rightarrow (1,2)_{-0.04} \rightarrow (1,3)_{-0.04} \rightarrow (1,2)_{-0.04} \rightarrow (1,3)_{-0.04} \rightarrow (2,3)_{-0.04} \rightarrow (3,3)_{-0.04} \rightarrow (4,3)_{+1}$$

2. ADAPTIVE DYNAMIC PROGRAMMING (ADP)

ADP is a smarter method than Direct Utility Estimation as it runs trials to learn the model of the environment by estimating the utility of a state as a sum of reward for being in that state and the expected discounted reward of being in the next state.

$$U^\pi(s) = U^\pi(s) + \alpha(R(s) + \gamma U^\pi(s') - U^\pi(s))$$

Where $R(s)$ = reward for being in states, $P(s'|s, \pi(s))$ = transition model, γ = discount factor and $U^\pi(s)$ = utility of being in state's'.

It can be solved using **value-iteration algorithm**. The algorithm converges fast but can become quite costly to compute for large state spaces. ADP is a model-based approach and requires the transition model of the environment. A model-free approach is Temporal Difference Learning.

3. TEMPORAL DIFFERENCE LEARNING (TD)

TD learning does not require the agent to learn the transition model. The update occurs between successive states and agent only updates states that are directly affected

$$U^\pi(s) = U^\pi(s) + \alpha(R(s) + \gamma U^\pi(s') - U^\pi(s))$$

Where α = learning rate which determines the convergence to true utilities.

While ADP adjusts the utility of s with all its successor states, TD learning adjusts it with that of a single successor states'. TD is slower in convergence but much simpler in terms of computation.

ACTIVE LEARNING

ADP with exploration function

As the goal of an active agent is to learn an optimal policy, the agent needs to learn the expected utility of each state and update its policy. Can be done using a passive ADP agent and then using value or policy iteration it can learn optimal actions. But this approach results into a greedy agent. **Hence, we use an approach that gives higher weights to unexplored actions and lower weights to actions with lower utilities.**

$$U_{i+1}(s) \leftarrow R(s) + \gamma \max_{a \in A(s)} f \left(\sum_{s'} P(s'|s, a) U_i(s'), N(s, a) \right)$$

Where $f(u, n)$ is the exploration function that increases with expected value u and decreases with number of tries n

$$f(u, n) = \begin{cases} R^+ & \text{if } n < N_e \\ u & \text{otherwise} \end{cases}$$

R^+ is an optimistic reward and N_e is the number of times we want an agent to be forced to pick an action in every state. **The exploration function converts a passive agent into an active one.**

GENERALIZATION

IN

REINFORCEMENT

LEARNING

GENERALIZATION IN REINFORCEMENT LEARNING

- The study of generalisation in deep Reinforcement Learning (RL) aims to produce RL algorithms whose policies generalise well to novel unseen situations at deployment time, avoiding overfitting to their training environments.
- Tackling this is vital if we are to deploy reinforcement learning algorithms in real world scenarios, where the environment will be diverse, dynamic and unpredictable.
- This survey is an overview of this nascent field. We provide a unifying formalism and terminology for discussing different generalisation problems, building upon previous works.
- We go on to categorise existing benchmarks for generalisation, as well as current methods for tackling the generalisation problem. Finally, we provide a critical discussion of the current state of the field, including recommendations for future work.

- Among other conclusions, we argue that taking a purely procedural content generation approach to benchmark design is not conducive to progress in generalisation, we suggest fast online adaptation and tackling RL-specific problems as some areas for future work on methods for generalisation, and we recommend building benchmarks in underexplored problem settings such as offline RL generalisation and reward-function variation.

POLICY SEARCH

- Reinforcement learning is **a branch of machine learning** dedicated to training agents to operate in an environment, **in order to maximize their utility** in the pursuit of **some goals**.
- Its underlying idea, **states Russel**, is that intelligence is an **emergent property** of the interaction between an **agent and its environment**.
- This property guides the agent's actions by orienting its choices in the conduct of some tasks.
- We can say, analogously, that **intelligence is the capacity of the agent to select the appropriate strategy in relation to its goals**. Strategy, a teleologically-oriented subset of all possible behaviours, is here connected to the idea of “policy”.
- **A policy is, therefore, a strategy that an agent uses in pursuit of goals**. The policy dictates the actions that the agent takes as a function of the agent's state and the environment.

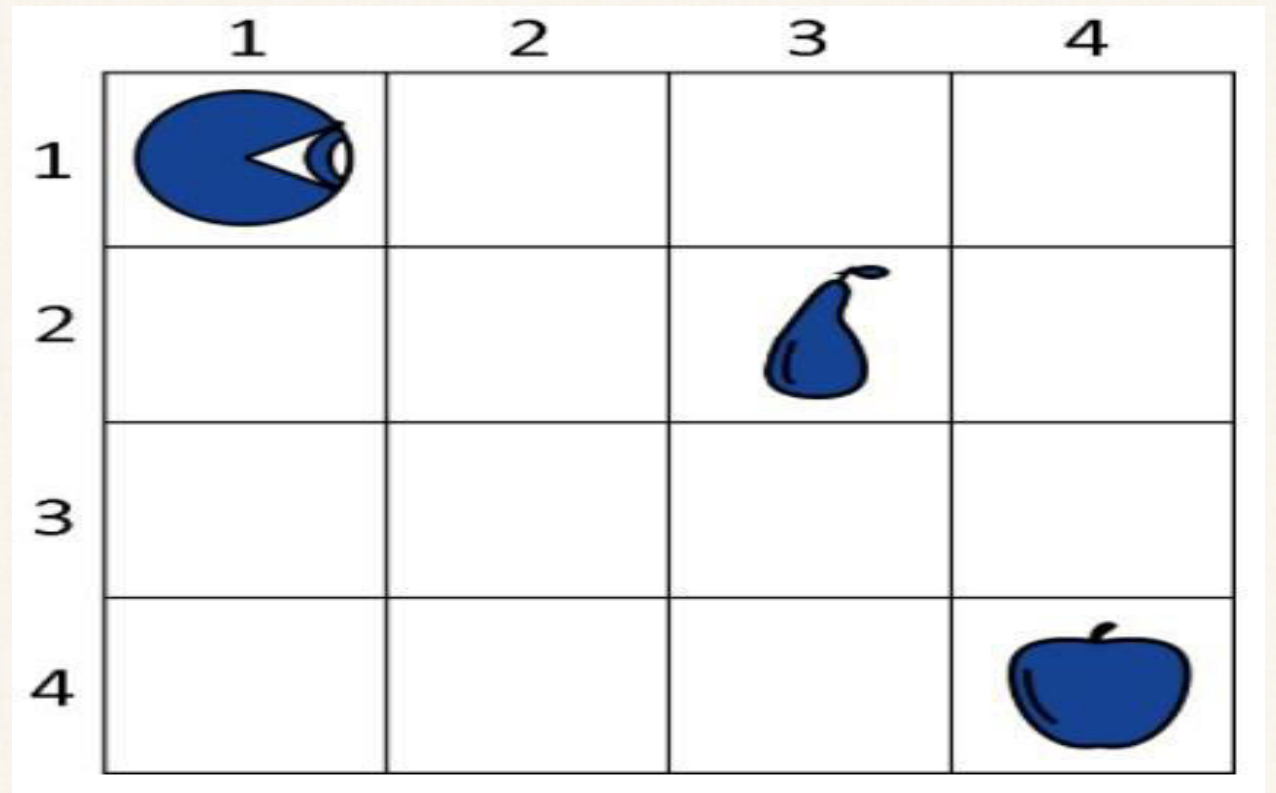
MATHEMATICAL DEFINITION OF A POLICY

- With formal terminology, we define a policy π in terms of the Markov Decision Process to which it refers. A Markov Decision Process is a tuple of the form (S,A,R,P) , structured as follows.
- The first element is a set S containing the internal states of the agent. Together, all possible states span a so-called state space for the agent. In the case of the grid worlds for agent simulations, S normally consists of the position of the agent on a board plus, if necessary, some parameters.
- The second element is a set A containing the actions of the agent. The actions correspond to the possible behaviors that the agent can take in relation to the environment. Together, the set of all actions spans the action space for that agent.
- An action can also lead to a modification of the state of the agent. This is represented by the matrix P containing the probability of transition from one state to another. Its elements, $P_a(S,S')$, contain the probabilities $\Pr(S'/S,A)$ for all possible actions $a \in A$ and pairs of states (S,S') , .

- The fourth element $R(s)$ comprises the reward function for the agent. It takes as input the state of the agent and outputs a real number that corresponds to the agent's reward.
- We can now formally define the policy, which we indicate with $\pi(s)$. **A policy $\pi(s)$ comprises the suggested actions that the agent should take for every possible state $s \in S$.**

EXAMPLE OF A POLICY IN REINFORCEMENT LEARNING

Let's now see an example of policy in a practical scenario, to better understand how it works. In this example, an agent has to forage food from the environment in order to satisfy its hunger. It then receives rewards on the basis of the fruit it eats:



The internal state of the agent corresponds to its location on the board, in this case, $s_t=(x,y)$ and $S_0=(1,1)$.

The action space, in this example, consists of four possible behaviours: $A=$ up, down, right. The probability matrix P contains all pairwise combinations of states (S,S') , for all actions in A . It's Bernoulli-distributed, and looks like this:

$$P_{\text{down}}((1,1), (1,2)) = 1; P_{\text{down}}((1,1), (1,3)) = 0; \dots; P_{\text{up}}((4,4), (4,3)) = 1$$

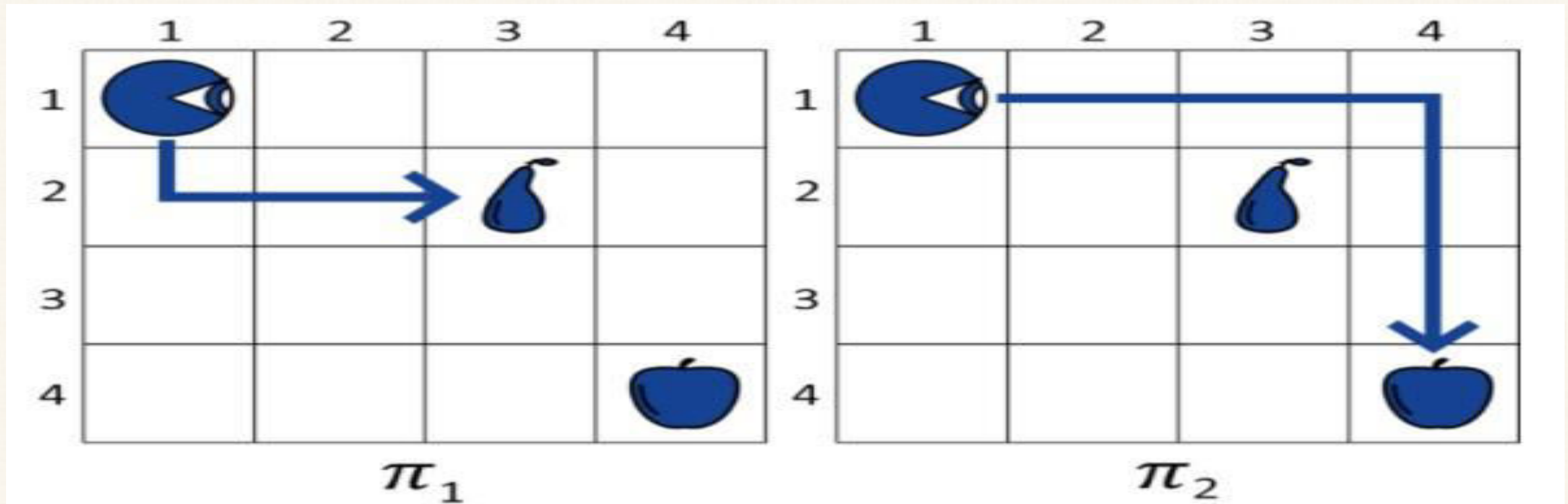
The reward function is defined in this manner. If it's in an empty cell, the agent receives a negative reward of -1, to simulate the effect of hunger. If instead, the agent is in a cell with fruit, in this case, (3,2) for the pear and (4,4) for the apple, it then receives a reward of +5 and +10, respectively.

- $R(\text{No fruit}) = -1$
- $R(\text{Pear}) = +5$
- $R(\text{Apple}) = +10$

EVALUATION OF THE POLICIES

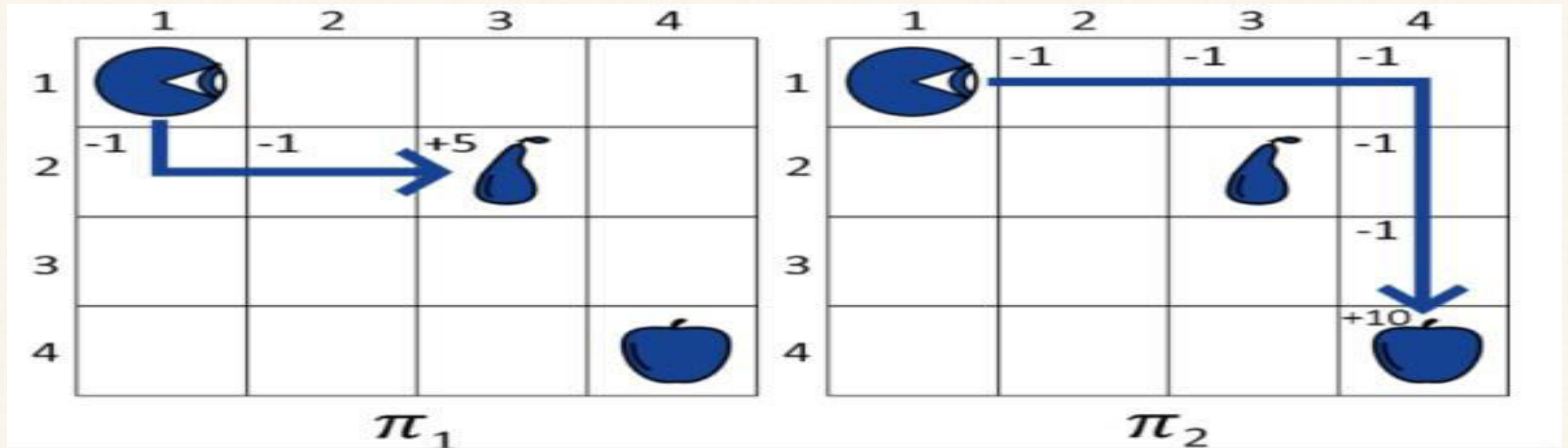
The agent then considers two policies π_1 and π_2 . If we simplify slightly the notation, we can indicate a policy as a sequence of actions starting from the state of the agent at s_0 :

1. π_1 = down, right, right $\rightarrow$ Pear
2. π_2 = right, right, right, down, down, down $\rightarrow$ Apple



The agent then has to select between the two policies. By computing the utility function U over them, the agent obtains:

- $U(\pi_1) = -1 - 1 + 5 = +3$
- $U(\pi_2) = -1 - 1 - 1 - 1 - 1 + 10 = +5$

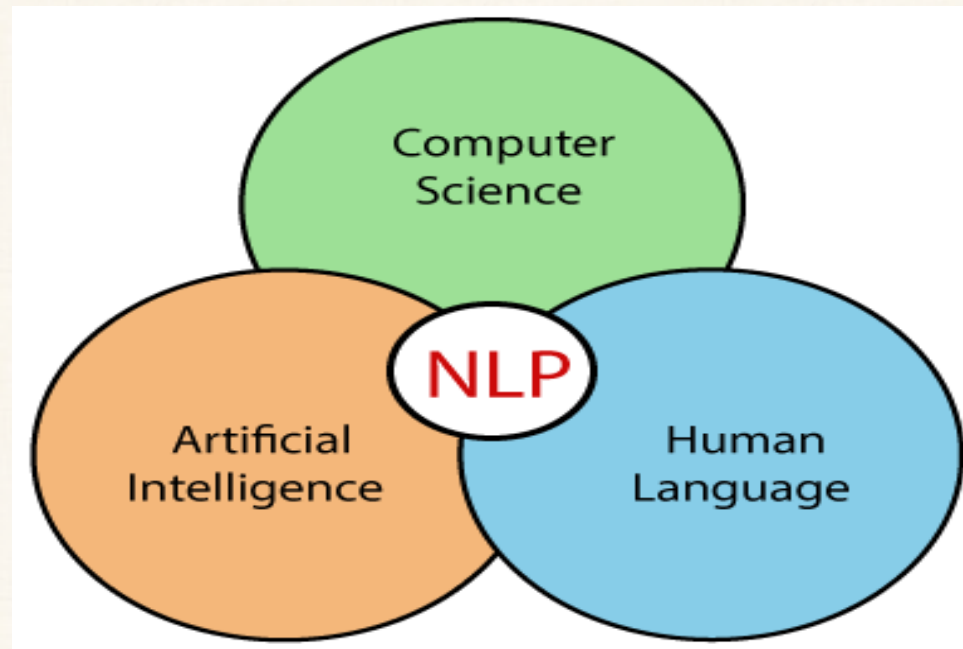


The evaluation of the policies suggests that the utility is maximized with π_2 , which then the agent chooses as its policy for this task.

NATURAL
LANGUAGE
PROCESSING

WHAT IS NLP?

- NLP stands for **Natural Language Processing**, which is a part of **Computer Science**, **Human language**, and **Artificial Intelligence**.
- It is the technology that is used by machines to understand, analyse, manipulate, and interpret human's languages. It helps developers to organize knowledge for performing tasks such as **translation, automatic summarization, Named Entity Recognition (NER), speech recognition, relationship extraction, and topic segmentation.**



HISTORY OF NLP

- **(1940-1960) - Focused on Machine Translation (MT)**
- **1948** - In the Year 1948, the first recognisable NLP application was introduced in Birkbeck College, London.
- **1950s** - In the Year 1950s, there was a conflicting view between linguistics and computer science
- **In 1957**, Chomsky also introduced the idea of Generative Grammar, which is rule based descriptions of syntactic structures.
- **(1960-1980) - Flavored with Artificial Intelligence (AI)**
- In the year 1960 to 1980, the key developments were:
- **Augmented Transition Networks (ATN)**
- Augmented Transition Networks is a finite state machine that is capable of recognizing regular languages.
- **Case Grammar** was developed by **Linguist Charles J. Fillmore** in the year 1968.
- **SHRDLU** is a program written by **Terry Winograd** in 1968-70.
- **LUNAR** is the classic example of a Natural Language database interface system that is used ATNs and Woods' Procedural Semantics.
- Till the year 1980, natural language processing systems were based on complex sets of hand-written rules. After 1980, NLP introduced machine learning algorithms for language processing.

ADVANTAGES OF NLP

- NLP helps users to ask questions about any subject and get a direct response within seconds.
- NLP offers exact answers to the question means it does not offer unnecessary and unwanted information.
- NLP helps computers to communicate with humans in their languages.
- It is very time efficient.
- Most of the companies use NLP to improve the efficiency of documentation processes, accuracy of documentation, and identify the information from large databases.

DISADVANTAGES OF NLP

A list of disadvantages of NLP is given below:

- NLP may not show context.
- NLP is unpredictable
- NLP may require more keystrokes.
- NLP is unable to adapt to the new domain, and it has a limited function that's why NLP is built for a single and specific task only.

COMPONENTS OF NLP

There are the following two components of NLP -

1. Natural Language Understanding (NLU)

Natural Language Understanding (NLU) helps the machine to understand and analyse human language by extracting the metadata from content such as concepts, entities, keywords, emotion, relations, and semantic roles.

NLU mainly used in Business applications to understand the customer's problem in both spoken and written language.

NLU involves the following tasks -

- It is used to map the given input into useful representation.
- It is used to analyse different aspects of the language.

2. NATURAL LANGUAGE GENERATION (NLG)

Natural Language Generation (NLG) acts as a translator that converts the computerized data into natural language representation. It mainly involves Text planning, Sentence planning, and Text Realization.

DIFFERENCE BETWEEN NLU AND NLG

NLU	NLG
NLU is the process of reading and interpreting language.	NLG is the process of writing or generating language.
It produces non-linguistic outputs from natural language inputs.	It produces constructing natural language outputs from non-linguistic inputs.

LANGUAGE MODELS

- Formal languages, such as the programming languages Java or Python, have precisely defined language models.
- A **language** can be defined as a set of strings; “print(2 + 2)” is a legal program in the language Python, whereas “2)+(2 print” is not.
- Since there are an infinite number of legal programs, they cannot be enumerated; instead they are specified by set of rules called a **grammar**.
- Formal languages also have rules that define the meaning or **semantics** of a program; for example, the rules say that the “meaning” of “2 + 2” is 4, and the meaning of “1/0” is that an error is signaled.
- Natural languages, such as English or Spanish, cannot be characterized as a definitive set of sentences.
- Everyone agrees that “Not to be invited is sad” is a sentence of English, but people disagree on the grammaticality of “To be not invited is sad.”

- Therefore, it is more fruitful to define a natural language model as a probability distribution over sentences rather than a definitive set.

$$P(S = \text{words})$$

- Natural languages are also **ambiguous**. Because we cannot speak of a single meaning for a sentence, but rather of a probability distribution over possible meanings.
- Finally, natural languages are difficult to deal with because they are very large, and constantly changing.

***N*-gram character models**

- Ultimately, a written text is composed of **characters**—letters, digits, punctuation, and spaces in English (and more exotic characters in some other languages).
- Thus, one of the simplest language models is a probability distribution over sequences of characters.

- We write $P(C1:N)$ for the probability of a sequence of N characters, $C1$ through CN .
- In one Web collection, $P(\text{“the”}) = 0.027$ and $P(\text{“zgq”}) = 0.0000000002$.
- A sequence of written symbols of length n is called an n -gram (from the Greek root for writing or letters), with special case “unigram” for 1-gram, “bigram” for 2-gram, and “trigram” for 3-gram.
- A model of the probability distribution of n -letter sequences is thus called an **n -gram model**. (But be careful: we can have n -gram models over sequences of words, syllables, or other units; not just over characters.)
- An n -gram model is defined as a **Markov chain** of order $n - 1$.
- In a Markov chain the probability of character c_i depends only on the immediately preceding characters, not on any other characters.

$$P(c_i \mid c_{1:i-1}) = P(c_i \mid c_{i-2:i-1})$$

- We can define the probability of a sequence of characters $P(c_{1:N})$ under the trigram model by first factoring with the chain rule and then using the Markov assumption:

$$P(c_{1:N}) = \prod_{i=1}^N P(c_i | c_{1:i-1}) = \prod_{i=1}^N P(c_i | c_{i-2:i-1})$$

- We call a body of text a **corpus** (plural *corpora*), from the Latin word for *body*.
- What can we do with n-gram character models? One task for which they are well suited is **language identification**.

$$\begin{aligned} \ell^* &= \operatorname{argmax}_{\ell} P(\ell | c_{1:N}) \\ &= \operatorname{argmax}_{\ell} P(\ell) P(c_{1:N} | \ell) \\ &= \operatorname{argmax}_{\ell} P(\ell) \prod_{i=1}^N P(c_i | c_{i-2:i-1}, \ell) \end{aligned}$$

Smoothing n -gram models

- The major complication of n -gram models is that the training corpus provides only an estimate of the true probability distribution.
- For common character sequences such as “_th” any English corpus will give a good estimate: about 1.5% of all trigrams. On the other hand, “_ht” is very uncommon—no dictionary words start with ht.
 - The process of adjusting the probability of low-frequency counts is called **smoothing**.
- The simplest type of smoothing was suggested by **Pierre-Simon Laplace** in the 18th century: he said that, in the lack of further information, if a random Boolean variable X has been false in all n observations so far then the estimate for $P(X = \text{true})$ should be $1/(n+2)$.
- That is, he assumes that with two more trials, one might be true and one false. Laplace smoothing (also called add-one smoothing) is a step in the right direction, but performs relatively poorly.
- A better approach is a **back-off model**, in which we start by estimating n -gram counts, but for any particular sequence that has a low (or zero) count, we back off to $(n - 1)$ -grams.
- **Linear interpolation smoothing** is a back-off model that combines trigram, bigram, and unigram models by linear interpolation. It defines the probability estimate as

$$\hat{P}(c_i|c_{i-2:i-1}) = \lambda_3 P(c_i|c_{i-2:i-1}) + \lambda_2 P(c_i|c_{i-1}) + \lambda_1 P(c_i)$$

- where $\lambda_3 + \lambda_2 + \lambda_1 = 1$. The parameter values λ_i can be fixed, or they can be trained with an expectation–maximization algorithm.
- It is also possible to have the values of λ_i depend on the counts: if we have a high count of trigrams, then we weigh them relatively more; if only a low count, then we put more weight on the bigram and unigram models.

Model Evaluation

- With so many possible n-gram models—unigram, bigram, trigram, interpolated smoothing with different values of λ , etc.—how do we know what model to choose? We can evaluate a model with cross-validation.
- Split the corpus into a training corpus and a validation corpus. Determine the parameters of the model from the training data. Then evaluate the model on the validation corpus.
- The evaluation can be a task-specific metric, such as measuring accuracy on language identification.
- Alternatively we can have a task-independent model of language quality: calculate the probability assigned to the validation corpus by the model; the higher the probability the better.
- This metric is inconvenient because the probability of a large corpus will be a very small number, and floating-point underflow becomes an issue.
- A different way of describing the probability of a sequence is with a measure called **perplexity**,

$$\text{Perplexity}(c_{1:N}) = P(c_{1:N})^{-\frac{1}{N}}$$

- Perplexity can be thought of as the reciprocal of probability, normalized by sequence length.
- It can also be thought of as the weighted average branching factor of a model. Suppose there are 100 characters in our language, and our model says they are all equally likely. Then for a sequence of any length, the perplexity will be 100.
- If some characters are more likely than others, and the model reflects that, then the model will have a perplexity less than 100.

N-gram word models

- Now we turn to n-gram models over words rather than characters.
- All the same mechanism applies equally to word and character models. The main difference is that the **vocabulary**—the set of symbols that make up the corpus and the model—is larger.
- There are only about 100 characters in most languages, and sometimes we build character models that are even more restrictive, for example by treating “A” and “a” as the same symbol or by treating all punctuation as the same symbol.
- But with word models we have at least tens of thousands of symbols, and sometimes millions.
- The wide range is because it is not clear what constitutes a word.
- In English a sequence of letters surrounded by spaces is a word, but in some languages, like Chinese, words are not separated by spaces, and even in English many decisions must be made to have a clear policy on word boundaries: how many words are in.
- Word n-gram models need to deal with **out of vocabulary** words.
- With character models, we didn’t have to worry about someone inventing a new letter of the alphabet.
- But with word models there is always the chance of a new word that was not seen in the training corpus, so we need to model that explicitly in our language model.

TEXT

CLASSIFICATION

TEXT CLASSIFICATION

- We now consider in depth the task of **text classification**, also known as **categorization**: given a text of some kind, decide which of a predefined set of classes it belongs to. Language identification and genre classification are examples of text classification
- **spam detection** classifying an email message as spam or not-spam(ham).
- A training set is readily available: the positive (spam) examples are in my spam folder, the negative (ham) examples are in my inbox.
- Note that we have two complementary ways of talking about classification.
- In the language-modeling approach, we define one n-gram language model for $P(\text{Message} \mid \text{spam})$ by training on the spam folder, and one model for $P(\text{Message} \mid \text{ham})$ by training on the inbox.
- Then we can classify a new message with an application of Bayes' rule:

$$\operatorname{argmax}_{c \in \{spam, ham\}} P(c | message) = \operatorname{argmax}_{c \in \{spam, ham\}} P(message | c) P(c)$$

- where $P(c)$ is estimated just by counting the total number of spam and ham messages. This approach works well for spam detection, just as it did for language identification.
- If there are 100,000 words in the language model, then the feature vector has length 100,000, but for a short email message almost all the features will have count zero.
- This unigram representation has been called the **bag of words** model.
- You can think of the model as putting the words of the training corpus in a bag and then selecting words one at a time.
- The notion of order of the words is lost; a unigram model gives the same probability to any permutation of a text.
- Higher-order n-gram models maintain some local notion of word order.
- It can be expensive to run algorithms on a very large feature vector, so often a process of **feature selection** is used to keep only the features that best discriminate between spam and ham.

CLASSIFICATION BY DATA COMPRESSION

- Another way to think about classification is as a problem in **data compression**.
- A lossless compression algorithm takes a sequence of symbols, detects repeated patterns in it, and writes a description of the sequence that is more compact than the original.
- For example, the text “0.142857142857142857” might be compressed to “0.[142857]*3.”
- To do classification by compression, we first lump together all the spam training messages and compress them as a unit.
- We do the same for the ham. Then when given a new message to classify, we append it to the spam messages and compress the result.
- We also append it to the ham and compress that. Whichever class compresses better—adds the

INFORMATION RETRIEVAL

- Information retrieval is the task of finding documents that are relevant to a user's need for information.
- The best-known examples of information retrieval systems are search engines on the World Wide Web.
- A Web user can type a query such as “AI book” into a search engine and see a list of relevant pages.
- An **information retrieval** (henceforth **IR**) system can be **characterized** by :
- A **corpus of documents**. Each system must decide what it wants to treat as a document: a paragraph, a page, or a multipage text.
- **Queries posed in a query language**. A query specifies what the user wants to know. The query language can be just a list of words, such as [AI book]; or it can specify a phrase of words that must be adjacent, as in [“AI book”]; it can contain Boolean operators as in [AI AND book]; it can include non-Boolean operators such as [AI NEAR book].

- **A result set.** This is the subset of documents that the IR system judges to be **relevant** to the query.
- **A presentation of the result set.** This can be as simple as a ranked list of document titles or as complex as a rotating color map of the result set projected onto a three dimensional space, rendered as a two-dimensional display.
- First, the degree of relevance of a document is a single bit, so there is no guidance as to how to order the relevant documents for presentation.
- Second, Boolean expressions are unfamiliar to users who are not programmers or logicians.
- Third, it can be hard to formulate an appropriate query, even for a skilled user.

IR scoring functions

- Most IR systems have abandoned the Boolean model and use models based on the statistics of word counts. We describe the **BM25 scoring function**.
- A scoring function takes a document and a query and returns a numeric score; the most relevant documents have the highest scores.
 - In the **BM25** function, the score is a linear weighted combination of scores for each of the words that make up the query.

Three factors affect the weight of a query term:

- First, the **frequency** with which a query term appears in a document (also known as TF for term frequency). For the query documents that mention “farming” frequently will have higher scores.
- Second, the inverse document frequency of the term, or **IDF**. The word “in” appears in almost every document, so it has a high document frequency, and thus a low inverse document frequency, and thus it is not as important to the query.
- Third, the **length of the document**. A million-word document will probably mention all the query words, but may not actually be about the query. A short document that mentions all the words is a much better candidate.

$$BM25(d_j, q_{1:N}) = \sum_{i=1}^N IDF(q_i) \cdot \frac{TF(q_i, d_j) \cdot (k + 1)}{TF(q_i, d_j) + k \cdot (1 - b + b \cdot \frac{|d_j|}{L})}$$

IDF(qi) is the inverse document frequency of word qi, given by

$$IDF(q_i) = \log \frac{N - DF(q_i) + 0.5}{DF(q_i) + 0.5}$$

IR system evaluation

- How do we know whether an IR system is performing well? We undertake an experiment in which the system is given a set of queries and the result sets are scored with respect to human relevance judgments.
- Traditionally, there have been two measures used in the scoring.
 - recall
 - precision.

- **Precision** measures the proportion of documents in the result set that are actually relevant.
- In our example, the precision is $30/(30 + 10) = .75$. The false positive rate is $1 - .75 = .25$.
- **Recall** measures the proportion of all the relevant documents in the collection that are in the result set.
- In our example, recall is $30/(30 + 20) = .60$. The false negative rate is $1 - .60 = .40$.
- In a very large document collection, such as the World Wide Web, recall is difficult to compute, because there is no easy way to examine every page on the Web for relevance.
- All we can do is either estimate recall by sampling or ignore recall completely and just judge precision.

IR refinements

- There are many possible refinements to the system described here, and indeed Web search engines are continually updating their algorithms as they discover new approaches and as the Web grows and changes.
- One common refinement is a better model of the effect of document length on relevance.
- Singhal *et al.* (1996) observed that simple document length normalization schemes tend to favor short documents too much and long documents not enough.
- They propose a *pivoted* document length normalization scheme; the idea is that the pivot is the document length at which the old-style normalization is correct; documents shorter than that get a boost and longer ones get a penalty.

- The **BM25** scoring function uses a word model that treats all words as completely independent, but we know that some words are correlated.
- Many IR systems attempt to account for these correlations.
- The next step is to recognize **synonyms**, such as “sofa” for “couch.” As with stemming, this has the potential for small gains in recall, but can hurt precision.
- As a final refinement, IR can be improved by considering **metadata**—data outside of the text of the document. Examples include human-supplied keywords and publication data.
- On the Web, hypertext **links** between documents are a crucial source of information.

The PageRank algorithm

- **PageRank** was one of the two original ideas that set Google's search apart from other Web search engines when it was introduced in 1997. (The other innovation was the use of anchor text—the underlined text in a hyperlink).
- PageRank was invented to solve the problem of the tyranny of TF scores: if the query is [IBM], how do we make sure that IBM's home page, ibm.com, is the first result, even if another page mentions the term “IBM” more frequently?
 - The idea is that ibm.com has many in-links (links to the page), so it should be ranked higher: each in-link is a vote for the quality of the linked-to page.
- But if we only counted in-links, then it would be possible for a Web spammer to create a network of pages and have them all point to a page of his choosing, increasing the score of that page.

- Therefore, the PageRank algorithm is designed to weight links from high-quality sites more heavily.
- What is a high quality site? One that is linked to by other high-quality sites.
- The definition is recursive, but we will see that the recursion bottoms out properly. The PageRank for a page p is defined as:

$$PR(p) = \frac{1-d}{N} + d \sum_i \frac{PR(in_i)}{C(in_i)}$$

- where $PR(p)$ is the PageRank of page p , N is the total number of pages in the corpus, in_i are the pages that link in to p , and $C(in_i)$ is the count of the total number of out-links on page in_i .
- The constant d is a damping factor. It can be understood through the **random surfer model** : imagine a Web surfer who starts at some random page and begins exploring.

The HITS algorithm

- The Hyperlink-Induced Topic Search algorithm, also known as “Hubs and Authorities” or HITS, is another influential link-analysis algorithm .
- HITS differs from PageRank in several ways.
- First, it is a query-dependent measure: it rates pages with respect to a query.
- Given a query, HITS first finds a set of pages that are relevant to the query. It does that by intersecting hit lists of query words, and then adding pages in the link neighborhood of these pages
- Both PageRank and HITS played important roles in developing our understanding of Web information retrieval.
- These algorithms and their extensions are used in ranking billions of queries daily as search engines steadily develop better ways of extracting yet finer signals of search relevance.

Question answering

- Information retrieval is the task of finding documents that are relevant to a query, where the query may be a question, or just a topic area or concept.
- **Question answering** is a somewhat different task, in which the query really is a question, and the answer is not a ranked list of documents but rather a short response—a sentence, or even just a phrase.
- There have been question-answering NLP (natural language processing) systems since the 1960s, but only since 2001 have such systems used Web information retrieval to radically increase their breadth of coverage.

INFORMATION

EXTRACTION

INFORMATION EXTRACTION

- Information extraction is the process of acquiring knowledge by skimming a text and looking for occurrences of a particular class of object and for relationships among objects.
- A typical task is to extract instances of addresses from Web pages, with database fields for street, city, state, and zip code; or instances of storms from weather reports, with fields for temperature, wind speed, and precipitation.
- In a limited domain, this can be done with high accuracy. As the domain gets more general, more complex linguistic models and more complex learning techniques are necessary.

Finite-state automata for information extraction:

- The simplest type of information extraction system is an **attribute-based extraction** system that assumes that the entire text refers to a single object and the task is to extract attributes of that object.
- For example, the problem of extracting from the text “IBM Think Book 970. Our price: \$399.00” the set of attributes {Manufacturer=IBM, Model=ThinkBook970, Price=\$399.00}.
- We can address this problem by defining a **template** (also known as a pattern) for each attribute we would like to extract. The template is defined by a finite state automaton, the simplest example of which is the **regular expression**, or regex.

<code>[0-9]</code>	matches any digit from 0 to 9
<code>[0-9]+</code>	matches one or more digits
<code>[.] [0-9] [0-9]</code>	matches a period followed by two digits
<code>([.] [0-9] [0-9]) ?</code>	matches a period followed by two digits, or nothing
<code>[\$] [0-9]+ ([.] [0-9] [0-9]) ?</code>	matches \$249.99 or \$1.23 or \$1000000 or ...

- One step up from attribute-based extraction systems are **relational extraction** systems, which deal with multiple objects and the relations among them.
- Thus, when these systems see the text “\$249.99,” they need to determine not just that it is a price, but also which object has that price.
- A typical relational-based extraction system is **FASTUS**, which handles news stories about corporate mergers and acquisitions.
- A relational extraction system can be built as a series of **cascaded finite-state transducers**.
- That is, the system consists of a series of small, efficient finite-state automata (FSAs), where each automaton receives text as input, transduces the text into a different format, and passes it along to the next automaton.

FASTUS consists of five stages

1. Tokenization
2. Complex-word handling
3. Basic-group handling
4. Complex-phrase handling
5. Structure merging

1. **FASTUS's** first stage is **tokenization**, which segments the stream of characters into tokens (words, numbers, and punctuation). Some tokenizers also deal with markup languages such as **HTML, SGML, and XML**.
2. The second stage handles **complex words**, including collocations such as “set up” and “joint venture,” as well as proper names such as **“Bridgestone Sports Co.”**

3 The third stage handles **basic groups**, meaning noun groups and verb groups. The idea is to chunk these into units that will be managed by the later stages.

4 The fourth stage combines the basic groups into **complex phrases**.

5 The final stage **merges structures** that were built up in the previous step.

Probabilistic models for information extraction

- When information extraction must be attempted from noisy or varied input, simple finite-state approaches fare poorly.
- It is too hard to get all the rules and their priorities right; it is better to use a probabilistic model rather than a rule-based model.
- The simplest probabilistic model for sequences with hidden state is the hidden Markov model, or HMM.
- HMM models a progression through a sequence of hidden states, $\mathbf{x}_t$, with an observation $\mathbf{e}_t$ at each step.
- To apply HMMs to information extraction, we can either build one big HMM for all the attributes or build a separate HMM for each attribute. We'll do the second.

Conditional random fields for information extraction

- One issue with **HMMs** for the information extraction task is that they model a lot of probabilities that we don't really need.
- Modeling this directly gives us some freedom. We don't need the independence assumptions of the Markov model—we can have an $\mathbf{x}_t$ that is dependent on $\mathbf{x}_1$.
- A framework for this type of model is the **conditional random field**, or **CRF**, which models a conditional probability distribution of a set of target variables given a set of observed variables.
- Like Bayesian networks, CRFs can represent many different structures of dependencies among the variables.
- One common structure is the **linear-chain conditional random field** for representing Markov dependencies among variables in a temporal sequence.
- Thus, **HMMs** are the temporal version of naive Bayes models, and linear-chain CRFs are the temporal version of logistic regression.

Ontology extraction from large corpora

- So far we have thought of information extraction as finding a specific set of relations (e.g., speaker, time, location) in a specific text (e.g., a talk announcement).
- A different application of extraction technology is building a large knowledge base or ontology of facts from a corpus.
- This is different in three ways:
- First it is open-ended—we want to acquire facts about all types of domains, not just one specific domain.
- Second, with a large corpus, this task is dominated by precision, not recall—just as with question answering on the Web .
- Third, the results can be statistical aggregates gathered from multiple sources, rather than being extracted from one specific text.

Automated template construction

- Fortunately, it is possible to *learn* templates from a few examples, then use the templates to learn more examples, from which more templates can be learned, and so on.
- In one of the first experiments of this kind, Brin (1999) started with a data set of just five examples
 - (“Isaac Asimov”, “The Robots of Dawn”)
 - (“David Brin”, “Startide Rising”)
 - (“James Gleick”, “Chaos—Making a New Science”)
 - (“Charles Dickens”, “Great Expectations”)
 - (“William Shakespeare”, “The Comedy of Errors”)
- Clearly these are examples of the author–title relation, but the learning system had no knowledge of authors or titles.
- The words in these examples were used in a search over a Web corpus, resulting in 199 matches. Each match is defined as a tuple of seven strings,

(*Author, Title, Order, Prefix, Middle, Postfix, URL*) ,

- where *Order* is true if the author came first and false if the title came first, *Middle* is the characters between the author and title, *Prefix* is the 10 characters before the match, *Suffix* is the 10 characters after the match, and *URL* is the Web address where the match was made.

Machine reading

- Automated template construction is a big step up from handcrafted template construction, but it still requires a handful of labeled examples of each relation to get started.
- To build a large ontology with many thousands of relations, even that amount of work would be onerous; we would like to have an extraction system with *no* human input of any kind—a system that could read on its own and build up its own database.
- Such a system would be relation-independent; would work for any relation. In practice, these systems work on *all* relations in parallel, because of the I/O demands of large corpora.
- They behave less like a traditional information extraction system that is targeted at a few relations and more like a human reader who learns from the text itself; because of this the field has been called **machine reading**.

INTRODUCTION

Communication is the intentional exchange of information brought about by the production SIGN and perception of signs drawn from a shared system of conventional signs. Most animals use signs to represent important messages: food here, predator nearby, approach, withdraw, let's mate.

PHRASE STRUCTURE GRAMMARS

- The n-gram language models were based on sequences of words.
- The big issue for these models is **data sparsity**—with a vocabulary of, say, trigram probabilities to estimate, and so a corpus of even a trillion words will not be able to supply reliable estimates for all of them.
- We can address the problem of sparsity through generalization.
- Despite the exceptions, the notion of a **lexical category** (also known as a **part of speech**) such as *noun* or *adjective* is a useful generalization—useful in its own right, but more so when we string together lexical categories to form **syntactic categories** such as *noun phrase* or *verb phrase*, and combine these syntactic categories into trees representing the **phrase structure** of sentences: nested phrases, each marked with a category .

GENERATIVE CAPACITY

- Grammatical formalisms can be classified by their **generative capacity**: the set of languages they can represent.
- Chomsky (1957) describes four classes of grammatical formalisms that differ only in the form of the rewrite rules.
- The classes can be arranged in a hierarchy, where each class can be used to describe all the languages that can be described by a less powerful class, as well as some additional languages.
- Here we list the hierarchy, most powerful class first:

1. **Recursively enumerable** grammars use unrestricted rules: both sides of the rewrite rules can have any number of terminal and nonterminal symbols, as in the rule $A B C \rightarrow D E$.
 - These grammars are equivalent to Turing machines in their expressive power.

2. Context-sensitive grammars are restricted only in that the right-hand side must contain at least as many symbols as the left-hand side.

- The name “context sensitive” comes from the fact that a rule such as $A X B \rightarrow A Y B$ says that an X can be rewritten as a Y in the context of a preceding A and a following B .
- Context-sensitive grammars can represent languages such as $\{a^n b^n c^n \mid n \geq 1\}$ (a sequence of n copies of a followed by the same number of b s and then c s).

3. In context-free grammars (or **CFGs**), the left-hand side consists of a single nonterminal symbol. Thus, each rule licenses rewriting the nonterminal as the right-hand side in *any* context.

- CFGs are popular for natural-language and programming-language grammars, although it is now widely accepted that at least some natural languages have constructions that are not context-free (Pullum, 1991).

- Context-free grammars can represent

$a^n b^n$, but not $a^n b^n c^n$.

Regular grammars are the most restricted class. Every rule has a single nonterminal on the left-hand side and a terminal symbol optionally followed by a nonterminal on the right-hand side.

- Regular grammars are equivalent in power to finite state machines. They are poorly suited for programming languages, because they cannot represent constructs such as balanced opening and closing parentheses .
- The closest they can come is representing a^*b^* , a sequence of any number of a s followed by any number of b s.
- There have been many competing language models based on the idea of phrase structure; we will describe a popular model called the **probabilistic context-free grammar**, or PCFG.
- A **grammar** is a collection of rules that defines a **language** as a set of allowable strings of words. Probabilistic means that the grammar assigns a probability to every string.

VP $\rightarrow$ Verb [0.70]

VP NP [0.30]

- Here VP (*verb phrase*) and NP (*noun phrase*) are **non-terminal symbols**. The grammar also refers to actual words, which are called **terminal symbols**.
- This rule is saying that with probability 0.70 a verb phrase consists solely of a verb, and with probability 0.30 it is a VP followed by an NP.

The lexicon of

- First we define the **lexicon**, or list of allowable words. The words are grouped into the lexical categories familiar to dictionary users: nouns, pronouns, and names to denote things; verbs to denote events; adjectives to modify nouns; adverbs to modify verbs; and function words: articles (such as *the*), prepositions (*in*), and conjunctions (*and*).
- Each of the categories ends in . . . to indicate that there are other words in the category.

$\mathcal{E}_0$

The Grammar of

The next step is to combine the words into phrases.

A grammar for $\mathcal{E}_0$ with rules for each of the six syntactic categories and an example for each rewrite rule.

$\mathcal{E}_0 :$	S	$\rightarrow$	$NP\ VP$	[0.90]	I + feel a breeze
			$S\ Conj\ S$	[0.10]	I feel a breeze + and + It stinks
	NP	$\rightarrow$	$Pronoun$	[0.30]	I
			$Name$	[0.10]	John
			$Noun$	[0.10]	pits
			$Article\ Noun$	[0.25]	the + wumpus
			$Article\ Adjs\ Noun$	[0.05]	the + smelly dead + wumpus
			$Digit\ Digit$	[0.05]	3 4
			$NP\ PP$	[0.10]	the wumpus + in 1 3
			$NP\ RelClause$	[0.05]	the wumpus + that is smelly
	VP	$\rightarrow$	$Verb$	[0.40]	stinks
			$VP\ NP$	[0.35]	feel + a breeze
			$VP\ Adjective$	[0.05]	smells + dead
			$VP\ PP$	[0.10]	is + in 1 3
			$VP\ Adverb$	[0.10]	go + ahead
	$Adjs$	$\rightarrow$	$Adjective$	[0.80]	smelly
			$Adjective\ Adjs$	[0.20]	smelly + dead
	PP	$\rightarrow$	$Prep\ NP$	[1.00]	to + the east
	$RelClause$	$\rightarrow$	$RelPro\ VP$	[1.00]	that + is smelly

Figure 23.2 The grammar for $\mathcal{E}_0$, with example phrases for each rule. The syntactic categories are sentence (S), noun phrase (NP), verb phrase (VP), list of adjectives ($Adjs$), prepositional phrase (PP), and relative clause ($RelClause$).

SYNTACTIC ANALYSIS (PARSING)

➤ **Parsing** is the process of analyzing a string of words to uncover its phrase structure, according to the rules of a grammar.

1. Have the students in section 2 of Computer Science 101 take the exam.

2. Have the students in section 2 of Computer Science 101 taken the exam?

- If the algorithm guesses wrong, it will have to backtrack all the way to the first word and reanalyze the whole sentence under the other interpretation.
- To avoid this source of inefficiency we can use dynamic programming: *every time we analyze a substring, store the results so we won't have to reanalyze it later.*
- For example, once we discover that “the students in section 2 of Computer Science 101” is an NP, we can record that result in a data structure known as a **chart**.
- Algorithms that do this are called **chart parsers**.
- There are many types of chart parsers; we describe a bottom-up version called the **CYK algorithm**, after its inventors, **John Cocke, Daniel Younger, and Tadeo Kasami**.

CYK algorithm

function CYK-PARSE(*words*, *grammar*) **returns** P , a table of probabilities

$N \leftarrow \text{LENGTH}(\textit{words})$

$M \leftarrow$ the number of nonterminal symbols in *grammar*

$P \leftarrow$ an array of size $[M, N, N]$, initially all 0

/ Insert lexical rules for each word */*

for $i = 1$ **to** N **do**

for each rule of form $(X \rightarrow \textit{words}_i [p])$ **do**

$P[X, i, 1] \leftarrow p$

/ Combine first and second parts of right-hand sides of rules, from short to long */*

for $\textit{length} = 2$ **to** N **do**

for $\textit{start} = 1$ **to** $N - \textit{length} + 1$ **do**

for $\textit{len1} = 1$ **to** $N - 1$ **do**

$\textit{len2} \leftarrow \textit{length} - \textit{len1}$

for each rule of the form $(X \rightarrow Y Z [p])$ **do**

$P[X, \textit{start}, \textit{length}] \leftarrow \text{MAX}(P[X, \textit{start}, \textit{length}],$

$P[Y, \textit{start}, \textit{len1}] \times P[Z, \textit{start} + \textit{len1}, \textit{len2}] \times p)$

return P

Figure 23.5 The CYK algorithm for parsing. Given a sequence of words, it finds the most probable derivation for the whole sequence and for each subsequence. It returns the whole table, P , in which an entry $P[X, \textit{start}, \textit{len}]$ is the probability of the most probable X of length $\textit{len}$ starting at position $\textit{start}$. If there is no X of that size at that location, the probability is 0.

Learning probabilities for PCFGs

- A PCFG has many rules, with a probability for each rule.
- This suggests that **learning** the grammar from data might be better than a knowledge engineering approach.
- Learning is easiest if we are given a corpus of correctly parsed sentences, commonly called a **treebank**.
- The Penn Treebank is the best known; it consists of 3 million words which have been annotated with part of speech and parse-tree structure, using human labor assisted by some automated tools.

```

[ [S [NP-SBJ-2 Her eyes]
  [VP were
    [VP glazed
      [NP *-2]
      [SBAR-ADV as if
        [S [NP-SBJ she]
          [VP did n't
            [VP [VP hear [NP *-1]]
              or
              [VP [ADVP even] see [NP *-1]]
              [NP-1 him]]]]]]]]
  .]

```

Figure 23.6 Annotated tree for the sentence “Her eyes were glazed as if she didn’t hear or even see him.” from the Penn Treebank. Note that in this grammar there is a distinction between an object noun phrase (*NP*) and a subject noun phrase (*NP-SBJ*). Note also a grammatical phenomenon we have not covered yet: the movement of a phrase from one part of the tree to another. This tree analyzes the phrase “hear or even see him” as consisting of two constituent *VP*s, [VP hear [NP *-1]] and [VP [ADVP even] see [NP *-1]], both of which have a missing object, denoted *-1, which refers to the *NP* labeled elsewhere in the tree as [NP-1 him].

Comparing context-free and Markov models

- The problem with PCFGs is that they are context-free.
- That means that the difference between $P(\text{“eat a banana”})$ and $P(\text{“eat a bandanna”})$ depends only on $P(\text{Noun} \rightarrow \text{“banana”})$ versus $P(\text{Noun} \rightarrow \text{“bandanna”})$ and not on the relation between “eat” and the respective objects.
- A Markov model of order two or more, given a sufficiently large corpus, *will* know that “eat a banana” is more probable.
- We can combine a PCFG and Markov model to get the best of both. The simplest approach is to estimate the probability of a sentence with the geometric mean of the probabilities computed by both models.
 - Another problem with PCFGs is that they tend to have too strong a preference for shorter sentences.

AUGMENTED GRAMMARS AND SEMANTIC INTERPRETATION

Lexicalized PCFGs

- To get at the relationship between the verb “eat” and the nouns “banana” versus “bandanna,” we can use a **lexicalized PCFG**, in which the probabilities for a rule depend on the relationship between words in the parse tree, not just on the adjacency of words in a sentence.
- Of course, we can’t have the probability depend on every word in the tree, because we won’t have enough training data to estimate all those probabilities.
- It is useful to introduce the notion of the **head** of a phrase—the most important word. Thus, “eat” is the head of the VP “eat a banana” and “banana” is the head of the NP “a banana.”
- We use the notation $VP(v)$ to denote a phrase with category VP whose head word is v . We say that the category VP is **augmented** with the head variable v .

$VP(v) \rightarrow Verb(v) NP(n)$	$[P_1(v, n)]$
$VP(v) \rightarrow Verb(v)$	$[P_2(v)]$
$NP(n) \rightarrow Article(a) Adjs(j) Noun(n)$	$[P_3(n, a)]$
$Noun(\mathbf{banana}) \rightarrow \mathbf{banana}$	$[p_n]$
$\dots$	$\dots$

Formal definition of augmented grammar rules

- Augmented rules are complicated, so we will give them a formal definition by showing how an augmented rule can be translated into a logical sentence.
- The sentence will have the form of a definite clause, so the result is called a **definite clause grammar**, or DCG.

$$Article(a, s_1) \wedge Adjs(j, s_2) \wedge Noun(n, s_3) \wedge Compatible(j, n) \\ \Rightarrow NP(n, Append(s_1, s_2, s_3)) .$$

Case agreement and subject–verb agreement

- We splitting NP into two categories, NPS and NPO, to stand for noun phrases in the subjective and objective case, respectively.
- We would also need to split the category Pronoun into the two categories PronounS (which includes “I”) and PronounO (which includes “me”).

$\mathcal{E}_1 :$	S	$\rightarrow$	$NP_S VP \mid \dots$
	NP_S	$\rightarrow$	$Pronoun_S \mid Name \mid Noun \mid \dots$
	NP_O	$\rightarrow$	$Pronoun_O \mid Name \mid Noun \mid \dots$
	VP	$\rightarrow$	$VP NP_O \mid \dots$
	PP	$\rightarrow$	$Prep NP_O$
	$Pronoun_S$	$\rightarrow$	I you he she it ...
	$Pronoun_O$	$\rightarrow$	me you him her it ...
			...
$\mathcal{E}_2 :$	$S(head)$	$\rightarrow$	$NP(Sbj, pn, h) VP(pn, head) \mid \dots$
	$NP(c, pn, head)$	$\rightarrow$	$Pronoun(c, pn, head) \mid Noun(c, pn, head) \mid \dots$
	$VP(pn, head)$	$\rightarrow$	$VP(pn, head) NP(Obj, p, h) \mid \dots$
	$PP(head)$	$\rightarrow$	$Prep(head) NP(Obj, pn, h)$
	$Pronoun(Sbj, 1S, \mathbf{I})$	$\rightarrow$	I
	$Pronoun(Sbj, 1P, \mathbf{we})$	$\rightarrow$	we
	$Pronoun(Obj, 1S, \mathbf{me})$	$\rightarrow$	me
	$Pronoun(Obj, 3P, \mathbf{them})$	$\rightarrow$	them
			...

Figure 23.7 Top: part of a grammar for the language $\mathcal{E}_1$, which handles subjective and objective cases in noun phrases and thus does not overgenerate quite as badly as $\mathcal{E}_0$. The portions that are identical to $\mathcal{E}_0$ have been omitted. Bottom: part of an augmented grammar for $\mathcal{E}_2$, with three augmentations: case agreement, subject–verb agreement, and head word. *Sbj*, *Obj*, *1S*, *1P* and *3P* are constants, and lowercase names are variables.

Semantic interpretation

To show how to add semantics to a grammar, we start with an example that is simpler than English: the semantics of arithmetic expressions.

$$\begin{aligned} \textit{Exp}(x) &\rightarrow \textit{Exp}(x_1) \textit{Operator}(op) \textit{Exp}(x_2) \{x = \textit{Apply}(op, x_1, x_2)\} \\ \textit{Exp}(x) &\rightarrow (\textit{Exp}(x)) \\ \textit{Exp}(x) &\rightarrow \textit{Number}(x) \\ \textit{Number}(x) &\rightarrow \textit{Digit}(x) \\ \textit{Number}(x) &\rightarrow \textit{Number}(x_1) \textit{Digit}(x_2) \{x = 10 \times x_1 + x_2\} \\ \textit{Digit}(x) &\rightarrow x \{0 \leq x \leq 9\} \\ \textit{Operator}(x) &\rightarrow x \{x \in \{+, -, \div, \times\}\} \end{aligned}$$

Figure 23.8 A grammar for arithmetic expressions, augmented with semantics. Each variable x_i represents the semantics of a constituent. Note the use of the $\{test\}$ notation to define logical predicates that must be satisfied, but that are not constituents.

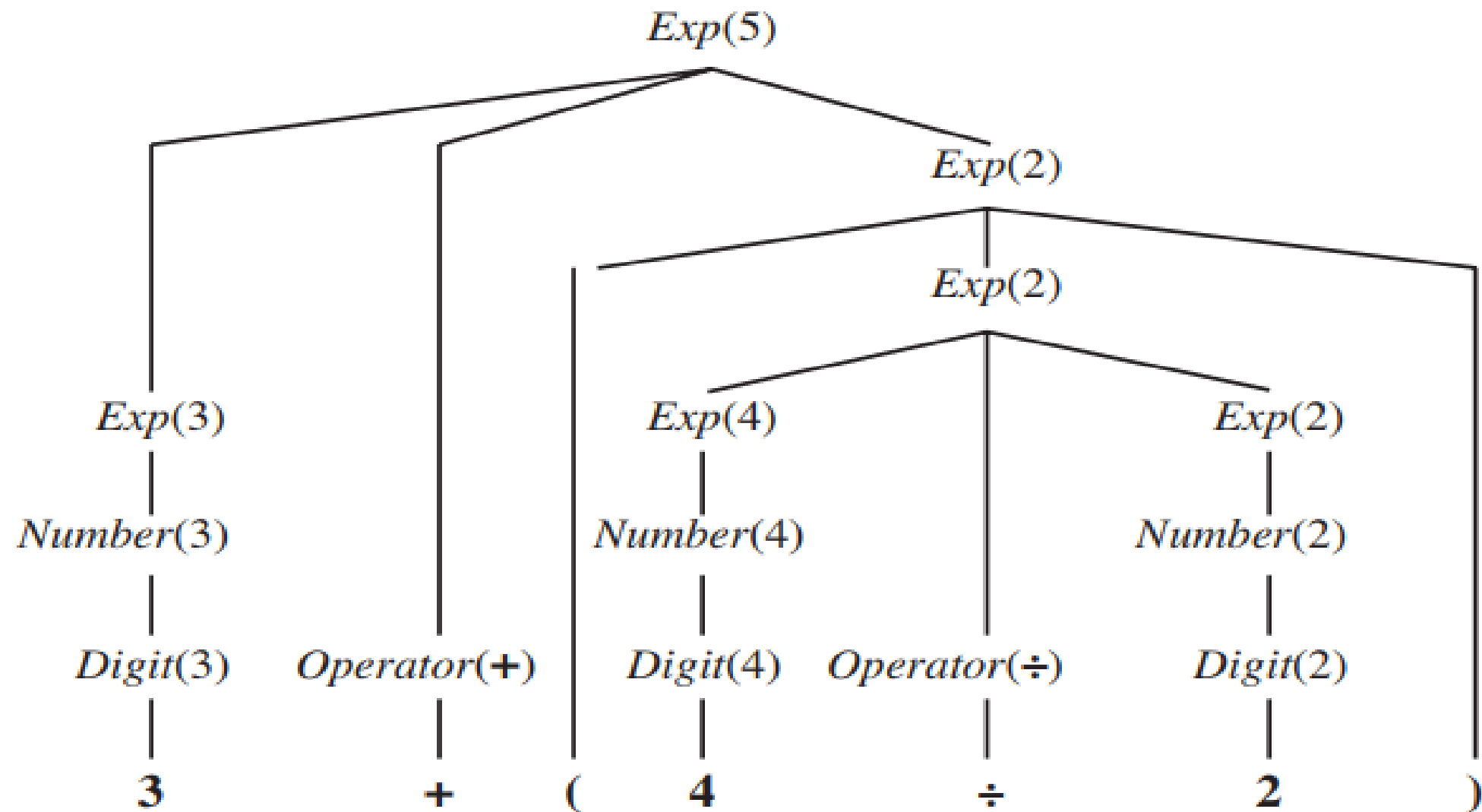


Figure 23.9 Parse tree with semantic interpretations for the string "3 + (4 ÷ 2)".

MACHINE TRANSLATION

- **Rough translation**, as provided by free online services, gives the “gist” of a foreign sentence or document, but contains errors.
- **Pre-edited translation** is used by companies to publish their documentation and sales materials in multiple languages.

The original source text is written in a constrained language that is easier to translate automatically, and the results are usually edited by a human to correct any errors.

- **Restricted-source translation** works fully automatically, but only on highly stereotypical language, such as a weather report.

Machine translation systems

- Some systems attempt to analyze the source language text all the way into an interlingua knowledge representation and then generate sentences in the target language from that representation.
- This is difficult because it involves **three unsolved problems**:
 - ✓ creating a complete knowledge representation of everything;
 - ✓ parsing into that representation; and
 - ✓ generating sentences from that representation.
- Other systems are based on a **transfer model**.
- They keep a database of translation rules (or examples), and whenever the rule (or example) matches, they translate directly.
- Transfer can occur at the lexical, syntactic, or semantic level.

Interlingua Semantics
Attraction(NamedJohn,μNamedMary,μHigh)

English Semantics
Loves(John,μMary)

French Semantics
Aime(Jean,μMarie)

English Syntax
S(NP(John),μVP(loves,μNP(Mary)))

French Syntax
S(NP(Jean),μVP(aime,μNP(Marie)))

English Words
John loves Mary

French Words
Jean aime Marie



Statistical machine translation

- This approach does not need a complex ontology of interlingua concepts, nor does it need handcrafted grammars of the source and target languages, nor a hand-labeled treebank.
- All it needs is data—sample translations from which a translation model can be learned. To translate a sentence in, say, English (e) into French (f), we find the string of words f^* that maximizes

$$f^* = \operatorname{argmax}_f P(f | e) = \operatorname{argmax}_f P(e | f) P(f) .$$

- Here the factor $P(f)$ is the target **language model** for French; it says how probable a given sentence is in French. $P(e|f)$ is the **translation mode**.
- All that remains is to learn the phrasal and distortion probabilities. We sketch the procedure;

SPEECH RECOGNITION

- **Speech recognition** is the task of identifying a sequence of words uttered by a speaker, given the acoustic signal.
- It has become one of the mainstream applications of AI—millions of people interact with speech recognition systems every day to navigate voice mail systems, search the Web from mobile phones, and other applications.
- Speech is an attractive option when hands-free operation is necessary, as when operating machinery.
 - Speech recognition is difficult because the sounds made by a speaker are ambiguous and, well, noisy.
- Several issues that make speech problematic :

- First, **segmentation**: written words in English have spaces between them, but in fast speech there are no pauses in “wreck a nice” that would distinguish it as a multiword phrase as opposed to the single word “recognize.”
- Second, **coarticulation**: when speaking quickly the “s” sound at the end of “nice” merges with the “b” sound at the beginning of “beach,” yielding something that is close to a “sp.”
- Another problem that does not show up in this example is **homophones**—words like “to,” “too,” and “two” that sound the same but differ in meaning.

$$\operatorname{argmax}_{word_{1:t}} P(word_{1:t} \mid sound_{1:t}) = \operatorname{argmax}_{word_{1:t}} P(sound_{1:t} \mid word_{1:t}) P(word_{1:t}) .$$

- Most speech recognition systems use a language model that makes the Markov assumption—that the current state Word t depends only on a fixed number n of previous states—and represent Word t as a single random variable taking on a finite set of values, which makes it a Hidden Markov Model (HMM).

Acoustic model

- An analog-to-digital converter measures the size of the current—which approximates the amplitude of the sound wave—at discrete intervals called the **sampling rate**.
- The precision of each measurement is determined by the **quantization factor**; speech recognizers typically keep 8 to 12 bits.
- **A phoneme** is the smallest unit of sound that has a distinct meaning to speakers of a particular language.
- For example, the “t” in “stick” sounds similar enough to the “t” in “tick” that speakers of English consider them the same phoneme.
- First, we observe that although the sound frequencies in speech may be several kHz, the *changes* in the content of the signal occur much less often, perhaps at no more than 100 Hz.

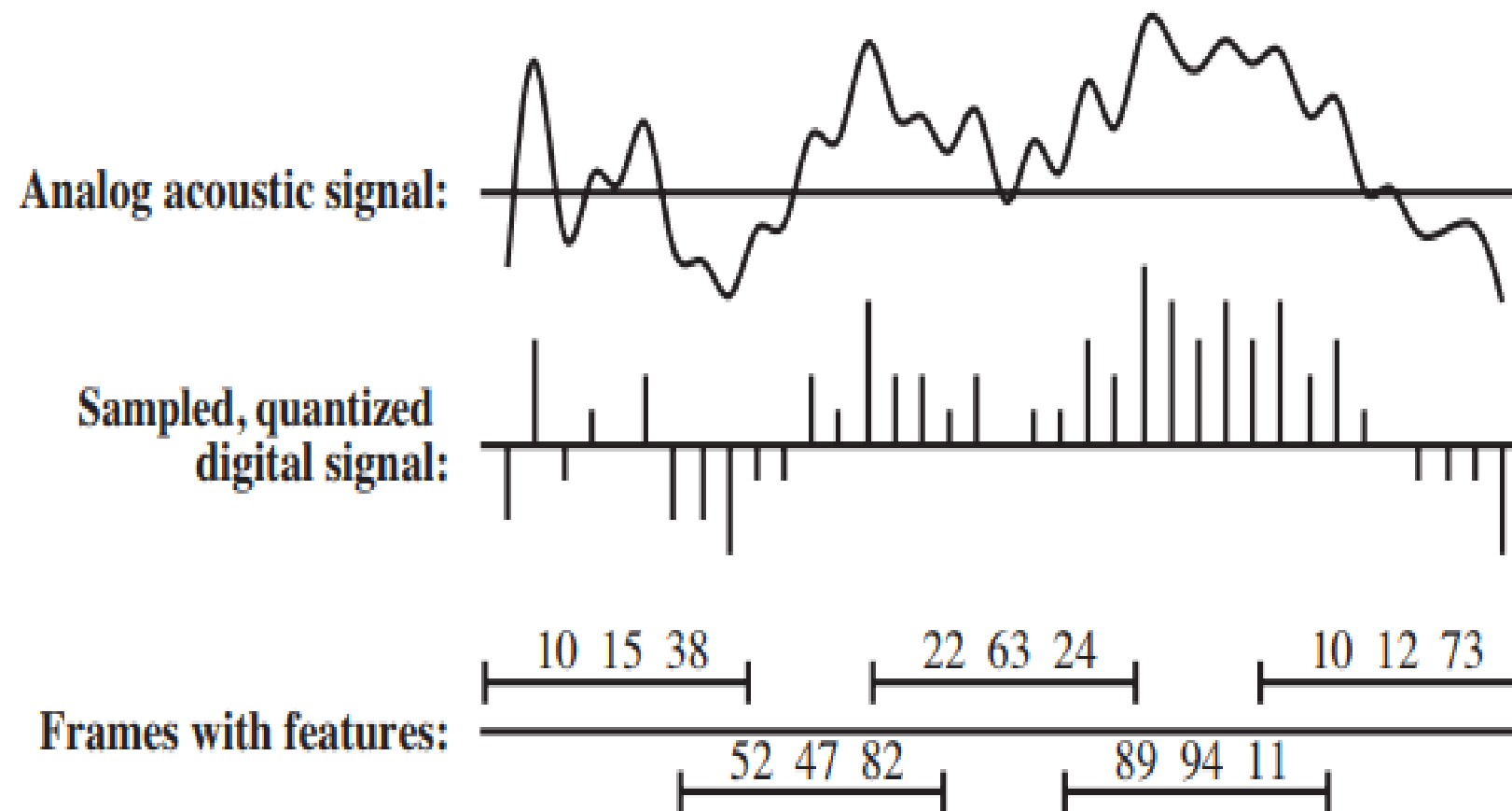


Figure 23.15 Translating the acoustic signal into a sequence of frames. In this diagram each frame is described by the discretized values of three acoustic features; a real system would have dozens of features.

Language model

- For general-purpose speech recognition, the language model can be an n-gram model of text learned from a corpus of written sentences.
- However, spoken language has different characteristics than written language, so it is better to get a corpus of transcripts of spoken language.
- For task-specific speech recognition, the corpus should be task-specific: to build your airline reservation system, get transcripts of prior calls.
 - It also helps to have task-specific vocabulary, such as a list of all the airports and cities served, and all the flight numbers.

IMAGE FORMATION

Imaging distorts the appearance of objects. For example, a picture taken looking down a long straight set of railway tracks will suggest that the rails converge and meet. As another example, if you hold your hand in front of your eye, you can block out the moon, which is not smaller than your hand. As you move your hand back and forth or tilt it, your hand will seem to shrink and grow in the image, but it is not doing so in reality. Models of these effects are essential for both recognition and reconstruction.

Images without lenses: The pinhole camera

In cameras, the image is formed on an image plane, which can be a piece of film coated with silver halides or a rectangular grid of a few million photosensitive **pixels**, each a complementary metal-oxide semiconductor (CMOS) or charge-coupled device (CCD).

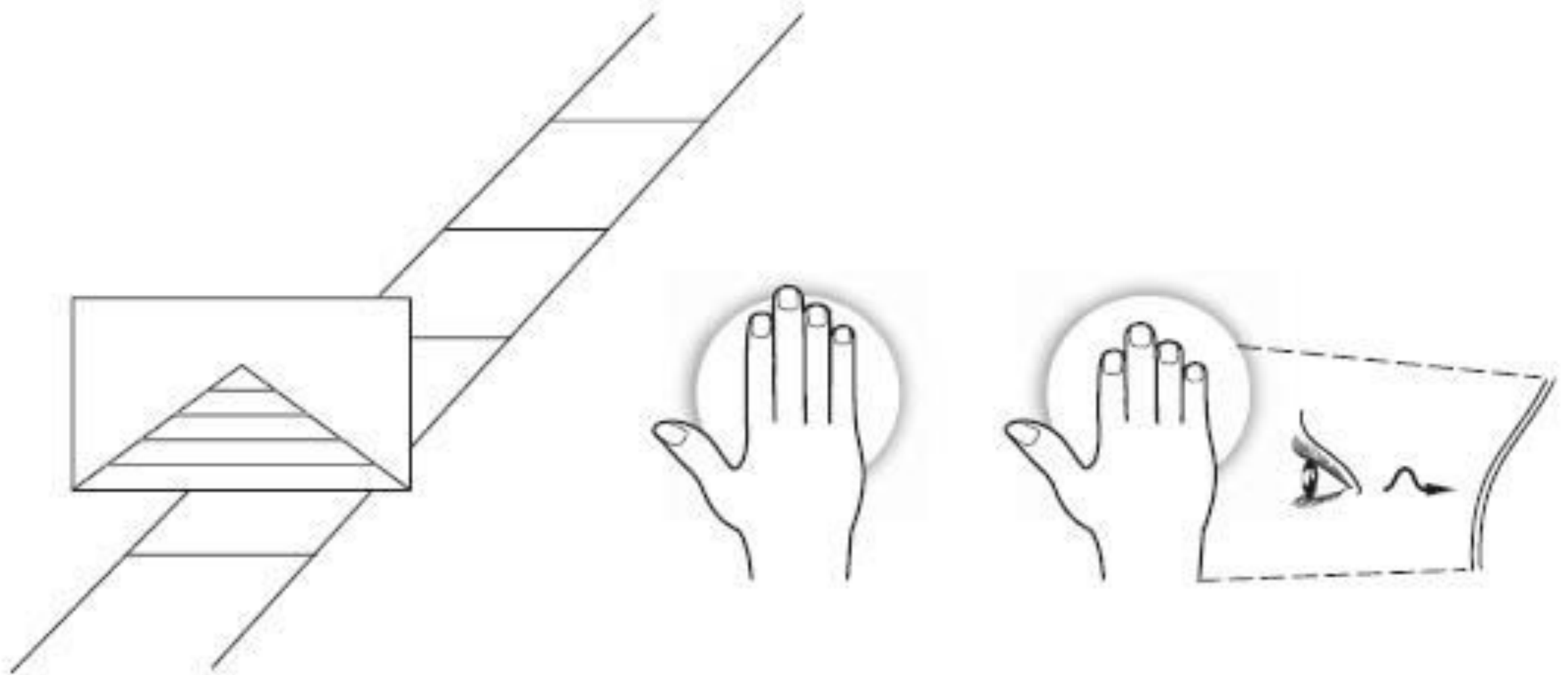
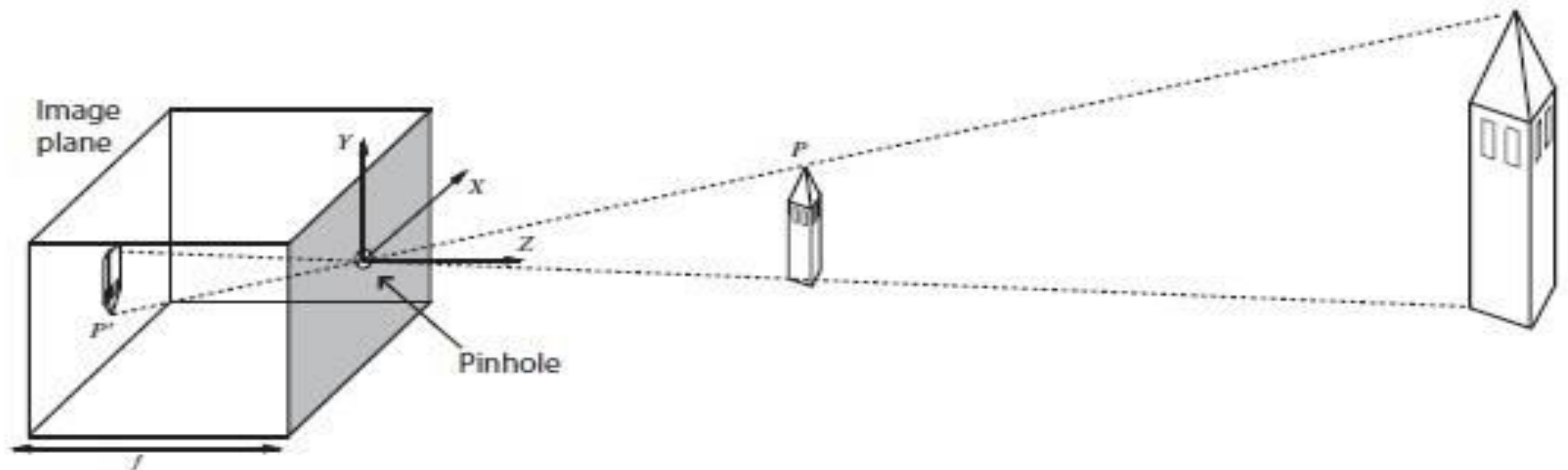


Figure 24.1 Imaging distorts geometry. Parallel lines appear to meet in the distance, as in the image of the railway tracks on the left. In the center, a small hand blocks out most of a large moon. On the right is a foreshortening effect: the hand is tilted away from the eye, making it appear shorter than in the center figure.

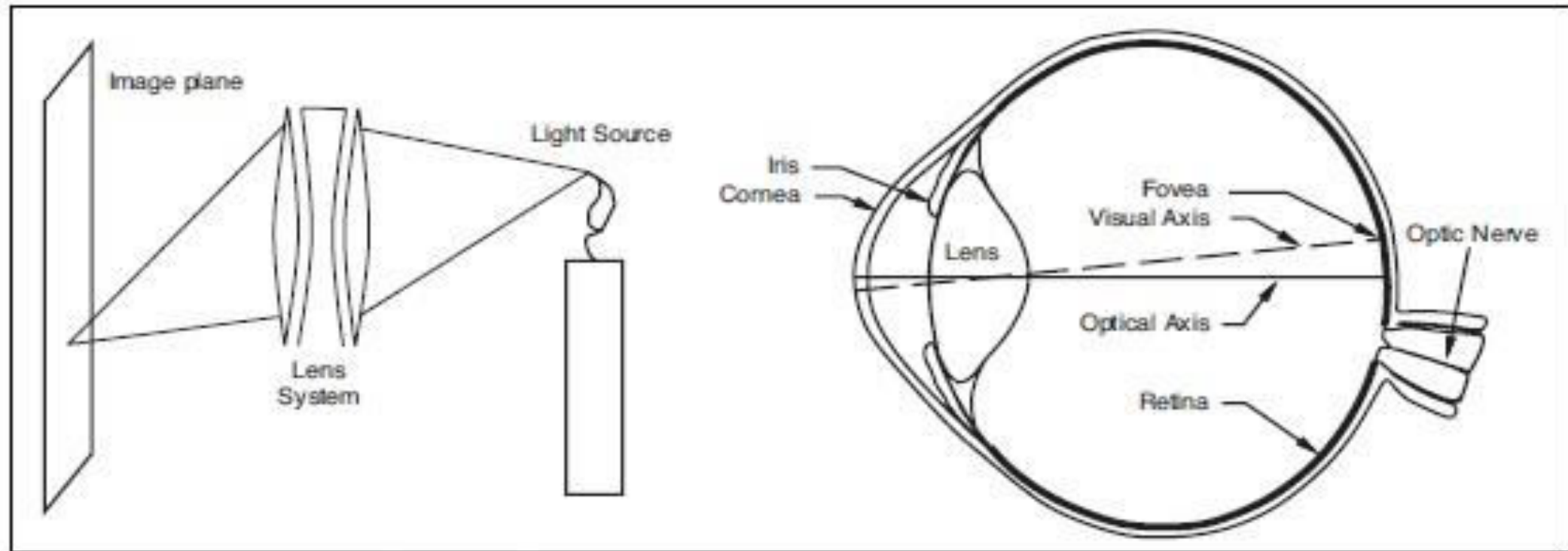
$$\frac{-x}{f} = \frac{X}{Z}, \quad \frac{-y}{f} = \frac{Y}{Z} \quad \Rightarrow \quad x = \frac{-fX}{Z}, \quad y = \frac{-fY}{Z}.$$



Each light-sensitive element in the image plane at the back of a pinhole camera receives light from a small range of directions that passes through the pinhole. If the pinhole is small enough, the result is a focused image at the back of the pinhole. The process of projection means that large, distant objects look the same as smaller, nearby objects. Note that the image is projected upside down.

$$\left(f \frac{X_0 + \lambda U}{Z_0 + \lambda W}, f \frac{Y_0 + \lambda V}{Z_0 + \lambda W} \right)$$

Lens systems



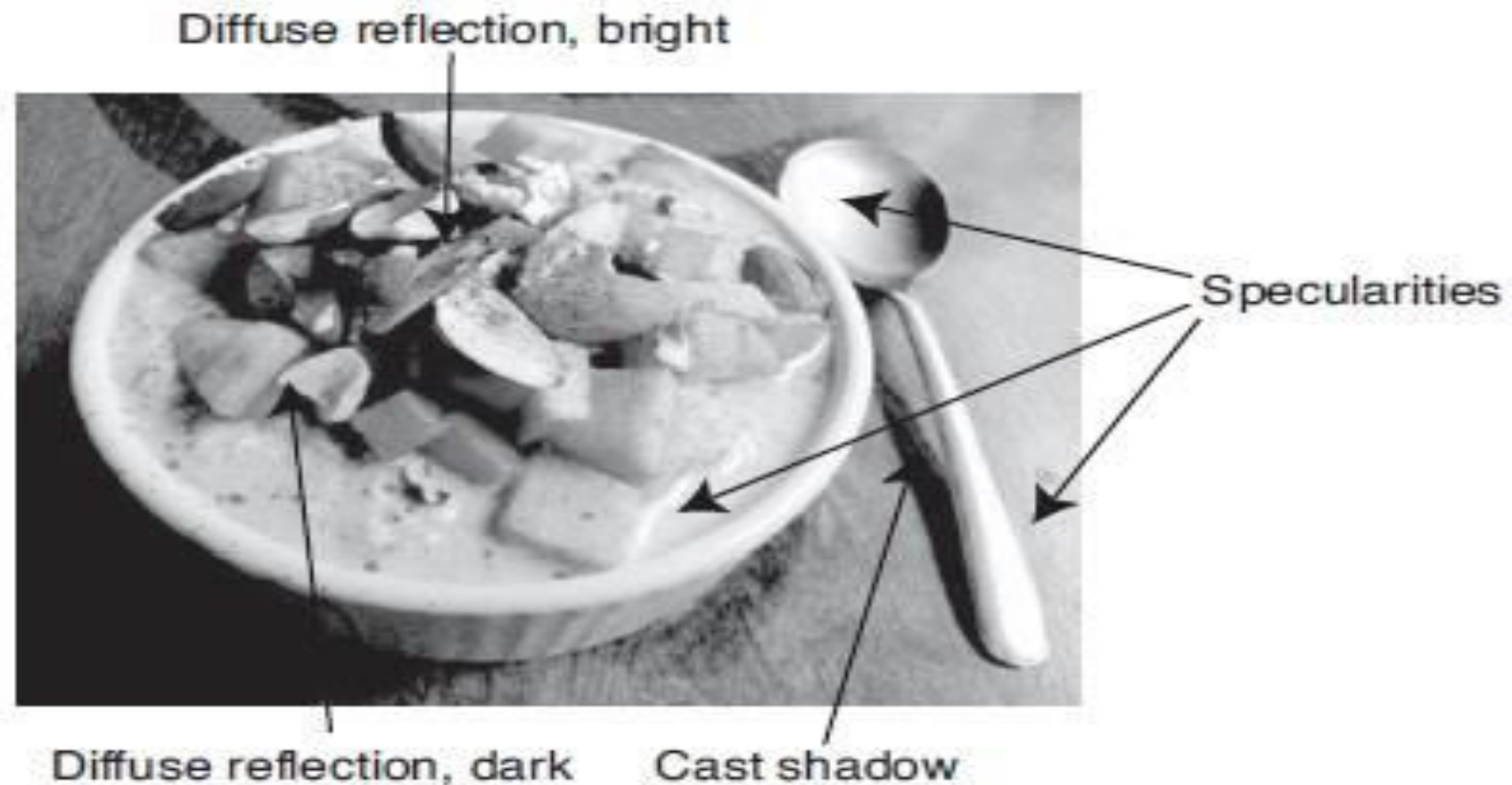
Lenses collect the light leaving a scene point in a range of directions, and steer it all to arrive at a single point on the image plane. Focusing works for points lying close to a focal plane in space; other points will not be focused properly. In cameras, elements of the lens system move to change the focal plane, whereas in the eye, the shape of the lens is changed by specialized muscles.

Scaled orthographic projection

The appropriate model is **scaled orthographic projection**. The idea is as follows: If the depth Z of points on the object varies within some range $Z_0 \pm \Delta Z$, with $\Delta Z \ll Z_0$, then the perspective scaling factor f/Z can be approximated by a constant $s = f/Z_0$. The equations for projection from the scene coordinates (X, Y, Z) to the image plane become $x = sX$ and $y = sY$. Scaled orthographic projection is an approximation that is valid only for those parts of the scene with not much internal depth variation. For example, scaled orthographic projection can be a good model for the features on the front of a distant building.

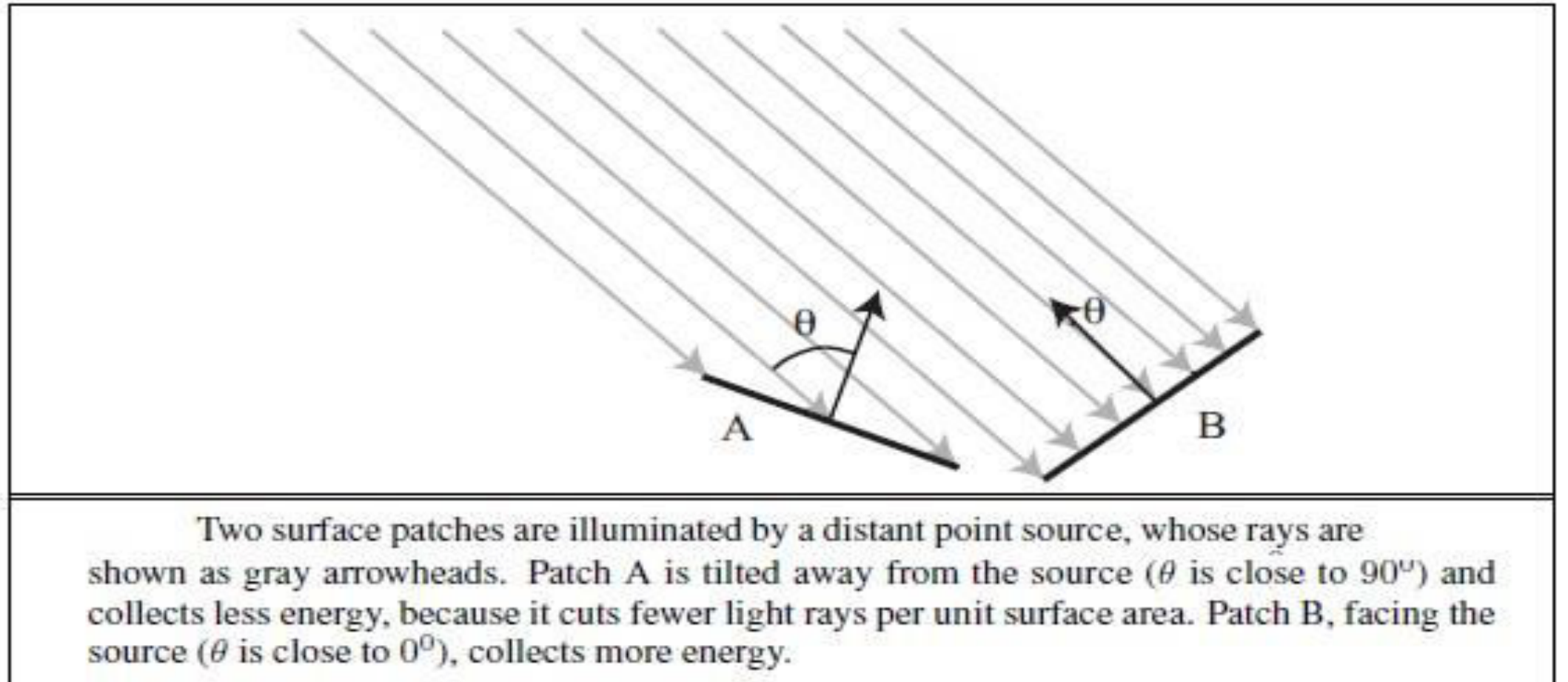
Light and shading

The first cause is **overall intensity** of the light. Even though a white object in shadow may be less bright than a black object in direct sunlight, the eye can distinguish relative brightness well, and perceive the white object as white. Second, different points in the scene may **reflect** more or less of the light. Third, surface patches facing the light are brighter than surface patches tilted away from the light, an effect known as **shading**. Most surfaces reflect light by a process of **diffuse reflection**.



A variety of illumination effects. There are specularities on the metal spoon and on the milk. The bright diffuse surface is bright because it faces the light direction. The dark diffuse surface is dark because it is tangential to the illumination direction. The shadows appear at surface points that cannot see the light source. Photo by Mike Linksvayer (mlinksva on flickr).

The main source of illumination outside is the sun, whose rays all travel parallel to one another. We model this behaviour as a **distant point light source**. This is the most important model of lighting, and is quite effective for indoor scenes as well as outdoor scenes. The amount of light collected by a surface patch in this model depends on the angle θ between the illumination direction and the normal to the surface.

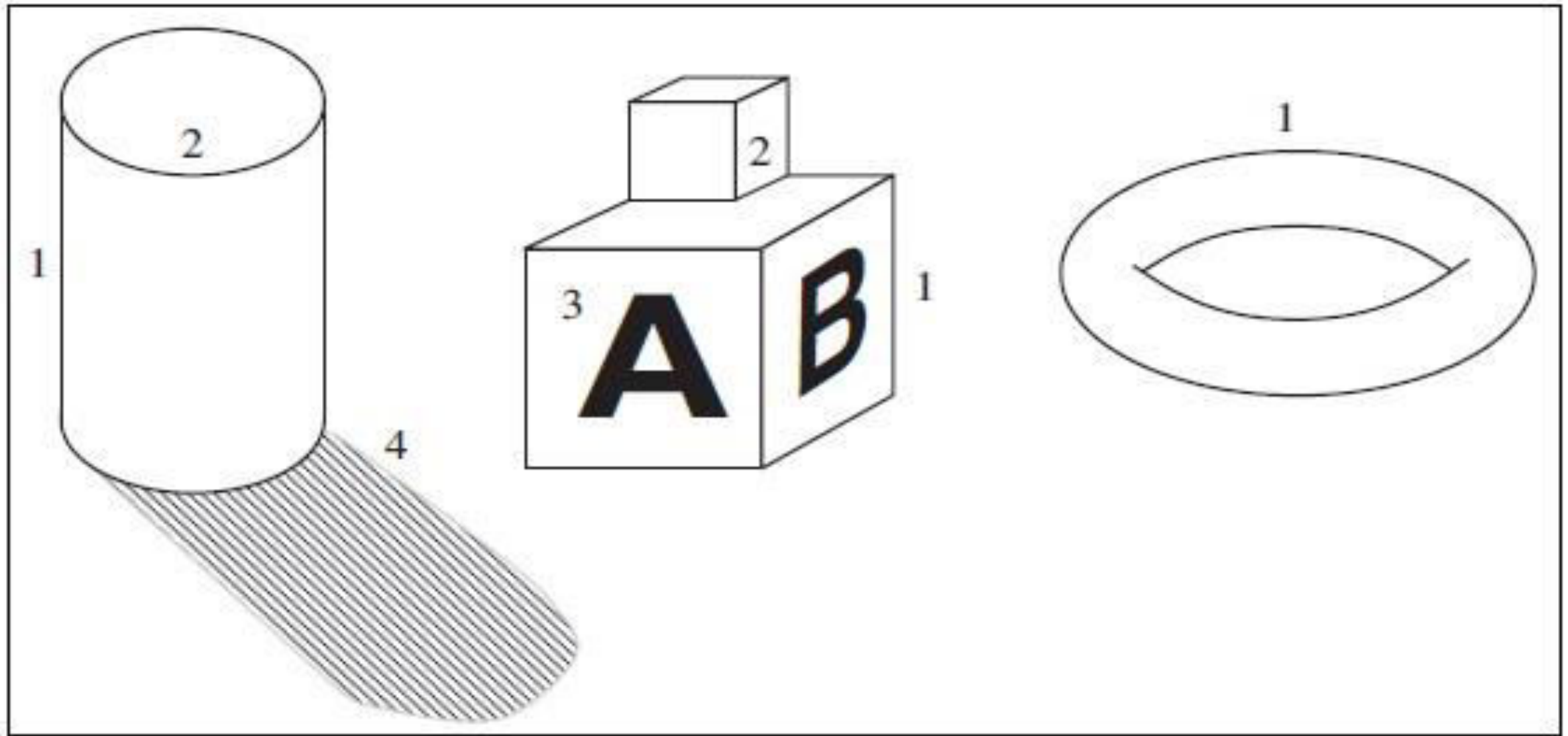


Colour

The **principle of trichromacy** states that for any spectral energy density, no matter how complicated, it is possible to construct another spectral energy density consisting of a mixture of just three colours—usually red, green, and blue—such that a human can't tell the difference between the two. That means that our TVs and computer displays can get by with just the three red/green/blue (or R/G/B) colour elements. It makes our computer vision algorithms easier, too. Each surface can be modelled with three different albedos for R/G/B. Similarly, each light source can be modelled with three R/G/B intensities. We then apply Lambert's cosine law to each to get three R/G/B pixel values. This model predicts, correctly, that the same surface will produce different coloured image patches under different-coloured lights. In fact, human observers are quite good at ignoring the effects of different coloured lights and are able to estimate the colour of the surface under white light, an effect known as **colour constancy**.

EARLY IMAGE-PROCESSING OPERATIONS

we will study three useful image-processing operations: edge detection, texture analysis, and computation of optical flow. These are called “early” or “low-level” operations because they are the first in a pipeline of operations. Early vision operations are characterized by their local nature (they can be carried out in one part of the image without regard for anything more than a few pixels away) and by their lack of knowledge: we can perform these operations without consideration of the objects that might be present in the scene. This makes the low-level operations good candidates for implementation in parallel hardware—either in a graphics processor unit (GPU) or an eye. We will then look at one mid-level operation: segmenting the image into regions.



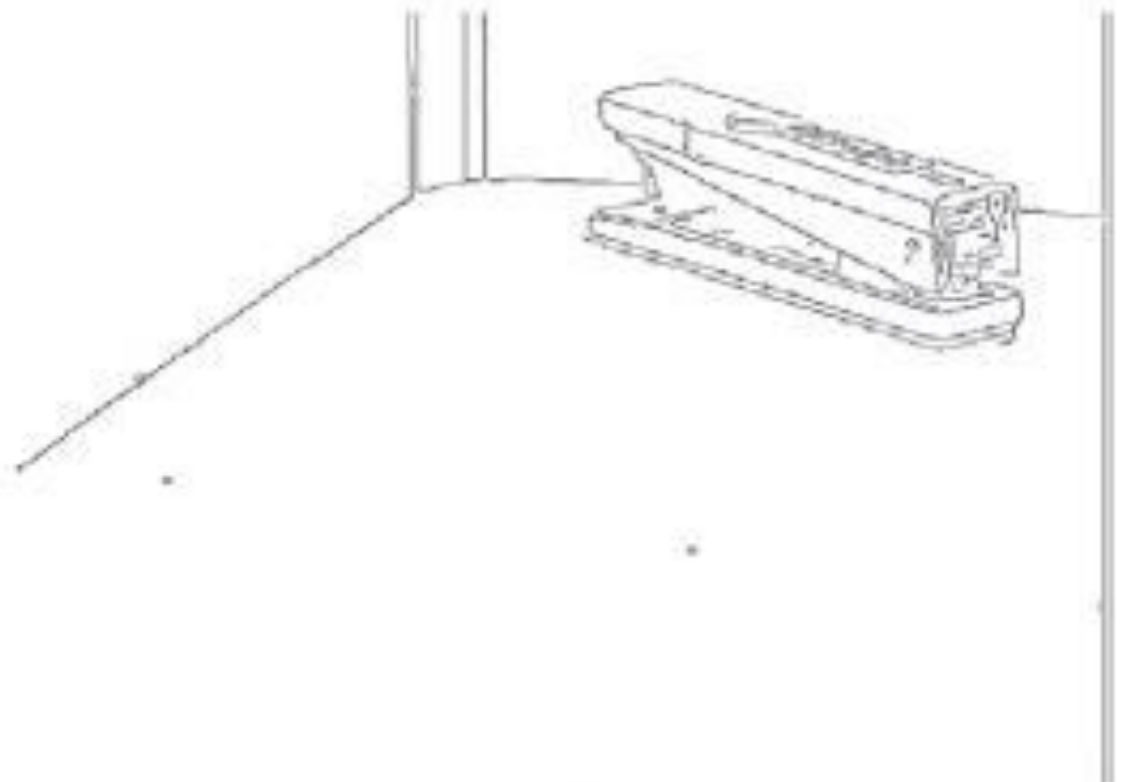
Different kinds of edges: (1) depth discontinuities; (2) surface orientation discontinuities; (3) reflectance discontinuities; (4) illumination discontinuities (shadows).

Edge detection

Edges are straight lines or curves EDGE in the image plane across which there is a “significant” change in image brightness. The goal of edge detection is to abstract away from the messy, multimega byte image and toward a more compact, abstract representation, the motivation is that edge contours in the image correspond to important scene contours. In the figure we have three examples of depth discontinuity, labelled 1; two surface-normal discontinuities, labelled 2; a reflectance discontinuity, labelled 3; and an illumination discontinuity (shadow), labelled 4. Edge detection is concerned only with the image, and thus does not distinguish between these different types of scene discontinuities; later processing will.



(a)



(b)

(a) Photograph of a stapler. (b) Edges computed from (a).

(a) shows an image of a scene containing a stapler resting on a desk

(b) shows the output of an edge-detection algorithm on this image.

INTRODUCTION

- **Robots** are physical agents that perform tasks by manipulating the physical world. To do so they are equipped with **effectors** such as legs, wheels, joints, and grippers. Effectors have a single purpose: to assert physical forces on the environment. Robots are also equipped with **sensors**, which allow them to perceive their environment.
- Present day robotics employs a diverse set of sensors, including cameras and lasers to measure the environment, and gyroscopes and accelerometers to measure the robot's own motion.
- Most of today's robots fall into one of three primary categories. **Manipulators**,
- The second category is the **mobile robot**. Mobile robots move about their environment using wheels, legs, or similar mechanisms.
- They have been put to use delivering food in hospitals, moving containers at loading docks, and similar tasks. **Unmanned ground vehicles**, or **UGVs**, drive autonomously on streets, highways, and off-road.

- The **planetary rover** shown in Figure 25.2(b) explored Mars for a period of 3 months in 1997.
- Other types of mobile robots include **unmanned air vehicles** (UAVs), commonly used for surveillance, crop-spraying



(a)

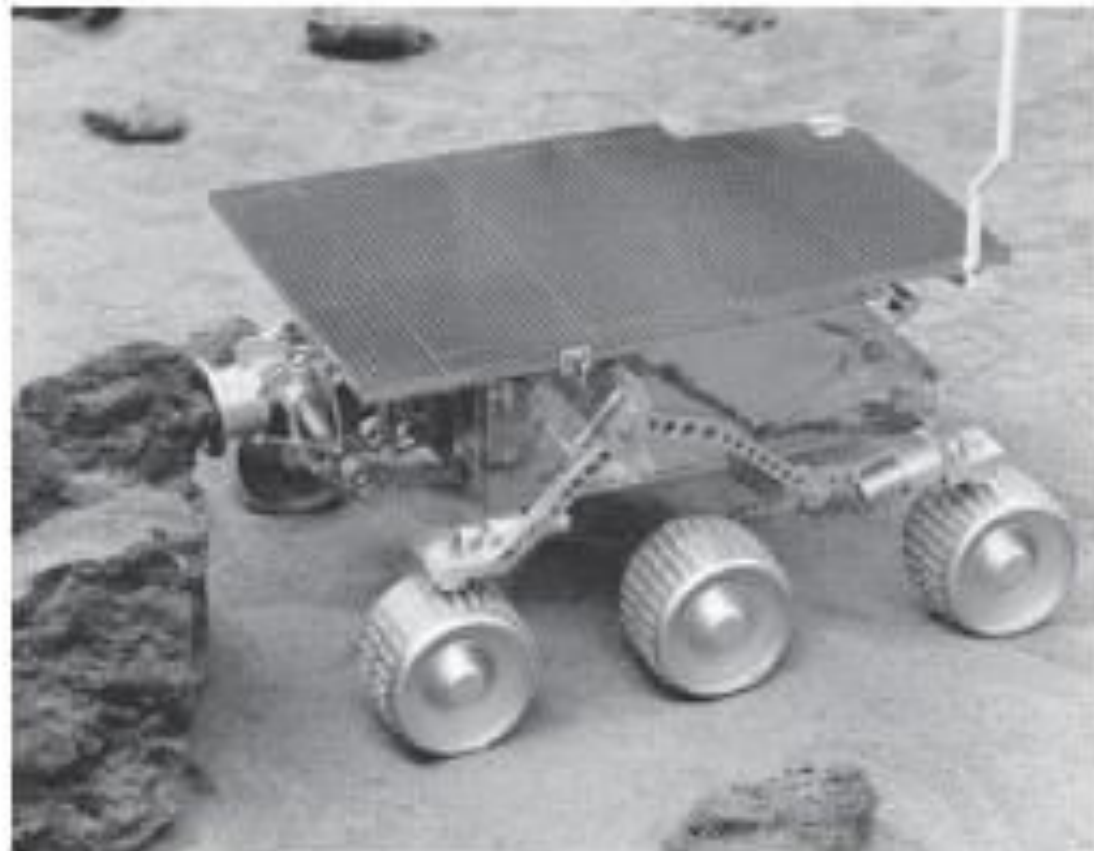


(b)

Figure 25.1 (a) An industrial robotic manipulator for stacking bags on a pallet. Image courtesy of Nachi Robotic Systems. (b) Honda's P3 and Asimo humanoid robots.



(a)



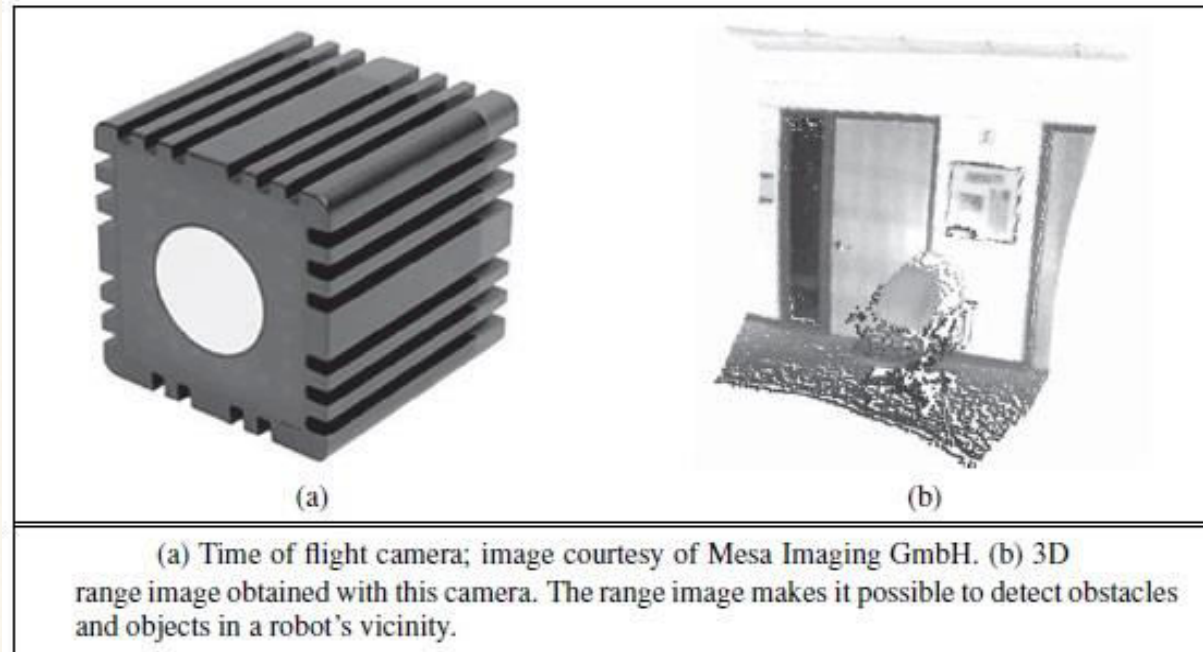
(b)

(a) Predator, an unmanned aerial vehicle (UAV) used by the U.S. Military. Image courtesy of General Atomics Aeronautical Systems. (b) NASA's Sojourner, a mobile robot that explored the surface of Mars in July 1997.

- The third type of robot combines mobility with manipulation, and is often called a **mobile manipulator**. **Humanoid robots** mimic the human torso. shows two early humanoid robots, both manufactured by Honda Corp. in Japan. Mobile manipulators can apply their effectors further afield than anchored manipulators can, but their task is made harder because they don't have the rigidity that the anchor provides.
- UAV commonly used by the U.S. military. **Autonomous underwater vehicles** (AUVs) are used in deep sea exploration. Mobile robots deliver packages in the workplace and vacuum the floors at home.

ROBOT HARDWARE

- **Sensors:** Sensors are the perceptual interface between robot and environment
- **Passive sensors**, such as cameras, are true observers of the environment: they capture signals that are generated by other sources in the environment.
- **Active sensors**, such as sonar, send energy into the environment.
- **Range finders** are sensors that measure the distance to nearby objects.
- In the early days of robotics, robots were commonly equipped with **sonar sensors**.

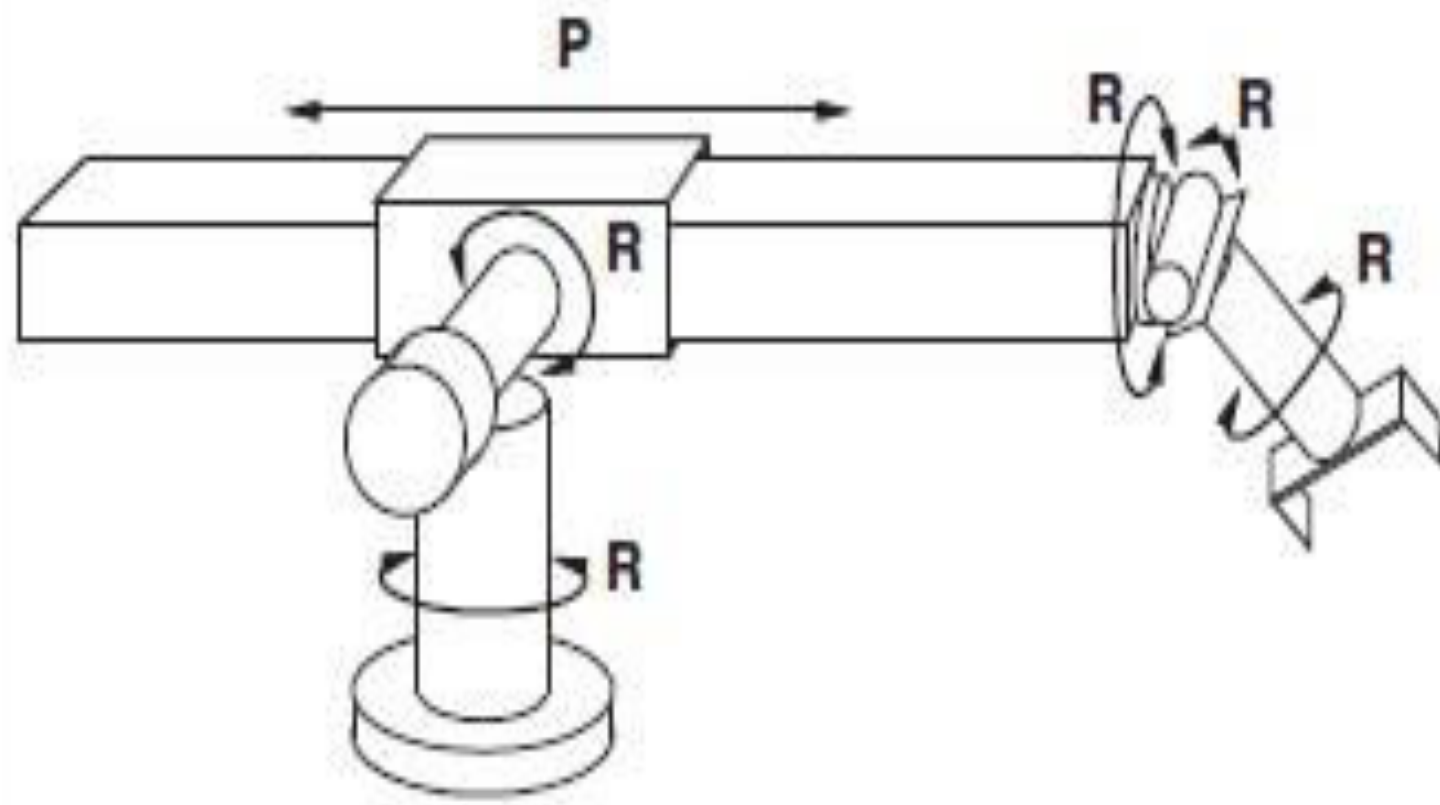


- **Stereo vision** relies STEREO VISION on multiple cameras to image the environment from slightly different viewpoints.
- shows a **time of flight camera**. This camera acquires range images like the one at up to 60 frames per second.
- These sensors are called **scanning lidars** (short for *light detection and ranging*).
- On the other extreme end of range sensing are **tactile sensors** such as whiskers
- A second important class of sensors is **location sensors**.
- Outdoors, the **Global Positioning System** is the most common solution to the localization problem.
- **Differential GPS** involves a second ground receiver with known location, providing millimetre accuracy under ideal conditions.
- The third important class is **proprioceptive sensors**, which inform the robot of its own motion.
- **Inertial sensors**, such as gyroscopes, rely on the resistance of mass to the change of velocity. They can help reduce uncertainty.

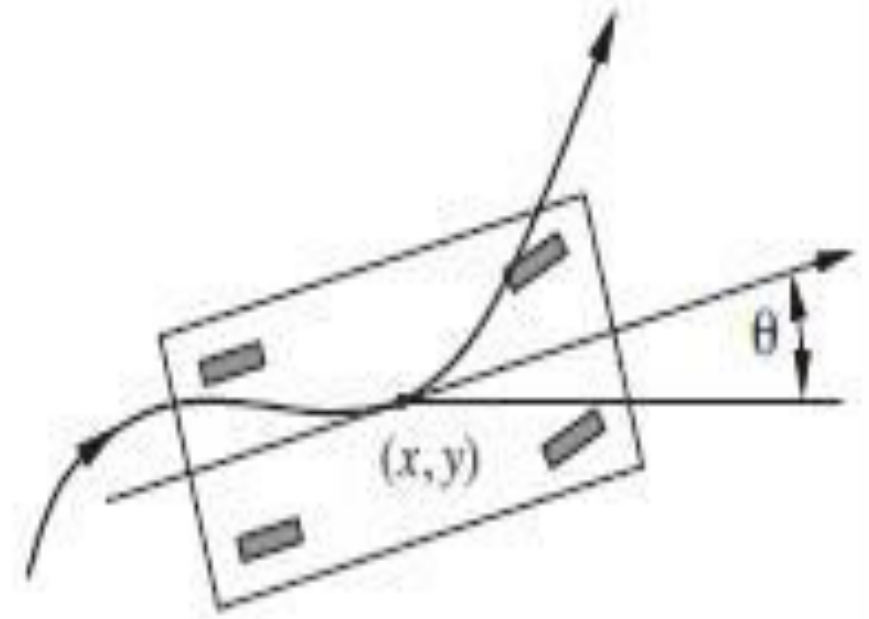
EFFECTORS

- To understand the design of effectors, it will help to talk about motion and shape in the abstract, using the concept of a **degree of freedom (DOF)**. We count one degree of freedom for each independent direction in which a robot, or one of its effectors.
- These six degrees define the **kinematic state** or **pose** of the robot. The **dynamic state** of a robot includes these six plus an additional six dimensions for the rate of change of each kinematic dimension, that is, their velocities.

created by five **revolute joints** that generate rotational motion and one **prismatic joint** that generates sliding motion. You can verify that the human arm as a whole has more than six degrees of freedom by a simple experiment: put your hand on the table and notice that you still have the freedom to rotate your elbow without changing the configuration of your hand. Manipulators that have extra degrees of freedom are easier to control than robots with only the minimum number of DOFs. Many industrial manipulators therefore have seven DOFs, not six.



(a)



(b)

(a) The Stanford Manipulator, an early robot arm with five revolute joints (R) and one prismatic joint (P), for a total of six degrees of freedom. (b) Motion of a nonholonomic four-wheeled vehicle with front-wheel steering.

Differential drive robots possess two independently actuated wheels (or tracks), one on each side, as on a military tank. If both wheels move at the same velocity, the robot moves on a straight line. If they move in opposite directions, the robot turns on the spot. An alternative is the **synchro drive**



(a)



(b)

(a) Mobile manipulator plugging its charge cable into a wall outlet. Image courtesy of Willow Garage, © 2009. (b) One of Marc Raibert's legged robots in motion.

Legged robots have been made to walk, run, and even hop—as we see with the legged robot. This robot is **dynamically stable**, meaning that it can remain upright while hopping around. A robot that can remain upright without moving its legs is called **statically stable**



(a)



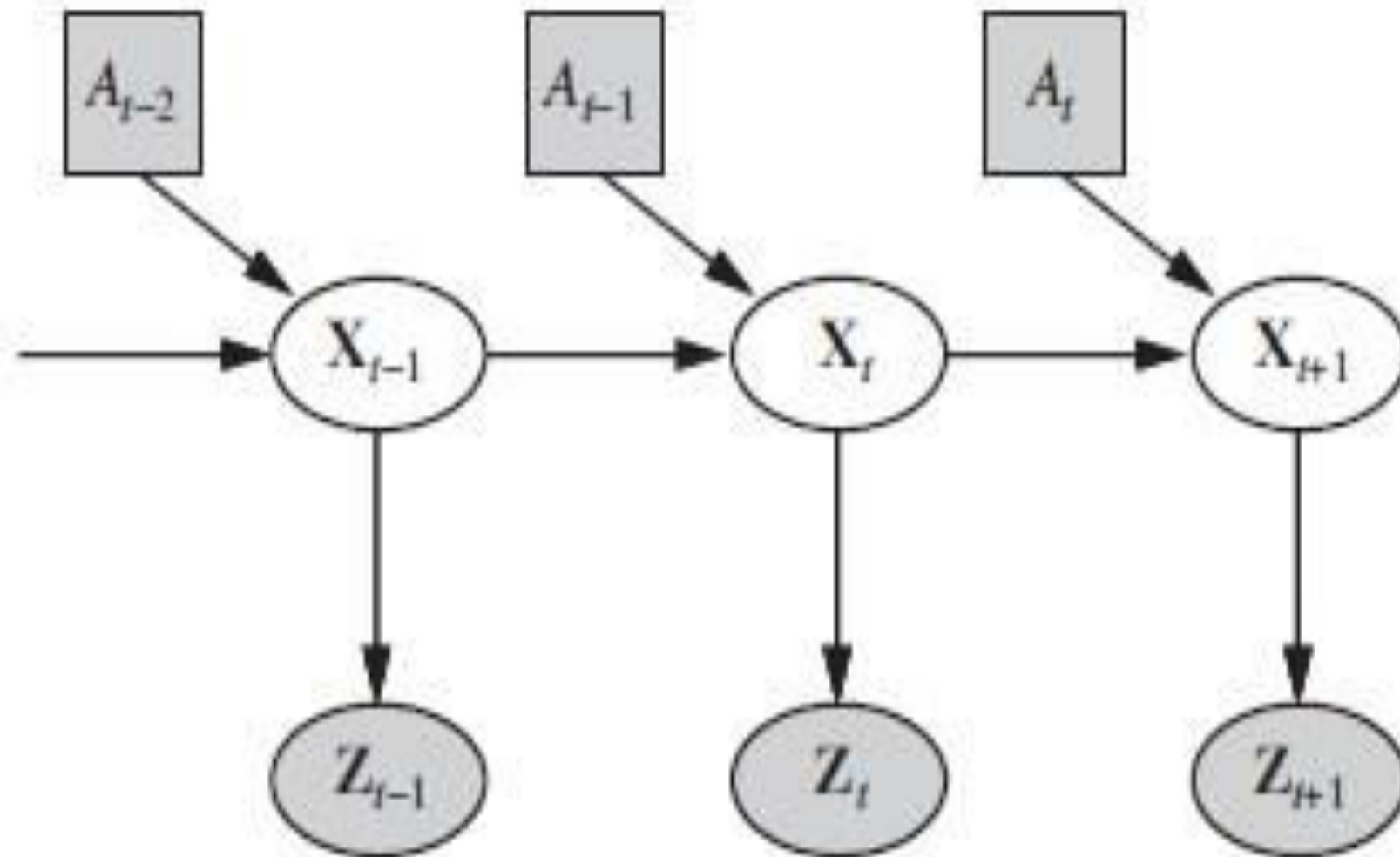
(b)

(a) Four-legged dynamically-stable robot “Big Dog.” Image courtesy Boston Dynamics, © 2009. (b) 2009 RoboCup Standard Platform League competition, showing the winning team, B-Human, from the DFKI center at the University of Bremen. Throughout the match, B-Human outscored their opponents 64:1. Their success was built on probabilistic state estimation using particle filters and Kalman filters; on machine-learning models for gait optimization; and on dynamic kicking moves. Image courtesy DFKI, © 2009.

ROBOTIC PERCEPTION

Perception is the process by which robots map sensor measurements into internal representations of the environment. Perception is difficult because sensors are noisy, and the environment is partially observable, unpredictable, and often dynamic. In other words, robots have all the problems of **state estimation (or filtering)**. As a rule of thumb, good internal representations for robots have three properties: they contain enough information for the robot to make good decisions, they are structured so that they can be updated efficiently, and they are natural in the sense that internal variables correspond to natural state variables in the physical world.

we saw that Kalman filters, HMMs, and dynamic Bayes nets can represent the transition and sensor models of a partially observable environment, and we described both exact and approximate algorithms for updating the **belief state**



Robot perception can be viewed as temporal inference from sequences of actions and measurements, as illustrated by this dynamic Bayes network.

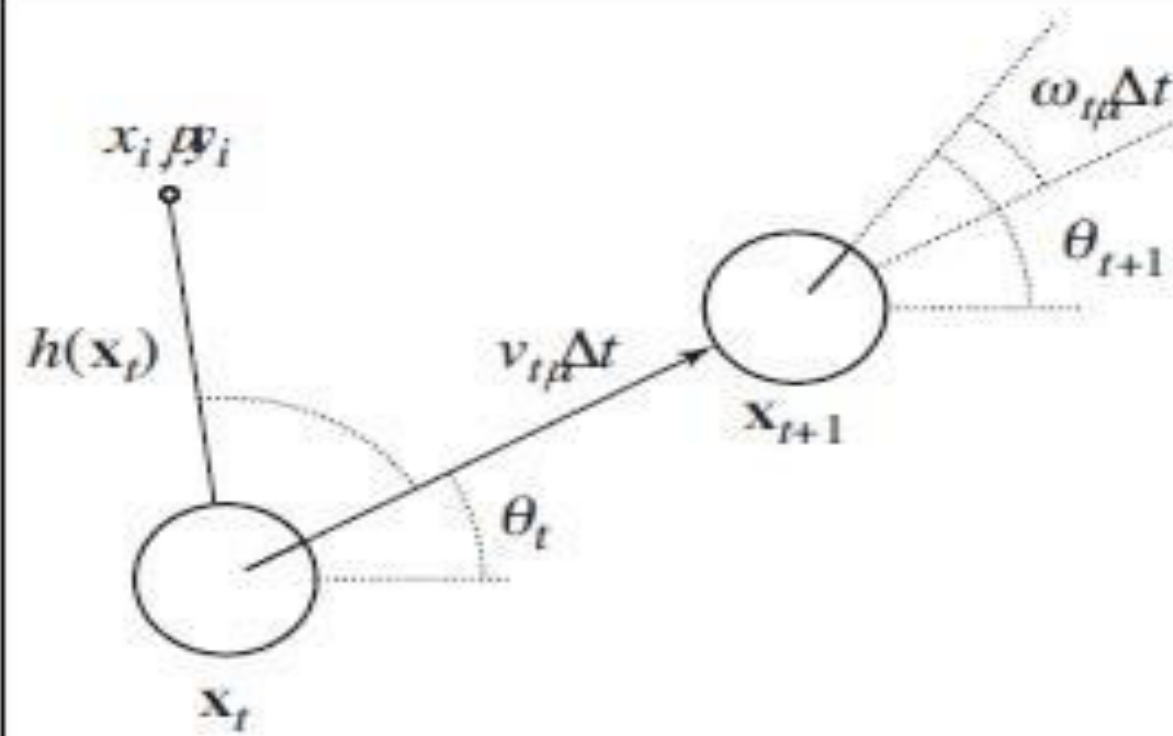
We would like to compute the new belief state, $\mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{z}_{1:t+1}, a_{1:t})$, from the current belief state $\mathbf{P}(\mathbf{X}_t \mid \mathbf{z}_{1:t}, a_{1:t-1})$ and the new observation $\mathbf{z}_{t+1}$. We did this in Section 15.2, but here there are two differences: we condition explicitly on the actions as well as the observations, and we deal with *continuous* rather than *discrete* variables

$$\begin{aligned} \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{z}_{1:t+1}, a_{1:t}) \\ = \alpha \mathbf{P}(\mathbf{z}_{t+1} \mid \mathbf{X}_{t+1}) \int \mathbf{P}(\mathbf{X}_{t+1} \mid \mathbf{x}_t, a_t) P(\mathbf{x}_t \mid \mathbf{z}_{1:t}, a_{1:t-1}) d\mathbf{x}_t . \end{aligned}$$

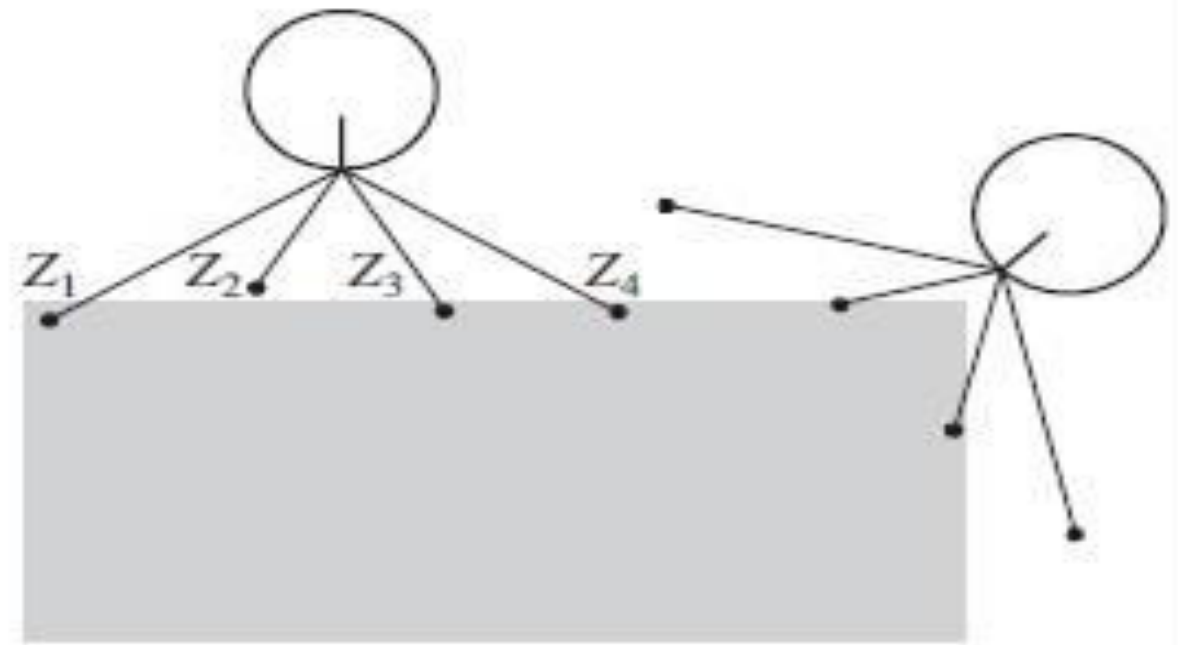
LOCALIZATION AND MAPPING

Localization is the problem of finding out where things are—including the robot itself.

Knowledge about where things are is at the core of any successful physical interaction with the environment. For example, robot manipulators must know the location of objects they seek to manipulate; navigating robots must know where they are to find their way around.



(a)



(b)

(a) A simplified kinematic model of a mobile robot. The robot is shown as a circle with an interior line marking the forward direction. The state $\mathbf{x}_t$ consists of the (x_t, y_t) position (shown implicitly) and the orientation θ_t . The new state $\mathbf{x}_{t+1}$ is obtained by an update in position of $v_t \Delta t$ and in orientation of $\omega_t \Delta t$. Also shown is a landmark at (x_i, y_i) observed at time t . (b) The range-scan sensor model. Two possible robot poses are shown for a given range scan (z_1, z_2, z_3, z_4) . It is much more likely that the pose on the left generated the range scan than the pose on the right.